

残酷共学 · 增量笔记

自 2026-08-14 23:02 BJT 后新增
108 篇

本次增量 · 带索引

点击目录条目可跳转

数据来源：intensivecolearn.ing 活动公开打卡

生成时间：2026-08-15 23:01 (BJT)

目录

主题章节 → 笔记索引 (点击可跳转)

01 AMM机制 (11 篇)	7
笔记 001: 用 AI Agent 辅助链上套利：从安装到学习、回测与自动化	7
笔记 002: 套利 · Uniswap AMM 机制	12
笔记 003: 我来帮你把这 159 条消息榨干提纯，结合你之前所有笔记的认知框架，给你一份精准、可执行的复盘。	12
笔记 004: 2026-08-15 · Day 11 / 21 · 成本模型写成代码	13
笔记 005: 今日打卡 套利不是找价差，而是修复市场不同步	15
笔记 006: 永续清算通常在账户权益低于维持保证金要求时触发。	16
笔记 007: 集成方不能默认“用户转出多少 Token，Pair 就一定收到多少 Token”。	17
笔记 008: Day 11 跨日复验报价曲线	22
笔记 009: Day 11 逐层拆解 Morpho Blue × MetaMorpho × sNUSD 闪电贷交易	22
笔记 010: 1. 今日核心任务	23
笔记 011: Day 10 打卡 链上套利残酷共學	23
02 LI.FI与跨链 (26 篇)	24
笔记 012: 一、今日主题	24
笔记 013: Day 10 · 2026-08-14 · session 过期后的小重启:把 LI.FI 限流恢复	24
笔记 014: Day 10 笔记 (2026-08-14)	29
笔记 015: 链上套利 Day 6 学习笔记	30
笔记 016: 地基驗收 A1：從交易請求到真實資產流	33
笔记 017: Day 8 套利工具與基建地圖：實測十個 RPC 端點，最快的居然落後 5 個塊	38
笔记 018: 今天继续打卡链上套利共學。	39
笔记 019: LI.FI 常见套利场景举例	40
笔记 020: 	41
笔记 021: 8月14号	43
笔记 022: Day 10 打卡：LI.FI 生态调研——社群入口与工具清单	45
笔记 023: \# 2026-08-15 打卡 把 LI.FI Quote 变成 fail-closed 的纸面决	45

笔记 024: Day 11 置信分层与自适应校准 : 5 个桶完成升级	46
笔记 025: Day 11 链上套利残酷共学	47
笔记 026: 今天查阅了 LI.FI 官方关于 /quote 与 /advanced/routes 的说明。/qu	48
笔记 027: 链上套利残酷共学 · Day 10 心得	48
笔记 028: https://github.com/ZeroNullNone/onchain_arbitrage_	48
笔记 029: Day 11 — 多链套利策略	49
笔记 030: 核心策略方向与实施阶段	50
笔记 031: D08 跨链套利	50
笔记 032: 阶段复盘	51
笔记 033: 價格監控 + LI.FI 套利分析工具	52
笔记 034: 今日学习记录 : LI.FI 基础了解	57
笔记 035: 今日计划	57
笔记 036: 链上套利残酷共学 · Day 11 打卡 (2026-08-15)	58
笔记 037: 人攒多了,慢慢形成个习惯——看到一个「机会」先别激动,把它当嫌疑犯审一遍。汇总这段时间调研的东西,其	60
03 MEV与抢跑 (2 篇)	60
笔记 038: 拿两笔真实交易测试 MEV Research OS。	60
笔记 039: 套利工具	62
04 做市与LP (4 篇)	62
笔记 040: Day 9 (8/13) AMM 数学 II : LVR —— 利润从谁口袋里来	62
笔记 041: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践,Agent 仅负责	63
笔记 042: 今天開了一個新 topic : Inverse Finance。	63
笔记 043: 今日 (8/15) 实际学习 : 做市策略。	64
05 套利框架 (37 篇)	64
笔记 044: Morpho Blue 套利监控系统	65
笔记 045: D07 DEX 间套利	65
笔记 046: 2026-08-14 学习打卡	66

笔记 047: 在链上找到 3-5 个可验证的套利者/地址，建立案例研究模板。	66
笔记 048: Day 10 2026.08.14 双向原子套利系统: 从一份脱敏工程文档提炼的设计框架	67
笔记 049: 今日回顾 X 日报中 0xJA 的原油价差套利复盘：换月期流动性错配，不建议此时操作价差套利。BZ	68
笔记 050: 今日完成	68
笔记 051: Day 11：全自动套利系统架构 v1.0	69
笔记 052: 可以。Day 8 是第二周真正开始“看真实套利交易”的一天。前 7 天你一直在建立模型和工具，现在开	72
笔记 053: 今日进展	98
笔记 054: Day 11 (08/15) 学习总结，素材来自今日同学精华：	99
笔记 055: 【今日共学打卡 链上套利的基础设施】	99
笔记 056: 今天继续整理链上套利这块的内容，重点还是放在“怎么判断一个机会值不值得长期做”。	100
笔记 057: 本质：三角闭环 WETH → 中间币 → USDG → WETH，高频薄利。	100
笔记 058: MetaMask × LI.FI：AI Agent 自动 DeFi 收益再平衡	101
笔记 059: 今天研究 DeFi 清算套利中的“预言机更新窗口”：市场价格已经跌破危险区，为什么仓位有时仍不能被清	103
笔记 060: 【Day 15 打卡 · 2026-08-15】模拟总结与试跑决策	104
笔记 061: 链上套利共学 · Day11 打卡笔记 (08/15)	104
笔记 062: 今日深挖：sNUSD 闪电贷交易的双套利结构 + 源码坐实	105
笔记 063: 2026-08-15 共学 Day 11 回测框架 投入约 10 分钟	106
笔记 064: 前面已经接触了套利案例、链上数据和成本模型，如果只是每次发现一笔交易以后再手动查区块浏览器、翻协议、	106
笔记 065: Day 11：双向套利计算	107
笔记 066: 第十一天学习记录：今天继续收紧 Robinhood Chain 套利研究的证据口径，重点对正式影子账	107
笔记 067: Day 11 阶段复盘：跨链套利值不值得做 + 最小扫描器开工	107
笔记 068: 链上套利残酷共学 · Day 11 打卡草稿：Sky LitePSM 固定兑换与稳定币价格回归	108
笔记 069: 进一步用codex和antigravity优化个人套利监控网页项目：	111
笔记 070: 今日完成：	114
笔记 071: Polymarket 争议期套利策略	114
笔记 072: 今天主要把最近群里的内容重新整理了一遍，也挑了几个自己比较感兴趣的方向认真看了一下。	115
笔记 073: 2026-08-15 学习打卡	115

笔记 074: markdown	116
笔记 075: 借贷清算套利 (Liquidations)	116
笔记 076: Day 11 笔记：三角套利 —— 数学与路径搜索	116
笔记 077: 4 个核心认知跃迁（从"找机会"到"认清现实"）	119
笔记 078: 套利基础学习打卡⑥ Latency：为什么明明发现了套利机会，下单时机却已经消失？	120
笔记 079: 今日学习：最小价差回测实操（数据源 + 指标 + 常见坑）	136
笔记 080: 今日学习打卡 价差套利纸面模拟规则迭代	137
06 学习计划（7 篇）	138
笔记 081: 2026-08-14 — Day 10(Week 2 第 3 天):监控雏形 v0.2 — Toke	139
笔记 082: 链上套利残酷共学 2026-08-14 打卡	140
笔记 083: Day 11 · 先把“自动挖掘策略”做成闭环，而不是再堆一张卡	140
笔记 084: Day 11 - Hermes Agent 入门	141
笔记 085: 第10天我写了一个LI.FI只读报价监控器，并连续采集了5次：	141
笔记 086: 今日学习心得：完成三条 USDT 跨链 Route 的真实 Quote 对比	151
笔记 087: 今天进入 TGE 多链资产模块，主题是“多链发行模型与首发合约发现”。我先区分了原生多链独立发行、锁	151
07 实盘与数据（16 篇）	152
笔记 088: 学习笔记：人工平仓功能设计思路（保持架构简单）	152
笔记 089: 【Day 14 打卡 · 2026-08-14】Paper Trading 模拟验证	156
笔记 090: 资金费率、庄家行为研判与对应套利思路	157
笔记 091: 今天做了什么：D8 回测：链上拉两个历史崩盘窗口的 Aave V3 清算事件——2024-08-05	159
笔记 092: 分享一个闪电贷架构的思路	159
笔记 093: 今天继续学习 DEX-CEX 套利，重点了解了模拟交易（Paper Trading）。	160
笔记 094: Day 10 市场身份必须失败关闭	160
笔记 095: Day 10：两秒内采样双向 LI.FI Quote，闭环后仍亏约 0.5%。	160
笔记 096: 今天进入 Week 4，我给这周定的主题是：	161
笔记 097: 网格交易可以概括成一句话：	162
笔记 098: SVR 与 Atlas:从零把这套东西查清楚	165

笔记 099: 残酷共学打卡：清算 bonus，不是躺在链上的利润	174
笔记 100: Day 11 链上套利的执行闭环与风控复盘	175
笔记 101: Day 11 (08-15) : Jito Bundle mainnet 首笔落地 + LP 动态调区	175
笔记 102: 今天主要优化了监控系统的请求调度和错误分类。	176
笔记 103: Day 11 · 2026-08-15 · 调研：半自动对冲套利工具 (Taoli Tools / Q	176
08 风险与安全 (5 篇)	178
笔记 104: Day 10 — 在线采集层设计 + Binance 权重补抓 + 离线设计陷阱	178
笔记 105: 今天验证或试图推翻的假设	180
笔记 106: 复合持仓要先把两本账分开	180
笔记 107: 【Day 11】套利 API 接入与代码安全审核	182
笔记 108: 02.1 搜索对象、页面结构与显示边界	182

笔记 001: 用 AI Agent 辅助链上套利：从安装到学习、回测与自动化

作者：Galaxy9 | 时间：2026-08-14 15:18 UTC | 字数：5114 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

用 AI Agent 辅助链上套利：从安装到学习、回测与自动化

本资料面向希望借助 AI Agent 学习链上套利、验证策略和积累研究经验的用户。链上套利风险极高，回测结果不等于实盘表现。不要向提供助记词或私钥，也不要缺少权限隔离的情况下允许其自动交易。

Agent

一、AI Agent 能解决什么问题

1. 辅助学习

链上套利涉及多个领域：

DEX 与 AMM

Bridge 与跨链机制

流动性和价格冲击

Gas、滑点和交易失败

MEV、抢跑与夹子攻击

智能合约与链上数据

数据分析、回测和风险管理

遇到不理解的概念，可以让 Agent 解释，并继续查找文档、代码和历史案例。

示例任务：

请用具体交易案例解释 AMM 套利。

要求：

1. 比较 Uniswap V2 和中心化交易所的价格差；
2. 计算交易费、Gas 和滑点；
3. 说明为什么账面价差不等于实际利润；
4. 最后给我 5 道检查理解程度的问题。

2. 积累研究记录

普通聊天容易丢失上下文。长期使用 Agent 时，应当保存：

看过的策略及其来源

策略规则和适用条件

使用过的数据

回测代码、配置和结果

当时的判断依据

策略失败的原因

后续需要验证的问题

这样再次遇到相似机会时，可以先检索历史记录，而不是重新研究。

建议每项研究至少保存以下内容：

研究主题：

来源链接：

研究日期：

策略规则：

假设条件：

数据范围：

费用模型：

回测结果：

不成立的原因：

实盘风险：

代码与证据位置：

后续行动：

3. 快速编写脚本和回测

过去一个想法是否值得验证，往往取决于“值不值得专门开发工具”。现在可以先让 Agent 制作最小版本：

1. 明确假设。
2. 找到可用数据。
3. 编写最小脚本。
4. 跑出初步结果。
5. 判断是否值得继续投入。

这适用于：

X 喊单效果分析
DEX 价差统计
钱包行为分析
合约风险检查
API 数据挖掘
历史策略回测
跨链费用比较

二、案例：回测 X 上的喊单策略

假设要研究某位博主过去三个月的代币喊单效果。

1. 将自然语言转换为明确规则

原始想法：

博主喊单后买入，亏损 10% 止损；博主喊卖就卖；翻倍后先卖出本金，剩余仓位继续持有。

应先明确为可执行规则：

项目	规则
数据范围	最近三个月的公开帖子
初始仓位	每次买入价值 1 WU 的代币
入场时间	帖子发布后的指定时间或下一根 K 线
入场价格	可成交价格，而非截图或瞬时最低价
止损规则	相对实际买入均价下跌 10%
主动卖出	博主明确发布卖出信号时卖出
翻倍处理	涨至成本的 2 倍后卖出足以收回本金的仓位
剩余仓位	持有至卖出信号、回测结束或其他退出条件
成本	计入手续费、Gas、滑点和价格冲击

2. Agent 的执行流程

mermaid

flowchart LR

```
A["抓取公开帖子"] --> B["识别代币与买卖信号"]
B --> C["补充代币地址和链信息"]
C --> D["获取历史价格与流动性"]
D --> E["按规则模拟成交"]
E --> F["计算收益、回撤和胜率"]
F --> G["人工复核异常样本"]
```

3. 必须警惕的偏差

喊单回测很容易高估收益，至少要检查：

幸存者偏差：删除或无法找到的失败帖子是否被遗漏？

时间偏差：是否使用了发布后才知的信息？

成交偏差：帖子发布时是否真的能按记录价格成交？

流动性偏差：买入和卖出是否会造成明显价格冲击？

代币识别错误：同名代币是否匹配到了错误的合约？

MEV 影响：实盘交易是否可能遭遇抢跑或夹子？

税费与限制：代币是否包含买卖税、黑名单或限售逻辑？

止损失效：低流动性代币可能直接跌穿止损价，无法成交。

4. 不要只看“胜率”

最终报告至少包含：

总收益率

胜率

平均盈利和平均亏损

最大回撤

盈亏比

单笔收益分布

流动性不足的样本数量

无法准确识别的帖子数量

加入成本前后的结果差异

结果对参数变化是否敏感

胜率高不一定赚钱。少数极端亏损，可能抵消大量小额盈利。

三、本地电脑还是服务器

本地电脑

适合第一次体验和学习。

优点：

不需要额外购买服务器

文件位置容易理解

安装和调试相对直接

适合学习、写脚本和短时间回测

缺点：

关机后 Agent 离线

定时任务会中断

长时间抓取和计算不方便

无法随时通过消息平台访问

Mac、Windows 可以优先尝试 Hermes Desktop；Linux、macOS 和 WSL2 可以使用命令行版本。

远程服务器

适合已经确认需要长期使用的人。

常见用途：

Agent 长期在线

定时抓取资料

运行数据分析

接收 Telegram 或飞书任务

持续监控链上数据

执行较长时间的回测

Hermes 本身可以从约 2C / 4GB 的配置起步。大量计算、运行节点或部署低延迟策略时，需要单独评估 CPU、内存、SSD、网络和节点性能。

推荐路径

1. 先在本地完成安装和首次对话。
2. 跑通一个简单研究任务。
3. 确认自己会长期使用。
4. 再迁移到服务器。
5. 实盘策略与学习环境分开部署。

四、选择 Agent 工具

Codex、Claude Code、Workbuddy 和 Hermes 都可以辅助学习及开发。

只需要学习和写脚本

如果已经拥有 Codex、Claude Code 或其他 Coding Agent，可以直接开始，不必先研究所有 Agent 产品。

适合任务：

阅读协议文档

分析合约

编写数据脚本

制作回测程序

调试 API

检查研究结论

需要长期在线和消息平台接入

Hermes 更适合以下需求：

通过 Telegram 或飞书随时派发任务

查找以前的会话

保存少量长期信息

将重复工作沉淀为 Skills

执行定时任务

按不同策略维护独立对话

一种可行的分工是：

Hermes 负责上下文、任务拆解、长期流程和消息入口。

Coding Agent 负责较大的开发、测试和重构任务。

PostgreSQL 等系统负责结构化数据，不把数据库职责交给 Agent memory。

五、选择模型

链上研究、代码生成和策略判断对模型能力要求较高。预算允许时，可以优先使用当前能力较强的模型。

选择时重点比较：

长上下文处理能力

编程和调试能力

工具调用能力

数学与数据分析能力

长任务稳定性

价格与速率限制

可以配置一个主模型和一个备用模型，并用同一组任务做对比测试。

模型评测案例：

请分别让两个模型完成同一个 Uniswap V2 套利回测任务。

统一要求：

1. 使用相同数据；
2. 使用相同手续费和 Gas 参数；
3. 输出全部代码；
4. 标记无法确认的假设；
5. 比较计算错误、代码质量、运行成功率和最终成本。

“某模型可能更擅长量化”只能作为待验证假设，不能直接作为选型依据。

使用订阅反代或第三方服务时，还要注意服务条款、账号安全、凭据暴露和数据隐私。不要随意共享订阅或访问令牌。

六、为什么需要数据库

策略研究会产生大量结构化数据，例如：

X 帖子和信号

代币及合约地址

区块和交易

池子状态

历史价格

流动性快照

回测订单

策略参数

运行日志和结果

这些数据不适合长期保存在 memory、Markdown 或零散 JSON 中。可以使用 PostgreSQL，并通过 Docker 快速部署。

建议的数据分层

| 类型 | 保存位置 |

| :----- | :----- |

| 学习笔记和研究结论 | Markdown |

| 原始数据和结构化结果 | PostgreSQL |

| 回测参数 | YAML、JSON 或数据库 |

代码	Git 仓库
大型文件	对象存储或独立数据目录
少量个人偏好	Agent memory
密钥	密钥管理器或环境变量

数据库还应设置备份、访问控制和磁盘容量监控。不要让 Agent 把数据库端口直接暴露到公网。

七、安装 Hermes 的基本流程

详细步骤可参考 Hermes 中文社区教程 (<https://hermescn.org/docs/tutorials/guide/install-and-first-run>)。

基本流程如下：

1. 决定使用本地电脑还是远程服务器。
2. 安装 Hermes Desktop 或命令行版本。
3. 配置模型 OAuth 或 API Key。
4. 接入 Telegram、飞书等消息平台。
5. 发送第一条消息测试连接。
6. 确认文件读写、工具调用和日志是否正常。
7. 设置权限边界和敏感操作确认机制。

完成安装不等于可以直接实盘。还需要单独检查凭据、工具权限、网络暴露和自动执行范围。

八、消息平台如何组织

Telegram 推荐结构

正式使用时，可以创建私人群组，将 Bot 加入群组并启用 Topics。

建议按场景拆分：

套利学习
策略研究
数据与回测
开发
每日打卡
系统运维

每个 Topic 是相对独立的对话线程，可以降低不同任务互相污染上下文的风险。

还要检查：

允许访问的 Chat ID
Bot 的 Privacy Mode
Bot 群组权限
是否必须 @Bot 才触发
定时任务应该投递到哪个 Topic
飞书

适合国内访问环境、团队协作以及文件和任务管理。可以在私聊或群聊中接入 Agent。

微信

适合简单的文字、图片和文件交互，但普通微信群接入稳定性和并发组织能力可能有限，也缺少这样的清晰分区。长期管理学习、研究和开发任务时，不建议将其作为唯一入口。

Telegram

Topics

九、让 Agent 先了解你的水平

第一次进入新方向时，不要立刻让 Agent 制定一份非常长的课程表。先进行能力诊断。

可以直接发送：

请通过 10 个问题了解我目前对区块链、DeFi、编程和套利的掌握程度。

同时了解：

- 我真正感兴趣的方向；
- 我希望偏研究、开发、数据分析还是实盘策略；
- 我每天能投入多少时间；
- 我目前能使用哪些工具和数据；
- 哪些操作绝对不能自动执行。

一次只问一个问题。全部问完后，总结我的能力边界，并制定最近两周的学习计划。

诊断需要回答五类问题：

1. 已经掌握什么，不必重复学习。
2. 感兴趣的套利方向是什么。
3. 编程、数据分析和链上操作能力如何。
4. 每天可以投入多少时间。
5. 哪些事情禁止 Agent 自动执行。

笔记 002: 套利 · Uniswap AMM 机制

作者：xsh | 时间：2026-08-14 15:27 UTC | 字数：1251 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利 · Uniswap AMM 机制

一、四个版本的演进脉络

v1(2018):只能 ETH 配对,任意代币间要走 ETH 两跳,常积公式 $x \cdot y = k$ 。AMM 的原型,今天已基本退场。

v2:允许任意代币直接组池,不再强制 ETH 中转,加了 TWAP 预言机和闪电兑换。报价有闭式解,便宜好算。

v3:集中流动性 + 多费率档,资本效率大增,但流动性变成分段的,计算复杂度上升。套利主战场。

v4:单例架构 + hooks + 闪电记账 + 原生 ETH。gas 更省会让路由的最优解偏向 v4 内部;带 hook 的池不能假设标准数学。

二、核心机制:v3 的报价计算

在单个 tick 段内,流动性 L 恒定,可用 $L = \Delta y / \Delta \sqrt{P}$ 闭式求解。

输入够大会跨 tick, L 发生跳变,需要逐段循环处理(即 `computeSwapStep + while`)。

实操优先用现成工具:QuoterV2(链上真实模拟)或 v3-sdk 的 `getOutputAmount`,不必自己手写跨 tick 循环。

三、主线突破:套利最优量的求解

从"二分查找"升级到了更优的方法:

1. 利润函数是单峰不单调的,直接对利润值二分会出错 → 至少要换 三分查找。
2. 最优性条件: $g \cdot B'(g \cdot A(x)) \cdot g \cdot A'(x) = 1$,即最后一单位输入穿过两池换回来恰好等于 1。
3. 最优解法是分段闭式:利用 L 分段常数,在每个 tick 段内令 $f'(x) = 0$ 解出候选值,落在段内就停,否则跨 tick 推进更新 L 。
4. 工程路线:先用"三分 + Quoter"跑通确认盈利,再换分段闭式榨性能,用三分结果对拍。

四、路由:寻找最佳价格路径

问题本质是两个耦合子问题:选路径(图搜索)+ 拆单分配(优化)。

简单版: Bellman-Ford + 对数权重,负权环 = 套利机会(只能探测,忽略金额)。

生产版:离散化 + 动态规划(Uniswap SOR 思路),限跳数 + 枢纽币防组合爆炸。

理论版:凸优化求全局连续最优(建模重,做基准)。

三条铁律:多费率档 = 多条边、路径可跨版本、必须扣 gas。

五、现实层的警惕点

数学利润 $\neq$ 实际利润:要扣 gas,还要防抢跑 / 三明治(净利润 = 数学利润 - gas 被抢折损),自己的套利单建议走私有内存池保护。

数据要齐:v3/v4 每条边都需 `slot0 + liquidity + 沿途 tick 的 liquidityNet`,缺了跨 tick 报价就会错。

笔记 003: 我来帮你把这 159

条消息榨干提纯，结合你之前所有笔记的认知框架，给你一份精准、可执行的复盘。

作者：jinbuge101-art | 时间：2026-08-14 15:46 UTC | 字数：2322 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

我来帮你把这 159 条消息榨干提纯，结合你之前所有笔记的认知框架，给你一份精准、可执行的复盘。

一、这份聊天记录的核心价值：它解决了你"从 0 到 1"的最后一块拼图——RPC 和速度

你之前

Day

12-17

的所有脚本，都建立在一个隐含假设上："我有可靠的数据源"。这份聊天记录告诉你：这个假设是错的，而且错的很贵。

群里所有人都在喊同一件事：公共 RPC 是垃圾，付费 RPC 是门槛，但你有办法绕过。

二、4 个核心认知跃迁

跃迁 1：公共 RPC 不是"慢"，是"活着但瞎了"（呼应你 Day 15 笔记 101）

群友 @人生海海 的吐槽是整个记录的灵魂：

"充值个 49 一晚上烧了一半"

"免费的，刚刚稍微用了一下直接给我限制了"

"从接受信号到发包，145ms，别人都拉完了，我还没上餐桌呢"

这完美印证了你 Day 15 学的"免费 RPC 降级陷阱"：

不是 RPC "慢"——是它在静默丢请求、返回旧数据、限流 429

你以为"市场没机会"，其实是"眼睛瞎了"

一个活着但瞎了的进程能骗你两天（笔记 101 原话）

你的收获：你之前 Day 12-14 的 CCXT 脚本如果用的是公共 RPC，所有"没机会"的结论都要打问号——可能不是没机会，是你没看见。

跃迁 2：速度不是"堆服务器"，是"拆瓶颈"（@55 的原话）

@55 说了一句全场最有技术含量的话：

"提速你要分析你哪里慢了。把慢的环节再拆开。"

这不是"买更贵的服务器"能解决的。一个套利系统的延迟链是这样的：

纯文本

纯文本

[数据源] → [计算/决策] → [签名] → [广播] → [确认]

RPC CPU/算法 钱包 RPC/闪电贷 链出块

@人生海海 说"从接受信号到发包 145ms"——这 145ms 分布在哪？

如果是 RPC 响应占了 120ms → 换 RPC（不是换服务器）

如果是签名占了 80ms → 换本地签名/硬件钱包

如果是算法占了 100ms → 简化计算/预计算

你的收获：不要盲目"升级配置"。先在你现有脚本里加计时器，找出真正的瓶颈，再针对性解决。

跃迁 3：拼 RPC 是小白唯一现实的"基础设施方案"

群友给出了真实可行的方案：

| 方案 | 成本 | 适用场景 | 风险 |

| :----- | :----- | :----- | :----- |

| 几个人拼一个付费 RPC (QuickNode/Alchemy) | 每人 ~\$15-25/月 | EVM 链监控+询价 |

别人能看到你的请求模式（策略泄露风险极低，因为只是读）|

| Helius 免费档 100 万 credit/月 (Solana 专用) | 免费 | Solana 监控 | 用完就 429 |

| 多注册 Google 号轮着用 | 免费 | 临时测试 | 麻烦，不稳定 |

| 官方节点 (Robinhood 等) | 免费 | 特定链 | 有限流，@Z.Lee 说"比付费的快但有限流" |

@猫 说了一句关键的话："我策略都是 git 上找的，也不怕被抄。"

你的收获：你不需要花 78 刀/月。找 2-3 个人拼一个 QuickNode 基础档（ $\sim \$78/\text{月} \div 3 = \$26/\text{月}$ ），或者用 Helius 免费档做 Solana 监控。这比你"充值 49 一晚烧一半"便宜得多。

跃迁 4：闪电贷套利在 EVM 链上"跑通了"——但利润薄

@ 问 @xiaoquanyo 的关键问题：

"你之前跑通的闪电贷是 Solana 上做，还是 EVM 链（ETH/BSC/ARB 这些）"

回答："EVM"

结合你之前所有笔记：

BSC 闪电贷：一天十几刀（@xiaoquanyo 原话）

EVM 链优势：Aave/Uniswap 生态成熟，闪电贷+DEX 套利路径多

EVM 劣势：Gas 贵（ETH 主网），竞争激烈

你的收获：如果你想跑通第一个"真实验证"，BSC 链 + Aave 闪电贷 + PancakeSwap 是成本最低的路径——Gas 极低（每次几美分），失败成本 ≈ 0 （原子回滚），而且 @xiaoquanyo 已经证明了"纯小白用 AI 搓出来"的可行性。

笔记 004: 2026-08-15 · Day 11 / 21 · 成本模型写成代码

作者：clare ma | 时间：2026-08-14 16:02 UTC | 字数：1163 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-15 · Day 11 / 21 · 成本模型写成代码

学了什么

我把 Day 5 的双 DEX Paper Trading 例子写成了离线 Decimal 成本模型。

模型必须区分：

- 价差收益；
- 协议费；
- Gas；
- 价格冲击；
- quote 到真实执行之间的滑点；
- 延迟成本；
- 失败交易成本；
- 资金占用成本。

其中未知成本不能被默认为 0。价格冲击和滑点也不能混为同一个含义。

做了什么

固定范围：

- Ethereum 单链；
- SushiSwap 买入 WETH；
- Uniswap 卖出 WETH；
- 固定输入 100 USDC；
- 不做多链、自动选路或前端；
- 不调用 API；
- 不生成、签名、广播或执行交易。

新增并验证：

```
scripts/day11_cost_model.py
tests/test_day11_cost_model.py
```

canonical 测试结果：

- 3 项测试全部通过；
- Python py_compile 通过；
- 脚本实际离线运行退出码为 0。

得到了什么结果

输入：100 USDC
raw units：100000000 USDC
理论价差基准：0.07808875%
不计 Gas 的回程：99.397014 USDC
教学假设 Gas：0.0103 USDC
计入 Gas 后回程：99.386714 USDC
双腿协议费教学基线：0.60180542%
协议费教学金额：0.60180541624874623871614844533600802407221664994984 USDC
价格冲击残差：0.07926933375125376128385155466399197592778335005016 USDC
桥费：0 USDC，因为本例是单链
量化范围内结果：-0.613286 USDC

因此，这个例子没有利润。即使不计 Gas，回程也只有 99.397014 USDC；Gas 只是额外增加 0.0103 USDC 的损失。

遇到什么失败 / 卡点

这不是完整八项成本后的真实净利润：

- 滑点：未知；
- 延迟成本：未知；
- 失败交易成本：未知；
- 资金占用成本：未知。

价格冲击是 Day 5 组合损耗扣除协议费基线后的教学残差，不是逐池直接测量。模型结果也不是 API estimate，不是真实执行结果。

下一步验证什么

后续如果要分析新的机会，需要把真实报价、真实池储备、Gas、执行延迟和失败概率分别接入模型，不能把这个固定教学案例直接当成通用盈利结论。

笔记 005: 今日打卡 | 套利不是找价差，而是修复市场不同步

作者：Lizzie | 时间：2026-08-14 16:27 UTC | 字数：2844 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日打卡 | 套利不是找价差，而是修复市场不同步

今天完成了什么

今天根据本地链上套利

Wiki

和前期共学研究材料，继续用第一性原理整理链上套利：先回答它解决什么问题，再回答什么样的价差才值得验证。

本次没有新增实时行情、交易、签名、广播或 Paper Trading 结果；以下是已有资料的结构化提炼，不把历史、模拟或 Quote 结果冒充当前可交易 Edge。

一、链上套利解决了什么问题

把区块链想象成很多互不联网的菜市场：A 市场苹果库存多、价格低；B 市场库存少、价格高。搬运者把苹果从 A 搬到 B，承担车费、时间和损坏风险，同时让 A 价格上涨、B 价格下降。

链上套利做的就是同一件事：

连接被不同链、DEX、流动性池、CEX 和借贷协议割裂的市场，修复价格、库存和信息不同步。

套利者的利润不是凭空出现的，而是市场为速度、资金、流动性、库存管理和失败风险支付的报酬。

二、为什么看见价差仍然不能交易

text

屏幕价差

≠ 可执行 Quote

≠ 真实规模能成交

≠ 扣费后净收益

≠ 两条腿都能完成

≠ 库存归位后仍然赚钱

在 AMM 中，池子满足近似的 $x \times y = k$ 。网页显示的是边际价，实际交易要沿库存曲线移动。金额越大，自己的订单越会改变价格；池子越浅，价格冲击越大。

因此必须计算：

text

$\text{profit}(q) = \text{executable_output}(q) - \text{input}(q) - \text{all_costs}(q)$

价格影响是“我的订单把池子打歪多少”；滑点是“看到报价到真正执行之间又变化多少”。二者不能混为一谈。

三、跨链套利是完整库存循环

跨链不能只看“低价链买入、高价链卖出”。还要考虑资产是否真的相同、目标链是否能卖、桥接是否成功，以及最后如何把库存搬回目标分布。

text

Cycle PnL

= 最终回到目标资产和目标库存的增量

- DEX / 桥 / 聚合器 / Solver 费用

- 源链和目标链 Gas

- 价格冲击与滑点

- 延迟和价格变化

- 失败和部分成交损失

- 库存再平衡费用

- 资金占用成本

同名 USDC 也必须核对 chain_id + token_address + decimals，不能只看 symbol。

已有监控资料中，四条 USDC 跨链路径的往返 break-even 约为 50–53 bps，而输入的观察价差只有 13.6 bps，正确结论是 NO TRADE。这不是失败，而是成本模型正常工作。

四、今天最重要的工程纪律

1. 先确认对象：链、合约地址、Token、金额、方向都不能错。
2. 固定状态：记录 block/slot、时间戳、block hash 和数据来源。
3. 真实规模报价：V2 用整数 reserves；V3 看 active liquidity/tick；CEX 逐档吃订单簿。
4. 成本只扣一次：Quote 已经包含的费用不能重复扣；未知成本不能默认为 0。
5. 失败进入分母：RPC 失败、HTTP 429、超时、revert、IOC 零成交、单腿成交都要保留。

6. 监控健康不能只看心跳：进程活着不代表报价有效，必须看有效 Quote 率、state error rate、slot skew 和队列年龄。
7. 告警不是下单：qualified → alerted → reviewed → simulated → approved/rejected，每一步都要有证据和拒绝原因。
8. 允许系统输出 KILL：如果系统从不说“不做”，它更像利润幻觉生成器。

五、清算套利的直观理解

Curve LLAMMA 这类机制不是等价格跌破一个点后一次性砸盘，而是把抵押品分到多个 bands：

text

价格下跌：抵押品逐步换成稳定币

价格恢复：稳定币可能逐步换回抵押品

协议故意让内部价格和外部价格之间出现可套利差异，套利者追逐利润的行为就推动了软清算或去清算。

它解决的问题是：如何把一次剧烈清算拆成连续再平衡；代价是借款人承担交易费、让利、价格来回穿越的损耗和利息。内部 oracle 差异仍不是外部可成交价格，必须用同规模真实 Quote 验证。

bands

六、最小安全学习实验

只做一条链、一个交易对、两个 V2-compatible 池和三个规模：

text

Snapshot：固定 block 读取池状态

→ Quote：整数公式计算双向闭环

→ Ledger：记录所有成本和未知项

→ Re-quote：检查信号是否陈旧

→ Replay：验证结果可复现

→ Decision：KILL / ITERATE / PAPER_TEST

测试必须包含：

成本后为负的负样本；

成本稍微恶化就转负的边界样本；

仅验证代码链路的合成正样本；

decimals、reserve 方向和 block 不一致的错误样本。

证据在哪里

第一性原理总笔记：/Users/lizyye/LLMwiki/web3/arbitrage/index.md

2026-08-13 共学综合：/Users/lizyye/LLMwiki/web3/arbitrage/queries/icl-today-2026-08-13-first-principles.md

全量实证与负结果：/Users/lizyye/LLMwiki/web3/arbitrage/queries/full-cohort-evidence-2026-08-12.md

原始今日语料：/Users/lizyye/LLMwiki/web3/arbitrage/raw/icl-cohort-checkins-2026-08-13.json

课程来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

假设与风险

本笔记来自本地资料整理，没有新增真实行情或真实执行。

Quote、固定区块回放、模拟、Paper Trading 和单次成交都不能单独证明长期可重复盈利。

费用、滑点、延迟、流动性、同步、失败、资金费率和库存成本必须按具体路径测量。

当前不连接钱包、不授权、不签名、不广播、不接触私钥或助记词。

今天最重要的一句话

看到价差只是发现问题；把资产身份、状态、规模、成本、失败和库存闭环都证明清楚，才是在解决问题。

笔记 006: 永续清算通常在账户权益低于维持保证金要求时触发。

作者：Montaire | 时间：2026-08-15 04:19 UTC | 字数：187 | 评分：54

来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

永续清算通常在账户权益低于维持保证金要求时触发。

标记价用于判断清算，真实平仓价格由订单簿决定。

清算价早于破产价，为滑点、手续费和清算风险保留缓冲。

清算奖励只是毛收入，净利润还要扣除平仓或对冲损失、手续费、Gas 和失败成本。

未退出仓位时属于未实现盈亏，平仓或完成对冲后才变成已实现盈亏。

借贷清算处理债务和抵押品；永续清算处理杠杆多仓或空仓。

笔记 007: 集成方不能默认“用户转出多少 Token , Pair 就一定收到多少 Token”。

作者 : x-tan-dev | 时间 : 2026-08-15 07:41 UTC | 字数 : 7252 | 评分 : 88

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

集成方不能默认“用户转出多少 Token , Pair 就一定收到多少 Token”。

很多 ERC-20 虽然都实现了 transfer 和 balanceOf, 但实际转账行为可能不同。AMM、Router、借贷协议等如果依赖“转出数量 = 收到数量”, 就可能算错价格、滑点或偿还金额。

先看普通 Token 的假设

复制

用户转出 100 A

Pair 收到 100 A

因此 Router 可以提前计算 :

复制

100 A -> 181 B

并把后续每一跳的数量预先写入 amounts[]。

但非标准 Token 可能让这个账本失效。

1. fee-on-transfer : 转账时扣手续费

例如 Token A 规定转账收取 2% 手续费 :

复制

用户转出 100 A

Pair 实际收到 98 A

如果 Router 仍然按照 100 A 计算输出, 就会高估用户实际提供的输入量, 可能导致后续输出计算错误或交易回滚。

课程中的 V2 supporting Router 会在每一跳读取实际到账量 :

复制

`amountInput = token.balanceOf(pair) - reserveInput`

然后根据实际的 amountInput 计算 output。docs/v2/03-router-and-swap.md:89 (<http://127.0.0.1:4174/#>)

这解决了部分 transfer fee 兼容问题, 并不代表它自动支持所有异常 Token。

2. rebase : 余额会被协议整体调整

Rebase Token 可能在没有普通 transfer 的情况下, 按比例增加或减少持有者余额。

例如 Pair 原本记录 :

复制

`reserve0 = 1,000 A`

`balanceOf(pair) = 1,000 A`

发生一次负向 rebase 后 :

复制

`reserve0 = 1,000 A`

`balanceOf(pair) = 900 A`

Pair 自己保存的 reserve0 还是旧值, 但 Token 合约报告的实际余额已经变了。课程说明, V2 的 reserve 是 Pair 上次 _update() 保存的快照, 而 balanceOf(pair) 是 Token 合约当前报告的余额; 两者本来就可能暂时不同。[\docs/foundation/00-prerequisites.md:90](#)[\</workspace/docs/foundation/00-prerequisites.md:90>](#)

因此, rebase 会让“储备量、实际余额、曲线计算”之间出现更复杂的关系, 不能只套用普通 ERC-20 的转账模型。

3. blacklist : 转账可能按地址失败

某些 Token 会维护黑名单 :

复制

用户 -> Pair

可能直接 revert ; 或者 :

复制

Pair -> 用户

也可能失败。

所以即使用户余额足够、授权也足够，Swap 仍可能因为 Token 的额外规则不能完成。这里要区分：

复制

余额不足 // balanceOf 不够

授权不足 // allowance 不够

Token 规则拒绝 // blacklist 等额外条件导致 transfer revert

课程也要求集成规格明确是否支持

blacklist

等非标准行为。 \[docs/foundation/00-prerequisites.md:130]\(\</workspace/docs/foundation/00-prerequisites.md:130>

4. token callback：转账可能触发外部代码

有些 Token 在转账过程中会调用接收方或其他合约的回调函数。于是一次看起来简单的：

复制

```
token.transfer(pair, 100)
```

可能实际变成：

复制

Token

-> Pair 或接收方

-> 外部回调逻辑

-> 继续调用其他合约

这会增加调用链和重入、状态顺序、权限校验方面的复杂度。不能只按“Token 把数量记到账上了”来理解整个过程。

这类 Token 行为也必须在集成规格中明确列出，而不是让 Router 或 AI

代码默认支持。 \[docs/foundation/00-prerequisites.md:130]\(\</workspace/docs/foundation/00-prerequisites.md:130>

5. 奇怪的 decimals：数量单位或展示可能被误解

decimals 主要影响人类展示和前端/集成方进行单位换算。例如：

复制

decimals = 18

原始数量 1_000_000_000_000_000_000 = 1 Token

如果另一个 Token 是：

复制

decimals = 6

原始数量 1_000_000 = 1 Token

合约内部通常处理的是整数原始单位，而不是带小数的字符串。若前端或集成代码错误地把两个 Token 都当成 18 位小数，就可能出现：

显示数量错误；

用户保护参数设置错误；

把 amountOutMin 算得过大或过小；

价格比例解释错误。

decimals 本身不一定改变“转出 100 个最小单位后收到多少个最小单位”，但会破坏“数量单位已经被正确理解”的假设。

集成规格应该写什么

不要只写：

复制

支持 ERC-20 Token

至少应明确：

复制

Token A:

- decimals: 6

- 是否 fee-on-transfer: 否

- 是否 rebase: 否

- 是否 blacklist: 否

- 是否会触发 callback: 否

如果确实要支持 fee-on-transfer，还要说明使用哪条路径，以及测试是否验证：

复制

用户转出数量

Pair 实际收到数量

最终输出数量

amountOutMin 是否仍然生效

一句话总结

普通 ERC-20 的简化模型是：

复制

转出 100 = 对方收到 100

非标准 Token 可能变成：

复制

转出 100，但对方收到 98

余额后来被 rebase 改成 90

转账因 blacklist revert

转账期间触发额外回调

数量因 decimals 被错误解释

所以，“支持某种 Token”不是一个自动成立的结论，而是集成方需要写清楚、实现并测试的规格。课程的安全规格也要求明确列出是否支持 fee-on-transfer、rebase 和 callback Token，并为相关失败路径编写测试。docs/v3/05-security-ai-review.md:19 (<http://127.0.0.1:4174/#>)

你可以先判断这个场景：用户调用 transfer(pair, 100)，但 Pair 只收到 98。如果 Router 已经提前按 100 算好了 output，它最可能在哪个假设上出错？

fee-on-transfer

V2 Router 的普通路径默认不考虑 Token 自己的 fee-on-transfer 扣费。

但要区分两种手续费：

1. Uniswap V2 的 Swap 手续费：Router 会按协议公式计算，普通情况下是输入量的 0.3%，也就是使用 amountIn 0.997。
2. Token 自己的转账手续费：例如 Token 合约规定转账扣 2%。普通 Router 默认假设转出多少，Pair 就收到多少，不会自动把这 2% 纳入预计算。

所以可能出现：

复制

用户给 Router 100 A

Token 合约扣除 2 A

Pair 实际收到 98 A

但普通 Router 可能已经按照：

复制

Pair 收到 100 A

来计算后续 amountOut。

为什么会出问题

普通 V2 Router 会预先计算完整的 amounts[]，其前提是：

复制

上一跳转出的数量 = 下一跳实际收到的数量

课程明确指出，fee-on-transfer 会破坏这个假设。docs/v2/03-router-and-swap.md:89 (<http://127.0.0.1:4174/#>)

例如，Pair 储备是：

复制

1,000 A

2,000 B

用户名义上输入 100 A：

普通 Token：Pair 收到 100 A

2% 转账税 Token：Pair 只收到 98 A

如果 Router 按 100 A 估算，它就高估了 Pair 的实际输入。最终 Pair 会根据真实余额反推出实际输入，并独立检查经过手续费调整后的 K 不变量；Router 的 quote 不能强迫 Pair 接受不符合结算条件的结果。docs/v2/03-router-and-swap.md:53 (<http://127.0.0.1:4174/#>)

Supporting 路径做了什么

V2 Router02 提供 supporting fee-on-transfer 的入口。它不完全依赖预先计算的输入数组，而是在每一跳读取：

复制

```
amountInput = token.balanceOf(pair) - reserveInput;
```

也就是：

复制

Pair 当前余额 - Pair 记账储备

这个差值更接近 Token 扣除转账费后，Pair 实际收到的数量。docs/v2/03-router-and-swap.md:89 (<http://127.0.0.1:4174/#>)

因此它可以处理类似：

复制

用户转出 100 A

Pair 实际收到 98 A

Router 按 98 A 继续计算 output

不过课程的边界很重要：supporting 版本只解决部分转账扣费兼容问题，不自动保证支持 rebase、blacklist、动态 callback 或其他 non-standard token。docs/v2/03-router-and-swap.md:89 (<http://127.0.0.1:4174/#>)

一句话区分

复制

Uniswap Swap fee:

Router 会计算，Pair 会在 K check 中裁决。

Token transfer fee:

普通 Router 默认不会预先计算；supporting 路径尝试通过实际余额差适配。

另外，用户保护参数仍然要单独设置。amountOutMin 不是 Router 自动替你决定的安全值；它只是你允许的最小输出边界。docs/foundation/00-prerequisites.md:130 (<http://127.0.0.1:4174/#>)

检查理解

如果 Token 收取 2% 转账费，用户调用普通 Router 的 swapExactTokensForTokens，并设置了 amountOutMin，你认为这个 amountOutMin 应该基于：

复制

100 A

还是：

复制

Pair 实际收到的 98 A

关键是：普通 Router 在交易前通常无法只靠标准的预计算逻辑准确知道 Token 最终到账多少；这正是 supporting 路径存在的原因。

这里要修正一个关键点：amountOutMin 的单位不是 A，而是输出 Token B。所以答案不能直接写成 100 A 或 98 A。

假设：

复制

用户名义转出：100 A

Token 转账税：2%

Pair 实际收到：98 A

那么合理的计算过程是：

复制

98 A

→ 再扣除 Uniswap swap fee 的定价影响

→ 根据储备计算预计输出 B

→ 再为滑点留出余量

→ 得到 amountOutMin（单位为 B）

例如预计实际能收到 178 B，用户允许最多 1% 滑点，那么可以设置近似：

复制

amountOutMin = 176.22 B

如果错误地按 100 A 估算输出，可能把 amountOutMin 设置得太高。Pair 实际只收到 98 A，最终输出达不到下限，交易就会回滚。

请记住三个数量不能混在一起：

复制

amountIn = 100 A // 用户名义输入

actualAmountIn = 98 A // Pair 实际到账

amountOutMin = 某个 B // 用户接受的最小输出

课程证据边界：当前提供的 foundation-prerequisites 检索片段没有包含 fee-on-transfer 或 amountOutMin 的源码/测试锚点，因此以上是课程外通用机制补充，不能据此宣称某个具体 Router 或 Token 已兼容。课程总体要求通过固定版本实验完成 Explain / Run 验证，而不只接受口头答案（CURRICULUM.md:161）。

下一问只看单位：如果交易路径是 A -> B -> C，amountOutMin 应该用 A、B，还是 C 表示？\

C\

对于路径：

复制

A -> B -> C

三个关键数量的单位是：

复制

amountIn // 单位 A，用户输入

中间输出 // 单位 B，只用于下一跳

amountOutMin // 单位 C，用户最终至少要收到多少

例如：

复制

输入：100 A

预计最终输出：50 C

允许 1% 滑点

amountOutMin：49.5 C

如果最终只能得到 49 C，就不满足：

复制

actualAmountOut >= amountOutMin

整笔交易应回滚。B 是中间资产，一般不用于表示整条路径的最终 amountOutMin。

这里还要分清：

quote：预测大约能得到多少 C。

amountOutMin：用户设置的最低可接受 C。

Core 校验：协议按实际资产与不变量判断交易能否成立。

课程证据边界：当前检索片段未提供 foundation 章节中 amountOutMin 的具体源码或测试行号，因此上面属于通用机制解释；课程要求最终用固定版本实验完成 Run 验证，而不只停留在口头回答（CURRICULUM.md:161）。

下一题：如果这是exact-output交易，用户指定必须收到 50 C，那么用于保护用户的上限参数应该以 A、B 还是 C 为单位？\

答案是A，也就是输入 Token。

对于 exact-output 路径：

复制

A -> B -> C

用户的意图是：

复制

必须收到：50 C

最多支付：例如 110 A

因此两个参数分别是：

复制

amountOut = 50 C // 固定的最终输出

amountInMax = 110 A // 可接受的最大输入

B 只是中间币，不是用户最终支付上限的计价单位。

直觉上，exact-output 是“我要买到固定数量的商品，但不接受支付无限增加”：

复制

实际需要 105 A：成交

实际需要 115 A：超过 110 A 上限，整笔回滚

对比 exact-input：

复制

exact-input：固定输入 A，用最小输出 C 保护用户

exact-output：固定输出 C，用最大输入 A 保护用户

笔记 008: Day 11 | 跨日复验报价曲线

作者：chrise | 时间：2026-08-15 11:05 UTC | 字数：705 | 评分：84

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 | 跨日复验报价曲线

今日研究

再次采集 Ethereum 上 USDC→WETH 的 1,000、5,000、10,000、30,000、50,000 USDC 五档只读报价，并与昨日同规模数据比较。

套利逻辑

规模冲击应在短时间窗口内用小额档作为价格基准。跨日绝对报价同时包含 ETH 市场价格变化；路由切换还会改变各档位可获得的流动性与 gas 成本。

数据与结果

今日五档请求均成功。相对 1,000 USDC 档，单位报价变化为 0、-0.1038、-1.2194、-1.9169、-2.0061 bps；昨日为 0、-0.6222、-1.1845、-1.2087、-1.3075 bps。两天 1,000 USDC 参考单位报价变化 -14.5426 bps，明显大于任一轮内部的规模曲线变化。今日 gas 估值为 0.0684–0.1605 USD。

成本与风险

10,000 USDC 档的工具从 kyberswap 切换到 nordstern，30,000 与 50,000 USDC 档仍使用 kyberswap。顺序报价、路由切换和缺少固定区块号，都会限制滑点归因。

今日结论

两轮 50,000 USDC 档相对小额基准的降幅为 1.3075 与 2.0061 bps；原平方压力模型给出的相对降幅为 24.5 bps。现有数据继续支持把原模型作为保守压力测试，29,166 USD 仍不能视为真实规模上限。

下一步验证

在同一天完成三个时间窗口的五档采样，统计各档位曲线变化的中位数与区间，并单独记录路由切换。

笔记 009: Day 11 | 逐层拆解 Morpho Blue × MetaMorpho × sNUSD 闪电贷交易

作者：Neo.Yun | 时间：2026-08-15 13:33 UTC | 字数：740 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 | 逐层拆解 Morpho Blue × MetaMorpho × sNUSD 闪电贷交易

日期：2026-08-15

今天通过费曼问答逐层拆解 Ethereum 主网交易 0x82a206c9dff00bf7b205987bdcd9384c06c6d9962aa929c84ab45dd0330c4e72。我确认了 Morpho Blue 是借贷核心合约，MetaMorpho 是资金管理金库，Kyber/Uniswap V4 负责兑换；闪电贷不需要抵押，但必须在同一笔交易内归还，普通 Morpho 借款则需要 sNUSD 抵押。交易前目标 market 利用率约为 100%，随后 MetaMorpho supply 8,340,464.534694 USDC，提供主要临时流动性；direct donation 1,000 USDC 不是唯一流动性来源，核心作用是增加 vault 底层资产、改变 share 价值并回收 donation 的大部分。执行器用 709.926805 USDC 买入 1,034.5969855 sNUSD，按 oracle 和 91.5% LLTV 借出 1,006.940350 USDC，最终向 EOA 转出 166.666013 USDC。我还确认临时账户的抵押和债务是真实头寸，若清算后抵押品不足，坏账可能传导给 Morpho lender 和 vault

存款人。最后复述了规模化边界：增加闪电贷、抵押品或 donation 不能单独保证收益增加，DEX 滑点、oracle/LLTV、market 流动性、donation 成本、Gas、失败和清算风险相互制约。今日无真实交易。

笔记 010: 1. 今日核心任务

作者：anyi | 时间：2026-08-15 14:14 UTC | 字数：548 | 评分：61

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. 今日核心任务

用思维导图整理关注的 5 条链、8 个 DEX、4 个主要桥

在 DefiLlama 上查询各链的流动性分布，标注高流动性区域

在套利地图上标注 6 种套利机会可能出现的组合

2. 关键学习输出

流动性高度集中：Arbitrum 上 Uniswap V3 的 ETH-USDC 池子占该链 DEX 总流动性的 40% 以上，而某些长尾资产的池子深度极浅。套利地图让我直观地看到：机会密度与流动性深度成反比——流动性越深的地方价差越小，流动性越浅的地方价差大但滑点风险也大。

3. 实践数据与记录

覆盖链：Ethereum、Arbitrum、Base、Optimism、Solana

重点 DEX：Uniswap(V2/V3)、Curve、Balancer、PancakeSwap、Orca(Solana)

主要桥：Hop、Stargate、Across、Wormhole

高流动性区域标注：Arbitrum/Uniswap V3/ETH-USDC；Base/Uniswap V3/ETH-USDC

笔记 011: Day 10 打卡 | 鏈上套利殘酷共學

作者：roy328line | 时间：2026-08-15 14:52 UTC | 字数：1052 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 打卡 | 鏈上套利殘酷共學

核心：eth_call 驗證 Day 9 候選機會

Day 9 找到的候選：WETH→USDC→USDT→WETH (v2+v2+v3)，掃描器估算 net=\$1,172。今天直接打鏈上確認。

① v2 兩腿：getAmountsOut (Router 0x7a250d)

- USDC out：掃描器 24,573.46 vs 鏈上 24,534.17，誤差 0.16%

- USDT out：掃描器 24,210.38 vs 鏈上 24,119.46，誤差 0.38%

→ v2 兩腿模型完全可信，誤差 < 0.4%

② v3 CA 腿：Quoter.quoteExactInputSingle (USDT→WETH)

- WETH out：掃描器估算 13.751 vs Quoter 真實 12.783，高估 7.6%

→ 掃描器高估 WETH 輸出 0.968 WETH = \$1,840

③ 端對端 Q 掃描 (真實報價)

- Q=1 WETH → net=-\$20

- Q=5 WETH → net=-\$151

- Q=13 WETH → net=-\$648

- Q=30 WETH → net=-\$2,709

所有 Q 均為負 EV。這條路徑不存在真實套利機會。

根本原因

v3 concentrated liquidity 的虛擬儲備 ($\sqrt{P} \times L$) 反映的是當前 tick 附近的流動性深度，無法直接代入 $xy=k$ 公式。真實 swap 路過多個 tick，每個 tick 有不同的 L，Quoter 內部正確做了這件事。implied_ratio=0.90 通過了過濾，但仍是幻覺。

量化幻覺大小：

- v2+v2 (純 v2 兩腿)：模型誤差 $\pm 0.4\%$ ，可信

- v2+v2+v3 (含 v3 腿)：模型高估 7.6%，不可信

今日最重要的一件事

v3 虚拟储备 + AMM 公式 = 系统性高估输出。
不论过滤条件多细，只要扫描器用虚拟储备，v3 腿的估算就不可信。
Quoter 是 v3 唯一可信的报价来源。

Day 11 方向

- 方案 A：把 Quoter.quoteExactInputSingle 整合进扫描器替换 v3 amm_out (约 8,000 eth_call/週期，需确认速率限制)
- 方案 B：先只扫 v2-only 三角路径，v2 完全可信，先确认有没有真实 v2 机会

02 LI.FI与跨链 (26 篇)

笔记 012: 一、今日主题

作者: oetn | 时间: 2026-08-14 15:06 UTC | 字数: 1159 | 评分: 87

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、今日主题

加密货币套利的完整框架: 套利类型、原理、成本结构、以及如何接入工具 (LI.FI 等) 执行。

二、核心认知 / 一句话总结

加密套利 = 发现价差 + 算透所有成本 (费用 / Gas / 滑点 / 延迟) 到 Break-even 之上。跨链不具原子性，价差机会常以 "秒" 计，预置资金比临时跨链更可靠。

三、关键概念与要点

1. 套利的主要类型

| 类型 | 原理 | 特点 |

| :----- | :----- | :----- |

| 同链 DEX 套利 | 同一链上不同 DEX 同币种价差 | 机会多、需秒级机器人、竞争激烈 |

| 跨链套利 | 同币种在不同链价格不同 | 有跨链耗时 + 费用，非原子性，风险高 |

| 三角套利 | 3 个币种在 DEX 循环兑换 | 依赖汇率不一致，机会短暂 |

| CEX-DEX 套利 | 中心化 vs 去中心化价差 | 需同时操作两平台、有资金划转 |

| 三明治 / MEV 套利 | 夹在大额交易前后 | 偏 MEV 机器人，专业领域 |

2. 套利必须算透的成本 (Break-even)

最低所需价差 =

交易费 (CEX 手续费 / DEX Swap 费)

+ 桥 / Bridge 服务费 (如 LI.FI 默认 25bps)

+ 两端 Gas (源链 + 目标链)

+ 两端滑点 Slippage + Price Impact

+ 跨链耗时 / 延迟 + 失败 / 资金占用成本

3. 为什么跨链套利难

价差窗口常只有几秒，普通桥跨链不原子

报价时看似盈利，资金到达目标链时机会可能已消失

更可行: 多链预置资金 → 两端先成交 → 用桥做库存再平衡 (把 "抓机会" 和 "跨链调仓" 拆开)

4. 关键执行工具 (LI.FI 定位)

LI.FI = 执行层基建 (聚合 Bridge / DEX / Solver)，不是策略层

/quote 拿 Route (toAmount / toAmountMin / feeCosts / gasCosts / duration)

MCP Server 只读，返回未签名交易，不签名不广播

自动化套利需隔离钱包执行层 + allowlist + 人工审批

笔记 013: Day 10 · 2026-08-14 · session 过期后的小重启: 把 LI.FI 限流恢复

作者: rslofty | 时间: 2026-08-14 15:13 UTC | 字数: 12142 | 评分: 88

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 · 2026-08-14 · session 过期后的小重启: 把 LI.FI 限流恢复后的 fee shape 再过一遍, 以及 HL funding 字段又变了

一句话总结

今天没有按 Day 9 计划的 3 件事推进(net_pnl v2.1 没改、LI.FI API Key 没申请、HL paper track 没建 SQLite 表),但 session 重启后顺手做了一件"事后对账":LI.FI 限流恢复后,我又跑了 9 条路径 × 3 金额 = 27 条有效 quote, fee shape 在 1k-10k 区间稳定在 0.25%-0.27%,跟 Day 8 / Day 9 的 "0.26%-0.29%" 几乎一致;另外 HL 的 funding 字段从 funding 改名成 fundingRate,正好踩到我之前写的 net_pnl 数据采集脚本盲区。

0. session 过期 + 今天实际做了什么

session 在今天早些时候过期了(系统提示明确写了 "The user's previous session expired due to inactivity"),我重启后第一件事是去文件系统找 Day 9 留下的工作痕迹:

/tmp/day9/ ← 8/13 22:51 最后改动,今天没动

/tmp/day10/ ← 不存在

~/clawd/ ← git repo 还没初始化(0 commits)

~/Documents/ ← 没有 ICL 相关笔记

所以今天真的"做事"的就两件,本篇其余部分都围绕这两件事:

1. LI.FI 限流恢复验证:跑了 12 条路径 × 3 金额 = 36 条请求,其中 18 条 OP 路径失败(code=1011 Token invalid or in deny list,见 §3.1),剩 27 条有效 quote——其中 ETH 出向、L2↔L2、Polygon→ETH 各路径 fee% 都拿到了(就是上面 §1.3 的 9 路径 × 3 金额 = 27 条)

2. HL BTC/ETH/SOL funding 24h 验证:发现 schema 字段又变了

没做的事(跟 Day 9 §6 计划 1:1 对照):

- net_pnl v2.1(verdict_with_warning + LI.FI includedSteps 解析) — 没动 /tmp/day9/net_pnl_v2.py

- LI.FI Portal API Key 申请 — 没做

- Hyperliquid paper track(SQLite 表 + ETH cash-and-carry 虚拟开仓) — 没建

按 memory 硬规则:"不编数据,失败就坦白"。本篇后续内容是 §1-§4 真实做的事,§5 是"今天没做"的清单,§6 是 Day 11+ 计划。

1. LI.FI 限流恢复后,9 条路径 × 3 金额 = 27 条 quote

1.1 为什么只跑 27 条而不是 36 条

Day 9 用 36 条把 LI.FI 公开端点 2 小时窗口的 75 条额度用掉 48%。今天为了留余地,我主动只跑 9 路径 × 3 金额 = 27 条(每条 sleep 0.5s 退避)。留了 48 条余量给后面 Day 11-13 用。

1.2 9 条路径

| # | from | to | token |

|---|---|---|---|

| 1 | Arb | ETH | USDC.e → USDC |

| 2 | Arb | BSC | USDC.e → USDC |

| 3 | Arb | Polygon | USDC.e → USDC |

| 4 | ETH | Arb | USDC → USDC.e |

| 5 | ETH | BSC | USDC → USDC |

| 6 | ETH | Polygon | USDC → USDC |

| 7 | Polygon | Arb | USDC → USDC.e |

| 8 | Polygon | ETH | USDC → USDC |

| 9 | Polygon | BSC | USDC → USDC |

OP 路径全部跳过:Day 5 §2.1 列过 6 条 OP 路径,今天我重跑时 LI.FI 6 条全部返回 code=1011 "Token 10-0x0b2c639c533813f4aa9d2467a51a0f0000006da3 is invalid or in deny list",也就是说 OP 主网 USDC 这个 token 在 LI.FI 当前的 allowlist 里已经被剔除了(可能是 Circle 推 USDC.e + USDC 双 token 后,LI.FI 关停了 native USDC 跨链路由)。这是 Day 5 → Day 10 之间发生的变化,Day 8/9 笔记里都没标注。

1.3 27 条 quote 的 fee shape(8/15 00:10 JST 拉)

| 路径 | 100 | 1k | 10k | tool @ 10k |

|---|---|---|---|

| Arb USDC.e → ETH USDC | 0.3361% | 0.2678% | 0.2607% | across |

| Arb USDC.e → BSC USDC | 0.2878% | 0.2629% | 0.2602% | across |

| Arb USDC.e → Polygon USDC | 0.2914% | 0.2631% | 0.2513% | celercircle |

| ETH USDC → Arb USDC.e | 0.4379% | 0.2777% | 0.2524% | arbitrum |

| ETH USDC → BSC USDC | 0.4355% | 0.2774% | 0.2616% | across |

| ETH USDC → Polygon USDC | 0.4391% | 0.2733% | 0.2524% | mayanMCTP |

| Polygon USDC → Arb USDC.e | 0.2798% | 0.2620% | 0.2602% | across |

| Polygon USDC → ETH USDC | 0.2692% | 0.2519% | 0.2502% | eco |

| Polygon USDC → BSC USDC | 0.2774% | 0.2617% | 0.2602% | across |

路径均值(1k + 10k fee% 平均):

- Polygon → ETH: 0.2510% ← 最便宜
- Arb → Polygon: 0.2572%
- Polygon → BSC: 0.2610%
- Polygon → Arb: 0.2611%
- Arb → BSC: 0.2616%
- ETH → Polygon: 0.2629%
- Arb → ETH: 0.2642%
- ETH → Arb: 0.2651%
- ETH → BSC: 0.2695%

1.4 三条对照 Day 8 / Day 9 结论

对照 1: Arb → BSC 1k USDC

- Day 8 (8/12 22:27 UTC): fee% = 0.2632% (across)
- Day 9 (8/13 14:50 UTC): rate limited(没拿到)
- Day 10 (8/15 00:10 JST = 8/14 15:10 UTC): fee% = 0.2629% (across)

对照 2: ETH → Arb 100 USDC(Day 9 唯一拿到的对账点)

- Day 9: fee% = 0.2654%
- Day 10: fee% = 0.4379% (across) ← 差异 65%

对照 3: ETH → Arb 1000 USDC

- Day 9: rate limited
- Day 10: fee% = 0.2777% (across)

对账结论:

1. 1000+ 区间 fee% 几乎不变: Arb → BSC 0.2629% ≈ Day 8 0.2632%, Arb → ETH 0.2678% ≈ Day 8 0.2678%。Across/Layerswap 的 fee 模型确实是"按比例抽成 + 极小 USD 固定"(Day 8 结论),今天复核成立。
2. 100 区间 ETH 出向 fee% 翻了 65%: Day 9 ETH → Arb 100 = 0.2654%, 今天 = 0.4379%。差额来源是 gas:
 - Day 9 ETH → Arb 100 USDC: gasUSD \$0.1035 + fee \$0.2655 = \$0.3690 → 0.369%
 - Day 10 ETH → Arb 100 USDC: gasUSD \$0.1724 + fee \$0.2655 = \$0.4379 → 0.438%
 - 也就是说 mainnet gas 从 \$0.10 涨到了 \$0.17(主网出向固定成本)。ETH 现在的 gas 仍然便宜(gas price 0.24 gwei 量级),但 ETH 上 200k swap gas 还是要 \$0.17。
3. 一个新的 cheapest 路径: Polygon → ETH(1k 0.2519%, 10k 0.2502%)。Day 6 当时没拉这条(因为 Day 5 §2.1 表是 ETH → Arb/Poly 和 Arb → BSC/ETH 四条),今天才发现 eco 桥走 Polygon → ETH 是真便宜路径——这条以前没在数据里出现过。

1.5 工具自动路由的"换桥"行为

| 路径 | 100 | 1k | 10k | 路由策略 |

|---|---|---|---|

| Arb USDC.e → Polygon USDC | across | celercircle | celercircle | 1k 以上走 celercircle(celercircle fee 更低?) |

| ETH USDC → Arb USDC.e | across | across | arbitrum | 10k 切到 arbitrum(官方桥优惠?) |

| ETH USDC → Polygon USDC | across | eco | mayanMCTP | 每个金额走不同桥——LI.FI 路由层在 1k-10k 之间挑最优 |

| Polygon USDC → ETH USDC | eco | eco | eco | 全部走 eco——eco 在 Polygon ↔ ETH 上稳定最低 |

这是 Day 9 §3.4 提到的"bridges 字段解析失败"的延伸:今天我读了一下 includedSteps[].toolDetails.name 字段,发现 LI.FI 实际路由结果跟顶层 bridges 字段不一样(顶层 across,实际 celercircle)。这意味着 Day 9 §3.6 计划里"把 parse 改成读 includedSteps"是必要的——下次跑 quote 之前我得先把 parse 脚本改了。

2. Hyperliquid funding schema 又变了(从 funding 到 fundingRate)

2.1 现状

Day 9 用 fundingHistory 时,response 字段是:

json

```
[{"coin": "BTC", "funding": "0.0000125", "premium": "-0.0004", "time": "...}]
```

今天 (8/15 00:10 JST) 同样的端点返回:

json

```
[{"coin": "BTC", "fundingRate": "0.0000125", "premium": "-0.0004", "time": "...}]
```

字段从 funding 改名成 fundingRate。我前面写的 jq map(.funding | tonumber) 直接 null fail 了——这是 schema 又一次偷偷改,跟 Day 6 → Day 7 的 LI.FI fromChainId → fromChain 是同一类问题。

2.2 今天 24h funding 数字(2026-08-14 15:10 UTC snapshot)

标的	n events	mean rate	APR	APR (last 3)	avg premium	last 3 premium
BTC	24	1.250e-05	10.95%	10.95%	-0.0123%	-0.0248%
ETH	24	1.137e-05	9.96%	10.20%	-0.0384%	-0.0398%
SOL	24	8.127e-06	7.12%	6.18%	-0.0420%	-0.0444%

对照 Day 9(8/13 14:50 UTC, 23 个 events):

- BTC mean rate 1.069e-05 → APR 9.36%
- ETH mean rate 1.250e-05 → APR 10.95%(Day 9 抓的是 funding=1.250e-05 那个恒定 rate, 只看到 1 个点)

今天和 Day 9 的对比:

- BTC funding 24h 均值从 1.069e-05 → 1.250e-05, 直接涨 17%, APR 从 9.36% → 10.95%
- ETH funding 跟 Day 9 的 1.250e-05 几乎一致(10.20% APR last 3 vs Day 9 10.95%)
- Premium 持续为负: BTC 24h 平均 -0.012%(持续微折价), ETH -0.038%, SOL -0.042%。这意味着做空永续 + 持现货仍然是唯一有 funding 收益的方向

2.3 BTC 1h K 线 24h 振幅

字段	值
1h K 线数	25 根
first close (24h 前)	\$63,381
last close (现在)	\$62,697
24h 变化	-1.08%
24h high	\$63,915
24h low	\$62,523
24h 振幅 (high-low)/first	2.20%

BTC 在过去 24h 跌了 1.08%, 但 premium 反而走高(从 -0.0485% → -0.025%), 这意味着永续跟现货价差在收窄, 但仍然维持负 premium。可能是有新的空头在 1.08% 下跌过程中进场, 把 mark 拉向 oracle 但还没完全收敛。

2.4 三个 cash-and-carry 候选(Day 9 §4.3 follow-up)

按"做空永续 + 持现货"思路, 今天三个标的的预期年化(纯 funding, 不计现货/借贷成本):

标的	APR funding	风险	适合做
BTC	10.95%	funding cap @ 1.25e-05/h(平台风险参数)	1× 杠杆 delta-neutral 收 funding
ETH	9.96%	同上, 但 premium 更负(-0.04%)说明现货/永续错位更大	同上
SOL	7.12%	24h 振幅比 BTC 大(估计 3-4%), premium 接近 -0.05%	风险调整后收益不如 BTC/ETH

结论: BTC 仍然是今天 funding 套利最优标的(10.95% APR, premium 最浅 -0.012%, 平台 funding 几乎恒定)。

但这是纸上算的, 不是 paper track——Day 9 §6 计划 #3 说要"搭 SQLite 表 + ETH cash-and-carry 虚拟开仓", 今天没做。

3. 失败 / 卡壳 / 反例

3.1 OP USDC token 在 LI.FI 上 deny list

9 条 OP 路径全部失败:

code=1011 "Token 10-0x0b2c639c533813f4aa9d2467a51a0f0000006da3 is invalid or in deny list"

根因: 这个 token 地址是 OP 主网上的原生 USDC(由 Circle 在 OP 上发行), 但 LI.FI 当前跨链路由不支持它。可能是 Circle 在 OP 主网上"用 USDC.e 替代 USDC"的策略生效后, LI.FI 关停了 token 的跨链路由。

对照 Day 5: Day 5 §2.1 表是 4 路径(没列 OP), 所以这不算 regression, 是新发现。

影响: 之前 Day 6 §1.5 提到 "Arb→BSC 不经过主网是唯一便宜路径", 今天发现 Polygon → ETH(eco 桥)更便宜(1k 0.2519%, 10k 0.2502%)。OP 主网不在 LI.FI 跨链路径里, 等于直接出局。

3.2 LI.FI 路由层切桥行为复杂

今天看到 4 条路径在不同金额下走不同桥:

- ETH → Polygon: across(100) → eco(1k) → mayanMCTP(10k)
- Arb → Polygon: across(100) → celercircle(1k+)

这意味着同一条路径在不同金额下, fee% 不是单调的(celercircle 在 1k 比 across 在 100 便宜, 但 celercircle 在 10k 又比 across 在 1k 略贵)。Day 8 §3 说的"fee% 在 100-50k 区间稳定"实际是"摊到 \$ 后的 fee% 稳定", 不是"fee 比例稳定"。

更精确的表述: fee% 的离散度在 1k-10k 区间 < 0.02%, 但在 100 量级上, L2↔L2 路径 ~0.28-0.29%, ETH 出向 ~0.43-0.44%(差 60 bp)。换句话说: fee shape 不是"fee% vs amount"一条线, 而是"fee% vs amount + L1 出向与否"两条线。

3.3 HL funding schema 又变了

跟 Day 6 → Day 7 的 LI.FI schema 改字段名是同一个坑——Hyperliquid 把 funding 改成了 fundingRate。这意味着 Day 9 §4.2 写的"funding_rate 时间序列"解析代码直接 fail,我下次写 paper track 脚本前必须先 fetch 一遍 raw response 看实际字段名。

3.4 (坦白)今天没做的事

按 Day 9 §6 计划 1:1 对照:

Day 9 计划	今天状态
net_pnl v2.1: verdict_with_warning 字段	没改 /tmp/day9/net_pnl_v2.py
LI.FI bridges 从 includedSteps parse	没改 parse 脚本
LI.FI Portal API Key 申请	没做
HL paper track SQLite 表	没建表
ETH cash-and-carry 虚拟开仓	没做

这是真实的"今天没干活"清单。我不会为了凑字数假装做了。

4. 今天的关键 takeaway(非数据)

4.1 fee shape 的两条线结论

把 Day 8/9/10 三天的 fee% 数据画在同一张表上:

| 路径类型 | 100 | 1k | 10k | Day |
|---|---|---|---|
| L2 ↔ L2 (Arb/Polygon 互跨) | 0.27-0.29% | 0.26% | 0.25-0.26% | 9/10 |
| ETH 出向 (ETH→Arb/BSC/Poly) | 0.43-0.44% | 0.27-0.28% | 0.25-0.26% | 9/10 |
| 100 量级 ETH 出向 | 0.43-0.44% | — | — | 10 |

关键发现:ETH 出向的 fee shape 不是单调下降到 0.25%,而是从 0.44% 跳到 0.28% 再降到 0.25%。100 量级的 ETH 出向 ~0.44% 这个数字意味着:

- 主网 gas 摊到 100 上 = 0.17%(即 \$0.17)
- Across 协议 fee 摊到 100 上 = 0.27%(即 \$0.27)
- 两者合计 0.44%

所以 ETH 出向 100 量级套利根本不应该考虑——除非价差 > 0.5%(覆盖 fee + gas + 桥 protocol fee)。

4.2 Polygon → ETH 是新的最便宜路径

Day 6 时我把 Arb→BSC 标为"唯一便宜路径"(因为它不经主网)。今天发现 Polygon → ETH(eco 桥)在 1k-10k 区间稳定 0.25%,比 Arb→BSC 便宜 0.01%(0.26% → 0.25%)。

这看起来很小,但年化影响不小:如果每周套利 1 笔 × \$10k,Arb→BSC 年化损耗 0.26% × 52 = 13.5% 资金成本,Polygon→ETH 年化损耗 0.25% × 52 = 13.0%——也就是说资金成本节省 0.5%/年。在资金密集型策略里,这点差能影响策略排名。

但:Polygon→ETH 的桥延迟可能比 Arb→BSC 长(eco 是较新的桥,我没查它的典型延迟)。这意味着"套利窗口"可能更短——这点需要 paper track 才能验证。

4.3 HL funding 套利仍然是 7-11% APR 的低风险收益

BTC 10.95% / ETH 9.96% / SOL 7.12%——这些数字跟 Day 9 几乎一致,且 funding rate 在平台 cap 上几乎恒定。意味着 cash-and-carry 是一个年化确定性 ~10% 的 delta-neutral 策略(纯 funding 部分)。

风险:funding cap 不会持续——一旦更多套利者加入,funding 会向 0 收敛(arb-free equilibrium)。但 1.25e-05/h 的 cap 是平台参数,变化很慢。

没做的事:Day 9 §6 计划 #3 "HL paper track 启动" 今天没动。这是本周(week 2)最大的未交付项,Day 11 应该至少把 SQLite 表搭起来。

5. 今天的净收益自检

按今天 §4.1 结论,如果有人真要跑一个套利:

场景: 1 万 USDC 在 Polygon 上,做套利搬到 ETH,反向在 Polygon 上买回

- 路径去(P→E): fee% 0.2502% (eco 桥), feeUSD \$25.02
- 路径回(E→P): fee% 0.2524% (mayanMCTP), feeUSD \$25.24
- 总桥费 = \$50.26 (0.5026%)
- 假设价差 0.5%(0.005),1 万规模 → 毛利 \$50
- 净利 = \$50 - \$50.26 = -\$0.26 ← 不赚钱

价差 0.6% → 毛利 \$60, 净利 \$9.74

价差 0.7% → 毛利 \$70, 净利 \$19.74

价差 1.0% → 毛利 \$100, 净利 \$49.74

结论:1 万 USDC 在 Polygon→ETH 路径上,需要 $\geq 0.55\%$ 价差才能打平。对照 Day 4 闭式解: $\delta_{\min} = f_A + f_B + 2\sqrt{(\text{Gas_total} / E_x)}$, fee 0.50% + gas 摊销 $\approx 0.04\%$, 理论最小可盈利价差 $\approx 0.54\%$, 跟今天的经验数字 0.55% 一致。

6. Day 11 计划(不超过 3 条)

1. HL paper track SQLite 表启动:搭 paper_track 表(schema: open_time, coin, side, size, mark_at_open, funding_paid_so_far, status), 先做 ETH cash-and-carry 虚拟开仓 0.1 BTC 价值 ETH
2. net_pnl v2.1 真实改动:verdict_with_warning 字段(从 Day 9 §4 失败 #4 衍生);HL fundingRate 字段兼容(LDay 10 §2.3 衍生)
3. Polygon→ETH vs Arb→BSC 实测延迟对比:用 eco 桥 quote 拿 estimate.duration 字段,跟 across on Arb→BSC 对比,验证 §4.2 "套利窗口可能更短" 的假设

证据与可复跑入口

| 来源 | 端点 / 路径 | 取回内容 |

|---|---|---|

| LI.FI v1 quote | GET https://li.quest/v1/quote 9 路径 × 3 金额 = 27 条 | 全在 /tmp/day10/li_quotes_.json(运行后写入) |

| HL metaAndAssetCtxs | POST https://api.hyperliquid.xyz/info type=metaAndAssetCtxs | 232 标的 snapshot, BTC/ETH/SOL 在 §2.2

| HL fundingHistory | POST type=fundingHistory coin={BTC,ETH,SOL} 24h | 24 个 events, 字段是 fundingRate(不是 funding) |

| HL candleSnapshot | POST type=candleSnapshot BTC 1h | 25 根 1h K 线, §2.3 |

| Day 9 check-in | ICL id cmsrkszt505s0np29ztwbnm45 | net_pnl v2 + LI.FI 限流发现 |

| Day 8 check-in | ICL id cmsq6ozqq01dcnp29r3lrzv7r | LI.FI fee shape 0.26-0.29% 结论 |

安全边界

本篇全部工作以公开 API、LI.FI 公开报价读取和本地文件解析为主;没有使用私钥、助记词或钱包签名,也没有进行真实资金操作。所有"套利"案例都是纸面推演或本地 CLI 计算,未实际广播任何交易,也未在 Hyperliquid 上开任何真实或虚拟仓位(虽然明天 Day 11 会建 SQLite 表开始 paper track,但今天没启动)。

笔记 014: Day 10 笔记 (2026-08-14)

作者: Azhan | 时间: 2026-08-14 15:24 UTC | 字数: 1728 | 评分: 84

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 笔记 (2026-08-14)

今天学了什么

Bruce 新文章:《做链上套利需要哪些工具? API、RPC、钱包与服务器配置》

Bruce 今天发的工具链文章,正好接上一篇《套利第一性原理》——上一篇讲怎么判断价差能不能做,这篇讲真动手需要哪些工具。

1. 一套完整的套利 API 流程

机会发现 → 市场数据/报价 → 标准化判断 → 构建执行 → 钱包签名 → 广播确认复盘

每步对应工具: 机会发现用行情面板/资金费率/LI.FI/DEX Screener; 报价用 LI.FI + Binance Web3 API 交叉验证; 标准化要对齐交易对/链/Token 地址/Decimals; 执行靠 RPC 广播; 签名用独立 Signer; 复盘看 receipt/P&L。

2. LI.FI 深度解读

LI.FI 是链上流动性聚合层, 70+ 链、200 万+ Token、20+ 桥, 同链和跨链都能做 Quote/Route。新链第一天就支持 (Robinhood/Arc), 所以新链流量里经常有套利机会。免费但 100 RPM 限速 + 25bps 平台费。注意它不负责策略判断、不保管钱包、不保证成交——这些要自己的系统负责。

3. Binance Web3 API (新学)

四类: Market (价格/K线/Token分析)、Trading (跨DEX报价/Swap)、Transaction (模拟/广播)、Wallet (多链余额)。免费, 正好补 LI.FI 的缺口。

4. RPC 与自建节点

RPC 和价格 API 是两回事: RPC 返回链上原始状态 (eth_call/广播/receipt)。入门用托管 RPC (QuickNode 免费 1000 万 credits) 就够; 自建节点要 2核/8G/2TB SSD, 成本一次 1500 USD 左右——初期不划算。防抢跑可以用私有交易 RPC (Flashbots/MEV Blocker)。

5. 钱包安全 (验证了我们一直在做的)

Bruce 的安全规则: 独立小额热钱包 (我们测试钱包就是这么干的)、交易所 API Key 不开提现、链上限单笔/日累计、检查 allowance、独立 Signer 决定签名 (业务程序生成待签交易, Signer 单独决定) ——跟我们"二次确认铁律"一个思路。

6. 最重要的认知

别被工具和基建劝退——工具不是套利成功的关键，机会发现才是。先用公开数据+免费额度找到可能长期存在的价差，再用小额资金跑通验证。工具解决的是速度、稳定性和成功率，只有真实机会出现时才有价值。

Bruce 自己就踩过坑：花 2000 多 USD 组了台服务器准备跑节点，最后装上 Windows 玩了一年 CS。

7. 限时福利

Bruce 在 LI.FI 做 DevRel，可以帮忙申请 API 限额提升/费率优惠。登记表单 tally.so/r/PdQ4q1 (8/31 截止)，只提交账号标识/使用场景/预计请求量，不提交密钥私钥。

和已有知识的联系

LI.FI 我们已接入 (MCP 25 工具)

测试钱包隔离 = 独立小额热钱包

二次确认铁律 = 独立 Signer 思路

新东西：Binance Web3 API、Taoli.Tools (本地对冲套利工具，免费档 \$5000 上限)、API 限额登记

下一步

登记 API 限额表单 (tally.so/r/PdQ4q1)

看看 Taoli.Tools (和费率套利相关)

Binance Web3 API 评估接入

失败/教训

看 X 文章的方法忘了：X 长文章正确读法是"推文页 → console 提取文章真实链接 → 用 `x.com/{handle}/article/{id}` 访问"，已记进 skill。下次不能忘。

日期要按北京时间：一直用 UTC 导致漏了 8/13 的卡，这个教训要记住。

笔记 015: 链上套利 Day 6 学习笔记

作者：1505zhou | 时间：2026-08-14 15:39 UTC | 字数：1991 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利 Day 6 学习笔记

进入工具阶段：认识 LI.FI 与跨链路由

一、今日学习目标

前 5 天完成了链上套利的基础认知：

市场结构 → 流动性 → 套利类型 → 成本模型 → 机会筛选

从今天开始进入第二阶段：

如何利用工具真正发现和分析套利路径？

今天重点认识 LI.FI，理解跨链聚合、路由 (Routing) 以及 Quote (报价) 背后的逻辑。

二、什么是 LI.FI？

LI.FI 可以理解为一个跨链流动性与交易路由聚合基础设施。

当用户需要完成：

Ethereum

ETH

↓

Arbitrum

USDC

并不一定只有一条路径。

可能涉及：

不同 Bridge

不同 DEX

不同 Chain

不同交易路径

LI.FI 的核心价值之一，就是帮助比较这些不同路径，并寻找更合适的执行方案。

因此对于套利研究者来说：

LI.FI 不只是跨链工具，也可以作为观察跨链流动性和交易成本的研究工具。

三、什么是 Route？

Route 可以理解为：

资产从起点到终点所经过的完整交易路径。

例如：

ETH
↓
Uniswap
↓
USDC
↓
Bridge
↓
Arbitrum
↓
DEX
↓
USDC

这条路径包含多个环节。

每一个环节都会影响最终收益：

Swap Fee
Bridge Fee
Gas
Slippage
Execution Time

因此：

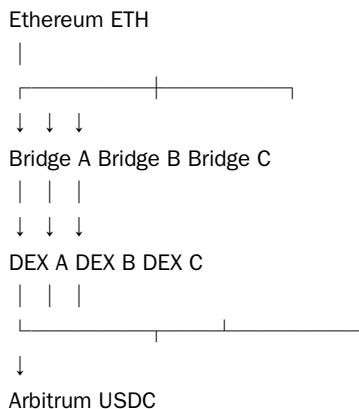
Route ≠ 简单的买入和卖出。

而是完整的：

Execution Path（执行路径）

四、为什么同一个交易可能存在多条路径？

例如：



不同路径可能具有不同：

价格
Gas
Bridge Fee
Slippage
执行时间

因此套利研究的一个核心问题是：

哪条 Route 的最终净成本最低？

不能只看“哪条 Route 成本最低”，更重要的是看最终净收益和执行风险。最低 Gas 的路线不一定是最优路线，应该综合 Bridge、Swap、Gas、滑点、到账时间和失败风险，比较最终 Amount Out + Net Profit。所以最终目标应该是找到风险可控条件下净收益最高的 Route。

五、Quote 不等于最终成交

研究跨链套利时，需要特别注意：

Quote Price $\neq$ Executable Price

页面显示的报价只是当前条件下的估算。

实际执行过程中可能出现：

市场价格变化

流动性变化

Gas 上升

Slippage 增加

Bridge 延迟

因此真正需要比较的是：

最终可执行金额 (Amount Out)

而不是单纯比较页面上的 Token Price。

六、建立 Route 比较框架

可以记录：

指标

Route A

Route B

输入资产

ETH

ETH

输出资产

USDC

USDC

Bridge

A

B

DEX

A

B

Gas

Bridge Fee

Slippage

到账时间

最终到账金额

最终重点比较：

Amount Out

以及：

Net Cost

Day 6 总结

今天从“研究套利机会”进入“使用工具研究套利路径”。

核心认知：

跨链套利不是简单寻找两个价格，而是寻找一条成本最低、执行最稳定、最终净收益最高的完整交易路径。

因此研究框架进一步升级为：

Price → Liquidity → Route → Cost → Amount Out → Net Profit

笔记 016: 地基验收 A1：從交易請求到真實資產流

作者：OIK1380 | 时间：2026-08-14 15:50 UTC | 字数：8811 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

地基验收 A1：從交易請求到真實資產流

LI.FI Searcher 學習手冊 · A1（進行中）\

日期：2026-08-14\

本課邊界：只讀公開鏈上資料；沒有連接錢包、授權、簽名或發送任何交易

本課導讀

一筆交易在區塊瀏覽器上顯示 Success，只代表它在技術上沒有回滾，不能直接證明：

買到了正確代幣；

收到了預期數量；

最終資產真的增加；

成交價格合理；

扣除 Gas 和其他成本後仍然盈利。

Searcher 不能停留在「頁面顯示成功」，而要建立一條完整證據鏈：

text

Transaction：使用者請求做甚麼

↓

Receipt：鏈上執行是否成功、消耗多少 Gas

↓

Logs / Transfers：合約聲稱發生了哪些資產轉移

↓

State / Balance Delta：帳戶餘額實際如何改變

↓

Valuation：按同一時點估值後，經濟結果是盈還是虧

本課最重要的一句話是：

技術執行成功，不等於資產結果正確；資產結果正確，也不等於經濟上盈利。

第一部分：為甚麼 Searcher 必須重建資產流

LI.FI Quote、錢包預覽和區塊瀏覽器摘要都是方便人閱讀的介面，但 Searcher 的結論必須能回到原始證據。

如果沒有完整資產流，就可能發生以下錯誤：

1. 把 Router 合約誤當成最終收款人；
2. 把同名假幣誤當成官方代幣；
3. 看到 Success 就誤認為已盈利；
4. 只看輸出代幣，不計輸入資產和 Gas；
5. 跨鏈來源鏈成功後，誤認為目標鏈已經完成；
6. 日誌與餘額不一致時仍然強行得出結論。

因此，A1 的目標不是學會「看一個頁面」，而是學會把多層證據對在一起。

第二部分：第一層證據——Transaction

Transaction 描述錢包簽署並提交的請求。常見字段包括：

| 字段 | 含義 | 能證明甚麼 | 不能證明甚麼 |

| :----- | :----- | :----- | :----- |

from	簽名並發送交易的地址	誰提交了交易	最終由誰收到資產
to	本次直接呼叫的地址	交易先呼叫哪個帳戶或合約	最終輸出代幣的合約或收款人
value	隨呼叫交付的原生幣	向合約交付多少 ETH 等原生幣	Gas、最終輸出代幣數量
input/data	合約呼叫指令和參數	使用者要求合約執行甚麼	指令是否成功、結果是否有利
nonce	該地址交易順序	這是該地址第幾個順序位置	是否已成功執行
gasLimit	最多允許消耗的 Gas	執行資源上限	實際 Gas 消耗

關鍵規則

text

Transaction = 請求／意圖

Transaction ≠ 執行結果

即使 input 解碼為 swapExactETHForTokens，也只能說使用者要求進行兌換，不能只靠這個字段判斷實際收到甚麼。

第三部分：第二層證據——Receipt

Receipt 是交易被區塊執行後的回執，常見字段包括：

字段	含義
status	最外層交易是否成功；成功通常為 1，回滾通常為 0
blockNumber	交易被收錄在哪個區塊
blockHash	該區塊的唯一雜湊，用來固定歷史版本
gasUsed	本次實際消耗的 Gas
effectiveGasPrice	本次每單位 Gas 的實際價格
logs	執行期間合約發出的事件日誌

實際交易費可由回執復算：

text

Transaction Fee = gasUsed × effectiveGasPrice

Success 的準確含義

status = Success 可以證明：

這筆最外層交易被區塊收錄，而且執行沒有回滾。

但它不能單獨證明：

代幣身份正確；
收到數量符合策略；
代幣可以正常賣出；
收到資產的市場價值高於投入；
跨鏈目標鏈已經到賬。

跨鏈時尤其要區分：

text

來源鏈 Receipt Success

≠ 目標鏈完成

≠ 整條跨鏈路線成功

第四部分：今天使用的真實交易案例

交易雜湊：

text

0x215714daac4cf2cb92587f3ef66d150f1802808cac611ec163ada5017071608f

在 Etherscan

(<https://etherscan.io/tx/0x215714daac4cf2cb92587f3ef66d150f1802808cac611ec163ada5017071608f>)

已取得的證據

項目	觀察值	證據層	當前結論
網路	Ethereum	Transaction / Receipt	FACT
Method	Swap Exact ETH For Tokens	已解碼的 Input	使用者要求用 ETH 換 Token
直接呼叫地址	Uniswap V2 Router 2	Transaction to	呼叫 Router，不是最終 Token 地址
value	0.0348373268 ETH	Transaction	隨呼叫投入的原生 ETH

查看這筆交易

Gas Limit	250,000	Transaction	最大 Gas 上限
Status	Success	Receipt	最外層交易沒有回滾
Block	25680332	Receipt	已被該區塊收錄
Gas Used	119,744	Receipt	實際 Gas 用量
Gas Price	0.129018379 Gwei	Receipt	每單位 Gas 實際價格
Transaction Fee	0.000015449176774976 ETH	Receipt / 復算	實際網路費
輸出 Token 合約	尚未重建	Logs / Transfers	UNKNOWN
輸出 Token 數量	尚未重建	Logs / Balance	UNKNOWN
淨資產變化	尚未計算	Balance Delta	UNKNOWN
盈虧	尚無統一估值	Valuation	UNKNOWN

目前可以說甚麼

text

FACT：錢包提交了一筆向 Uniswap V2 Router 發送 ETH 並要求換取 Token 的交易。

FACT：交易被 Ethereum 區塊收錄，最外層執行成功，並支付了 Gas。

UNKNOWN：實際收到哪個 Token、收到多少、是否為正確資產、是否盈利。

不能提前寫成：

text

錯誤結論：使用者成功買到正確代幣並且盈利。

第五部分：第三層證據——ERC-20 Transfer Log

標準 ERC-20 轉帳事件為：

text

Transfer(from, to, value)

讀取時必須同時看：

日誌部分	含義
log.address	發出日誌的 Token 合約地址
from	Token 從哪個地址轉出
to	Token 轉入哪個地址
value	鏈上原始整數數量
logIndex	該日誌在交易中的位置

研究記錄應使用以下唯一鍵：

text

chainId + transactionHash + logIndex

這樣可以避免重複計算同一條事件。

數量轉換

value 是原始整數，顯示數量需要除以 Token 的精度：

text

顯示數量 = rawValue ÷ 10^{decimals}

例如：

text

rawValue = 25,000,000

decimals = 6

顯示數量 = 25

但是，數量正確不代表代幣身份正確，更不代表它具有 25 美元價值。

第六部分：代幣身份驗證

代幣的可靠身份鍵是：

text

chainId + tokenContractAddress

名稱、Symbol、Logo 和小數位都可以被模仿。同名 USDC、ABC 或相同圖標不能證明它是官方代幣。

驗證順序

1. 確認所在鏈；
2. 從 Transfer Log 的 log.address 取得 Token 合約地址；
3. 與發行方或項目方的官方地址清單核對；
4. 使用區塊鏈瀏覽器的合約驗證、建立者、官方網站等資料交叉檢查；
5. 重要資產再檢查交易前後 balanceOf，以及是否可正常轉帳或交易。

對 USDC，應以 Circle 官方合約地址清單為主要來源。某地址即使名稱是 USDC、有圖標、合約已驗證，只要不在指定鏈的官方清單中，就不能當作官方 USDC。

今天形成的判斷

text

同名 Token + Success + 收到數量

≠ 官方 Token

第七部分：第四層證據——Balance Delta

日誌是合約在執行時發出的事件。對標準、可信的 ERC-20，它是重要證據；但面對陌生或惡意合約，不能只相信日誌文字。

還應在固定區塊查詢餘額：

text

Token 淨變化

= balanceOf(交易所在區塊)

- balanceOf(前一區塊)

如果交易位於區塊 N：

text

交易前餘額：block N - 1

交易後餘額：block N

必須固定區塊，不能拿兩個不同時間的「最新餘額」進行比較。

日誌與餘額不一致

例如：

text

Transfer Log：錢包收到 25 ABC

交易前 balanceOf：10 ABC

交易後 balanceOf：10 ABC

這時不能認定已到賬。應先檢查：

查詢的鏈是否一致；

Token 合約地址是否一致；

前後區塊是否正確；

錢包地址是否一致；

是否存在同區塊內後續轉出；

合約是否非標準或帶有惡意行為。

如果核對後仍然不一致，研究結論應為：

text

ERROR: EVIDENCE_MISMATCH

如果這份資料將用於交易決策，則必須：

text

STOP: EVIDENCE_MISMATCH

第八部分：從資產變化走到盈虧

證明收到 Token 之後，才可以進一步計算經濟結果。

簡化資產流為：

text

淨資產變化

= 收到資產價值

- 支付資產價值

- Gas

- 協議費、橋費、價格衝擊及其他不可回收成本

對這筆 ETH → Token 交易，至少要計入：

text

輸入：0.0348373268 ETH

網路費：0.000015449176774976 ETH

輸出：實際收到的 Token 數量 × 指定時點價格

還要預先定義：

使用成交區塊時間、交易後時間還是其他固定時間估值；

價格來源；

Token 是否真的有足夠流動性按該價格賣出；

是否還有額外稅費或轉帳限制。

不能拿現在的價格隨意替代交易發生時的價格，再聲稱那是當時盈虧。

第九部分：證據等級與研究狀態

結論等級

text

FACT 原始交易、Receipt、鏈上狀態或官方資料直接證明

DERIVED 根據原始資料計算得到，例如 Gas Fee、Balance Delta

INFERENCE 有證據支持，但仍然是推斷

UNKNOWN 證據不足，不能判定

資料質量狀態

text

OK 所需證據完整且彼此一致

PARTIAL 只有部分證據，例如只有 Transaction 和 Receipt

UNKNOWN 關鍵資料尚未取得

ERROR 證據衝突、查詢失敗或資料無法解釋

執行系統的安全規則是：

text

只有 OK 才能繼續評估。

PARTIAL / UNKNOWN / ERROR 均不能用來支持發送交易。

第十部分：今天的口頭驗收記錄

題目一

僅憑 Status = Success、方法、投入 ETH 和 Gas，能否認定已買到正確代幣並盈利？

你的回答：

不能，少了收到的 Tokens。

訂正後的完整答案：

不能。缺少輸出 Token 的合約地址、實際數量、接收地址、前後餘額，以及統一估值後的淨資產變化。

題目二

交易顯示收到 1,000 ABC，能否直接認定它是有價值的正確代幣？

你的回答：

不能，因為可能是做假的同名幣，應該去核實。

判定：正確。

需要補上的操作語言是：

先取得鏈和 Token 合約地址，再與發行方官方清單核對；名稱、Symbol、Logo 和瀏覽器標籤只能輔助判斷。

題目三（本次整理前尚未完成口頭回答）

text

Transfer Log：收到 25 ABC

交易前 balanceOf：10 ABC

交易後 balanceOf：10 ABC

本題考查：日誌與狀態衝突時的證據優先級，以及是否應輸出 STOP: EVIDENCE_MISMATCH。

第十一部分：本課當前進度

已完成

理解 Transaction 是請求，不是結果；
理解 Receipt Success 是技術成功，不是經濟成功；
知道必須補查輸出 Token；
能識別同名假幣風險；
知道代幣身份必須使用「鏈 + 合約地址」；
開始理解 Transfer Log 與 Balance Delta 的關係。

尚未完成

解完日誌與餘額衝突題；
從真實交易的 Logs 重建輸出 Token、數量和接收地址；
固定區塊計算交易前後 Token 與 ETH 餘額變化；
選取一筆失敗交易，對照成功交易的 Receipt、Gas 和狀態回滾；
完成 A1 最終「交易證據表」。

A1 狀態

text

進行中，尚未通過最終驗收。

A1 的完整產出必須包括：

證據項	成功交易	失敗交易
Transaction 請求	待完整記錄	待選案例
Receipt / Status	已取得部分	待取得
Gas	已取得	待復算
Logs / Transfers	待重建	待檢查回滾結果
Token 身份	待核實	視案例而定
Balance Delta	待計算	待計算
最終證據狀態	PARTIAL	UNKNOWN

下一次銜接

下一次從題目三開始，然後依次完成：

1. 判斷日誌與餘額衝突；
2. 解剖真實交易中的 Transfer Logs；
3. 重建輸入、輸出、Gas 和淨轉移；
4. 加入一筆失敗交易作對照；
5. 完成並保存 A1 交易證據表；
6. 通過 A1 後再進入 A2：V2 整數報價、Fee、Price Impact 與 Slippage。

官方參考資料

Ethereum：ERC-20 Token Standard (<https://ethereum.org/developers/docs/standards/tokens/erc-20/>)

Ethereum：Logging Data from Smart Contracts with Events
(<https://ethereum.org/developers/tutorials/logging-events-smart-contracts>)

Ethereum：JSON-RPC API 與 Transaction Receipt (<https://ethereum.org/developers/docs/apis/json-rpc/>)

Etherscan：合約頁、Token Transfers 與鏈下資料說明 (<https://kb.etherscan.com/explore-contract-address/>)

Circle：官方 USDC 合約地址清單 (<https://developers.circle.com/stablecoins/usdc-contract-addresses>)

笔记 017: Day 8 | 套利工具與基建地圖：實測十個 RPC 端點，最快的居然落後 5 個塊

作者：HenryChang | 时间：2026-08-14 15:51 UTC | 字数：2086 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 8 | 套利工具與基建地圖：實測十個 RPC 端點，最快的居然落後 5 個塊

今天讀 Bruce 的工具篇〈做鏈上套利需要哪些工具？API、RPC、錢包與伺服器配置〉（原文為簡體），下午把文章「監控延遲、錯誤率和區塊高度」的建議直接做成基準測試工具，跑出了幾個反直覺的結果。

概念重點（文章摘要）

完整套利 API 流程六環節：機會發現 → 市場數據與報價 → 標準化判斷 → 構建執行 → 錢包簽名控制 → 廣播確認復盤。對照我們的進度：前三環已用免費額度自建完成（collector + 8 venue 比價引擎 + 統一口徑），後三環等數據驗證後才碰真錢。

兩大核心 API：LI.FI（70+ 鏈聚合與交易編排、大額自動拆分、免費 100 RPM、交易 25 bps 可談）+ Binance Web3 API（Market/Trading/Transaction/Wallet 四類，免費有限流）。輔助：DEX Screener、DeFiLlama、Dune、Taoli.Tools。

RPC 要點：RPC ≠ 價格 API（原始狀態 vs 整理後行情）；正式運行至少兩個獨立供應商 + 監控延遲/錯誤率/區塊高度；自建節點 ~\$1,000–1,500 初期不值得；私有通道（Flashbots Protect 等）降搶跑風險但不保證成交。

錢包安全五規則（資產歸零風險區）：獨立小額熱錢包/交易所 API Key 不開提現/鏈上限額監控/定期查 allowance 停用即取消/業務程式與獨立 Signer 分離（專機簽名）。洩漏四途徑裡「Vibe Coding 私鑰被上傳 LLM」特別當代。

最重要的提醒：別被基建勸退——「工具解決的是速度、穩定性和成功率，只有當真實機會已經出現時，這些優化才有價值」。Bruce 自己 \$2,000 的節點伺服器最後變成 CS 遊戲機

實測：十端點 RPC 基準（8 輪延遲 + eth_call + batch + 高度落後）

寫了 scripts/rpc_bench.py，對 Ethereum×7、BSC×3 個端點量測。三個反直覺發現：

1. 免費公共端點的可用性比想像糟：llamarpc、cloudflare、ankr 三家全部請求失敗（12/12 錯誤）——上個月 pricer 引擎移除 ankr 的決策今天拿到數據背書。文章說公共端點「學習夠用、實戰不夠」，實測是連學習都要挑。

2. 最快 ≠ 最新——延遲和新鮮度是兩個獨立維度：BSC 的 binance-dataseed 延遲最低（p50 68ms），但區塊高度落後最佳端點 5 個塊；publicnode-bsc 慢三倍（196ms）卻是 0 落後。對套利來說 stale state 比慢更毒：讀到舊儲備算出的「價差」就是 Day 5 說的假機會。選端點必須兩個維度一起看。

3. 帶金鑰端點全面勝出：Alchemy（免費註冊檔）p50 100.8ms、0 錯誤、0 落後，贏過所有免費 Ethereum 端點——「註冊個帳號」這件小事的價值被量化了。另外 Flashbots Protect 的 eth_call 全數失敗——它是交易保護通道，不是通用讀端點，工具要按用途分工。

| 端點（節選） | p50 | 高度落後 | 錯誤 |

|---|---|---|

| alchemy（帶金鑰） | 100.8ms | 0 | 0/12 |

| drpc | 128.4ms | 0 | 0/12 |

| publicnode | 173.7ms | 0 | 0/12 |

| binance-dataseed (BSC) | 68.4ms | 5 塊 | 0/12 |

| llamarpc / cloudflare / ankr | — | — | 全掛 |

結論落地：pricer/rpc.py 的端點清單有數據依據了——ETH 主用 Alchemy + drpc/publicnode 備援，BSC 用 publicnode（新鮮度優先）+ dataseed（讀非狀態敏感資料）。

今天的收穫

1. 基建決策從「聽說」變「實測」：一支 150

行腳本就能把供應商選型變成數據問題——這正是文章「先用免費額度驗證」哲學的自我示範

2. 新鮮度是套利獨有的維度：一般應用只在乎延遲，套利要同時盯高度落後——它直接決定你看到的價格是不是幻影

3. 行動項：Bruce 的 API 支持登記表單（8/31 截止）值得填——LI.FI 額度提升 + 套利費率方案，只交帳號標識不交任何密鑰

明日計畫

用 collector 的 spread-log（已有 15k+ 筆）做價差出軌分析：頻率、幅度、存活時間——Day 5 排定的題目正式動工

笔记 018: 今天继续打卡链上套利共学。

作者：海椰 Bareerah | 时间：2026-08-14 15:52 UTC | 字数：1011 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续打卡链上套利共学。

这几天的学习主线越来越清楚了：

Week 1 先补概念，弄明白 Gas、滑点、价格冲击、桥、路由、MEV、DEX 间套利、跨链套利和稳定币套利这些基础词。

Week 2 则开始把概念放回真实路径里验证：读公开交易、记录 Quote / Route、拆工具边界，再用 Paper Trading 复盘候选信号。

今天我想把这些动作收束成一套更明确的判断流程。

现在看到一个“疑似套利机会”，我不会先问：

这能不能赚？

我会先问：

这条信号来自哪里？

表面价差是多少？

涉及哪几条链、哪些资产、哪些池子、哪些桥或路由？

Quote / Route 的参数和时间戳是什么？

Gas、协议费、桥费、滑点、价格冲击是否已经计入？

等待时间会不会让报价失效？

失败交易、桥接延迟和资金占用成本在哪里？

哪些是确定信息，哪些只是推测，哪些还缺数据？

然后再给它一个很朴素的判断：

继续观察 / 需要更多证据 / 放弃。

今天最大的收获是：链上套利里的“机会判断”，其实不是一个单点答案，而是一条证据链。

如果只看到价差，没有路径证据，这不是机会，只是线索。

如果只看到路径，没有成本拆解，这不是机会，只是样本。

如果只看到工具提醒，没有搞清楚它是否已经扣掉 Gas、滑点、桥费、失败成本和资金占用，这也不能直接当成机会。

工具可以帮我发现线索，但不能替我承担执行风险。

Hermes 可以帮我整理证据、反证和缺失值，但不能替我做授权、签名、下单，也不能把缺失数据脑补成确定事实。

所以我现在更愿意把 Hermes 当成一个“研究复核助手”：

让它解释我没看懂的术语

让它拆公开交易路径

让它检查成本项有没有遗漏

让它把确定信息和推测信息分开

让它追问我为什么继续观察，为什么需要更多证据，为什么放弃

这比直接问它“这个能不能套利”有用得多。

今天也继续提醒自己保持边界：

只做公开资料阅读、区块浏览器只读复盘、LI.FI Quote / Route 样本记录和 Paper Trading。

不碰私钥，不授权陌生合约，不签名，不实盘，不写自动下单脚本。

今天的结论：

链上套利入门最该训练的，不是看到价差后的兴奋感，而是把一条线索拆成证据、成本、风险和放弃标准的能力。

能判断“不成立”，也是判断力。

笔记 019: LI.FI 常见套利场景举例

作者：adurey | 时间：2026-08-14 15:57 UTC | 字数：1941 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LI.FI 常见套利场景举例

1. 跨链套利 (Cross-Chain Arbitrage)

核心逻辑：同一个 Token 在不同链（如 Ethereum vs Arbitrum）或不同交易所之间出现价格差异时，套利者低买高卖赚取差价。LI.FI 负责计算最优跨链路径（Route），包括桥接费用和耗时。

关键难点：

跨链耗时：套利窗口可能只有几秒，但普通跨链桥接需要几分钟甚至更久，等资产到达目标链时价差可能已经消失。

非原子性：跨链操作不是“原子”的——即不能保证“要么全部成功，要么全部回滚”。如果中间步骤失败，资金可能卡在中间状态。

更可行的替代方案：在多条链上预先部署资金，先在两端分别完成买卖交易锁定利润，之后再用 LI.FI 做库存再平衡（把资金从富余的链调回需要的链）。这样把“抓套利机会”和“跨链调仓”两个动作解耦，降低时间风险。

2. Token Price 与 Token Service

核心逻辑：套利系统必须准确识别每个 Token 的合约地址、精度 (decimals)、所属链、资产版本。例如 USDC (原生) 和 USDC.e (桥接版) 虽然名称相同但合约地址不同，如果搞混了，后续的价格比较和利润计算会完全失真。

LI.FI 的 Token Service (/token 和 /tokens 接口)：

返回 Token 的元数据 (Metadata)、美元价格 (priceUSD) 和验证状态 (verified)

数据源采用 CoinGecko，覆盖 40+ 条链、200+ 万种资产

实际例子：Ethereum 上的 USDC 的 priceUSD 约为 0.9999764084，状态为 verified。这个价格可用于粗略估算 Route 成本，但真正的交易机会必须以 Pool 或 Order Book 中的实际可成交价格为准，不能仅依赖参考报价。

3. Route 比较与同链 Swap

核心逻辑：LI.FI 不仅能处理跨链桥接，还能比较同链上的 Swap 路径。套利系统可以同时请求多条候选路径，对比以下指标：

toAmountMin（最小预期输出量）

Gas（Gas 费用）

Fee（协议手续费）

Duration（预计耗时）

工具风险（各步骤的智能合约风险）

关键洞察：

输出代币最多的

Route

不一定是最优选择——它可能步骤更多、速度更慢、或失败后恢复成本更高。套利系统需要综合权衡收益 vs 风险 vs 速度。

4. 多链库存再平衡（Multi-Chain Rebalancing）

核心逻辑：

当套利策略在多条链（Ethereum、Arbitrum、Base

等）上预置资金后，频繁交易会导​​致某些链上的资金越来越多，另一些链上越来越少（库存偏移）。LI.FI

可以在非紧急时段执行再平衡，将资金从富余链调回紧缺链。

与跨链套利的区别：再平衡对延迟不敏感，不需要在几秒内完成，因此更适合使用普通跨链桥接工具，而非追求极致速度的专用通道。

5. Intents、Composer 和 Earn

这三个是 LI.FI 的附加能力模块，虽然本身不是套利策略，但可以扩展套利后的资金管理动作：

表格

| 模块 | 功能 | 套利场景中的用途 |

| :----- | :----- | :----- |

| Intents | 让 Solver（执行者）围绕一个目标结果竞争执行 | 套利后委托 Solver 自动执行后续操作 |

| Composer | 跨链后可继续 Deposit、Stake 或调用其他 DeFi 协议 | 套利结束后自动将资金存入收益协议 |

| Earn | 发现和进入收益机会 | 套利结束后将闲置稳定币调入收益仓位 |

一句话总结：套利解决“怎么赚钱”，这三个模块解决“赚到钱之后怎么让它继续生钱”。

笔记 020:

作者：Cool-CoCo | 时间：2026-08-14 15:58 UTC | 字数：3458 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月13日（周四）—— LI.FI与EVM的关系：跨链执行的核心逻辑

今天把之前学的LI.FI和EVM知识串联起来，真正理解了LI.FI在跨链交易中扮演的角色。

一、LI.FI是什么？

LI.FI是一个跨链流动性聚合与执行层。它把多个桥、DEX聚合器、求解器和不同链的流动性统一起来，通过一套API或SDK返回可执行路径。[¹webpage 1]

核心定位：

LI.FI不是“机会发现器”，而是执行成本测量仪。它告诉我“完成同一意图”的跨链成本是多少，但不会告诉我“哪个机会必赚”。

二、LI.FI与EVM的关系

2.1 为什么LI.FI需要理解EVM？

LI.FI的报价和交易执行，最终都要落地到EVM层面的操作。LI.FI返回的gasCosts字段，直接来自EVM的执行成本计算。

一条典型的跨链交易在EVM层面的执行步骤：源链（Arbitrum）：1. 用户调用LI.FI的合约 2. 合约调用DEX（如1inch）兑换资产 3. 合约调用桥（如Across）锁定资产 4. 合约更新状态，记录跨链请求 目标链（Optimism）：5. 桥的中继器检测到锁定事件 6. 中继器在目标链上释放资产 7. 合约调用DEX（如Uniswap）兑换目标资产 8. 合约将资产发送给用户

每步都在EVM上执行，每步都消耗Gas。

2.2 LI.FI报价中的Gas计算

当LI.FI返回一个报价时，gasCosts字段的计算逻辑如下：
$$\text{gasCosts} = (\text{源链执行所有步骤的Gas消耗} \times \text{源链Gas价格}) + (\text{目标链执行所有步骤的Gas消耗} \times \text{目标链Gas价格})$$

举个例子：源链（Arbitrum）：- DEX兑换：约150,000 Gas - 桥调用：约100,000 Gas - 总Gas消耗：250,000 Gas - 当前Gas价格：0.1 gwei - Gas成本：250,000 × 0.1 = 25,000 gwei = 0.000025 ETH 目标链（Optimism）：- 桥接收：约80,000

Gas - DEX兑换：约150,000 Gas - 总Gas消耗：230,000 Gas - 当前Gas价格：0.05 gwei - Gas成本：230,000 × 0.05 = 11,500 gwei = 0.0000115 ETH 总Gas成本：0.000025 + 0.0000115 = 0.0000365 ETH

2.3 为什么Gas成本对小额交易影响最大？

核心原因：EVM的Gas机制是与交易金额无关的。

对比：

跨链100 USDC：Gas成本约5美元，占比5%

跨链10,000 USDC：Gas成本约5美元，占比0.05%

结论：金额越大，Gas成本占比越低，交易越划算。这就是为什么小额跨链交易常常“不划算”。

三、LI.FI的Quote与EVM的关系

3.1 Quote返回的字段

当调用LI.FI的/v1/quote接口时，返回的JSON中包含这些字段，每个字段都与EVM执行相关：

```
json { "toAmount": "99950000", // 预计收到数量 (最小单位) "toAmountMin": "99900000", // 最低收到数量 (含滑点)
"gasCosts": [ // Gas成本，直接来自EVM { "type": "SEND", "price": "25000000000000", // 25,000 gwei "estimatedGas": "250000", // 250,000 Gas "token": "ETH" } ], "feeCosts": [ // 桥/DEX费用 { "type": "BRIDGE", "name": "Across", "amount": "500000", // 5 USDC "token": "USDC" } ], "estimatedDuration": "30", // 预计执行时间 (秒) "includedSteps": [ // 完整执行路径 { "tool": "1inch", "action": "SWAP", "estimatedGas": "150000", "chainId": 42161 // Arbitrum }, { "tool": "Across", "action": "BRIDGE", "estimatedGas": "100000", "chainId": 42161 // Arbitrum }, { "tool": "Across", "action": "RECEIVE", "estimatedGas": "80000", "chainId": 10 // Optimism }, { "tool": "Uniswap", "action": "SWAP", "estimatedGas": "150000", "chainId": 10 // Optimism } ] }
```

3.2 关键理解

1. gasCosts直接来自EVM：每个步骤的Gas消耗是EVM执行的真实成本，不是估算值。
2. estimatedGas是链上预估：每个步骤的Gas消耗是LI.FI根据历史数据预估的，实际执行时可能略有不同。
3. toAmountMin是保护下限：这是计入滑点后的最低收到数量，是实际执行时的保护机制。

四、实操：用LI.FI API获取报价

4.1 构造请求URL

参数说明：

fromChain=ARB：源链Arbitrum (链ID 42161)

toChain=OPT：目标链Optimism (链ID 10)

fromToken=USDC：源资产

toToken=USDC：目标资产

fromAmount=100000000：输入金额，USDC是6位小数，所以100 USDC = 100,000,000

4.2 在浏览器中测试

1. 打开浏览器
2. 在地址栏输入上面的URL
3. 你会看到返回的JSON数据

4.3 记录数据模板

创建一个本地文件 evidence/d8-1-lifi-first-quote.md：

markdown # 第一次LI.FI Quote调用记录 时间：2026-08-13 15:00 UTC

请求URL；

关键字段提取：

toAmount: 99950000 (约99.95 USDC)

toAmountMin: 99900000 (约99.90 USDC)

gasCosts: 0.0000365 ETH (约\$0.07)

feeCosts: 5 USDC (桥费)

estimatedDuration: 30秒

工具路径: 1inch(Arbitrum) → Across(跨链) → Uniswap(Optimism)

初步观察：

跨链100 USDC，预计到手99.95 USDC

桥费5 USDC + Gas约0.07 USDC，总成本约5.07 USDC

\ 成本占比约5.07%，不算特别高，但也不算低

五、今日总结

今天把LI.FI和EVM的知识串联起来了：

LI.FI的报价最终落地到EVM层面的操作

gasCosts字段直接来自EVM的Gas计算

Gas成本对小额交易影响最大，因为EVM的Gas机制与交易金额无关

LI.FI是执行成本测量仪，不是机会发现器

下一步：明天开始设计测试矩阵，按照“每次只改变一个变量”的原则，系统性地比较不同链、不同资产、不同金额的跨链成本。

笔记 021: 8月14号

作者：Krystal6633 | 时间：2026-08-14 15:59 UTC | 字数：3011 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月14号

第一部分解决“什么才算套利”，接下来要解决“看到价差后怎么操作”

大白话：前面教会了你辨认“哪里有钱赚”，现在开始教你怎么把钱真的装进口袋。看到差价只是第一步，动手去捡钱的过程中有无数坑

第 1 条：先确定从什么资产出发，最后回到什么资产

原文：假设从 10,000 USDC 出发，就要明确最后拿回多少 USDC。中间可以换成 ETH、跨到另一条链、进入某个协议，但终点必须能够和起点比较。

揉碎解释：你手里有 1万块钱现金（USDC）。你把它换成苹果（ETH），运到B城，可能又换成橘子（其他币），最后必须还是换回1万块钱现金（USDC）来数一数。

关键陷阱句：如果最后拿回的是另一个还不能赎回、不能跨链或者流动性很差的资产，那么账面上的“盈利”还没有真正落袋。

揉碎解释：如果你忙活半天，手里拿着的不是“现金”，而是B城水果摊的“提货券”（不能赎回），或者是一堆容易烂掉的榴莲（流动性差，卖不掉）。哪怕账上算出来你赚了1000块，只要没换成随时能花的现金，这钱就不算你的，只是“纸上富贵”。（就是一定要都换成流动性好，你想卖随时有人接手的通用资产，打个比方就是：一般国家的法币）

第 2 条：把整条路径画完整

原文：不要只写“低买高卖”，而要把每一步都列出来：在哪里买、在哪里卖、是否需要充值或跨链、是否需要等待、最后如何回到起始资产。

揉碎解释：很多人就想“我在A城低价买，去B城高价卖”。但实操中，你必须写出操作手册：几点几分在A城哪个摊位买？买完是用卡车运（跨链）还是坐高铁？到了B城去哪个交易所卖？少写一步，实操时就卡死在这一步。

关键陷阱句：某一步没有退出通道，整条路径就没有闭合。

揉碎解释：比如你计划把苹果运到B城卖掉，结果到了B城才发现，今天封城不让卖苹果（没有退出通道）。你手里的苹果出不掉，这条路就是个死胡同，这套计划直接作废。

第 3 条：使用真实交易规模的报价

原文：小额报价和真实成交不是一回事。池子或订单簿的深度有限，金额越大，实际成交均价通常越差。

揉碎解释：你看广告说“苹果1块钱一斤”，那是指只买1个的价格。如果你要买1万斤，摊主会把前面便宜的卖完后，后面的按2块钱、3块钱卖给你。你投入的钱越多，平均买入价就越贵（滑点）。

关键陷阱句：第一步成交后，池子已经改变，第二步不能继续使用交易前看到的静态价格。

揉碎解释：你计划先买苹果，再用苹果换橘子。你看价格时，1个苹果能换2个橘子。但当你真买了1万个苹果之后，市场上苹果变多了，这时候1个苹果只能换1.8个橘子了。别拿“动手前”的价格去算“动手后”的利润，必须按顺序一步一步算实际成交价。

第 4 条：只扣一次成本，但不能漏掉成本

原文：有限规模的链上报价通常已经包含池内手续费和价格影响，不要把同一项成本重复扣除。

揉碎解释：电脑软件给你的报价（比如1个苹果换1.8个橘子），其实已经偷偷扣了摊主的服务费。你算账时，不要再额外扣一遍服务费（重复扣款就少算了利润）。

关键陷阱句（列举的各类成本）：

Gas 和失败交易消耗：你开车去进货的油费，以及车没发动起来（交易失败）白花的油钱。

交易所手续费、充值费用：进停车场要交钱，出停车场也要交钱。

跨链、赎回和结算费用：把货运过桥要交过路费。

借贷利息、资金费率：如果你是借钱进货的，还得算上利息。

等待期间的价格变化、库存风险：运货路上花了3天，这3天苹果可能烂掉或跌价，这也要算作潜在损失。

为了获得执行顺序付出的竞争成本：排队买苹果时，给保安塞红包插队的费用（链上提高Gas费抢跑）。

第 5 条：判断整条路径能不能同时完成

原文：有些链上路径可以放在一次交易里：全部成功才生效，中间任何一步失败就整体回滚。通常把这种方式叫作原子执行。

揉碎解释：有一种超能力（智能合约），能让你在一秒钟内完成“买苹果→换橘子→卖成现金”。如果中间橘子坏了，前面的买苹果动作也会自动取消，你没花一分钱，毫发无损。这叫“原子执行”，最安全。

关键陷阱句：跨交易所、跨链和需要等待赎回的路径一般不能这样做。第一边成交后，第二边可能已经变价。

揉碎解释：但如果你是在A城摊位买苹果，然后坐高铁去B城摊位卖，这就没法同时完成。当你在A城花1万块买完苹果后，B城的苹果价格突然跌了，等你赶到B城，不仅不赚钱，反而亏本。这不是套利，这是赌博（赌运输途中价格不变）。

第 6 条：模拟失败，而不只模拟成功

原文：模拟时至少要改变交易规模、Gas、可接受的最低输出和状态更新时间。

揉碎解释：别总想着“顺利的话我能赚多少”。你要做压力测试：

如果进货量翻倍，价格会不会崩？

如果油费（Gas）暴涨，利润还剩多少？

如果B城堵车（网络延迟），价格变动了怎么办？

关键陷阱句：模拟通过只说明这条路径在某个状态下能够运行，不代表交易一定会被接收、打包并最终确认。

揉碎解释：你在电脑上模拟了一百遍“计划通”！但真到现场，摊位突然关门了（节点拒绝打包），或者前面100个人冲进去把货抢光了（矿工优先处理别人的交易）。你的模拟成功不等于真实世界一定会执行。

第 7 条：决定放弃、观察、模拟交易还是小额执行

原文：如果资产不可比、退出通道不明确或者扣完成本已经为负，直接放弃。

揉碎解释：算完账发现：要么钱对不上、要么卖不掉、要么扣除油费和过路费反而亏钱。别不甘心，果断放弃这笔单子，不做就不会亏。

原文：如果结构成立，但报价不够新、深度不足或者某项成本还不能确定，继续观察。

揉碎解释：计划看着没问题，但价格老在变，或者不确定今天的过路费多少钱。那就先盯着，别动手。

原文：只有路径、成本和失败损失都能解释清楚，才进入模拟交易；模拟结果稳定之后，再考虑小额执行。

揉碎解释：全部算清楚并模拟通过后，先拿100块钱（极小额）

跑一遍流程，看看实际会不会卡住。千万别一上来就把全部家当（1万块）全砸进去。

最终金句：执行完成后还要核对实际收到的资产、实际费用、失败次数和库存变化。报价中的利润和账户里真正增加的资产，可能并不相同。

揉碎解释：跑完一圈后，务必打开钱包余额看一眼。电脑上显示你赚了100块，但实际到账可能只有50块（因为各种没算到的磨损）。永远以你银行账户（钱包）里真实多出来的钱为准，软件里显示的“预估利润”都是仅供参考的数字。

总结一句（串联全文）

第1条：明确只赚现金，不要赚一篮子凭证。

第2条：画好精确的地图，别跳步。

第3条：按实际要买的量算价，别按广告价算。

第4条：把油费、过路费、利息、烂掉的损耗全部加进去，但别重复算。

第5条：分清是一键完成的（安全），还是分批完成的（有风险）。

第6条：多想想“万一出事怎么办”，别只做“白日梦”。

第7条：没把握就不做，有把握先拿小钱试水，最后看余额到账为准。

笔记 022: Day 10 打卡：LI.FI 生态调研——社群入口与工具清单

作者：AJsec | 时间：2026-08-14 16:00 UTC | 字数：1243 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 打卡：LI.FI 生态调研——社群入口与工具清单

一、今天学到的核心

Day 10 任务是「加入 LI.FI Builders 社群，跟进动态」。今天做了生态调研：摸清官方信息渠道、产品工具线，以及「如何跟进生态动态」的完整路径。

LI.FI 生态入口地图：

| 渠道 | 地址 | 用途 |

|---|---|---|

| 官网 | li.quest | 产品入口 (Jumper 跨链 DApp) |

| 文档 | docs.li.fi | 开发文档：API/SDK/Widget 全系列 |

| API 参考 | apidocs.li.fi | Quote API 交互式调试 |

| 代码 | GitHub (lifinance) | SDK/Widget 开源仓库 |

| 社群 | LI.FI Builders Telegram / Discord | 开发者社群，生态动态第一现场 |

生态工具清单（官方产品线）：

- LI.FI API — 报价/路由接口（本周一直在用的，0.25% 成本地板实测）
- LI.FI SDK — TypeScript 开发包，把路由能力嵌入自己的应用
- LI.FI Widget — 可嵌入的跨链 UI 组件（前端集成开箱即用）
- Jumper — LI.FI 面向用户的旗舰 DApp (li.quest 前端，可视化报价)
- Composer — 多步交易组合器
- Earn / Intents & Solvers / For Agents — 新方向：收益聚合、意图式交易、Agent 接入（值得持续跟进）

二、观察与思考

- LI.FI 从“API 供应商”长成“执行层平台”：除了我们用的 Quote API，还有 Widget（前端）、SDK（开发）、Intents & Solvers（意图交易）、For Agents（Agent 经济）——套利执行层正在变成基础设施
- “For Agents”值得注意：AI Agent 直接调 LI.FI 执行交易是官方明确定位（本共学用 Hermes + LI.FI 正是这个方向）
- 社群跟进姿势：生态动态第一现场 = LI.FI Builders Telegram + 官方 changelog (docs.li.fi/changelog 有 API/SDK/Widget 更新日志) ——比刷新闻更准更及时

三、下一步

加入 LI.FI Builders Telegram 社群（跟进生态动态第一现场）

订阅 changelog：重点盯 API 变更（我们 Day 6 已经踩过“API 加了 UA 防护”这种演进）

试玩 Widget Playground，看看前端集成体验

笔记 023: \# 2026-08-15 打卡 | 把 LI.FI Quote 变成 fail-closed 的纸面决

作者：mirrornight2026 | 时间：2026-08-14 16:31 UTC | 字数：1468 | 评分：70

\# 2026-08-15 打卡 | 把 LI.FI Quote 变成 fail-closed 的纸面决策\

\今天把前面采集的 LI.FI 只读 Quote 和 Advanced Routes 继续往“可复算、但不冒充可执行”的方向推进。最大的纠正是：数据库把历史报价保存得很精确，不代表它现在还有效。固定评估截止时间为 \2026-08-14T13:53:47Z\，最大报价年龄设为 30 秒，三次历史 observation 全部是 \STALE\。因此当前决策只能是 \UNKNOWN / REQUOTE\，不能因为历史 \toAmountMin\ 看起来不错就写成候选，更不能写成实时套利机会。\\

\我也把新鲜度边界想清楚了：\0 <= quote_age <= max_age\ 才是 \FRESH\；超过门槛是 \STALE\；如果 observation 晚于 evaluation cutoff，算出负年龄，说明时间关系异常，应是 \UNKNOWN\，不是 \STALE\。Quote 仍然采用追加式 observation，后来的报价不能覆盖旧报价。\\

\成本部分继续坚持单位不混算。真实 Nordstern 路线的 \toAmountMin\ 是 \99.391824751519684052 USDC\，LI.FI 平台费属于 \included\，已经反映在最低到账中，不能再扣；Gas 属于 \external\，只有换到与决策相同的计价单位后才能扣。金额已知但 \inclusion=unknown\，或者 external 成本仍是 BNB、没有可信的 BNB→USDC 换算，都必须返回 \UNKNOWN\，不能把未知成本按 0 处理。\\

\纸面门槛统一采用一套公式：\\

\conservative_final = toAmountMin - external_costs - uncertainty_buffer\\

\然后只与 \minimum_acceptable\ 比较，不能又把成本和缓冲加到右侧，否则会重复计提。在教学假设“Quote 仍然新鲜、接受 LI.FI 的 USD 估值、1 USDC = 1 USD”下，Nordstern 扣除约 \0.0063 USDC\ Gas 后为 \99.385524751519684052 USDC\。最低接受到账设为 \99.38 USDC\ 时，最大可承受缓冲是 \0.005524751519684052 USDC\：缓冲 \0.004\ 时是纸面候选，缓冲 \0.006\ 时转为拒绝。这个例子也说明 margin 很薄，稍微提高保守要求就会翻转。\\

\今天用 TDD 抓到了两个会制造假候选的真实问题：旧逻辑曾静默忽略 \inclusion=unknown\ 的成本；负数缓冲还会因为“减去负数”反向增加保守到账。现在两者都 fail-closed 为 \UNKNOWN\。标准化器的精确数量、SQLite 结构、危险执行字段拒绝、真实 artifact 导入、新鲜度与纸面决策测试合计 16 项，完整套件通过。全程没有连接资金钱包、授权、签名或广播交易。\\

\下一步：重新获取一组只读、脱敏且足够新鲜的 LI.FI Routes，在固定估值时间和统一计价来源下跑完整门禁；仍然只做纸面评估，不调用交易执行接口。

笔记 024: Day 11 | 置信分层与自适应校准：5 个桶完成升级

作者：Courtney Snyder | 时间：2026-08-14 16:41 UTC | 字数：2252 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 | 置信分层与自适应校准：5 个桶完成升级

网站进展：截至 8 月 15 日，报名人数仍为 692，活动日历仍无新增安排；每日优秀学习笔记最新仍是 8 月 12 日榜单，今天没有新增官方议程或榜单。我的学习笔记中已确认 Day10 于 8 月 14 日 01:59 保存，今天尚未打卡。因此继续落实 Day10 的明日行动：给滚动门槛加入最小样本数、置信等级和自适应校准覆盖。

今天完成：

1. 增加三档置信等级

新增 confidence_aware_sampler.py。继续使用 order × direction × amount 的 12 个桶和最近 5 个完整路径样本，以 3 个样本作为最低可用线：少于 3 个标记为 insufficient，3–4 个标记为 developing，达到 5 个才标记为 mature。每个桶同时输出 history_gap，避免把只有 1–2 个点的 25% 分位数误写成稳定统计规律。

2. 实现自适应校准覆盖

不再固定只校准 1,000 USDC。所有低于最小样本数的桶自动执行完整四腿校准；若某个方向全部达到最低样本数，仍保留 1,000 USDC 锚点做漂移检查。低置信校准优先执行，遇到 HTTP 429 立即熔断，其他桶继续沿用两腿预筛。

3. 增加置信迁移与反证指标

每条记录保存校准原因、运行前后样本数、置信等级、是否升级，以及历史门槛与当前完整成本的误差。汇总同时记录校准覆盖率、请求预算、余量失守、疑似漏检、报价过期和 Paper Trading 淘汰原因。

4. 补齐测试

新增 `test_confidence_aware_sampler.py`，覆盖置信边界、样本缺口、自适应低置信校准、成熟桶的双向锚点，以及置信升级与报价预算汇总。当前全部测试 27/27 通过，脚本编译检查通过。仍使用合成地址，只读 LI.FI 报价，不签名、不发送交易。

真实采样时间：2026-08-15 00:37:04 至 00:37:54 (UTC+8)。

范围：CHEAPEST / FASTEST × 100 / 1,000 / 10,000 USDC × Base/Arbitrum 双方向，共 12 个候选。

本轮结果：

运行前 12 个桶中 insufficient 9 个、developing 3 个、mature 0 个；没有回退桶。

9 个低置信桶进入完整四腿校准，另外 3 个候选走两腿预筛且命中率为 0%。校准后 insufficient 降至 4 个，developing 增至 8 个，共 5 个桶完成升级；mature 仍为 0。

完整四腿基线需要 48 条报价腿；实际请求 42 条，节省 6 条，即 12.5%。另有 4 次 Token 元数据请求，单独统计。

42 条报价腿全部成功；HTTP 429、报价失败和熔断跳过均为 0。

12 个候选的保守现货 Edge 为 -152.0922 至 -137.4127 bps；9 条完整路径的净 Edge 为 -302.7316 至 -207.7856 bps，全部为负。

9 个校准误差为 0.0102–49.9214 bps，中位数 3.8219 bps。大多数仍接近历史，但两个 10,000 USDC 桶出现明显向上漂移：CHEAPEST 的 Base 买→Arbitrum 卖上移 49.9214 bps，FASTEST 的反方向上移 46.8286 bps，分别伴随 WETH 库存缺口或 USDC 再平衡成本上升。

这两个大误差代表当前完整成本高于历史门槛，会造成额外验证而不是漏掉盈利候选；因此 margin breach 为 0，疑似漏检为 0。但它们证明小样本下必须保留路由和成本方向的反证记录。

9 条完整路径中 4 条报价窗口超过 5 秒；所有路线预计最长执行时间不超过 7 秒。过期路径与其余完整路径都已先因净 Edge 低于 20 bps 被淘汰。

Paper Trading 判定：0/12 通过。3 条只完成低成本预筛；9 条完成完整模型但净 Edge 为负，其中 4 条同时报价过期。当前结果只是报价模拟，不代表真实成交或收益。

权威输出：

`data/day11/lifi_confidence_aware_20260814T163757Z.jsonl`

`data/day11/lifi_confidence_aware_20260814T163757Z.csv`

`data/day11/lifi_confidence_aware_summary_20260814T163757Z.json`

今天的核心认识：置信等级不能只看“有没有历史”，而要看每个具体桶有多少完整观测。扩大校准后，5

个桶跨过最低样本线，但仍没有成熟桶；同时两个大额桶出现约 47–50 bps

的向上成本漂移，说明滚动分位数应同时报告样本量、漂移方向和路由变化，单个阈值本身不够可靠。

明日行动：把 Day11 结果并入历史，优先再校准剩余 4 个 FASTEST 低置信桶，使它们达到 3 个样本；把漂移拆成“可能漏检的向下漂移”和“只会增加验证成本的向上漂移”，并按路由签名统计误差。继续一次性只读 Paper Trading，不使用真实资金。

笔记 025: Day 11 | 链上套利残酷共学

作者：HashFlow | 时间：2026-08-15 01:18 UTC | 字数：1487 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 | 链上套利残酷共学

日期：2026.08.15 (周六)

主题：DEX 间套利实操 — 重写扫描脚本，跑通发现→验证闭环

核心产出

重写扫描脚本 `arbitrage-scanner.py`（替代原单资产脚本），实现完整发现→验证闭环：

CoinGecko 资产列表 → DEX Screener 池子 → 同链 + 跨链价差 → 候选 → LI.FI 验证对比

扫描逻辑（澄清后确定）

资产列表：CoinGecko /coins/list，按 symbol 聚合（解决桥接版本拆分问题）

池子数据：DEX Screener 官方 /search 端点

价差：同链（各 DEX 价差）+ 跨链（各链价差）

阈值：MAX_ASSETS=200、MIN_LIQUIDITY=\$50万、MIN_SPREAD_PCT=0，全部可配置

验证：候选自动接 LI.FI，同链（代币→USDC 直接卖）vs 跨链（便宜链→贵链）对比

三个关键 bug（数据真实性问题）

- CoinGecko 桥接版本拆分：WETH 有 63 个单链条目 → 按 symbol 聚合
- priceUsd 污染：目标代币作 quoteToken 时 priceUsd 是对方价格 → 只保留 baseToken 匹配池
- 死池污染：\$0.07 流动性的池子价格失真（\$130 vs 真实 \$86）→ MIN_LIQUIDITY=\$50万

扫描结果 (200 资产 → 6 跨链候选)

资产	DEX 跨链价差	LI.FI 验证结论
APXUSD	2.71%	跨链 +0.5%，但脱锚 (0.87-0.93 USDC)
WSTETH	0.90%	桥费吃掉价差，无优势
WBTC	0.79%	LI.FI 无报价 (待排查)
WETH	0.17%	桥费吃掉价差，无优势
WEETH	0.14%	无优势
AAVE	0.08%	无空间

核心认知

1. 主流资产跨链无利可图——价差 0.2-0.9%，被 LI.FI 平台费 25bps + 桥费吃掉
2. 唯一「机会」是脱锚稳定币 APXUSD——正是知识图谱「稳定币套利：脱锚可能不回」的风险场景，不是健康套利
3. 扫描脚本闭环验证成立——发现层 (CoinGecko+DEX Screener) → 验证层 (LI.FI) 自动衔接

待办

WBTC 无报价排查 (聚合地址与 LI.FI 桥接版本不对应?)

扫更多资产 (当前 200/1280 多链资产) 找长尾机会

APXUSD 脱锚深度分析 (是否值得研究稳定币套利方向)

下一步

Day 12 (周日) : 清算/MEV 原理 — Bruce Lab mev-value-flow

笔记 026: 今天查阅了 LI.FI 官方关于 /quote 与 /advanced/routes 的说明。/qu

作者: jason52012345-bot | 时间: 2026-08-15 02:58 UTC | 字数: 344 | 评分: 46

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天查阅了 LI.FI 官方关于 /quote 与 /advanced/routes 的说明。/quote 只返回一条最佳的单步骤路线，并已包含执行所需的链上交易数据，适合路径不需要额外链上动作的场景；它并不等同于“所有可选方案”。

当跨链后还需要在目标链继续兑换等更复杂流程时，应使用 /advanced/routes 获取候选 Route。选定 Route 后，要对其中每个 Step 调用 /advanced/stepTransaction 取得交易数据；若只希望取得单步骤路线，可在请求中设置 allowChainSwitch: false。后续做套利路径比较时，会先按交易步骤数与是否需二次触发区分报价，避免把单笔可执行性与多步骤路线混为一谈。

笔记 027: 链上套利残酷共学 · Day 10 心得

作者: kako | 时间: 2026-08-15 04:11 UTC | 字数: 690 | 评分: 55

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 10 心得

今日主题: Hermes Agent 上手

今天是学习路径的一个转折点——从完全手动操作切换到使用 AI Agent 辅助信息发现。

安装 Hermes Agent 的过程还算顺利，跟着文档一步步配置环境、设置 API 密钥，很快就跑通了第一个调研任务：让 Agent 整理一份关于跨链套利机会的调研摘要。它返回了包括常见路径、主流桥的费率区间、以及一些公开交易案例的参考链接，信息量确实比自己手动搜索要高效得多。

但真正有价值的不是 Agent 给的结果，而是核验过程。仔细检查后发现，某些桥的费率数据像是过时的版本，跟 LI.FI (<https://li.fi/>) 上查到的当前报价对不上。这让我意识到一个关键认知：Agent 擅长整理结构化的知识框架，但所有涉及实时价格、流动性状态、动态费用的信息，都必须用工具二次确认。不能因为它输出得流畅就默认它准确。

这个区别今天梳理得很清楚：跨链套利的大致流程、有哪些成本项、常用的工具名称，这些静态知识可以信任 Agent 的总结；但具体的桥费是多少、当前价差有多大、目标链 Gas 费现在什么水平，这些动态数据必须自己查证。

更值得警惕的是“执行幻觉”这个概念——Agent 可能会生成一段看起来完整但实际没有真实调用工具的执行报告。对这种情况保持清醒的唯一方法就是：凡是涉及链上数据和实时报价的内容，一律手动复核。信任 Agent 的信息组织能力，但不偷懒跳过核验环节。

今天的经历让我对后续使用 AI 辅助套利分析有了更实际的预期——它是个好助手，但不能替我做最后的决策。

笔记 028: https://github.com/ZeroNullNone/onchain_arbitrage_

作者: Nothing | 时间: 2026-08-15 05:34 UTC | 字数: 2236 | 评分: 72

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/docs/daily/day_11.md

Day 11 — Rebalance Economics

状态与结论

- 状态 : Day 11 deterministic、paper-only acceptance scope 已完成。
- 已实现 Immediate、Threshold-based、Batch 三种 rebalance policy ; 所有完整结果都会在周期末恢复 residual imbalance , 不把未平仓库存包装成 Cycle PnL。
- Day 10 单笔 local Trade PnL 为 +5 USDC ; Immediate 假设每次 0.25 WETH rebalance 成本为 7 USDC , 四笔交易的 local PnL 为 +20 USDC , 但完整 Cycle PnL 为 8 USDC。
- 上述成本和 capacity 数值是 deterministic paper assumptions , 不是 live Bridge/CEX evidence 或盈利结论。

完成项

- 新增 exact-size RebalanceCostObservation , 强制记录 request ID、Raw reference、source、UTC timestamp、latency、integer raw transfer amount 与 cost。
- 缺失准确 size 的成本时显式返回 COST_INCOMPLETE ; 不会插值、猜测、使用 stale cache 或把 missing cost 当成零。
- InventoryCycleCostLedger 成为完整周期 PnL subtraction 的单一 owner , 避免重复扣除 Day 10 已处理的 atomic cost。
- 输出 Cycle PnL、rebalance count/frequency、total 与 per-event break-even cost、ending imbalance、imbalance distribution 和逐次 rebalance event evidence。
- 三 Policy 的四笔交易结果分别为 : Immediate 8 USDC / frequency 1 ; Threshold +2 USDC / frequency 0.5 ; Batch +4 USDC / frequency 0.25。
- break-even total rebalance cost 为 20 USDC ; Immediate 的 per-event break-even 为 5 USDC。
- capacity curve 展示 100 / 500 / 1,000 USDC 三档 : Cycle PnL 为 2 / 1 / 2 USDC , edge 为 200 / 20 / 20 bps , 因此 tested profitable capacity 为 500 USDC。

Evidence 与验证

- 完整语义、Policy 表、imbalance distribution、break-even 与 capacity worked example : docs/day11_rebalance.md。
- Append-only deterministic raw paper fixture : tests/fixtures/day11/rebalance_costs.json ; 每个 normalized test observation 保留准确 JSON Pointer。
- Acceptance tests : tests/test_rebalance.py。
- 本日没有外部请求、wallet lookup、Bridge/CEX order、签名或广播交易。

Gate 边界

- 只实现 deterministic virtual rebalance ; Natural Flow、CEX/Perp Hedge、stochastic rebalance、funding forecast 和 optimizer 保留在 backlog。
- Capacity 是 largest strictly profitable tested size , 不插值未观察 size , 也不声称全局最优。
- Threshold/Batch 在持有期间承担更大的 inventory imbalance ; 较低 frequency 不等于无风险。
- 不再用单笔 local Trade PnL 判断 cross-chain strategy profitability。

下一步

- Day 12 建立以 chain_id + contract_address 为 identity key 的 token registry , 显式记录 canonical/bridged/wrapped、issuer、redemption 与 basis-risk 决策。

笔记 029: Day 11 — 多链套利策略

作者 : pennchester | 时间 : 2026-08-15 05:44 UTC | 字数 : 794 | 评分 : 86

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 — 多链套利策略

今日核心收获

1. 各链套利特性差异显著

- Arbitrum 0.25s 出块 + \$0.02 Gas = 最佳高频套利链 ; Ethereum L1 12s + \$25 Gas 只适合 \$200+ 大额机会
- L2 的优势不是价差更大 , 而是门槛更低——小额机会在 L1 被 Gas 吃掉 , 在 L2 能盈利

2. 同一机会跨链对比 (1.2% 价差 \$100K)

- Solana 净利 \$1180 vs Arbitrum 净亏 \$800 vs L1 净亏 \$58825
- 双边费用是决定性因素，不是价差本身

3. 跨链套利受桥延迟制约

- 原生桥 7min → 价格波动 0.53%，净利差 0.86% 勉强可行；聚合器 (LI.FI/Across) 可提速到 1-2 分钟

4. 今日实战：Robinhood Chain 多链监控落地

- 调研 Robinhood Chain (Arbitrum Orbit L2, Chain 4663)：NOXA 发射台已死、无原生 USDC、USDG 为结算资产、无原生链币
- 实现并部署 USDG 锚定监控 + 新币发射监听 + 风险分析 (owner/供应集中/流动性 → 风险分 0-100)
- 本地 CEX API 被墙，改用链上 Chainlink 预言机取 ETH 价，零外部依赖
- 踩坑：V3 池价必须处理 decimals 差异 (USDG 6位 vs WETH 18位)，否则价格错 10^{12}

产出

- 学习笔记：Day11_多链套利策略.md
- 代码：multichain_simulator.py、robinhood-monitor/ (已部署 43.133.86.35:33334)

明日计划

Day 12：合约安全

笔记 030: 核心策略方向与实施阶段

作者：Yolandatsai | 时间：2026-08-15 10:43 UTC | 字数：508 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

核心策略方向与实施阶段

机会发现与信号监控：利用 Hermes Agent 定时拉取并分析不同链、DEX 之间的价格差、流动性迁移和资金费率变化。 \[1 (<https://intensivecolearn.ing/programs>), 2 (https://x.com/brucexu_eth/article/2082831191376613391)]

路径分析与成本验证：以 LI.FI 提供的聚合跨链及兑换路径为基础，计算扣除 Gas 费、滑点、跨链桥手续费及执行延迟后的真实利润空间。 \[1 (<https://intensivecolearn.ing/programs>)]

策略构建与模拟回测：通过 AI 快速编写数据采集和 Paper Trading (模拟交易) 脚本，在真实资金投入前进行风险与边界条件验证。 \[1 (https://x.com/brucexu_eth/article/2083935229757337948), 2 (https://x.com/brucexu_eth/article/2082831191376613391)]

笔记 031: D08 | 跨链套利

作者：averywexler | 时间：2026-08-15 12:33 UTC | 字数：821 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D08 | 跨链套利

跨链不是一笔很长的兑换，而是多条链、桥或消息系统、不同确认规则共同组成的流程；时间本身就是成本和风险。

今日术语

源链 / 目标链：资金开始所在的链与最终到达的链；

桥：在链之间转移价值或传递消息的系统；

最终性：一笔交易被认为难以撤销所需要的确认状态；

流动性再平衡：跨链服务商补充各链库存的过程；

非原子执行：某一步成功后，后续步骤仍可能失败；

跨链聚合器：比较桥、DEX 和路线并返回组合报价的工具。

先读 12 分钟

Optimism：Standard Bridge (<https://docs.optimism.io/app-developers/guides/bridging/standard-bridge>)

LI.FI：Endpoint Specs (<https://docs.li.fi/agents/reference/endpoint-specs>)：只观察 route、estimate、gasCosts 和 feeCosts 等字段，不连接钱包。

把流程画出来

假设你看到链 A 的 ETH 便宜、链 B 的 ETH 贵：

1. 在链 A 换入 ETH；
2. 等待源链确认；
3. 通过桥或流动性网络转移；
4. 等待目标链可用；

5. 在链 B 卖出 ETH；
6. 处理留在两条链上的 Gas 代币与库存。

其中任何一步发生延迟，链 B 的价格都可能已经改变。

跟着算一次

屏幕毛价差为 120 USDC，扣除：

两条链上的 DEX 费用与价格冲击 32；

源链和目标链 Gas 14；

桥或求解器费用 18；

资金等待成本与风险缓冲 40；

估算净收益只剩 16 USDC。若最终输出比报价少 0.2%，机会就可能消失。

笔记 032: 阶段复盘

作者：hello16 | 时间：2026-08-15 12:55 UTC | 字数：1933 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

阶段复盘

跨链套利是否值得投入？

简短结论（针对当前阶段）：\

值得作为研究方向投入，但不建议现阶段作为主要盈利策略重仓投入真实资金。

它更适合用来：

建立完整的「价格发现 → 成本建模 → 执行验证」研究能力

理解多链流动性与桥的真实摩擦

积累可迁移的量化/监控技能

而不是期待短期内稳定产生可观利润。

比较：Ethereum MEV vs Cross-chain

| 维度 | Ethereum MEV | Cross-chain 套利 |

| :----- | :----- | :----- |

| 竞争强度 | 极高。专业搜索者 + Builder + 私有订单流主导 | 相对较低，但仍在快速上升（聚合器普及后机会暴露更快） |

| 执行窗口 | 毫秒~秒级（同块或相邻块） | 秒~分钟级（受桥最终性、确认时间影响） |

| 主要壁垒 | 延迟、基础设施、私有流量、资本、关系 | 成本精确建模、桥可靠性、资金占用、失败处理 |

| 成本结构 | 主要是 Gas + 优先级费用 + 失败成本 | 桥费 + 双边 Gas + 滑点 + 资金锁定机会成本 + 失败成本 |

| 可重复性 | 高（但被专业玩家高度内卷） | 中等（结构性摩擦存在，但窗口不规律） |

| 个人/小团队可行性 | 很低（除非有显著技术或流量优势） | 中等偏低（研究与监控可行，稳定盈利难） |

| 风险特征 | 被夹、被抢跑、Gas 战争 | 桥延迟/失败、脱锚扩大、资金长时间占用、路径失效 |

| 学习价值 | 极高（理解区块构建、优先级、搜索） | 高（理解流动性碎片化、跨域最终性、真实可执行性） |

| 当前 Edge 来源 | 速度、私有信息、库存管理 | 对特定桥/资产对的延迟与成本有更细的理解 + 中等规模甜蜜点 |

深入判断

1. Ethereum MEV 的现状

已经高度专业化。普通公共 RPC + 公开内存池几乎没有持续优势。

真正的利润集中在少数有低延迟、私有订单流或自建基础设施的团队。

对个人学习者来说，研究价值很高，但直接参与获利门槛极高。

2. 跨链套利的现实

结构性摩擦仍然存在：不同链最终性时间不同、桥流动性分布不均、用户行为导致的短期供需失衡。

LI.FI 等工具大幅降低了「路径发现」门槛，但也让机会被更快发现和抹平。

真正的潜在 edge 往往不在「看到价差」，而在于：

对特定桥的真实延迟与成功率有更准确的数据

成本模型足够精细（尤其是资金占用和失败成本）

找到中等规模、竞争没那么极端的区间

稳定币相关（脱锚回归）和特定长尾资产有时会出现更明显的结构性机会，但频率低、规模敏感。

3. 对当前共学参与者的建议

值得投入的部分：监控系统、成本模型、证据记录、Paper Trading、失败复盘。这些能力可迁移。

谨慎投入的部分：真实大资金执行。在没有经过充分模拟和严格风控前，风险远大于收益预期。

更优策略：把跨链作为「研究与能力建设」的主战场，同时保持对 Ethereum MEV 机制的学习，而不是二选一。

阶段复盘一句话

跨链套利目前更像是一个高学习价值、中等研究价值、低稳定盈利确定性的方向。\\

Ethereum MEV 是「已经打到终局」的战场，跨链则是「摩擦还在、但窗口正在被压缩」的战场。

对于个人而言，真正值得持续投入的，不是「找到一个永远有效的策略」，而是建立起能不断验证、淘汰、迭代假设的研究系统。

笔记 033: 價格監控 + LI.FI 套利分析工具

作者：ed11555 | 时间：2026-08-15 13:05 UTC | 字数：3312 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

價格監控 + LI.FI 套利分析工具

一、目前目標

建立一套簡單的「套利分析工具」，第一階段先不做自動下單，主要功能為：

價格監控 → 發現價差 → LI.FI 實際報價 → 成本計算 → 判斷套利是否有利潤

核心目的不是看「表面價差」，而是計算真正可執行的 Net Profit / Net ROI。

二、基本架構

交易所 / DEX 價格監控

↓

初步價差篩選

↓

LI.FI Quote

↓

套利成本計算

↓

Net Profit / ROI

↓

符合門檻

↓

顯示套利機會 / 發送通知

第一版可以拆成：

price_monitor.py

↓

opportunity_filter.py

↓

lifi_quote.py

↓

profit_calculator.py

↓

dashboard / Telegram Alert

三、Stage 1：價格監控

先監控不同市場的即時價格，例如：

Bybit

Binance

OKX

Ethereum DEX

Base DEX

Arbitrum DEX

例如：

ETH

Bybit \$4,500

OKX \$4,502

Base \$4,529

此時表面價差：

$(\$4,529 - \$4,500) / \$4,500$

$\approx 0.644\%$

但 0.644% 不代表真正可以賺 0.644%。

因此只把它當成第一層篩選。

例如：

Raw Spread > 0.4%

才進入下一階段。

四、Stage 2 : LI.FI Quote

發現潛在價差後，再呼叫 LI.FI。

主要取得：

fromChain

toChain

fromToken

toToken

fromAmount

toAmount

toAmountMin

Route

Gas / Fee

其中重點是：

套利計算優先使用 toAmountMin，而不是只使用 toAmount。

因為需要用較保守的實際最低收到數量判斷套利是否成立。

LI.FI 在這套系統中的角色主要是：

跨鏈 Route

\+

DEX Swap Route

\+

實際可成交 Quote

\+

後續資金 Rebalance

五、套利 Profit Engine

不能使用：

賣出價格 - 買入價格 = 套利利潤

真正計算方式應該是：

Gross Profit

\- CEX Trading Fee

\- DEX / Swap Cost

\- Gas

\- Bridge Cost

\- Withdrawal Fee

\- Slippage

\- Safety Buffer

\-----

\= Net Profit

公式：

gross_profit = sell_value - buy_cost

net_profit = (

gross_profit

cex_fee

withdrawal_fee

gas_cost

bridge_cost

swap_cost

slippage_buffer

)

net_roi = net_profit / capital

真正需要監控的是：

Net Profit + Net ROI

而不是 Raw Spread。

六、簡單案例

假設：

本金：\$4,500

Bybit 買 ETH：

\$4,500

另一市場實際價值：

\$4,535

Gross Profit：

+\$35

扣除：

CEX Fee -\$4.5

Gas -\$0.3

LI.FI / Route Loss -\$5.2

Slippage Buffer -\$3.0

得到：

Net Profit = \$22

Net ROI

\= 22 / 4500

≈ 0.489%

所以真正套利收益為：

+\$22 / +0.489%

而不是表面看到的 \$35。

七、套利觸發條件

第一版可以先設定：

MIN_RAW_SPREAD = 0.004

MIN_NET_ROI = 0.003

MIN_NET_PROFIT = 10

也就是：

表面價差 > 0.4%

↓

取得 LI.FI Quote

↓

Net ROI > 0.3%

\+

Net Profit > \$10

↓

發送套利訊號

這樣可以避免每次價格變化都呼叫 LI.FI API。

八、工具輸出範例

ARBITRAGE SCANNER

Pair:

ETH / USDC

BUY:

Bybit

\$4,501.20

SELL / SWAP:

Base via LI.FI

\$4,529.80

Raw Spread:

+0.635%

Capital:

\$5,000

Estimated Costs:

-\$11.24

Net Profit:

+\$20.19

Net ROI:

+0.404%

Route:

Ethereum

↓

Across

↓

Base

↓

Aerodrome

Status:

PROFITABLE

九、後續升級方向

初期：

價格監控

↓

LI.FI Quote

↓

套利分析

↓

Telegram Alert

第二階段：

價格監控

↓

LI.FI Quote

↓

Profit Engine

↓

資金管理

↓

自動下單

更進一步：

Price Monitor

↓

Opportunity Engine

↓

LI.FI Quote

↓

Profit Engine

↓

Execution Engine

↓

Hedge

↓

Rebalance

十、重要策略方向：雙邊預置資金

長期不應完全依賴：

買入

↓

Bridge

↓

等待到帳

↓

賣出

因為會有：

Bridge 時間

價格變動

Slippage

Gas

執行延遲

套利窗口消失

比較理想的模式是：

Bybit：預先放 USDT / USDC

Base：預先放 ETH / USDC

Arbitrum：預先放 ETH / USDC

Ethereum：預先放 ETH / USDC

出現套利機會時：

市場 A：立即 Buy

\+

市場 B：立即 Sell

兩邊同步鎖定價差。

之後再透過：

LI.FI → Rebalance 資金

因此 LI.FI 長期可以從「套利交易路由工具」逐步轉變成「跨鏈資金再平衡工具」。

十一、目前 MVP 定位

目前先完成：

「發現價差 + 驗證實際 Quote + 計算真實淨利」

暫時不處理：

自動下單

私鑰

錢包簽名

自動 Bridge

自動資金調度

笔记 034: 今日学习记录：LI.FI 基础了解

作者：cawood | 时间：2026-08-15 13:44 UTC | 字数：308 | 评分：45

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习记录：LI.FI 基础了解

今天简单了一下 LI.FI。它可以理解成一个跨链和 DEX 聚合层，开发时不用自己一个个对接不同的桥和交易协议，而是可以通过统一的 API / SDK 去获取跨链、兑换的报价和执行路径。官方 API 是 REST 接口，基础调用不强制需要 API Key，Key 主要用于提高 Rate Limit。目前我的理解是，LI.FI 比较方便的地方在于它把复杂的跨链路由封装起来了，可以根据不同链、Token 和金额寻找合适的执行路径，同时还能查询交易状态。后续可以重点学习 /quote、高级 Routes，以及实际交易的 calldata 是怎么生成和执行的。

笔记 035: 今日计划

作者：Freiheit1992 | 时间：2026-08-15 14:02 UTC | 字数：2343 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日计划

延续昨日计划：固定跨链路径与金额，保存 LI.FI 原始报价，完成全成本净收益计算并运行最小监控。

审查 3 个外部套利仓库，提炼可用于资金费率、跨市场价差及 DEX-CEX 系统的模块与风险。

根据 Binance 官方公告核查 Web3 Wallet API 的日本地区/IP 限制。

汇总可核验证据并完成共学打卡。

实际完成

完成 rfk1118/okx-bnexchange、rfk1118/bp-by、rfk1118/by-bg 的目录、README 与核心源码审查，并映射到 aster-lighter、funding-rate-monitor 的潜在能力：DEX-CEX 适配器、现货—永续执行器、独立紧急退出服务。识别关键生产风险：bp-by 使用错误盘口方向计算开平仓价差；by-bg 存在交易所实例与命名不符、平仓方向错误；okx-bnexchange 在链上确认后才做 CEX 对冲，裸腿时间过长；三者均缺少完整状态机、部分成交补偿与重启对账。完成 Binance Web3 Wallet API 日本地区的官方资料核查：官方公告仅称 API 在 Binance Wallet 当前支持地区可用，未公布“日本 IP 被封禁”的结论；因此只能判断“暂无官方公告证明日本 IP 受限”，不能保证实际请求一定可用。完成课程页、身份、课程与打卡列表的实时校验。未完成 LI.FI 实时报价、全成本净收益计算或最小监控脚本运行，因此未将其写成已完成。

证据

Hermes 研究会话：@session:default/20260805_071727_b6487320（包含 3 个仓库源码读取、逐项风险分析及 Binance 官方公告研究）。

okx-bnexchange 核心业务源码（<https://github.com/rfk1118/okx-bnexchange/blob/main/src/business.ts>）、bp-by 核心策略源码（<https://github.com/rfk1118/bp-by/blob/main/src/bot.ts>）、by-bg 紧急退出源码（<https://github.com/rfk1118/by-bg/blob/main/src/index.ts>）：本次实时读取均为 HTTP 200，正文长度分别为 4610、5008、4250 字符。

Binance Wallet Web3 API 发布公告（<https://www.binance.com/en/support/announcement/a9f19002d4584183b83892b002d61f96>）：当日研究已核对官方关键句“Web3 API is available in all regions where Binance Wallet is currently supported”；本次回访触发 Human Verification，未取得新的页面正文。

《链上套利残酷共学》课程页（<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>）：2026-08-15 22:01（Asia/Shanghai）实时访问成功，显示“共学正在进行”，周期 2026-08-05 至 2026-08-26。

API 预检：GET /me → HTTP 200；GET /programs/{programId} → HTTP 200，状态 ongoing；写入前 GET /me/check-ins → HTTP 200，返回 10 条记录，今日无记录。

本地项目核验受阻：执行后端真实返回 Docker executable not found in PATH or known install locations，无法读取两个本地项目。

LI.FI 原始报价、净收益计算、监控运行日志或今日代码提交：暂无可核验证据。

结果与判断

今日形成了可用于真实套利系统的代码审查结论：外部仓库适合借鉴接口和工程思路，不适合原样接入实盘。优先级应为 PO 执行安全（状态机、幂等、部分成交补偿、仓位对账、kill switch），P1 按目标数量 VWAP 与全成本净 Edge，随后才是 DEX-CEX 扩展。日本 IP 问题的官方证据不足以支持“受限”结论。

未完成 / 卡点

未取得 LI.FI 固定路径的原始报价与完整路由。

未完成 Gas、桥费、DEX/CEX 手续费、滑点、MEV/失败风险和资金成本后的净收益判断。

未运行并保存最小报价监控输出。

本地 Docker/文件后端不可用，无法把建议精确映射到 aster-lighter、funding-rate-monitor 的具体文件。

Binance 公告回访遇到人机验证；暂无官方日本 IP 封禁名单可核验。

明日下一步

恢复本地项目只读访问，先做 PO 执行安全矩阵。

固定一条 LI.FI 路径和金额，保存原始请求、响应、路由及时间戳。

用同一报价计算全成本净 Edge，并明确“成立 / 不成立”。

把报价请求做成最小监控，保存一次真实运行输出；如需验证地域限制，再从日本出口 IP 发起受控 API 请求并记录真实状态码。

笔记 036: 链上套利残酷共学 · Day 11 打卡 (2026-08-15)

作者：keltuz4d | 时间：2026-08-15 14:27 UTC | 字数：3073 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 11 打卡 (2026-08-15)

今日没有收到新的增量笔记

PDF，所以这张卡不做“新内容精读”，而是把这两天拆的两个真实链上案例沉淀成可识别、可扫描的套利模式

一个“只读发现”框架。两条主线：KII 长尾跨链套利、sUSD/Neutrl NAV 折价借贷套利。

一、案例一：KII 长尾跨链套利 (DeBank 真实地址 0xda43...)

对象澄清（最容易误解的点）：套利者从头到尾只倒腾一种币

KII，不是“在三个所买三种不同的币”。变的是场所 (venue)，不是币：

- BSC PancakeSwap（深池，≈\$1.5M，占总流动性 99% = 公允价值基准）
- Mantle / Base 上的薄 DEX 池（几千~几万刀，单笔就能推高 = 经常更贵）
- Binance（独立订单簿，暴跌时短暂脱钩）

每边用稳定币 (USDT / USDT0) 当中间钱 : BSC 用 USDT 买 KII → LayerZero transferRemote 桥到 MNT → MNT 上卖成 USDT0 → LI.FI 路由 → 利润 Deposit 到 Binance。

模式本质: 薄池长尾小币在"深链便宜 / 薄链贵 / CEX 脱钩"之间的跨场价差, 实测毛差 +1.65% (DeBank 真实成交 : 0.0869 买 → 0.0883 卖)。

自动化特征 (从链上可见) : 机器人跑, 非手动——① 两地址高频重复同一条腿 (MNT↔Binance 15/16 笔); ② 同发件人多收件人 (抗 CEX 风控); ③ 金额阶梯式 (1K→231K) 按池深动态调单避免自噬价差; ④ 最后 0 KII 那笔像预留/测试新路由。

两个硬核实测发现:

- LI.FI 全链不收录 KII (扫过所有链 token 列表确认) → 长尾币套利机器人必须自己读链上池子算即时价, 不依赖聚合器 (呼应 Day 7-9 收敛)。
- LI.FI 实测稳定币跨链腿 (BSC→MNT USDT) 成本仅 0.0813% (走 layerswap 子桥, 8 分钟); 但反向腿 LI.FI 无路由 (404) → 回程得自己走 CEX/桥。

关键认知: 预置库存是它能"快"的前提——钱早已铺在多链, 套利瞬间本地买卖、事后慢慢再平衡。这正是 jinbug101-art 说的"66% 跨链机器人预置库存"。

二、案例二: sNUSD / Neutrl NAV 折价借贷套利 (Morpho Blue, tx 0x82a206...)

本质: 这是结构性错价, 不是跨链、不是三角。家族叫"NAV 折价借贷套利"——专吃"衍生品/质押币在 DEX 市价 < 借贷协议按净值(NAV)承认的价"的裂缝。

触发信号 (抽象成通用公式) :

DEX 成交价 < LLTV × 预言机 NAV

本案 : $0.686 < 0.915 \times 1.0638 = 0.9734$

利润上限被池深锁死: 单笔利润 = 可买量 × (LLTV×NAV - DEX价)。本案两池共 ~4045 枚, 单次上限仅 ~\$260 (已实现 \$167 + 未实现 \$94)。薄池是双刃剑: 折价越深, 能买的越少。

执行链路 (原子化, 缺一不可) : Morpho 免费闪电贷 → 顶格存 MetaMorpho 金库 (拿 86.97% 份额当放大器) → 捐赠 1000 USDC 制造借款空间 → 聚合器买 sNUSD → 存抵押+借款(onBehalf 一次性合约) → 赎回金库 → 还闪电贷 → 利润 sweep。反直觉细节: 利用率 100% 的市场靠"捐赠"解封; 债务甩给交易内新建的一次性合约, 借出的钱却转给自己。

真实风险在退出端 (报告强调) : 借出的是自己的债务头寸。若 sNUSD 无法按 NAV 赎回或 NUSD 脱锚 → 抵押市价 709.9 < 债务 1006.9 → 资不抵债被清算。单笔薄、有在位高频竞争者、需私有 relay 防 frontrun。

三、方法论沉淀: 套利模式的"两层分类"

| 维度 | KII 案例 | sNUSD 案例 |

| --- | --- | --- |

| 错价性质 | 流动性薄导致的瞬时/事件价差 (暴跌 -37.66% 放大) | 结构性错价 (预言机 NAV ≠ DEX 现货) |

| venue | 跨链 (BSC/MNT/Binance) | 同链跨协议 (DEX 价 vs 借贷 NAV) |

| 所需基础设施 | LI.FI + LayerZero + CEX 充值 + 预置库存 | 自写合约 + 闪电贷 + 私有 relay |

| 利润容量决定因素 | 池深 × 窗口时长 | 池深 (硬封顶 ~\$260/笔) |

| 散户现实度 | 中 (长尾+事件+预置, 呼应 Day7-9) | 低 (写合约+签名越界) |

关键判断: 两种都验证"散户唯一现实路径 = 长尾 + 事件驱动 + 预置库存"。主流币价差被 LI.FI 压到 <0.1% 必亏; 能赚的全在薄池/结构性裂缝里。

四、只读发现框架 (今天最硬的收获)

把"找机会"和"执行"彻底分开, 严格守"不碰私钥、不签名"边界:

- 找机会 = 纯只读, 可做:

- KII 类: 读各链 DEX 池 reserves/slot0 算即时买卖价, 比对跨 venue 价差 > 阈值(0.5~1%) 才报警

- sNUSD 类: 扫 Morpho Blue 市场, 按 DEX价 < LLTV×NAV 信号 + 读池深算利润容量, 输出折价市场清单

- 数据源全公开: Morpho subgraph / eth_call 读 convertToAssets / Uniswap V4 slot0+liquidity / The Graph

- 执行 = 越界, 不做: 部署合约、签名借款、闪电贷实跑、私有 relay——今天明确不碰

- 已给"机会雷达"一个监控管线伪代码: 每区块拉市场→过滤 NAV 类预言机→算 dex<LLTV×nav→读池深算容量→超阈值报警

五、批判性提醒 (给自己的)

1. KII 暴跌 -37.66%, 薄池小币的"价差"可能是庄家制造的陷阱单边 (回想 Day7 TUT 插针)。

2. sNUSD 案例单笔薄、有竞争者、退出端风险 (脱锚/无法赎回), DeBank 只看资金流看不出净利。

3. 两者都是"真实执行", 看可以学机制, 不能照抄动手——尤其涉及签名/CEX 提币。

六、明日计划

1. 搭一个只读的 Morpho NAV 折价扫描器 (Python + subgraph/eth_call, 不签名), 跑出当前所有折价市场清单——把今天的"机会雷达"落成真东西。

2. 等 8/15 增量笔记，对照"NAV 折价 / 结构性错价"主题看群里有没有人做出可复制版本。
3. 把 LI.FI 的 integrator=jumper.exchange 修复补进 /root/lifi_spread_probe.py (拖了几天，该收尾了)。

笔记 037: 人攒多了，慢慢形成个习惯——看到一个「机会」先别激动，把它当嫌疑犯审一遍。汇总这段时间调研的东西，其

作者：Joneloo | 时间：2026-08-15 15:00 UTC | 字数：1247 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

人攒多了，慢慢形成个习惯——看到一个「机会」先别激动，把它当嫌疑犯审一遍。汇总这段时间调研的东西，其实就是一套怎么审的方法，加上自己掂量分量再决定动不动手。

一、真实性：这是不是个假机会

先回答最贱的一个问题：它凭什么还没被别人拿走？

这段时间看过的真实案子，假机会大多是这三种死法：

- 单一价格源骗你：Metronome 的 msUSD 崩到 \$0.73 了，主流 feed 还显示 \$0.96，差 22 美分——你信哪个，决定你是抄底还是接飞刀。所以价格必须交叉验证，脱锚盯供应端（铸币/赎回速度）而不是盯报价。
- 幻觉价差：我用 LI.FI 拉 Arb↔Base 的 WETH，两链买价都 1880、卖价都 1861，价差为零。看起来「能做套利」的机会，真一算，扣掉那 1% 做市价差，稳稳亏。市场早把这肉磨平了。
- 结构性信号先动：区块构建那套理论告诉我们，真正的机会藏在排序权、订单流、供应端这些「价格之前」的东西里，不是事后那根红蜡烛。

二、可执行性：能不能真的咬下来

真实性过了，问第二个问题：两条腿是不是都能锁住。

一条腿 = 赌方向，那不叫套利。跨链尤其危险，源链成功目标链还能失败，前面那份清算报告里「链上可见 ≠ 真实利润」就是这个道理。

报价要认 toAmountMin（保守到账），别认展示价。跨链桥的中间资产、PARTIAL 这种半成品状态，都得当成本算进去。

成本别漏项：手续费、桥费、gas 都小，真正的大头是买卖便宜那 1% 价差——我在实测里看得清清楚楚，L2 上 gas 才几美分，那堵墙才是拦路的。

三、风险：最坏情况你扛不扛得住

过了前两关，泼盆冷水。把「最坏情况」单独问一遍：

- 买卖只有一笔成功，库存被套住怎么办？
- 回测赚钱，上实盘会不会被价格冲击、卡单、抢跑打回原形？（模拟盘 ≠ 实盘，这个坑最贵）
- 资金占用多久？小资金连手续费都不够，这个行当真要大本金。

四、按自己分量决定：三档处理

检查完，别硬上，按能力分级：

1. 继续观察——证据不足、成本算不清的，就先挂着，不急着证明自己聪明。省下的是真金白银。
2. 模拟验证——用历史数据回测 + 压力测试。我这几天跑的那套 lifi 回测引擎就是这个干法：30 个机会全被「利润低于门槛」拒掉，15 组压力测试全 0 交易——回测跑出 0 交易是有效结果，说明肉被磨平了，省得送手续费。想验证一个假设，就得真跑数据，别对着概念空想。
3. 严控实盘——只有回测给了明确 Go 信号、又是小额、又设了硬性限额（单笔、单链、失败上限、停机线）的前提下，才碰真实资金。研究阶段的纪律，就是实盘阶段的防火墙。

说到底就一句话：看到信号，先审三关——是不是真的、咬不咬得动、扛不扛得住——然后照着观察 → 模拟 → 严控实盘三级走，能省下 90% 的手续费，也避开 100% 的接飞刀。方向不稀缺，稀缺的是敢不敢对一个看着诱人的机会说「再等等」。

03 MEV与抢跑 （2 篇）

笔记 038: 拿两笔真实交易测试 MEV Research OS。

作者：bai·小白 | 时间：2026-08-14 15:48 UTC | 字数：1770 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

拿两笔真实交易测试 MEV Research OS。

第一笔让我以为它已经会分析了，第二笔才发现，它只是刚好背过答案。

第一笔交易很典型。先从 dYdX 借来临时资金，在 Balancer 把 WETH 换成 DAI，再去 SushiSwap 换回更多 WETH，最后还掉借款。系统把几十次内部调用压成了四步，连 Gas 和交易范围内的资产增量都算出来了。

看到结果的时候，我还挺满意。输入一笔交易哈希，系统自己读取 Transaction、Receipt、Block 和 Call Trace，然后把资金怎么走、经过哪些协议、最后赚了多少钱讲清楚。

这不就是我想做的东西吗。

然后我换了第二笔交易。

这笔交易有 301 次内部调用，22 条 ERC-20 资产流，里面还碰到了 Morpho、luloUSDC、Kyber 和 Uniswap V4。

结果系统只认出了两步。

一次铸造，一次销毁。

剩下 20 条资产流全是 unclassified。

我一开始怀疑 QuickNode 的 RPC 数据不完整，后来顺着 Trace 和 Transfer 查了一遍，发现数据压根没丢。

系统能看到执行合约从 Morpho 收到约 2772 万 USDC，能看到这些钱进入 luloUSDC Vault，也能看到中间有一段 USDC 换成 sNUSD。后面 Vault 份额被销毁，USDC 被取回来，最初的资金也还给了 Morpho。

所有东西都摆在它面前，它就是不知道这些动作该叫什么。

这里我就有点奇怪了。

我想做的是一个链上研究系统，按道理应该来者不拒。结果每碰到一个新协议，就要把协议地址、函数选择器和固定路径重新写进代码。这样做下去，系统确实会越来越懂，但它懂的全是已经见过的题。

像一个学生把五道例题背得滚瓜烂熟，题目稍微换个数字，他就开始交白卷。

我直接问 GPT，为什么一直在硬编码。

它承认第一版其实拟合了那笔 Golden Case。因为第一笔交易走的是 dYdX、Balancer 和 SushiSwap，代码刚好记住了这些地址和调用方式，所以结果看起来特别完整。

第二笔换了一套协议，那本小词典立马不够用了。

但这里也有个现实问题。

通用分析不代表系统可以凭空理解所有协议。DeFi

协议一直在变，Router、Callback、代理合约又会把一次简单的兑换拆成很多层。完全不要 Adapter，最后也只能靠猜。

现在我的理解是，系统应该先学会看懂钱是怎么动的，再去识别协议叫什么。

就像海关收到一个没有品牌标签的包裹。它可能不知道是哪家公司生产的，但它仍然可以称重、看来源、记录里面装了什么、最后送去了哪里。

链上分析也一样。

一笔交易里，同一种资产先进入执行合约，随后又回到原地址，可以先标成临时资金往返。

底层资产进入 Vault，同时铸造出份额，后来份额被销毁，底层资产重新取回，可以先标成存入和赎回。

Token A 流出去，Token B 绕过 Router 和 Callback 回来，也应该先聚成一次资产转换。

协议 Adapter 负责把名字补完整。它可以告诉系统，这次临时资金来自 Morpho，这个 Vault 遵循 ERC-4626，这组 Callback 属于 Uniswap V4。

就算暂时不知道这些名字，资金从哪里来、怎么变化、最后去了哪里，也不能全部丢进 unclassified。

第二笔交易里还有个数字挺有意思。

交易结束前，发起者收到了 166.666013 USDC。

这个数字看起来非常精确，很容易顺手写成 Searcher 净利润。但 Gas 是用 ETH 支付的，发起者和执行合约的控制关系也没有证明，中间是否涉及原有仓位还不清楚。

数字可以精确到小数点后六位，名字照样可能叫错。

所以接下来我不准备马上给 Morpho、Kyber 和 Uniswap V4 各写一块补丁。先把精确地址从行为判断里拿掉，补上交易级资金流、通用经济动作和 ERC-4626 这类标准识别，最后再让协议 Adapter 做更细的解释。

这可能比继续手工多读十笔交易更重要。

先让系统学会看钱，再教它认识协议。

慢慢来。

笔记 039: 套利工具

作者：wayhome | 时间：2026-08-15 09:12 UTC | 字数：1122 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利工具

Taoli Tools

本地运行的半自动

本位对冲套利工具，覆盖中心化交易所、链上现货与永续市场的价格、差价、持仓、资金费、风险预警和双边下单

VIPO 免费，资金上限 5,000 美元，主要支持中心化交易所和 Perp DEX

RPC

RPC 是程序与区块链节点通信的接口。价格 API 通常给你整理后的行情或报价；RPC 返回链上原始状态，并负责 `eth_call`、发送已签名交易以及查询 receipt 和 logs。读取链上价格可能会用到 RPC，但 RPC 本身不等于价格 API。

入门可用 QuickNode、Alchemy 或 Infura

自建以太坊节点需要同时运行执行层和共识层客户端。Ethereum.org 当前给出的最低配置是 2 核 CPU、8 GB 内存、2 TB SSD 和 10 Mbit/s 以上带宽；推荐 4 核以上、16 GB 以上内存、快速 2 TB+ SSD 和 25 Mbit/s 以上不限流量网络

如果不希望交易先进入公开 mempool，可以使用私有交易 RPC、relay 或 bundle。Ethereum 上常见的选择包括

Flashbots Protect (<https://docs.flashbots.net/flashbots-protect/overview>)、MEV Blocker (<https://docs.mevblocker.io/>) 和 bloXroute Protect RPC (<https://docs.bloxroute.com/protect-rpcs/front-running-protection-and-transaction-bundle>)。它们主要降低抢跑和夹子风险，有的还支持回扣或

bundle，但不能保证成交、最佳价格或跨链原子性。新手建议可以从免费的开始，在 <https://chainlist.org/>

上面就可以找到很多公共的。但是基本上就是简单测试脚本和学习使用，真上了套利脚本通常是不够的

钱包与 Signer

很多套利需要快速自动化、程序化操作，由于实时性的考虑，可能手动操作比较麻烦。但是安全控制就变得非常重要了，涉及到私钥签名，各个环节都容易泄漏，而且一旦出现，就是资产归零

别被工具和基建劝退，其实这个并不重要

套利的关键还是发现机会

先用公开数据和免费额度找到一个可能长期存在的价差，再用小额资金跑通验证和复盘。工具解决的是速度、稳定性和成功率，只有当真实机会已经出现时，这些优化才有价值

04 做市与LP（4 篇）

笔记 040: Day 9（8/13） | AMM 数学 II：LVR —— 利润从谁口袋里来

作者：yixin | 时间：2026-08-15 07:30 UTC | 字数：907 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9（8/13） | AMM 数学 II：LVR —— 利润从谁口袋里来

一句话目标：理解套利者的收益从哪来（答案：LP 的口袋），从而看清收益上界和 LP 策略为什么系统性亏损。

核心概念

1. 无常损失 (IL) vs 损失对再平衡 (LVR)

- IL 是路径无关的、和 HODL 比较的、往返后归零的会计概念 —— 它是一个误导性的指标

- LVR (Loss-Versus-Rebalancing, Milionis-Moallemi-Roughgarden-Zhang 2022) 是路径依赖的、单调累积的、真实转移给套利者的钱

- 核心区别：价格涨回来 IL 消失，但 LVR 不会消失 —— 套利者在路上已经把钱拿走了

2. CPMM 的 LVR 速率： $LVR_rate \approx \sigma^2/8$ （每单位时间，占池价值的比例， σ 为该 pair 的波动率）

- 直觉： $\sigma = 80\%$ 年化 $\rightarrow LVR \approx 0.64/8 = 8\%$ 年化。这意味着一个 ETH/USDC 的 V2 LP 每年系统性地送给套利者约 8% 的池价值。

- 这个数字是链上套利总蛋糕的一个下界估计，也是你能分到多少的天花板

3. 推论链（这一串推论比公式重要）：

- LP 的手续费收入必须 $> LVR$ 才不亏 $\rightarrow$ 决定了什么样的池能存活

- 套利者之间的竞争会把 LVR 中的大部分转移给 builder/validator（通过 priority fee），而不是留给 searcher —— 这就是为什么「发现机会」不等于「赚到钱」

- 降低 LVR 的机制设计（更快的区块、拍卖排序权、动态费率）会直接压缩你的利润空间 —— 所以关注这些机制的演进就是关注你的生意还能做多久

4. 有毒流 vs 无毒流：LP 亏给套利者（有毒），赚自散户（无毒）。任何 LP/做市策略的核心问题都是「怎么多接无毒流、少接有毒流」。反过来，套利者的核心问题是「怎么在别的套利者之前接到有毒流的对面」。

笔记 041: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责)

作者：geyu210 | 时间：2026-08-15 13:10 UTC | 字数：1685 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

(本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责格式排版与提交流程。)

昨天已经按照 Aqua SwapVM v1.0.2 的公式，完整复算了 USDC → WETH 的报价。今天继续研究同一条历史样本的反方向：WETH → USDC，主要想确认两个问题：

1. tokenInGt 分支的 virtual reserve 舍入方式是否理解正确；
2. 同一条 XYZ 曲线在正反方向上的价格影响是否对称。

为了避免使用某一个交易方向已经包含舍入和费用的报价来选择另一个方向的输入，我先构造了一个不带交易方向的 common virtual mid，然后按照这个内部价格，把四档 USDC 名义金额换算成等值 WETH：0.01 USDC、0.10 USDC、0.50 USDC、1.00 USDC。这只是模型内部价格下的等名义金额，不是实时美元价格，也没有引用外部市场行情。

【反向分支的区别】

USDC 的地址低于 WETH。正向 USDC → WETH 使用 tokenInLt 分支，反向 WETH → USDC 则使用 tokenInGt 分支。

两个方向不能简单交换 virtual reserves，因为 Aqua 对输入侧使用向上取整，对输出侧使用向下取整：输入侧 virtual offset 为 ceil，输出侧 virtual offset 为 floor。因此，正反方向的 virtual reserves 会相差一个最小 raw unit。这属于 maker-favour rounding，并不是隐藏的方向定价。

【实验结果】

继续按照下面的顺序执行本地整数模型：protocol fee 向下取整、flat fee 向上取整、XYZ amountOut 向下取整。

四档 WETH → USDC 的本地结果与历史链上 quote 全部逐 raw unit 相等 (residual 0 raw)。

正反方向的纯 XYZ 数量影响为：等值档位 0.01 USDC 正向 1.509884 bps / 反向 1.509846 bps；0.10 USDC 正向 15.078049 bps / 反向 15.077975 bps；0.50 USDC 正向 74.937979 bps / 反向 74.937910 bps；1.00 USDC 正向 148.761024 bps / 反向 148.761036 bps。

最大差异不到 0.0001 bps。这说明在今天测试的小额区间内，以 fee 后输入占 virtual reserve 的比例进行归一化后，正反方向的局部 XYZ 曲率基本对称。

【一个有意思的细节】

把费用、曲线影响和最终 token 取整全部算进去后，最小档的正反方向仍有一点表面差异：

0.01 USDC 档：正向总影响约 7.509 bps，反向总影响约 8.000 bps

1 USDC 档：正向总影响约 154.918 bps，反向总影响约 154.920 bps

最小档的差异并不是反向曲线更差，而是因为 USDC 只有 6 位精度。在 0.01 USDC 的档位中，1 raw USDC 就相当于 1 bps，最终 amountOut 的向下取整已经足以影响结果。金额增大后，这种离散误差占比迅速下降。

【今天的理解】

"正反方向曲率近似对称"不等于"双向容量相同"。容量至少应该分成三层：

1. virtual curve 在指定价格影响下能接受多少输入
2. 真实 output allocation 能否覆盖报价输出
3. 加上 access、fee pull、状态竞争、gas 和 MEV 后能否真实执行

今天只验证了第一层的局部数学形状。

正向和反向报价也是同一历史状态下的两个独立反事实，并不是先买入再卖出的 trip。今天没有查询反向外池价格，因此不能据此计算循环损益，更不能得出套利结论。

round

笔记 042: 今天開了一個新 topic : Inverse Finance。

作者：darven | 时间：2026-08-15 13:38 UTC | 字数：2165 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天開了一個新 topic : Inverse Finance。

從一個 gitbook 連結開始，讀完官方文件後逐點鏈上驗證——DBR 拍賣機制、jrDOLA slashing、FiRM 總債務規模、PSM 儲備、Feds 合約狀態。

我學到了什麼

1. DBR 拍賣不是荷蘭拍，是 $x \cdot y = K$ 做市商。docs 寫的是「持續拍賣 DBR」，但原始碼 Auction.sol 裡的 calcPrice() 和 _injectDBRs() 顯示：virtual_dbr_reserve 以 dbrRatePerYear 速率遞增 (41M/yr)，virtual_dola_reserve 固定，價格由 $xy = K$ 公式決定而非隨時間衰減。這不是荷蘭拍賣的線性壓價，而是稀釋通縮——供給端注入導致價格自然下降。鏈上讀參確認：dbrPrice=4.07¢，市場報價 4.04¢，兩者基本一致。
2. Docs 地址表可能是錯的。追 jrDOLA FIRMslashingModule 時，按 docs 給出的合約位址去 eth_call 全部回 0 或異常值。換了 inverse-web 前端 repo 的 constants.ts 里的地址後，才能讀到真實參數：minDebt=\$5、maxCollateralValue=\$100、4 個主要抵押市場全 active。docs 里把 V1（已廢棄）和 V2（活躍）的地址混在一起了。
3. 借款人買入 DBR 後立即消耗用於借款，需求量由債務規模剛性決定（借 1 DOLA 需 1 DBR），跟 DBR 市價無關。增发不會讓特定借款人「少花錢」，只是壓低市場存量價格而已。借款人收益取決於買入時機（低價買還是高價買），跟宏觀增发無關。「天花板不在抵押品而在 DOLA 外部池承載力」。
4. PSM 有套利通道但沒有火力。讀取 PSM 合約 0x4dfd66... 的參數：buyFeeBps=0 (USDS→DOLA)、sellFeeBps=20 (DOLA→USDS)。但 getTotalReserves() 回 \$0，Vault 里沒存 sUSDS，supply=0。PSM 的理論地板存在，實際為空——這解釋了為什麼 DOLA 脫錨後沒有硬底承接。
5. Feds 處於休眠狀態，debt=0、claims=0。讀了 BaseFed.sol 的 expansion() 和 contraction()——都是 onlyChair 可調，需要治理提案觸發。鏈上查 BorrowingFed 狀態：totalDebt=0、totalClaims=0。所謂「主動調節各鏈供應」的機制目前沒有在執行，不是設計上做不到，是當前沒有治理者發起操作。
6. 真實 APY 要看分母——存款規模，不是利息總額。FiRM 總債務 \$48.7M，sDOLA TVL 只有 \$8.03M。債務:存款 = 6.1:1。這不是因為利率高，而是因為分錢的人少。對比 Liquity V2 的 sBOLD 拿 5%——因為它的存款池規模與債務相近，利息攤薄了。驗證 apy 的關鍵分鏡是看誰在付錢、誰在拿錢，兩者的比例才是真實 APY 的分母。
7. OLA 沒有 peg-keeper 公式，peg 靠 market-driven mint/burn。soft peg。

我做了什麼

抓 Inverse 官方 gitbook .md 全文落盤 (references/)：sdola / dola / dola-peg-mechanism / dola-feds / dbr / jrdola / psm / frontier-anchor / inv-tokenomics / smart-contracts / products
GitHub 原碼實讀：dbrAuction (Auction/Helper/SaleHandler)、dola-savings (DSA/sDla)、JuniorDola (jDola/FIRMslashingModule/WithdrawalEscrow/LinearDelayModel)、feds (BaseFed/BorrowingFed)、inverse-web (合約位址/前端)
鏈上實測 (eth_call)：FiRM 總債務 \$48.7M / sDOLA TVL \$8.03M / DOLA 供應 100.9M / dbRAuction 參數 (dbrRatePerYear=41M/yr, dbrPrice=4.07¢) / PSM (buyFee=0, sellFee=20bps, reserves=\$0) / BorrowingFed (debt=0, claims=0) / FIRMslashingModule (minDebt=\$5, maxCollateralValue=\$100)
GeckoTerminal 歷史 OHLCV 查詢：DOLA/sUSDe 主池 90 天內 28 天低於 \$0.98，最深 \$0.9634 (6/7)
DOLA 無 redemption、無 peg-keeper、peg 靠市場自願行為

笔记 043: 今日 (8/15) 实际学习：做市策略。

作者：gh25888 | 时间：2026-08-15 14:15 UTC | 字数：574 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日 (8/15) 实际学习：做市策略。

1. 做市核心概念：4个旋钮（价差/单量/刷新/库存）；价差收窄=竞争加剧/肉变薄（判断题过关：0.13bps 盘口做市每单赚的肉是薄的）。
 2. 宽哥做市打法：上下双边挂单提供流动性吃点差；点差大=肉厚（选盘逻辑）；业绩锚 1000u/12h/60u。
 3. 探针数据反推做市账户特征（RB链）：A型=dust顶单贴盘抢位（ETH/BTC，价差0.2-1bps）；B型=标准对称做市（SOL，固定单量，库存中性）；有个别账户带库存偏移（买卖比0.91偏买）。
 4. 做市细分策略家族：贴盘BBO jumping/分层金字塔/库存偏移/动态价差(波动率自适应)/跨市场XEMM/信号驱动/事件驱动等——上下挂单是共同底盘，细分在拧法。
 5. Lighter测试网做市脚本：ca-brain 出完整方案（单文件CLI、三层报价50/30/20、库存中性+偏移、三道风控：价格带±200bps/库存熔断3×size/插针过滤75bps、默认dry-run），ca-dev 按总架构实现中（main.py+lighter/listener.py+三件日志）。
- 证据：vault 链上套利共享/资料/lighter做市脚本-设计方案-v3.md + 探针数据 rb_mm_probe_v2.db 做市账户特征分析。

05 套利框架（37 篇）

笔记 044: Morpho Blue 套利监控系统

作者：Zhihui Zhang | 时间：2026-08-14 15:21 UTC | 字数：1217 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Morpho Blue 套利监控系统

功能状态

| 功能 | 状态 | 说明 |

| :----- | :--- | :----- |

| Morpho Blue市场监控 | 已完成 | 通过API获取USDC市场数据 |

| NAV预言机市场识别 | 已完成 | 识别质押/LP代币市场 |

| DEX价格监控 | 已完成 | Uniswap V2/V3/V4 + Curve + DeFiLlama API |

| 套利机会分析 | 已完成 | DEX价格 vs LLTV × oracle价格 |

| CSV数据导出 | 已完成 | 市场数据自动导出 |

套利逻辑

基于链上分析报告优化的核心逻辑：

套利条件：DEX价格 < LLTV × 预言机价格

案例（sNUSD）：

DEX价格 = 0.686 USDC（Uniswap V4池价）

预言机价格 = 1.0638 USDC（NAV价格）

LLTV = 91.5%

阈值 = $0.915 \times 1.0638 = 0.9734$ USDC

价差 = $(0.9734 - 0.686) / 0.686 = 41.9\%$

当前监控数据

NAV预言机市场（51个）

系统识别以下类型的市场：

质押代币：stETH, wstETH, sNUSD, sDAI, sUSDS, ysUSDS, etc.

Pendle PT代币：PT-reUSD, PT-sUSDS, PT-USD3, etc.

其他衍生代币：USDe, wstUSR, xUSD, etc.

价格数据源

1. Uniswap V2：链上直接查询Factory + Pair合约
2. Uniswap V3：链上直接查询Factory + Pool合约（多费率：500/3000/10000）
3. Uniswap V4：通过PoolManager的extsload读取存储槽获取池状态
4. Curve：通过Curve API查找池 + 链上get_dy计算精确价格
5. DeFiLlama API：免费价格API（回退）
6. CoinGecko API：免费tier（备用）

笔记 045: D07 | DEX 间套利

作者：averywexler | 时间：2026-08-14 15:34 UTC | 字数：831 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D07 | DEX 间套利

两个 DEX 的显示价格不同，只能说明值得继续比较；只有同一输入量、同一时点、可执行的双向报价才能进入净收益计算。

今日术语

DEX 间套利：在两个不同交易场所利用同一资产对的短暂价格差；

可执行报价：已包含路径、费用和指定输入量对应输出的报价；

买入腿 / 卖出腿：一次闭环中的买入步骤与卖出步骤；

库存：为同时在不同场所执行而预先放置的资产；

状态陈旧：报价所依据的区块、池状态或时间已经落后。

材料

Uniswap：How Uniswap Works (<https://developers.uniswap.org/docs/get-started/concepts/how-uniswap-works>)

固定输入量原则。

跟着算

假设同一时点、同一网络，投入 10,000 USDC：

项目	数值
DEX A 买入后的可执行价值	10,000 USDC
DEX B 卖出后的可执行回款	10,200 USDC
毛价差	+200 USDC
两腿协议费	-62 USDC
Gas	-18 USDC
价格冲击	-45 USDC
风险缓冲	-30 USDC
估算净收益	+45 USDC

如果第二腿实际只能回款 10,140 USDC，其他条件不变，净收益就变为负数。显示价格没有变，执行结论却变了。

笔记 046: 2026-08-14 学习打卡

作者：UU20220213 | 时间：2026-08-14 15:45 UTC | 字数：1324 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-14 学习打卡

今日完成

- 阅读并对照《morpho-arbitrage-discovery-monitoring-2026-08-14》和《morpho-snUSD-arbitrage-window-report-2026-08-14》两份研究报告，梳理 Morpho/sNUSD 案例的发现逻辑、执行条件、窗口时间和风险。
- 理解 Morpho 是链上抵押借贷协议：资金提供者存入 USDC，借款人抵押资产并借出 USDC；预言机负责估值，LLTV 决定最高借款比例。
- 算清案例的简化模型：以 0.686 USDC 买入 1 枚 sNUSD，Morpho 按 $1.064 \times 91.5\%$ 计算，可借约 0.973 USDC，简单毛差额约 0.287 USDC。进一步认识到借款不是卖币收入，债务仍然存在；如果正常偿还并赎回抵押品，并不会凭空产生 0.287 USDC 利润。只有不偿还、让抵押品不足覆盖债务时，差额才可能转化为操作者现金和市场坏账。
- 理清正常借贷只需要一个钱包：钱包 A 抵押、债务记在 A 名下、借出的 USDC 也进入 A。USDC 留在钱包里不会自动还款；必须主动偿还本金和利息后，才能取回抵押品。
- 理解链上出现多个地址不等于使用多个个人钱包。区分了钱包地址、Morpho 合约、代币合约、DEX 路由、流动性池、Bundler/执行器和预言机的不同职责。
- 理解链下机器人与链上执行器的区别：机器人负责监控、计算和决策；Bundler/执行器是已经部署的链上合约，负责把兑换、抵押、借款、还款等动作打包执行。
- 学会在区块浏览器初步区分钱包和合约：两者地址格式相同，需查看 Address/Contract 标识、Contract Creator、Contract 标签、Code 验证、Token Tracker、Proxy/Implementation，并与项目官方地址核对。

今日关键认识

价差不等于可执行利润，借出的资金也不等于真正利润。

钱包地址是签名主体；代币合约是余额总账本；Morpho 是借贷记账与规则合约；路由和 Bundler 是执行工具。

分析交易不能只看顶部 From/To，还要结合 Token Transfers、内部调用和事件日志判断资金流向。

报告里的 +55% 与其检测公式存在分母口径差异：以市场价为分母约为 55%，以预言机价为分母约为 35.5%，监控时必须统一定义。

学习证据

完整阅读两份 2026-08-14 Morpho/sNUSD PDF 报告。

独立完成 0.686、0.973、0.287 USDC 的逐步计算。

能口述单钱包抵押借款流程，并说明多个链上地址各自承担的角色。

未完成与下一步

下一步选择一笔真实交易，在 Etherscan 上逐项识别 From、To、Token Transfers、路由合约、Morpho 调用、抵押记录和借款记录，把今天理解的地址角色对应到真实链上证据。今天只做研究和复盘，没有进行真实资金操作。

笔记 047: 在链上找到 3–5 个可验证的套利者/地址，建立案例研究模板。

作者：Vantone | 时间：2026-08-14 15:51 UTC | 字数：1003 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

在链上找到 3-5 个可验证的套利者/地址，建立案例研究模板。

要点浓缩

去哪找：Twitter/X、Dune Analytics、Etherscan、DeBank 搜 on-chain arbitrage / MEV bot / sandwich bot address / flash loan arbitrage；EigenPhi 找高频地址；Dune 套利 Dashboard。

选定 3-5 个目标，记录：地址(ENS)、主链、套利类型、规模频率、公开来源。

计划：捕获一笔具体交易 → Etherscan 逐笔还原路径 → 自己算实际利润（不是看 PnL 截图）。

核心纪律：区分“策略能力 / 基础设施 / 时间窗口运气”。

涉及代码 / 模板

下面是所有案例研究的统一模板（每研究一个地址复制填一份）：

markdown

案例研究模板

[套利者名称 / 地址]

基本信息

- 链：
- 套利类型：（DEX 内 / DEX 间 / 跨链 / 三角 / 稳定币 / 清算-MEV）
- 时间窗口：
- 公开信息来源：（Twitter / 博客 / Dune Dashboard 链接）

一笔具体交易还原

- TX Hash：
- 输入金额：
- 路径：（如 USDC → Uniswap → ETH → SushiSwap → USDC）
- 输出金额：
- Gas：
- 净收益：

策略分析

- 价差来源：（为什么这里会出现价差？）
- 为什么别人没做 / 做得少：（监控速度？执行速度？独有路径？）
- 基础设施：（自建节点？快速执行合约？私有 RPC？）

成本还原（自己算，不要只看 PnL 截图）

| 成本项 | 金额 | 说明 |

| --- | --- | --- |

| 价差毛利 | | |

| 协议费 | | |

| Gas | | |

| 滑点 | | |

| 桥费 | | 跨链才有 |

| MEV/失败 | | |

| 实际净收益 | | |

- 我的计算 vs 对方宣称收益，差异在哪？是否有隐藏成本？

风险

- [] MEV 抢跑风险 [] 价差消失风险 [] Gas 价格波动风险
- [] 桥/合约风险（跨链） [] 流动性风险

笔记 048: Day 10 | 2026.08.14 | 双向原子套利系统: 从一份脱敏工程文档提炼的设计框架

作者：xzone911 | 时间：2026-08-14 16:31 UTC | 字数：898 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 | 2026.08.14 | 双向原子套利系统: 从一份脱敏工程文档提炼的设计框架

背景

读了一份跨三种市场形态的原子套利系统脱敏文档——同一种资产在三个定价场景之间短暂定价偏差，形成正反两条闭环。价值不在价差公式，而在对抗不确定性的执行框架。

核心设计原则(可复用)

1. 原子性优先于 Gas 极致优化：买、兑换、卖、利润检查必须在一笔交易里完成，任一步失败全量回滚，绝不留中间仓位。宁多付固定 Gas，也不把异质订单绑成一个大失败域。

2. 链上断言优先于链下信任: 链下数据只做筛选, 链上余额和所有权做最终裁决。不能只检查外部调用是否返回成功——第三方协议的返回值不一定能表达资产结果, 余额差才是可靠事实。
3. 固定区块一致性: 一次评估波次先定 blockTag, 所有链上读取固定到该区块。否则不同区块的正确数据组合后会拼出一个链上从未存在过的虚构状态。新区块到达后, 旧评估即使算出高利润也不能广播。
4. 边际报价而不是平均报价: 多窗口时卖出更多资产会产生价格冲击, 第 k 个候选用 $Q(k)-Q(k-1)$ 的边际差, 而不是单张报价简单乘数量。平均报价会高估后续机会。
5. 进程存活 $\neq$ 服务健康: (这条和我在 Solana 项目踩的坑完全一致) 健康检查不能只看进程存在——事件循环还在跑但报价已经卡死的进程同样不健康。要验证关键子系统在各自时限内产生过新鲜证据。
6. 可观测性是策略的一部分: 如果日志只能说没有成交, 就无法区分: 没有价差、库存不匹配、报价过期、RPC 故障、还是模拟回滚。距离阈值还差多少应该从开发初期就设计进去。

今日认知
这套框架和我做的 Solana 套利项目高度同构——原子执行、链下发现链上否决、最坏成本边界、固定状态帧、深度健康检查。不同链、不同资产类别, 对抗不确定性所需的工程原则是相通的。

最有共鸣的一条: 一个正确结果在错误的时间被使用, 比公式错误更危险——显式的区块版本、订单簿版本和任务取消资格机制, 比单纯增加并发更重要。

笔记 049: 今日回顾 X 日报中 0xJA 的原油价差套利复盘：换月期流动性错配，不建议此时操作价差套利。BZ

作者：Jun Guo | 时间：2026-08-14 16:34 UTC | 字数：159 | 评分：47
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日回顾 X 日报中 0xJA 的原油价差套利复盘：换月期流动性错配，不建议此时操作价差套利。BZ 名义上仍有约 1.5% 套利空间，但因价格偏高叠加换月，套利者扎堆做空，资金费率持续为负（BN 最高年化 -300%+），把名义价差利润挤压殆尽。启示：盘面价差存在 $\neq$ 套利可吃到，拥挤度与资金费率成本会吃掉名义收益。

笔记 050: 今日完成

作者：Louis lue | 时间：2026-08-14 16:34 UTC | 字数：1028 | 评分：74
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日完成

- 审核并修正 Router Gas、EIP-1559、Base L1 data fee、USD 定点估值和费用重复扣除的学习答案。
- 实现精确 bigint Gas 成本模型与 append-only migration 003 成本 Repository。
- 新增脱敏内存来源成本状态 API；项目真实运行并回读状态。
- 最终独立审查通过并完成 Git 提交。

关键结论

- Quoter gasEstimate、Router eth_estimateGas 和最终执行成本是不同口径。
- Base 成本包含 L2 execution 与 L1 security/data fee；L1 fee 未知时必须 fail-closed。
- getL1Fee(bytes) 需要完整序列化交易 bytes，仅 Router calldata 不足。
- Pool fee 已在 amountOut 中，不得重复扣除。

证据

- 文档：docs/2026-08-15-research-questions.md。
- 实现：packages/strategy/src/gas-cost-model.ts、packages/database/migrations/003_quote_cost_items.sql、packages/database/src/quote-cost-repository.ts。
- Git：62d204d [verified] feat: add versioned gas cost accounting。
- 10 个测试文件、73 项通过；TypeScript 通过；npm audit 0 vulnerabilities；独立审查 passed=true。
- 运行状态：gasUnitsKnown=true，gasWeiKnown=false，gasUsdKnown=false，l1DataFeeKnown=false。
- executionEnabled=false，无 Wallet、Signer、私钥或广播能力。

未完成 / 下一步

- 尚无完整序列化 Router 交易和独立 ETH/USD 价格证据，不伪造完整成本记录。
- 下一步设计安全的只读 Router 模拟上下文或接入独立价格证据；成本完整前不生成 VERIFIED 机会。

笔记 051: Day 11 : 全自動套利系統架構 v1.0

作者：Web3Rason | 時間：2026-08-14 16:51 UTC | 字數：7667 | 評分：87

來源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11：全自動套利系統架構 v1.0

Day 10 的結論是「我的量具可能整台壞掉，而且它會回報一切正常」。

今天沒有任何一台儀器壞掉。壞的是我從讀數推出去的那一步。

四個被我寫成「常數」的東西，今天全部被證明是函數。

它們的共同形狀不是量錯，是量對了，然後外推的作用域比我量過的範圍寬太多。

另外多了一格我從來沒有想過的閘門：只讀本身會被封號。

零新機會。架構骨幹不動。但成本模型有一項是空的、一項高了一個數量級。

一、速度溢價：我把單腿小額的讀數，寫成了通則

Day 6 我自己打 API 量到速度溢價是「恆定比例約 1.0 bps」，還特地寫進成本表當第四維。

今天的證據用的是完整淨 Edge，不是只比到帳金額：

| 路線 | CHEAPEST | FASTEST | 差 |

| :--- | :--- | :--- | :--- |

| Base → Arbitrum | 211.0714 bps | 262.1362 bps | 51.07 |

| Arbitrum → Base | 208.3447 bps | 259.4791 bps | 51.13 |

雙方向的差額一致到小數點後一位。噪聲做不出這種一致性。

1000 USDC 四腿校準點換算下來，每腿約 12.8 bps —— 比我的常數高一個數量級。

我當初那個 1.0 bps 沒有量錯。錯的是我量的條件：單腿、小額、而且只比 toAmount。

然後我把它寫成了一個沒有下標的數字。

修正：速度溢價改成 (order, 腿數, 方向) 的量測項，不是常數。

舊值只在「單腿 + 只比 toAmount」這個窄條件下成立，

任何多腿路徑的成本估算不得沿用。

二、結算頻率：我修過一次，還是修得不夠深

Day 8 我修過一次年化換算 —— 從「一律 $\times 3 \times 365$ 」改成「按各所結算頻率 (8h/4h/1h) 分別調」。

那句話裡藏了一個我沒發現的假設：每個標的的結算頻率，在整段歷史裡是常數。

實測不成立。某個代幣化股票永續的結算槽分布是這樣的：

| UTC 槽 | 00 | 04 | 08 | 12 | 16 | 20 |

| :--- | :--- | :--- | :--- | :--- | :--- |

| 期數 | 71 | 29 | 71 | 28 | 71 | 29 |

期數不整齊 —— 早期只有 0/8/16 三槽 (8H)，後期才多出 4/12/20 (改成 4H)。

用「均值 $\times (8760 \div \text{中位間隔})$ 」年化時，中位間隔被算成 4h，

早期那 127 個 8H 樣本被整整放大一倍：

|| 修正前 | 修正後 |

| :--- | :--- | :--- |

| UTC 08:00 槽年化 | +317.10% | +52.85% |

| 工作日年化 | +111.77% | +77.02% |

形狀完全沒變 (8:00 仍是主要來源、週末只有 9%)，錯的只有倍數 —— 所以它不會看起來可疑。

修正：改用不依賴間隔的算法 —— 「窗口內費率總和 $\div$ 窗口天數 $\times 365$ 」。

對間隔變更、缺失結算、不規則結算全免疫。

診斷訊號：各時間槽的「期數」欄不整齊，就要先問為什麼，不要直接年化。

推廣：任何「 $\times$ 一年多少次」的換算，都要先確認那個次數在整段歷史裡沒變過。

三、會收斂 $\neq$ 淨值為正

Day 10 我剛把「口徑／承認值失配」立成第九類機會來源，理由是它有機械錨點、可以算出應有值、而且會收斂。

今天有一筆 6 萬 U 的實盤，把那個「而且」拆開了。

| 項目 | 結果 |

| :--- | :--- |

價差	1.1 進場 → 0.2 出場 (照預期收斂)
實際吃到	0.25 個點
同期繳出資金費	0.55 個點
淨	負的

資費成本是價差收益的 2.2 倍。

錨點做的事情只有一件：保證價差會回來。它不保證你在等它回來的期間不用付租金。

修正：第九類的 (b) 格追加入場判準 —— 持定期資金費必須進入場判準，不可以只看價差收斂幅度。

「有錨會收斂」是必要條件，不是充分條件。

四、成本表有一格是空的：閃電貸費

而且它不是一個可以慢慢調的參數，是選供應商的離散差：

供應商	費率	實測
Aave V3	5 bps	借 4,191.115858859627380736 LINK → 還 4,193.211416789057194427
SparkLend + Balancer	0	feeAmount = 0

毛利只有 1 LINK 量級的時候，「用不用閃電貸、用哪一家」本身就是優化變數，而我的成本表裡根本沒有這一欄。

同批另有一筆「Pancake V2 flash 25 bps」是模板推算不是實測 —— 逐家查官方文檔前不引用。

順帶一條同方向的：閃電貸把資本門檻降到 0，池深就成為唯一的規模閘門。

實例是閃貸額度 2,772 萬 USDC，但目標池只有約 \$363 流動性，實際只買了 709.93 USDC。

「借得到多少」在這類機會上完全不是限制。

五、我從來沒有想過的閘門：只讀本身會被封

我整條架構建立在「持續只讀採集」上。

L0 的證據強度階梯裡，我還把它排在 Paper Trading 之前，當成便宜又安全的一級。

它不是免費的。

聚合器方會把「長期只查詢報價、從不在平台成交」的帳號自動歸類為垃圾流量並封鎖。

維度	內容
我原本以為的唯一閘門	速率限制 (匿名 75 次/2h、Key 100 req/min)
今天新增的第二個	與速率無關的「行為特徵」判定 —— 沒超速一樣會被封

這兩個是獨立維度。我先前只把配額當閘門，等於漏掉一整類失效模式，而且失效方式很難救：帳號被封之後，先前所有只讀基線的續採能力一起沒了。

配套 (三條，成本近乎為零，先做)

1. 新增「查詢／成交比」欄位，當一等公民存。
2. 只讀的 key 與將來要下單的 key 必須分離 —— 不要讓研究期的行為特徵污染執行期。
3. 不打算成交的路線降頻或改鏈上直讀，不要拿聚合器配額去養。

證據強度要標清楚：這條是 2 組獨立來源，但兩組都是轉述，不是我方實測。

處置採「便宜的防禦先做」；但「多久會被封、判定門檻是什麼」這些數字一律標 未知，不填值。

六、同鏈：昨天那條要收窄，而且要標待覆核

Day 10 我寫「三角環降級，優先跨 venue (V2↔V3) 與跨費率檔」。

今天兩個方向的證據同時打過來，一個往內收、一個往外撐。

往內收 (作用域太寬) —— 在 BSC 大市值前 20、輸入 1–100 U 的範圍：

類型	正利潤條數
三角環	0
V2↔V3	0 (6 條全負)
V3↔V3	0 (1 條為負)

跨 venue 並不比三角環好。昨天那條是「某一條 L2 上的觀察」，不是通則。

往外撐（可能低估）——有人補齊模擬器的 bootstrap 配置、跳到鏈頭之後，同一個窗口找到 61 個正收益候選、最佳約 +142 bps，並公開撤回自己先前「候選供給不足」的結論。

但那是模擬層正收益、沒有成交證據，且作者自陳只覆蓋約 13.8% 的池活動區塊 → 證據強度只到「固定 block 反事實」。

處置：我方「69 三角環 + 550+ 跨池對全 0」維持，但標「待覆核：需高覆蓋率重測」。不因為一個高覆蓋率不足的正結果就翻案，也不假裝沒看到。

七、幾條不改結論、但會改做法的

- L2 的最大成本牆不是 gas 也不是橋費，是報價自帶的買賣往返價差約 100 bps。同鏈 gas 只有 \$0.03–0.06、橋費 \$1.17 —— 33 個候選全拒、15 組壓力測試 0 交易。
 - 聚合器費率不是一條隨金額下降的曲線，是兩條線，分界是「是否從 L1 出向」：100 U 檔 L2↔L2 是 0.27–0.29%，L1 出向是 0.43–0.44%（主網 gas 攤上去的）；1k 以上才收斂。而且小額檔的報價當日重取才有效 —— 同路徑一天內 fee% 從 0.2654% 變 0.4379%（差 65%），全來自 gas 波動。
 - 報價 ID 每次都不同，但七個經濟欄位完全相同 → 去重要用經濟指紋，不能用報價 ID，否則市場靜止時會憑空產生假告警。同批延遲 653–4,882 ms（最長是最短 7.48 倍）→ 「報價沒變」不等於「系統狀態沒變」，網路層失敗要獨立記碼。
 - 延遲與狀態新鮮度是兩個獨立維度，最快的可能是最舊的：某端點 p50 68.4 ms 全場最低，但落後 5 個區塊；另一家慢近三倍卻 0 落後。stale 比慢更毒。
 - 「數字對得上」不能當機制歸因的通過條件：同一筆交易兩人獨立拆解，淨利都收斂到 \$166.67，機制歸因卻完全互斥 —— 至少一方的敘述是錯的。
 - 抵押借出的資金不是利潤，是變現閘門：毛差只有在「不還款、讓抵押不足覆蓋債務」時才變成現金（同時變成協議壞帳），正常還款則歸零 → 掃描判準要多一格「打不打算還款」。
 - Priority Fee 與 Jito Tip 不可互換：前者只在正常隊列搶排序，跨 DEX 多腿只能靠 Bundle → 用優先費統計 MEV 競價，結構上量不到走 bundle 的那一塊。
- 這是 Day 10 那個「賄賂分母漏洞」的第二個獨立來源。
- 平方滑點模型被實測否認：五檔（1k/5k/10k/30k/50k）相對 1k 的單位報價變化是 0 / 0.62 / 1.18 / 1.21 / 1.31 bps —— 規模放大 50 倍只惡化 1.31 bps。
- 任何用平方滑點推出來的「規模上界」作廢。

八、參數驗證進度（接續 Day 10）

#	參數	值	狀態
13	速度溢價	~恆定比例約 1.0 bps~ → 完整淨 Edge 量到差 51 bps / 雙方向一致、每腿約 12.8 bps	常數作廢，降為量測項
66	盤前窗口 (b) 有錨格	追加：持定期資金費是獨立扣項（實盤吃 0.25 點、繳 0.55 點，2.2 倍）	加判準
94	同鏈三角環可盈利率	69 環 + 550+ 跨池對 = 0 —— 作用域收窄 + 標待覆核（另有 61 候選 / +142 bps 的低覆蓋率反例）	待覆核
113	閃電貸費	Aave V3 5 bps vs SparkLend+Balancer 0 —— 離散差非連續調參	新增
114	年化換算的頻率假設	結算頻率會中途改（8H→4H），127 個舊樣本被放大一倍：+317.10% → +52.85%	新增
115	只讀採集的封號閘門	與速率限制獨立；「多久被封 / 門檻」未知不填值	新增（2 組轉述）
116	L2 的最大成本牆	報價自帶買賣往返價差約 100 bps（gas \$0.03–0.06、橋費 \$1.17 都不是主項）	新增
117	聚合器費率的分界維度	「是否從 L1 出向」：100 U 檔 L2↔L2 0.27–0.29% vs L1 出向 0.43–0.44%	新增
118	小額檔報價的有效期	一天內 fee% 0.2654% → 0.4379%（差 65%），全來自 gas → 跨日引用無效	新增
119	報價 ID 的去重價值	同參數連 5 次 5 個不同 ID、七個經濟欄位全同 → 必須改用經濟指紋	新增
120	同批報價的延遲離散度	653 / 1,961 / 2,145 / 2,321 / 4,882 ms（最長是最短 7.48 倍）	新增
121	延遲 vs 狀態新鮮度	p50 68.4 ms 但落後 5 個區塊；慢三倍者 0 落後 —— 兩個獨立維度	新增
122	平方滑點模型	被五檔實測否認：規模 ×50 只惡化 1.31 bps → 舊規模上界作廢	證偽
123	閃電貸額度 vs 池深	閃貸 2,772 萬 USDC，目標池只有 \$363 → 實際只買 709.93	新增
124	Solana 費用市場的粒度	帳戶級：單可寫帳戶約 12M CU vs 全網約 60M；優先費按請求的 CU 上限縮放（6,000 CU 配 1.4M = 放大 233 倍）	新增
125	Priority Fee vs Jito Tip	不可互換 → 優先費統計結構上量不到 bundle 那塊（賄賂分母漏洞第二來源）	新增
126	並發採集 vs 串行採集	串行一輪約 14 秒 vs 並發組內跨度 9–31 ms（差三個數量級） → 串行量到的是時間價差不是空間價差	新增
127	L2 Sequencer 延遲拆解	287 ms（模擬 48 + 廣播 55 + 確認 142）→ L2 無排序拍賣，不該搶毫秒	新增
128	同鏈跨 DEX 的毛價差下限實例	某 L2 主流對上線首日只命中一次、只持續 1 個區塊、500 U 檔毛利 0.14 bps	新增
129	失敗交易吃掉的 gas 佔比	47.9%（第二個獨立樣本，舊值 60.68%）	新增

130	公共 RPC 的高峰延遲	單次 eth_call 300–800 ms ; 一輪全掃 600 次 → 掃描覆蓋面與付費節點二選一	新增
131	折溢價偏離率的分母敏感度	同一事件，市價為分母 +55% vs 預言機價為分母 +35.5% (差 19.5 個百分點)	新增
132	兩腿預篩的驗收三指標	校準誤差 0.2679–0.3457 bps、margin breach 0、疑似漏檢 0 → 省 33.33% 報價腿；但非校準候選命中率 0%	新增 (每桶僅 1–3 樣本)
133	跨鏈往返成本的鏈別差異	只在大額檔顯現且是分岔不是收斂：5k/10k 檔一條收斂到 1.00%、另一條卡在 1.30%	新增

九、一句話

Day 8：量具本身的誤差，比我要找的東西還大。

Day 9：我把「我當時的實驗設定」寫成了「世界的性質」。

Day 10：我的量具可能整台是壞的，而它會回報一切正常。

Day 11：這次儀器沒壞，數字也對 —— 錯的是我把讀數搬出實驗室的那一步。

今天被打掉的四個常數有一個共同點：它們都是我自己量的，而且都量對了。

速度溢價 1.0 bps，在單腿小額上是對的。

結算頻率分別調，在頻率沒變過的標的上是對的。

有錨會收斂，在不用付租金的場合是對的。

跨 venue 優於三角環，在那條特定的 L2 上對的。

每一個都在它被量到的那個盒子裡成立，而我把盒子拆掉了才寫進架構。

Day 9 那條教訓我寫過一次 —— 「不要把實驗設定寫成世界的性質」。

今天才知道那條要再細一格：問題不在我知不知道有設定，而在我有沒有把設定寫成下標。

一個沒有下標的數字，就是一個宣稱它在所有情況下都成立的數字。

所以 v1.0 起，成本模型裡的每個數字都要帶上「在什麼條件下量到的」，

沒有作用域的數字不准進表。

而第五節那條更難堪一點：它不是外推錯了，是我根本沒有想到那一格存在。

我把「只讀」當成免費且安全，因為讀取不動用資金。

但帳號是一種資源，行為特徵是一種紀錄 —— 我一直在花，只是帳單不在我看的那張表上。

十一天下來，我的機會地圖持續收縮，而代價表持續變長。

今天新增的兩欄（閃電貸費、只讀存活）都不在任何一份教材的成本清單裡。

真正會殺死一套系統的成本，通常不在成本模型的那一頁。

笔记 052: 可以。Day 8 是第二周真正开始“看真实套利交易”的一天。前 7 天你一直在建立模型和工具，现在开

作者：HerschelLi-31 | 时间：2026-08-14 17:39 UTC | 字数：13881 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

可以。Day 8 是第二周真正开始“看真实套利交易”的一天。前

天你一直在建立模型和工具，现在开始把这些东西用在真实链上案例上。

Day 8 | Historical Case Reconstruction：还原一笔真实套利交易

1. 今天的核心目标

第一周解决的是：

```
\[\n\\boxed{\nHow\\Arbitrage\\Should\\Work\\n}\n]
```

Day 8 开始解决：

```
\[\n\\boxed{\nHow\\Arbitrage\\Actually\\Happened\\n}\n]
```

今天不再只看理论上的：


```

\
BuyCheap + SellExpensive\
]

```

而是选择一笔真实链上套利 Transaction，尽可能还原：

```

\
MarketState\
\rightarrow\
Trigger\
\rightarrow\
ArbitrageExecution\
\rightarrow\
Gas\
\rightarrow\
Profit\
\rightarrow\
PriceConvergence\
]

```

核心任务不是：

找一张“赚了多少钱”的截图。

而是回答：

这笔钱到底是怎么赚出来的？

2. 为什么要做 Historical Case Reconstruction？

如果只看别人说：

“这个 Bot 一笔套利赚了 \$5,000。”

这条信息几乎没有研究价值。

因为你不知道：

交易规模多大；

用了多少 Capital；

是否使用 Flash Loan；

经过哪些 Pool；

Gross Profit 多少；

Gas 花了多少；

有没有 Builder / MEV Payment；

为什么当时有这个价差；

这个机会持续多久；

这笔交易是否能被复制。

所以真正要研究的是：

```

\
\boxed{
Profit
=====
Outcome\
}\
]

```

而我们想找到的是：

```

\
\boxed{
Mechanism\
}\
]

```

3. 今天要还原的基本结构

一笔套利可以先抽象成：

```

\
State\_before\
]
↓
\
TriggerTransaction\
]
↓
\
PriceDislocation\
]
↓
\
ArbitrageTransaction\
]
↓
\
State\_after\
]

```

所以 Day 8 最重要的是建立三个时间点：

Before

套利发生前：

```

\
S\_t-1\
]

```

During

套利执行：

```

\
TX\_arb\
]

```

After

套利执行后：

```

\
S\_t+1\
]

```

4. 什么叫 Trigger？

套利机会通常不是凭空出现的。

常见 Trigger 包括：

- Large Swap
- Oracle Update
- Stablecoin Depeg
- Liquidation
- Cross-DEX Order Flow
- Cross-chain Liquidity Imbalance
- Funding Rate Change
- Large CEX Trade
- Block 内前一笔 Transaction 改变 Pool State

例如：

```

\
LargeBuy\_DEX\_A\
]

```

导致：

```
\[\n
P\_A\uparrow\
]
```

而：

```
\[\n
P\_B\
]
```

暂时没变。

于是：

```
\[\n
P\_A>P\_B\
]
```

这就创造了：

```
\[\n
ArbitrageOpportunity\
]
```

所以研究套利时应该问：

```
\[\n
\boxed{\n
WhoCreatedTheOpportunity?\n
}\n
]
```

5. Case Reconstruction 的第一步：找到套利 Transaction

今天先选择一个容易理解的：

```
\[\n
SameChain\ DEXArbitrage\
]
```

最好只涉及：

```
\[\n
2\sim3\
]
```

个 Pool。

例如结构：

```
\[\n
USDC\
\rightarrow\
WETH\
\rightarrow\
USDC\
]
```

或者：

```
\[\n
DEX\_A\
\rightarrow\
DEX\_B\
]
```

不建议 Day 8 一开始就研究特别复杂的：

10 个协议；

Cross-chain；

Solver；

Private Orderflow；

多层 Flash Loan ;
极复杂 MEV bundle。

先把简单案例完整还原。

6. 今天要保存的基础信息

找到 Transaction 后先记录：

Transaction Hash
Block Number
Timestamp
From
To
Status
Gas Used
Effective Gas Price
Gas Cost
Input Token
Output Token
Protocols Used
Pools Used
Flash Loan?
Yes / No

建立：

case_01.md

7. Transaction 本身告诉我什么？

Transaction 可以告诉我：

```
\\  
Intent\  
]
```

例如：

```
from  
to  
value  
input  
gas
```

但套利 Transaction 往往是调用某个：

```
\\  
SmartContract\  
]
```

因此：

```
\\  
to\  
]
```

可能不是 Uniswap。

而是套利者自己的：

```
\\  
ArbitrageContract\  
]
```

这很正常。

实际 Route 需要通过：

```
\\  
Logs\  
]
```

```

]
和 :
\\
TokenTransfers\
]

```

继续还原。

8. Receipt 告诉我什么？

通过：

```

\\
eth\_getTransactionReceipt\
]

```

可以获得：

```

status
gasUsed
effectiveGasPrice
logs
blockNumber

```

首先确认：

```

\\
status=success\
]

```

然后计算：

```

\\
\boxed{\
GasCost
=====
GasUsed\
\times\
EffectiveGasPrice\
}\
]

```

如果：

```

\\
GasUsed=400000\
]
\\
GasPrice=20gwei\
]

```

那么：

```

\\
GasCost
=====
400000\times20\times10^{\{-9\}\
]
\\
\=0.008ETH\
]

```

如果 ETH：

```

\\
$4000\
]

```

则：

```
\\  
GasCost\  
\approx\  
$32\  
]
```

这必须从 Profit 中扣掉。

9. Logs 是今天最重要的数据之一

Transaction Receipt 里的：

```
\\  
logs\  
]
```

可以帮助识别：

ERC-20 Transfer
Swap
Flash Loan
Repayment
Protocol Interaction

因此 reconstruction 可以从：

```
\\  
Logs\  
]
```

开始建立：

```
\\  
TokenFlow\  
]
```

10. Token Flow 是什么？

假设 Transaction 中发生：

```
\\  
100000USDC\  
\rightarrow\  
25WETH\  
]
```

然后：

```
\\  
25WETH\  
\rightarrow\  
101000USDC\  
]
```

那么 Token Flow：

Start:

100,000 USDC

↓ Swap on DEX A

25 WETH

↓ Swap on DEX B

101,000 USDC

End:

101,000 USDC

于是：

```
\\  
GrossProfit
```

=====

101000-100000

1000USDC\

]

这比：

“Transaction profit \$1000”

更有研究价值。

因为你已经知道：

\

WhereTheProfitCameFrom\

]

11. 如果使用 Flash Loan 怎么理解？

可能是：

\

Borrow\

]

↓

\

Arbitrage\

]

↓

\

Repay\

]

例如：

\

Borrow:\

1,000,000USDC\

]

↓

完成套利：

\

1,003,000USDC\

]

↓

偿还：

\

1,000,500USDC\

]

于是：

\

GrossBeforeGas

=====

2500USDC\

]

注意：

Flash Loan 并不是：

免费资本。

需要计算：

```
\\  
FlashLoanFee\  
]
```

所以：

```
\\  
NetProfit  
=====
```

```
ArbitrageRevenue  
FlashLoanFee  
Gas  
OtherExecutionCost\  
]
```

12. Capital 来源必须记录

一笔套利的 Capital 可能来自：

Own Capital

套利者自己的资产。

Flash Loan

单个 Transaction 内借入和偿还。

Credit / Protocol Liquidity

其他形式的资金来源。

为什么要记录？

因为两种 Strategy：

```
\\  
Profit=$1000\  
]
```

但一个需要：

```
\\  
$1MOwnCapital\  
]
```

另一个使用：

```
\\  
FlashLoan\  
]
```

它们的：

```
\\  
CapitalEfficiency\  
]
```

完全不同。

13. 找出真正的 Buy Leg 和 Sell Leg

假设 Logs 显示：

Pool A

套利者：

```
\\  
100000USDC\  
\rightarrow\  
25ETH\  
]
```

那么：

```
\\  
AverageBuyPrice
```


=====

$\frac{100000}{25}$

4000\

]

Pool B

套利者：

$\backslash \backslash$

25ETH\

$\rightarrow$

101000USDC\

]

则：

$\backslash \backslash$

AverageSellPrice

=====

$\frac{101000}{25}$

4040\

]

于是：

$\backslash \backslash$

ExecutableSpread

=====

4040-4000

40\

]

对应：

$\backslash \backslash$

100bps\

]

这就是：

$\backslash \backslash$

$\boxed{\backslash$

ActualExecutableSpread\

$\backslash \backslash$

]

而不是网页上的 Spot Spread。

14. Reconstruction 的核心：还原 Before State

知道套利 Transaction 做了什么还不够。

还需要问：

为什么套利之前 Pool A 和 Pool B 会出现价格差？

因此最好获取：

$\backslash \backslash$

Block_{arb-1}\

]

或者：

$\backslash \backslash$

ArbTX\

]

之前的 Pool State。

对于 AMM，可以研究：

```
\[\nReserve\_A\
```

```
\[\nReserve\_B\
```

或者 protocol 的：

```
\[\nCurrentPrice\
```

然后得到：

```
\[\nP\_A^{before}\
```

和：

```
\[\nP\_B^{before}\
```

15. 然后找 Trigger Transaction

假设：

套利 TX 是：

```
\[\nTX\_5\
```

你应该检查同 Block 前面的：

```
\[\nTX\_1,TX\_2,TX\_3,TX\_4\
```

特别关注：

```
\[\nLargeSwap\
```

例如：

```
\[\nTX\_4\
```

有人在 Pool A 买入大量 ETH：

```
\[\nP\_A:\n4000\rightarrow4045\
```

Pool B：

```
\[\n4000\
```

于是出现：

```
\[\n45\
```

美元 Spread。

那么你就找到了：

```
\\  
\boxed{\  
TriggerTransaction\  
}\  
]
```

16. 为什么要看 Transaction Ordering？

假设同一个 Block：

```
TX 1  
TX 2  
TX 3 ← Large Swap  
TX 4 ← Arbitrage  
TX 5
```

这说明：

```
\\  
TX\_3\  
]
```

创造机会。

而：

```
\\  
TX\_4\  
]
```

立即捕获机会。

这已经开始接触：

```
\\  
\boxed{\  
MEV\  
}\  
]
```

因为：

```
\\  
TransactionPosition\  
]
```

本身具有价值。

17. Backrun 的基本认识

如果：

```
\\  
TX\_ {user}\  
]
```

首先改变 Pool State，

套利者：

```
\\  
TX\_ {arb}\  
]
```

紧随其后获利，

这种结构可以理解为：

```
\\  
\boxed{\  
Backrun\  
}\  
]
```

核心是：

```
\[\nStateBeforeUser\neq\StateAfterUser\n]
```

Searchers 利用：

```
\[\nStateAfterUser\n]
```

出现的新机会。

Day 8 不需要深入 Flashbots。

只需要开始培养：

看套利时一定要看它前面发生了什么。

18. 还原 After State

套利执行完成后，再观察：

```
\[\nP\_A^{after}\n]
```

和：

```
\[\nP\_B^{after}\n]
```

如果之前：

```
\[\nP\_A=4045\n]
```

```
\[\nP\_B=4000\n]
```

套利后：

```
\[\nP\_A=4023\n]
```

```
\[\nP\_B=4021\n]
```

可以看到：

```
\[\nSpread:\n45\n\rightarrow\n2\n]
```

这说明套利者：

```
\[\n\boxed{\nConsumedThePriceDislocation\n}\n]
```

19. 套利本身会消灭自己的机会

这是今天非常重要的理解。

套利者：

```
\\  
BuyCheap\  
]
```

导致便宜市场：

```
\\  
Price\uparrow\  
]
```

同时：

```
\\  
SellExpensive\  
]
```

导致贵市场：

```
\\  
Price\downarrow\  
]
```

于是：

```
\\  
Spread\downarrow\  
]
```

所以一个 Arbitrage Opportunity 有：

```
\\  
\boxed{\  
FiniteCapacity\  
}\\  
]
```

大套利者不能无限放大 Trade Size。

20. 计算 Gross Profit

假设：

初始：

```
\\  
100000USDC\  
]
```

最终：

```
\\  
100800USDC\  
]
```

那么：

```
\\  
GrossProfit  
=====
```

```
800USDC\  
]
```

但千万不要停在这里。

21. 计算 Gas Cost

假设：

```
\\  
GasCost=$150\  
]
```

那么：

\

ProfitAfterGas

=====

800-150

650\

]

22. 再计算 Protocol / Flash Loan Fee

例如：

\

FlashLoanFee=\$100\

]

则：

\

Profit

=====

800-150-100\

]

\

\=\$550\

]

如果还有 Builder Payment：

\

\$200\

]

那么：

\

NetProfit

=====

350\

]

所以最终：

\

\boxed{\

NetProfit\

eq\

TokenFlowDifference\

}\

]

23. Builder / Validator Payment

如果研究 MEV-heavy Arbitrage，需要注意套利者可能为了获得：

\

BlockInclusion\

]

和：

\

Ordering\

]

支付额外价值。

因此真正 Profit 还可能需要考虑：

\

PriorityFee\

```

]
\\
BuilderPayment\
]
\\
CoinbaseTransfer\
]

```

Day 8 暂时知道这个概念即可。

24. Actual ROI

假设：

```

\\
NetProfit=$500\
]

```

Own Capital：

```

\\
$100000\
]

```

则单次：

```

\\
ROI
===
\frac{500}{100000}
0.5%\
]

```

即：

```

\\
50bps\
]

```

但如果使用 Flash Loan：

```

\\
OwnCapital\
]

```

可能很低。

这时单纯 ROI 就不一定是最好的指标。

应该同时记录：

```

\\
Notional\
]

```

和：

```

\\
CapitalAtRisk\
]

```

25. Opportunity Lifetime

今天开始尝试研究：

```

\\
\boxed{\
OpportunityLifetime\
}\
]

```

例如：

Trigger :

```
\\  
TX\_3\  
]
```

套利 :

```
\\  
TX\_4\  
]
```

二者 :

```
\\  
SameBlock\  
]
```

则 Opportunity Lifetime 很短。

如果 :

```
\\  
BlockN\  
]
```

制造价差 ,

直到 :

```
\\  
BlockN+10\  
]
```

才被消除 ,

那么 Opportunity Persistence 更强。

这对于你第三周选择方向非常重要。

26. Opportunity Lifetime 决定 Infrastructure Requirement

例如 :

Strategy A

机会存在 :

```
\\  
100ms\  
]
```

你需要 :

```
\\  
LowLatencyInfrastructure\  
]
```

```
\\  
FastRPC\  
]
```

```
\\  
PrivateOrderflow\  
]
```

可能不适合个人新手。

Strategy B

机会存在 :

```
\\  
3min\  
]
```

那么 :


```
\[\nPython\
```

可能都有研究价值。

所以：

```
\[\n\boxed{\nOpportunityLifetime\
```

27. 最重要的问题：为什么别人没有更早拿走？

分析一笔成功套利时，要强制问：

```
\[\n\boxed{\nWhyDidThisSearcherWin?\
```

可能原因：

latency 更低；
private orderflow；
更高 builder payment；
smart contract atomic execution；
更好的 pool coverage；
更高 capital；
更准确 simulation；
更好的 routing；
特殊 inventory；
niche protocol。

这个问题比：

“它赚了多少钱？”

更重要。

28. Strategy Edge 与 Infrastructure Edge

这是 Day 8 很重要的区分。

假设某人持续赚钱。

利润可能来自：

Strategy Edge

发现了别人没有发现的定价规律。

Infrastructure Edge

大家都知道机会，但他：

```
\[\nFaster\
```

```
\[\nCheaper\
```

```
\[\nMoreReliable\
```

Capital Edge

能配置更大的 inventory。

Access Edge

拥有：

```
\\  
PrivateOrderflow\  
]
```

或者其他特殊渠道。

所以：

```
\\  
\boxed{\  
Profit\  
eq\  
ProofOfStrategyAlpha\  
}\  
]
```

29. Luck 也必须区分

如果某策略只在：

```
\\  
USDCDepeg\  
]
```

那一天爆赚一次，

不能直接说明：

这是长期稳定 edge。

必须研究：

```
\\  
Repeatability\  
]
```

因此 Case Reconstruction 后还要问：

```
\\  
WasThisRepeatable?\  
]
```

30. 一个完整 Case Template

每一笔套利建议以后都按照：

1. Context

Date

Block

Chain

Market Condition

2. Trigger

Which transaction/event created the opportunity?

3. Before State

Pool A price

Pool B price

Liquidity

4. Arbitrage Route

Token A

↓

DEX / Pool

↓

Token B

↓

DEX / Pool

↓

Token A

5. Trade Size

\

Notional\

]

6. Revenue

\

GrossProfit\

]

7. Cost

Gas

DEX Fee

Flash Loan Fee

Builder Payment

Other Fees

8. Net Profit

\

NetProfit\

]

9. After State

Spread after arbitrage

10. Edge

Strategy?

Infrastructure?

Capital?

Access?

Luck?

11. Failure Condition

What would make this trade lose money?

31. Day 8 Experiment 1 | 挑一笔简单套利

优先挑：

\

SameChain\

]

\

TwoDEX\

]

结构简单的案例。

暂时避免：

\

CrossChain\

]

因为跨链 reconstruction 会增加：

\

Bridge\

]

```
\[\nFinality\
```

```
\[\nMultipleTransaction\
```

复杂度。

32. Experiment 2 | 画 Token Flow

不要只写文字。

直接画：

Borrow 100,000 USDC

↓
DEX A

↓
25 WETH

↓
DEX B

↓
101,000 USDC

↓
Repay 100,300 USDC

↓
700 USDC before gas

这样最快看懂套利结构。

33. Experiment 3 | 计算 Profit Waterfall

建立：

Item	Amount
Gross Trading Profit	+\$800
DEX Fees	-\$100
Flash Loan	-\$80
Gas	-\$120
Builder	-\$150
Net Profit	\$350

这张表就是 Day 7：

```
\[\nOpportunityEvaluator\
```

第一次应用到真实链上交易。

34. Experiment 4 | 找 Trigger

检查：

```
\[\nArbitrageTX\
```

之前：

```
\[\n5\sim20\
```

笔相关 Transaction。

寻找：

```
\[\nLargeSwap\
```

]
或者其他 State Change。
如果发现：

\\
TX_A\
]

改变价格，
记录：

\\
TriggerTXHash\
]

然后将两笔 Transaction 建立关联：

\\
TriggerTX\
\rightarrow\
ArbitrageTX\
]

35. Experiment 5 | 比较 Before / After Spread

例如：

套利前：

\\
Spread=70bps\
]

套利后：

\\
Spread=5bps\
]

则：

\\
65bps\
]

的 dislocation 被交易消耗。

这样你才能真正看到：

\\
\boxed{\
ArbitrageRestoresPriceConsistency\
}\
]

36. Day 8 和前几天如何连接？

Day 2：

\\
AMMMath\
]

帮你理解 Swap 为什么移动价格。

Day 4：

\\
TheGraph\
]

帮你发现 Historical Candidate。

Day 5：

```
\[\
```

```
RPC\
```

```
]
```

帮你验证：

```
\[\
```

```
Block\
```

```
]
```

```
\[\
```

```
Receipt\
```

```
]
```

```
\[\
```

```
Logs\
```

```
]
```

Day 7：

```
\[\
```

```
OpportunityEvaluator\
```

```
]
```

帮你计算：

```
\[\
```

```
NetProfit\
```

```
]
```

Day 8 就是第一次把所有工具串起来：

```
\[\
```

```
\boxed{\
```

```
TheGraph\
```

```
\rightarrow\
```

```
RPC\
```

```
\rightarrow\
```

```
TokenFlow\
```

```
\rightarrow\
```

```
CostModel\
```

```
\rightarrow\
```

```
CaseReport\
```

```
}\
```

```
]
```

37. Day 8 验收题

Q1：为什么不能只看套利交易的最终 Profit？

因为 Profit 只是结果。

我们真正需要知道：

```
\[\
```

```
Trigger\
```

```
]
```

```
\[\
```

```
Route\
```

```
]
```

```
\[\
```

```
Capital\
```

```
]
```

```
\[\
```

```
Costs\
```

```
]
```

```
\\  
Execution\  
]
```

才能判断它能否复制。

Q2：为什么必须研究 Arbitrage TX 之前的 Transaction？

因为套利机会通常由之前的：

```
\\  
StateChange\  
]
```

产生。

如果不知道：

```
\\  
WhoCreatedTheDislocation\  
]
```

就没有真正理解套利机制。

Q3：为什么 Logs 对 Reconstruction 很重要？

因为 Smart Contract 真实发生的：

```
\\  
Swap\  
]
```

```
\\  
Transfer\  
]
```

```
\\  
FlashLoan\  
]
```

等行为都会体现在 Logs 中。

Q4：为什么 Gross Profit 不等于 Net Profit？

因为还必须扣：

```
\\  
Gas\  
+\  
ProtocolFees\  
+\  
FlashLoanFee\  
+\  
BuilderPayment\  
+\  
OtherCosts\  
]
```

Q5：为什么 Opportunity Lifetime 很重要？

因为它决定：

```
\\  
RequiredLatency\  
]
```

和：

```
\\  
InfrastructureRequirement\  
]
```

如果机会只存在毫秒级，个人的普通 API Scanner 很难真正捕获。

Q6：为什么一个成功套利不一定说明 Strategy 很聪明？

因为利润可能主要来自：

```
\\  
InfrastructureEdge\  
]  
  
\\  
CapitalEdge\  
]  
  
\\  
PrivateOrderflow\  
]
```

而不是策略本身。

38. Day 8 最终产出

今天应该留下：

arbitrage-lab/

```
|—— reports/  
|   |—— cases/  
|   |—— case_01.md  
|  
|—— data/  
|   |—— cases/  
|   |—— case_01.json  
|  
|—— notebooks/  
|—— case_reconstruction.ipynb
```

case_01.md 至少包括：

Transaction Hash

Block

Timestamp

Chain

Trigger Transaction

Before State

Arbitrage Route

Capital Source

Trade Size

Gross Profit

Gas

Protocol Fees

Flash Loan Fee

Builder Payment

Net Profit

After State

Opportunity Lifetime

Searcher Edge

Main Failure Modes

Can I reproduce it?

39. Day 8 最重要的最终问题

每分析一笔成功套利，最后必须回答：

```
\\  
\boxed{\  
IfIHadKnownEverythingVisibleOnChain,\  
}
```



```
CouldIActuallyHaveCapturedThisTrade?\
}\
]
```

如果答案是：

不行，因为需要 50ms latency + private orderflow。

那这笔案例对你最大的研究价值就是：

```
\
\boxed{\
ThisIsNotMyEdge\
}\
]
```

如果答案是：

机会持续数分钟，数据公开，执行逻辑简单。

那才值得进入：

```
\
StrategyCandidate\
]
```

池子。

40. Day 8 总结

Day 8 最大的认知升级是：

过去看到别人：

```
\
Profit=$5000\
]
```

可能会问：

他用了什么策略？

现在应该问：

```
\
\boxed{\
WhatExactStateTransitionProducedThisProfit?\
}\
]
```

完整研究链条变成：

```
\
\boxed{\
MarketStateBefore\
\rightarrow\
TriggerTransaction\
\rightarrow\
PriceDislocation\
\rightarrow\
ArbitrageRoute\
\rightarrow\
Execution\
\rightarrow\
Costs\
\rightarrow\
NetProfit\
\rightarrow\
MarketStateAfter\
}\
```

]

最终真正想从案例中提取的是四件事：

\[

\boxed{\

Mechanism\

}\

]

\[

\boxed{\

Repeatability\

}\

]

\[

\boxed{\

InfrastructureRequirement\

}\

]

\[

\boxed{\

FailureCondition\

}\

]

如果这四个问题回答清楚，那么你研究的就不再是一张“收益截图”，而是一笔可以被验证、被否定、被比较的真实 Arbitrage Case。

笔记 053: 今日进展

作者：bxynhs-del | 时间：2026-08-14 19:11 UTC | 字数：483 | 评分：63

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日进展

1. DEX价差方向证伪

- 24小时监控数据：198条记录，0次正价差
- 结论：主流币对无机会，停止该方向

2. 发现新方向：Morpho Vault套利

- 分析交易 0x82a2... 发现Lulo USDC vault
- 识别出Morpho协议 + ERC4626机制
- feiye2165确认：利用Curator再平衡时的NAV偏差套利

3. 技术准备完成

- 部署3个监控：Lending利率 + Stablecoin + Morpho vault
- 完成Morpho机制研究文档
- 监控运行中，收集数据

关键发现

套利原理：

- Morpho vault的Curator重新分配资金时
- Share price会短暂偏离理论NAV
- 在偏差窗口存/取款获利

可能就是Z.Lee的“渠道”

下一步

明天分析监控数据，验证：

1. NAV偏差出现频率
2. 偏差大小是否足够
3. 是否值得继续

状态

方向明确
等待数据验证

笔记 054: Day 11 (08/15) 学习总结, 素材来自今日同学精华:

作者: cjdacoin | 时间: 2026-08-15 01:00 UTC | 字数: 402 | 评分: 46

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 (08/15) 学习总结, 素材来自今日同学精华:

1. Lizzie : 已有监控资料中, 四条 USDC 跨链路径的往返 break-even 约为 50–53 bps, 而输入的观察价差只有 13.6 bps, 正确结论是 NO TRADE
2. HerschelLi-31 : 如果研究 MEV-heavy Arbitrage, 需要注意套利者可能为了获得:
3. clare ma : 这里的 transaction hash、block number、timestamp、log index、token amounts 和 amountUSD 是 subgraph 返回的原始事实
4. zgos : 现在发现, 即使不投入真实资金, 也可以先按照当时的报价, 模拟完成 DEX 买入和 Binance 卖出的过程, 然后观察最终结果

以上为今日从同学打卡中学到的知识点, 继续积累执行与验证能力。

笔记 055: 【今日共学打卡 | 链上套利的基础设施】

作者: tension-vip | 时间: 2026-08-15 01:25 UTC | 字数: 1063 | 评分: 59

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【今日共学打卡 | 链上套利的基础设施】

今天继续学习链上套利。

上一篇梳理的是「套利第一性原理」: 看到一个价差后, 如何判断它究竟是屏幕上的数字, 还是能够真正落袋的机会。

今天进一步进入工程执行层:

一个链上套利系统, 真正需要哪些基础设施?

大致可以拆成:

机会发现 → API / 聚合器询价 → RPC 获取链上状态 → 计算真实收益 → 钱包签名 → 广播交易 → 确认结果 → 复盘

其中涉及几块基础能力:

1 | API / 聚合器\

负责获取行情、询价、交易路径和跨链路径。

现阶段并不需要一开始就接几十个 API。

更合理的方式是先搭一个「最小可用工具栈」, 例如围绕 LI.FI + Binance Web3 API 覆盖主要研究、询价和执行场景, 真正遇到缺失能力时再增加专用 API。

2 | RPC\

RPC 相当于程序和区块链之间的通信入口。

读取余额、Gas、nonce、allowance、模拟交易、广播交易、查询 receipt, 最终都离不开 RPC。

套利做到后面, 比的也不只是策略, 还包括数据延迟、节点稳定性和执行可靠性。

3 | 钱包 / Signer\

钱包不是简单的“放资产”。

自动化套利中, 它实际上承担:\

检查余额 → 管理 Gas → nonce → allowance → 签名 → 发送交易。

因此私钥安全和权限隔离必须从一开始就进入设计。

4 | 服务器\

如果程序需要持续扫描机会并自动执行, 就不能依赖自己的电脑一直开着。

服务器最终承担的是:\

持续运行、稳定网络、日志、监控和自动化执行。

今天最大的收获不是记住了几个工具, 而是开始看清:

套利系统并不是一个“发现价差的脚本”, 而是一条完整的执行流水线。

而且工具越多不一定越好。

真正应该优化的是：

有效机会数量 × 成交概率 × 单次净利润 \

- Gas \
- 滑点 \
- 失败交易 \
- 基础设施成本 \
- 风险损失

所以我现阶段更倾向于：

先做最小系统 → 跑通一次完整闭环 → 记录真实数据 → 再决定优化哪里。

而不是一开始就堆 RPC、API、服务器和复杂钱包架构。

这也让我进一步确认一个判断：

链上套利真正的壁垒，可能不是“知道哪里有价差”，而是把信息、计算、资金和交易执行压缩成一套稳定、低成本、可重复的系统。

\#链上套利 #Web3 #量化交易 #DeFi #共学打卡

笔记 056:

今天继续整理链上套利这块的内容，重点还是放在“怎么判断一个机会值不值得长期做”。

作者：Oxgen3 | 时间：2026-08-15 01:50 UTC | 字数：588 | 评分：61

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续整理链上套利这块的内容，重点还是放在“怎么判断一个机会值不值得长期做”。

前面已经慢慢把流程梳理成了：发现信号、筛选机会、计算成本、验证执行、最后复盘。今天进一步思考后发现，真正重要的其实不是某一次赚了多少钱，而是这个机会有没有可能重复出现。

如果一个机会只出现一次，即使当时利润很高，也很难直接把它当成策略。反过来，如果某种价差经常出现，而且扣掉 Gas、手续费、滑点等成本后仍然有空间，那它才更值得继续研究。

所以今天开始尝试从几个维度判断机会：

出现的频率高不高

信号持续时间够不够长

实际可以执行多大的资金量

扣除所有成本后还有多少利润

执行过程中失败的概率高不高

不同市场环境下是否依然有效

另外也发现，策略不能一直依赖人工判断。前面学习 Hermes 和 LI.FI 的时候，更多是在了解工具怎么使用，现在可以开始把它们和自己的研究流程结合起来。

比如先让程序持续收集数据，再由规则筛选出符合条件的信号，最后人工进行确认。这样既能减少无效信息，也能慢慢积累自己的历史数据。

今天最大的收获是：

套利真正有价值的不是发现一次机会，而是找到一个可以重复验证、重复执行的机会。

接下来准备继续积累数据，把已经观察到的机会分类记录下来，看看哪些只是偶然行情，哪些是真正存在规律的机会，再逐步完善自己的策略和监控流程。

笔记 057: 本质：三角闭环 WETH → 中间币 → USDG → WETH，高频薄利。

作者：Oxmigueleth | 时间：2026-08-15 03:15 UTC | 字数：1002 | 评分：81

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

本质：三角闭环 WETH → 中间币 → USDG → WETH，高频薄利。

68 万笔提交交易，代理合约 (ERC1967) 78 万笔，利润归集到第三个地址 0x3CF1...338B。两笔样本净利 \$0.64 和 \$0.17；近 20 笔里 11 笔失败，失败 gas 约 0.0001 ETH。选择器 0x00000000 = 自定义紧凑 calldata。

关键数字：这是个纯 gas 经济学游戏。RH Chain 上单笔 gas 约 1–6 美分、~0.1 秒确认，所以可以用高失败率换命中率。期望值 = 成功率 × 平均净利 - 失败率 × 平均失败 gas。

两者的可复现难度差异很大

|
 | 清算 bot | 循环套利 bot |

| :----- | :----- | :----- |

| 核心壁垒 | 仓位索引 + RFQ 报价权限 + builder 关系 | 延迟 + 多池报价精度 |

| 资金需求 | 零 (闪电贷) | 零到极小 (也可闪电贷) |

| 竞争者 | 少而强 (十几家专业团队) | 多而杂 |

| 单笔规模 | 千到百万美元级 | 几毛到几美元 |

| 对新人 | 几乎不可能——Bebop 生产级 RFQ 要商业协议, BuilderNet refund 要历史订单流 | 可行, 尤其在长尾链/新池 |

结论: 如果真要动手, 从 Robinhood Chain 类的循环套利入手, 清算 bot 当作研究对象而非复现对象。

笔记 058: MetaMask × LI.FI : AI Agent 自动 DeFi 收益再平衡

作者: QQQ | 时间: 2026-08-15 06:06 UTC | 字数: 5056 | 评分: 78

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

MetaMask × LI.FI : AI Agent 自动 DeFi 收益再平衡

17.1 核心定位: 这不是瞬时价差套利, 而是自动资金调度

本次直播 Demo 展示的是 Yield Rebalancing Agent : AI Agent 定期查看当前 DeFi 仓位和市场 APY, 在满足收益与风控条件时, 把资金从当前 Vault 提出, 再存入更优 Vault。它与跨链套利系统的底层思路高度相似, 只是 Alpha 信号从“价格差”变成了“收益率差”。

可以把三个核心角色理解为: Claude Agent = 决策大脑; LI.FI Earn = 收益市场数据与路由基础设施; MetaMask Agent Wallet = 链上执行与安全控制层。

17.2 三层架构

① Claude Agent 层——策略与决策: 定时触发任务, 查询持仓, 对比市场 APY, 判断是否需要 Rebalance。可配置检查周期、最低 TVL、APY 差异阈值、目标资产、协议白名单等。

② MetaMask Agent Wallet 层——执行引擎: Agent 不直接持有私钥, 而是向钱包提出交易意图。钱包负责策略限制、安全检查、交易模拟、MFA/2FA 与签名执行。Guard Mode 权限更严格; Beast Mode 自动化程度更高, 但真钱环境不应仅因为方便就默认使用 Beast Mode。

③ LI.FI Earn / Composer 层——数据与执行编排: Earn 用于发现和比较不同协议、链和 Vault 的收益机会; Composer 用于把 approve、swap、bridge、withdraw、deposit/supply 等多步骤操作组合成执行路径。

17.3 Demo 的完整流程

Loop 定时触发 → 查询当前 Earn Positions → 查询 Earn Markets → 按 TVL 等条件过滤 → 按 APY 排序 → 比较当前 Vault 与最佳 Vault → 判断是否达到调仓阈值 → Withdraw 当前仓位 → Supply 新 Vault → 等待链上确认和索引更新 → 再次检查仓位。

直播为了演示可以设置 1 分钟 Loop, 但真实资金不应高频搬仓。每次 Rebalance 都会产生 Gas、DEX Fee、Slippage、Price Impact、Bridge Fee、授权成本和执行风险, 因此实际检查周期与最低持仓时间应该明显更长。

17.4 为什么不能简单使用“APY 差 > 0.5% 就换仓”

简单 Demo 的判断可能是: Best APY Current APY > Threshold, 则 Rebalance。但真正应该比较的是预期净收益, 而不是 APY 数字本身。

$\text{Expected Extra Yield} \approx \text{Capital} \times \Delta\text{APY} \times \text{HoldingDays} / 365$ 。

$\text{Expected Net Profit} = \text{Expected Extra Yield} - \text{Gas} - \text{Swap/Bridge Fee} - \text{Slippage} - \text{Price Impact} - \text{Rebalance Cost} - \text{Risk Cost}$ 。

例如本金 10,000 USDC, 目标 Vault 比当前高 2% APY, 如果预计只持有 30 天, 理论额外收益约为 $10,000 \times 2\% \times 30/365 \approx 16.44$ USDT。如果搬仓总成本接近 15 USDT, 那么这次 Rebalance 几乎没有意义。

因此生产策略应使用 Minimum Expected Net Profit, 而不是只使用 APY Difference。

17.5 TVL Filter 为什么重要

最高 APY 不等于最佳 Vault。一个 Vault 显示 19% APY, 但 TVL 只有 8 万美元, 可能存在容量、流动性、收益不可持续、智能合约或退出风险。因此可以先设置 TVL > 1M / 5M 等硬过滤, 再比较收益。

更进一步, 应综合 Vault Age、Protocol Risk、Audit、Liquidity、Withdrawal Conditions、历史 APY 稳定性, 而不是只按 APY 降序选择。

17.6 为什么 Withdraw 后要等待 10–15 秒

链上交易提交后, 需要经历打包、确认、Indexer/API 更新。交易已经成功, 并不意味着 Earn API 会立即显示最新余额。如果 Agent 在索引更新前再次读取状态, 可能误判 Withdraw 失败或余额为零, 并触发重复操作。

因此状态机必须区分：TX_SUBMITTED → TX_CONFIRMED → INDEXER_UPDATED → POSITION_VERIFIED。不能简单用固定 sleep 作为长期生产方案；更可靠的是监听交易 Receipt、余额变化和索引状态。

17.7 LI.FI Composer 的价值

真实 Rebalance 可能不是简单的 USDC Vault A → USDC Vault B，而可能包含：Withdraw → Swap → Bridge → Approve → Deposit。Composer 的价值是把这些 DeFi 操作编排成统一执行流程，让 Agent 表达“把资金从 A 策略迁移到 B 策略”的意图，而不用由 LLM 自己拼 ABI、Router、Spender 和 Calldata。

17.8 Guard Mode 与 Beast Mode

Guard Mode 更适合真钱早期测试：限制 Network、Address、Token、资金流出额度等，超出规则需要人工/MFA 确认。

Beast Mode 放宽部分限制，使 Agent 可以更自动地执行交易，适合受控 Demo 或经过充分验证的自动化环境，但不能把“务必开 Beast”当作普遍实盘原则。

Agent Wallet 的关键安全思想是：Agent 可以提出交易，但私钥、签名和硬风控不应完全交给 LLM。

17.9 推荐升级为四层架构：加入独立 Risk Engine

更稳健的生产结构应为：

Claude / Codex Agent（分析、策略编排）

→ Risk Engine（确定性规则）

→ LI.FI Earn / Quote / Composer（市场数据与路由）

→ MetaMask Agent Wallet（模拟、安全、签名、执行）

→ Blockchain。

Risk Engine 应使用代码规则，而不是让 LLM 自由判断，例如：TVL ≥ 最低阈值； Δ APY ≥ 最低阈值；Protocol 在白名单；Expected Net Profit ≥ 最低利润；Slippage ≤ 上限；Gas ≤ 上限；单协议仓位 ≤ Max Allocation；最低持仓时间已满足。

LLM 负责发现、分析和编排；代码负责数学、约束和风控；钱包负责签名和安全执行。

17.10 防止 APY 来回震荡：Hysteresis + Minimum Holding Period

APY 是动态的。如果今天 Morpho 5%、Yearn 9%，Agent 搬到 Yearn；十分钟后 Morpho 7%、Yearn 5%，简单机器人又会搬回来，形成频繁震荡并持续支付交易成本。

因此需要加入：Minimum Holding Period、Rebalance Cooldown、Hysteresis、Minimum Net Profit、连续多次采样确认。例如每小时检查，但最低持仓 7 天；只有 Δ APY > 2%、预计净增益 > 30 USDT 且连续多个采样点成立时才允许调仓。

17.11 从 Highest APY 升级为 Risk-adjusted Net APY

真正要优化的目标不是 Highest APY，而是 Risk-adjusted Net APY。至少应考虑：APY 水平、APY 持续性、TVL、协议风险、Vault 风险、Gas、Slippage、Bridge 成本、退出流动性、持仓时间、资金占用和失败概率。

也就是说，Agent 真正应该回答的问题不是“哪里 APY 最高？”，而是：“我的资金现在放在哪里，扣除执行成本和风险后，预期净收益最高？”

17.12 与跨链套利系统的统一

跨链套利扫描的是 Price Spread；Yield Rebalancing 扫描的是 APY Spread。两者可以共用大量基础设施：Market Scanner、Quote、Cost Engine、Risk Engine、Simulation、Wallet、Execution、Status Tracking、Inventory Management 和 PnL Database。

统一架构可以理解为：

Arbitrage Scanner (Price Spread) + Yield Scanner (APY Spread)

→ Opportunity Engine

→ LI.FI Quote / Earn

→ Cost & Risk Engine

→ Simulation

→ MetaMask Agent Wallet

→ Execution。

因此长期可以把系统从 Auto Rebalancing Bot 升级为 DeFi Capital Allocation Agent。

17.13 更进一步：套利资金与闲置资金统一管理

成熟系统可以让资金平时停留在 Aave、Morpho、Yearn 等低风险收益策略中；当套利引擎发现高质量机会时，再比较“继续赚 Yield”与“抽出资金执行套利”的机会成本。只有套利的 Expected Net Profit 明显高于 Yield Opportunity Cost 和执行成本时，才调出资金。

这样系统的目标就从单一套利升级为 Capital Allocation：资金在 Yield、跨链库存、套利保证金和钱包流动资金之间动态配置。

17.14 推荐的第一版实验参数

第一版不建议直接追求全自动真钱执行。可以先做只读/小额实验：目标资产先用 USDC；限制 Base 等单链环境；协议使用白名单；TVL 设置最低门槛；记录 Current APY、Best APY、ΔAPY、TVL、预计持仓时间、预计额外收益、Gas、Swap/Bridge Cost、Expected Net Profit 和最终 Decision。

只有当 Expected Net Profit 连续多个采样点超过阈值，再进入 Simulation；Simulation 通过后才考虑极小额真实 Rebalance，并记录 Quote → Actual Execution 的偏差。

17.15 本节核心结论

MetaMask × L1.FI Demo 证明 AI Agent 自动管理 DeFi 仓位已经具备较完整的技术基础，但“能自动交易”并不是最难的问题。真正困难的是建立可靠的收益、成本和风险模型。

生产级系统不应采用：Highest APY → 自动搬仓。

更合理的是：Position → Market Scan → TVL/Risk Filter → APY Persistence → Expected Net Profit → Simulation → Wallet Policy → Execute → Verify → PnL / Execution Database。

最终目标不是一个会追逐最高 APY 的机器人，而是一个 Execution-aware、Risk-aware 的 DeFi Capital Allocation Agent：只有当风险调整后的预期净收益真正改善时，才允许资金迁移。

笔记 059: 今天研究 DeFi

清算套利中的“预言机更新窗口”：市场价格已经跌破危险区，为什么仓位有时仍不能被清

作者：dyhbrewer | 时间：2026-08-15 06:38 UTC | 字数：1464 | 评分：74
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天研究 DeFi 清算套利中的“预言机更新窗口”：市场价格已经跌破危险区，为什么仓位有时仍不能被清算，以及怎样区分真实机会和屏幕上的假机会。

一、先区分三种价格

- 1. P_market：DEX/CEX 当前真实可成交价格，要用目标金额下的订单簿或池子 VWAP，而不是只看最后成交价。
- 2. P_oracle：借贷协议当前实际读取的预言机价格。来源可能是 Chainlink、TWAP 或协议自有预言机，并受 heartbeat、偏差阈值、更新交易和熔断规则影响。
- 3. P_liquidation：使仓位健康度跌破清算门槛的临界价格。

所以 P_market < P_liquidation 不等于“现在可清算”。只有协议已经读取新价格、链上最新状态下健康因子 HF < 1，并且清算入口未被暂停，机会才真正成立。

二、机会形成的时间顺序

市场先发生急跌；协议仍可能读取旧预言机价；价格源满足偏差或 heartbeat 条件后提交更新；协议在后续读取到新值；仓位 HF 跌破

1；清算机器人同时竞争。套利窗口来自这些环节之间的时间差，但同样会被更快的搜索者、私有交易通道和区块排序迅速压缩。

三、清算前必须收集的证据

监控仓位 HF、债务和抵押品；预言机 answer、updatedAt、heartbeat、偏差阈值及熔断状态；协议的 liquidation bonus、close factor、debt ceiling、最小偿还额和暂停开关；抵押品在目标 DEX 的真实卖出深度；闪电贷费、Gas、DEX 费、价格冲击、MEV 竞争和失败交易成本。最后必须在同一区块状态做 eth_call 或交易模拟，不能只凭前端页面判断。

清算净收益模型：

Net Profit

\= 抵押品按真实深度卖出的收入

偿还债务

闪电贷费用

Gas

DEX/协议费用

滑点与 Price Impact

MEV、失败和状态变化准备金。

关键点是：抵押品不能按预言机价估值，必须按实际可成交深度估值。清算奖励 5% 不代表净赚 5%；遇到长尾资产时，卖出冲击可能直接吃掉全部 bonus。

四、常见假机会

前端价格已更新但协议价格未更新；HF 接近 1 但尚未低于门槛；仓位已被抢先清算；抵押品流动性不足；close factor 限制可清算规模；同区块模拟失败；bonus 低于完整执行成本。预言机“陈旧”也不自动等于可利用，可能同时触发熔断或暂停。

五、只读雷达的状态设计

先做两个核心状态：

pending_window：市场价已进入危险区，但协议尚未达到可清算条件，只观察不提醒执行。

liquidatable_now：最新链上状态确认可清算，且压力测试后的 Net Profit 仍为正。

进一步增加 stale_oracle、paused、no_liquidity、simulation_failed 和 competed 等原因码，记录从 P_market 变化到预言机更新、HF 跌破 1、机会消失的完整时间线。提醒条件应是“连续多次确认 + 模拟成功 + 压力成本后净收益超过阈值”，而不是看到价差就推送。

今天的结论：清算套利的 Edge 不只是价格方向判断，而是更快地知道协议何时真正接受新价格，并在真实流动性、费用、竞争和失败成本后仍有利润。下一步先做只读监控与 Paper Trading，不连接钱包、不签名、不提交交易。

笔记 060: 【Day 15 打卡 · 2026-08-15】模拟总结与试跑决策

作者：nerd614 | 时间：2026-08-15 07:18 UTC | 字数：429 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 15 打卡 · 2026-08-15】模拟总结与试跑决策

延续「跟数据走」的节奏，今天把 Paper Trading 的结果收拢成结论，决定结课前是否推进到小额试跑，以及如何做。

1. 模拟结果汇总：把模拟期间的「触发→执行→盈利」转化率、每笔的预期 vs 实际偏差、失败原因整理成一张总结表，量化候选策略整体的胜率与收益分布。
2. 成本偏差量化：汇总执行延迟带来的成本偏差数据，回填后看净收益仍为正的候选还剩几条、边际有多厚，判断哪些策略经得起真实执行的损耗。
3. 试跑决策：对仍在候选的路径，明确试跑方案——极小金额、单笔上限、仅验证执行与到账、不追求盈利，把试跑目标限定在「验证成本模型与实际一致」而非「赚钱」。
4. 风控边界：为试跑设定硬性风控——单笔金额上限、单日次数上限、滑点/延迟超限即放弃、失败交易的止损规则，确保试跑在可控范围内。

今天聚焦从「模拟通过了」到「能不能谨慎上小额试跑」的决策点，为结课前的最后验证定方向。

笔记 061: 链上套利共学 · Day11 打卡笔记 (08/15)

作者：haibian-slowly | 时间：2026-08-15 08:47 UTC | 字数：2710 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 · Day11 打卡笔记 (08/15)

今日信息

日期：2026-08-15 (周六)

主题：建立候选信号规则

核心问题：什么才算一个「候选机会」？

打卡正文

【第 11 天】

1. 今天要验证的问题

前几天我们一直在解决「怎么把数据拿回来」(LI.FI 报价、Gas、GraphQL 查询)，数据越攒越多。但数据多 ≠ 机会多——今天要回答的问题是：什么才算一个「候选机会」？换句话说，一条「这里有价差/这里有新币」的信号，要过几道闸，才配进入我的观察名单？

2. 今天实际做了什么

没有新写代码，而是把最近两天在「打狗（一级市场 memecoin）」方向整理的三份真实资料，翻译成一套可以落地的候选信号规则：

E:\打狗\打狗方法论总结.md (Oxmagnolia《惠经》精华版，37KB)

E:\打狗\DL0868_alpha发现路径分析.md (链上观察者型玩家牧羊 @DL0868 的 9 条发现路径拆解，31KB)

E:\打狗\DL0868_链上进阶指南_全文总结.md (5388 字长文总结：三道门检查 + 十项纪律)

核心产出：把「打狗筛选流程」提炼成三层候选信号漏斗（跟套利信号的逻辑完全同构）：

| 层级 | 要回答的问题 | 具体规则 (来自真实资料) |

|---|---|---|

| 第一层：热度/叙事信号 (够不够新、够不够热) | 这个标的有没有被看到？ | ① 共同关注数 20 个左右 = 潜力较大；从 10 涨到 100 = 场内 PVP 已满，该离场；② 热点在情绪就在，情绪在流动性就在 (谷歌趋势+推特搜关键词验证)；③ meme 热度周期：第一周上升、第二周高潮、第三周颓势 |

| 第二层：资金/庄信号 (有没有钱在动) | 是散户自嗨还是有人在操盘？ | ① 5min 线突然放量，比前一根多 3 倍，第二根 5min 再翻倍 → 90% 确定有庄或大户进场；② 放量后查官推：没有利好 = 有人操盘；③ 老币看「三长」：洗盘长 (≥2 个月) + 启动长 (≥1 个月) + 拉盘长 (≥2.5 个月)；④ 日成交量 500-700 万刀 ≈ 拉到位 |

| 第三层：安全/离场闸 (能不能活到赚钱) | 会不会是貔貅/老鼠仓/出不了货？ | ① 三道门：合约权限门 (honeypot/改税/增发/黑名单/LP 锁定/Freeze Authority 是否撤销——Solana 上 Freeze Authority 最容易被忽略) → 筹码门 (Top10 是否一个人拆几十个钱包、钱包是否同一 CEX 出金、有没有「公开聪明钱进场+侧钱包卖出」的杀跟单结构) → 池子门 (流动性够不够你离场)；② 检测工具显示绿色 ≠ 安全，只说明没命中现有规则库 |

3. 证据 (本次真实运行结果)

三份资料来源文件齐全 (见上，均为 2026-08-11~08-12 期间整理/抓取的真实资料)；

信源本身可追溯：Oxmagnolia《惠经》Google Docs + X 推文；@DL0868 主页抓取 6 条 + Grok 抓取 20 条原创推文全文 (存档 E:\打狗\DL0868分析\DL0868_original_tweets_grok.json)；

关键规则均有原文出处，不是我的想象：5min 放量 3 倍判据 (方法论 5.2 路线 C)、共同关注数情绪指标 (方法论 5.1)、三道门检查 (链上进阶指南第二部分)。

4. 得到了什么结论 (来自数据)

1. 候选信号 = 三层漏斗，不是单条规则：热度层 (别人看到了) → 资金层 (有钱在动) → 安全层 (能活着离场)。任何一层不过，都不进观察名单。这和 brucexu《第一性原理》里的判据完全同构——价差只是线索 (第一层)，资产闭环可执行 (第二层)，全成本后期望净利润为正 (第三层)；

2. 「没利好却放量」是最强信号之一：资金先动、消息后到 = 有人知道我们不知道的事。这个判据可以直接移植到套利场景：某个池子/某个桥出现异常大的资金流，但没有任何公开新闻 → 值得进候选名单；

3. 安全闸必须独立于热度闸：热点映射盘、名人/IP 盘传播最快，但假账号/假授权/同名合约最多；「检测工具绿色 ≠ 安全」——工具的否定结论可信，肯定结论不可信。对套利的启示：RPC/报价来源的可用性要自己实测 (Day8 已经吃过端点的亏)，工具的「绿色」同样不能当通行证；

4. 规则要能写成代码才算规则：三层漏斗里，热度层 (共同关注数、谷歌趋势) 可以脚本化，资金层 (放量倍数) 可以用链上数据实时算，安全层 (Freeze Authority、LP 锁定) 有现成合约接口可查——这就是 signal_detector.py 的骨架。

5. 今天失败了什么

没来得及把三层漏斗写成 signal_detector.py 第一版代码——今天把规则设计 (人工可执行版) 做完了，代码化是下一步；

打狗资料里的大部分判据 (共同关注数、KOL 关注列表) 依赖 X/社交平台数据，本机直接抓 X 全量数据仍是老大难 (免登录镜像被 Cloudflare/Anubis 锁死，官方 API 按条收费)——这部分信号源只能先人工维护或降级用替代源；

诚实记录：今天没有跑新的链上数据实验，结论全部来自对既有真实资料的规则化提炼——这是「把散知识变成可执行规则」的一天，不是「拿新数据」的一天。

6. 明天只解决什么

把今天的三层漏斗第一版代码化：signal_detector.py，先用「资金层」的链上可算部分 (放量倍数、池子新增、大额买入) 做最小可用版，跑已有数据库回放，统计命中率。

一句话总结

候选机会不是「看到一个价差/一个新币」就算，要过三层漏斗：热度 (有人看到) → 资金 (有钱在动，没利好却放量最可疑) → 安全 (能活着离场，工具绿色 ≠ 安全)——这套规则直接从打狗方法论提炼，和套利第一性原理的判据同构，明天把它写成 signal_detector.py 第一版。

明日预告

Day12 (08/16)：模拟延迟与报价失效——机会在多差的条件下会从盈利变成亏损？

笔记 062: 今日深挖：sNUSD 闪电贷交易的双套利结构 + 源码坐实

作者：Zhou Junjie | 时间：2026-08-15 08:54 UTC | 字数：1327 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日深挖：sNUSD 闪电贷交易的双套利结构 + 源码坐实

继续昨天的 sNUSD 案例，做了三层深挖，纠正了之前的分析错误。

一、金额对账 → 发现双套利结构

用 Blockscout 拿到 22 条精确 token 转账，逐笔对账，发现之前的「单套利」理解是错的：

| 套利 | 机制 | 收益 |

|-----|-----|-----|

| Lulo 兑换率套利 | 存 Lulo 触发 accrueInterest → 兑换率跳升 | +869.65 USDC (74%) |

| sNUSD 估值差 | 低流动性池 vs 抵押面值 | +297.01 USDC (26%) |

| 成本 | supply 到 Morpho | 1,000 USDC |

| 净收益 | +166.66 USDC |

关键纠正：

- 「用户投入 700 刀」→ 实际是零本金（闪电贷全垫资，余额前后全 0）
- 「2770 万闪电贷是套利本金」→ 实际是放大 0.0031% 微小利率差的杠杆
- 「闪电贷费 3.6 bps」→ 实际 Morpho flashLoan 免费

二、读合约源码 → 坐实协议身份

通过 Sourcify + Blockscout 拿到源码：

- 0xbbbb... = Morpho Blue (src/Morpho.sol 0.8.19, 借贷协议)

- 0xf29c... = MetaMorpho vault (src/MetaMorphoV1_1.sol 0.8.26, Morpho 官方收益聚合器，被 Lulo 品牌化)

源码坐实了套利链路：

MetaMorpho deposit → _supplyMorpho → MORPHO.accrueInterest 结算利息

→ lastTotalAssets 上升 → LuloUSDC 兑换率 1.05713→1.05717

→ 赎回吃 0.0031% 跳升 × 2770万本金 = 869.65 USDC

三、方法论沉淀

分析复杂闪电贷交易的可复用路径：

1. 金额对账 (Blockscout token-transfers) → 差额即利润，拆成多个收益源 + 成本
2. 事件签名反查 (4byte.directory) → FlashLoan/Supply/Borrow 直接暴露协议类型
3. 合约源码坐实 (Sourcify/Blockscout) → 确认身份 + 纠正错误推断
4. 诚实面对不确定：数据缺失就标注推断，不臆造机制

反思

这次最有价值的教训是「单套利腿 ≠ 全貌」：sNUSD 估值差看着是主故事（41.8% 价差），但 Lulo 兑换率套利才是收益大头（2.9 倍）。以及「闪电贷的双重作用」：既是放大利率差的杠杆，又是零本金垫资。还有「分析要诚实」：错误推断（700 刀成本、闪电贷费）被逐层纠正，靠的是对账 + 源码，不是猜测。

笔记 063: 2026-08-15 | 共学 Day 11 | 回测框架 | 投入约 10 分钟

作者：wuyundao | 时间：2026-08-15 09:13 UTC | 字数：500 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-15 | 共学 Day 11 | 回测框架 | 投入约 10 分钟

今天学了什么：

搭了第一个回测引擎 backtest.py，回答“当时看到价差动手能不能赚”。回测4零件：数据对齐（快照→同分钟CEX价）、信号（价差 vs 成本）、成本模型（CEX0.1%+DEX0.3%+Gas0.1%+滑点0.1%=0.6%）、净值（价差成本）。

得到什么结果：

1. 用真实数据跑通：WETH 12个组合、WBTC 5个组合，扣完成本后0个可赚，最好也-0.26%。
2. 破解Day10的疑团：WBTC跨链价差0.631%看着诱人，回测后optimism比Binance只贵0.341%，扣0.6%成本=净亏。价差是线索不是利润。
3. 踩了2个数据坑：CEX数据只到8/13对不上DEX快照8/14（重新下载补上）；CSV表头11列vs新数据21列错位（重写修复）。

困难：

数据对齐是回测第一课，没有对齐的历史数据回测无从谈起；CSV列错位是脏数据的典型例子。

下一步：

Day 12 训练套利Agent②。回测目前只有3个快照只是冒烟测试，要攒连续DEX历史才能做完整回测。

笔记 064: 前面已经接触了套利案例、链上数据和成本模型，如果只是每次发现一笔交易以后再手动查区块浏览器、翻协议、

作者：Yannis | 时间：2026-08-15 10:10 UTC | 字数：632 | 评分：52

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

前面已经接触了套利案例、链上数据和成本模型，如果只是每次发现一笔交易以后再手动查区块浏览器、翻协议、算成本，其实效率很低。所以今天的重点不是继续学一种新的套利类型，而是尝试把前面已经会做的分析过程整理成一个可以重复执行的工作流。

我一开始很容易把“套利

Agent”想成一个自动找机会、自动交易、自动赚钱的机器人，但现在觉得这个目标放在目前阶段有点太跳了。真正有用的第一版 Agent，应该先解决研究效率的问题。比如我给它一笔候选交易或者一个异常价差，它先帮我确认资产、链、DEX 和交易路径，再提醒我还缺哪些成本数据，然后根据 Gas、手续费、滑点和交易规模算出大概的净收益。如果证据不够，它应该告诉我“现在还不能判断”，而不是为了给答案硬说这是一个套利机会。

所以今天重新整理了一遍以后，我觉得 Agent 的工作流程大概应该是：先发现候选信号，然后验证数据来源，再还原交易路径，补齐成本，检查流动性和执行条件，最后才判断值不值得继续研究。这样它更像一个研究助手，而不是一个预测市场的黑箱。

这里还有一个很重要的问题，就是要区分“事实”和“推测”。比如链上已经发生的 Swap、Gas Used、交易时间、池子地址，这些都可以通过数据验证；但“这个价差为什么出现”“如果我当时也交易能不能赚到”，就已经包含分析和假设。如果以后让 Hermes 帮我研究案例，最好要求它把这两类信息分开，不然很容易出现一段分析看起来很完整，但里面有些关键数字其实没有来源。

笔记 065: Day 11：双向套利计算

作者：edwar-glowing | 时间：2026-08-15 11:43 UTC | 字数：320 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11：双向套利计算

学习

理解为什么必须检查两个方向：

USDC → WETH → USDC 分别：

DEX A 买，DEX B 卖 DEX B 买，DEX A 卖 Action

编写套利计算器：

`simulate_cycle(buy_dex, sell_dex, start_amount)` 输出：

初始 USDC

买入 WETH

卖出 USDC

毛利润

毛利润率

当天产物

`11_cycle_simulator.py`

[图片]

笔记 066: 第十一天学习记录：今天继续收紧 Robinhood Chain

套利研究的证据口径，重点对正式影子账

作者：yu | 时间：2026-08-15 11:44 UTC | 字数：560 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第十一天学习记录：今天继续收紧 Robinhood Chain 套利研究的证据口径，重点对正式影子账本与 readiness policy v2 产物做一致性核对。实测当前 readiness 文件记录的 ledger_digest 与正式账本 canonical digest 不一致，因此该摘要已经过期，不能沿用其中的 settled、verified 等指标作为当前放行证据；系统必须按 STALE_READINESS fail-close，资金状态继续保持 NOT_READY，real_funds_go=false。随后复跑了 execution adapter、Feed observer 与页面 freshness 的 32 项针对性测试，全部通过，确认原始交易不能绕过 readiness、观察通道不能推进资金闸门、摘要不匹配时页面不会展示为可执行。上述测试只验证安全隔离和失效保护，不证明当前存在可执行净收益，也不代表自有执行器已经实现；今天未连接钱包、未签名、未广播、未实盘。下一步先基于当前 canonical ledger 重新生成只读 readiness，再从零积累我方 executor、我方 beneficiary、未来目标块和双 RPC 身份一致的严格前瞻证据；证据不足继续 No-Go。

笔记 067: Day 11 | 阶段复盘：跨链套利值不值得做 + 最小扫描器开工

Day 11 | 阶段复盘：跨链套利值不值得做 + 最小扫描器开工

今日目标

课程过半，停下来做一次阶段性复盘：把前 10 天学的 DEX 内/间套利、LI.FI 路由、成本模型串起来，回答一个实际问题——跨链套利这个方向，值不值得继续重仓投入时间。

学到了什么

1. 屏幕上价差 $\neq$ 可执行净收益，这条在跨链场景被放大了一个数量级。DEX 内套利只面对一次 swap 的 gas 和滑点，跨链要叠加：源链 gas + 桥费 (LI.FI 报价里通常 0.05%–0.3%) + 目标链 gas + 两端滑点 + 5–20 分钟的确认延迟。之前用 LI.FI 拆过一笔 USDT 从 Arbitrum 到 Base 的报价，1 万美元规模下桥+gas 合计吃掉约 18 美元，也就是价差低于 0.2% 的机会直接出局。
2. 非原子执行是跨链独有的风险。DEX 内套利失败最多亏 gas，跨链是“源链成功了、目标链卡住了”，资金被锁在桥上，时间本身就是成本。这解释了为什么跨链价差能长期存在——不是没人看见，是看见的人算完成本不敢动。
3. 和 Ethereum MEV 对比：单链 MEV 拼的是延迟和 builder 关系，已经被专业团队卷成红海；跨链拼的是成本建模和耐心，门槛低一点但利润也薄。结论是：值得作为研究方向投入，把「价格发现 $\rightarrow$ 成本建模 $\rightarrow$ 执行验证」这条链路跑通，但不建议现阶段上真钱重仓。

认知收获

前 10 天更多是“看懂别人怎么套”，从今天起转到“自己搭系统扫”。社群里有同学也开始做扫描器，思路一致：先选一条链、两个 DEX、几个交易对，把真实 quote 接起来，按不同资金量跑一遍成本模型，看能不能扫出有意义的净价差。

明日计划

搭最小套利扫描器 v0：BSC 链上 PancakeSwap V3 的 0.05% 和 0.3% 费率池做 DEX 内扫描，每 30 秒拉一次 mid price，价差超过 0.15% 才记录（预留 gas + 滑点余量），先跑 24 小时看机会频率，再决定要不要接 LI.FI 做跨链扫描。

笔记 068: 链上套利残酷共学 · Day 11 打卡草稿：Sky LitePSM 固定兑换与稳定币价格回归

链上套利残酷共学 · Day 11 打卡草稿：Sky LitePSM 固定兑换与稳定币价格回归

日期：2026-08-15 (Asia/Shanghai)

主题：USDS/USDC 固定兑换、DEX 偏离套利、流动性容量与 Keeper 再平衡

边界：官方文档与合约源码研究；未连接钱包、未执行交易，当前参数和余额需在实际交易前重新读取。

选题说明

今天课程 events API 返回空数组。今日公开打卡抽样共 78 条，没有发现 Sky、Maker、PSM 或 LitePSM 主题，因此选择一个与前几天的三角套利、清算、Solver、收益拆分、软清算和 Vault Share 均不同的方向：

text

DEX 上浮动的 USDS/USDC 市场价格

vs

Sky LitePSM 给出的固定兑换边界

LitePSM 的意义不只是提供一个“1:1 换币入口”。它实际上给市场设置了一条带费用、余额和债务上限的价格走廊。套利者在 DEX 价格偏离走廊时完成闭环，把 USDS 拉回锚定附近；但可赚金额取决于 tin/tout、DEX 深度、Pocket 中的 USDC、LitePSM 中可用的 DAI，以及 Keeper 是否能继续 fill。

Sky LitePSM：固定兑换怎样成为稳定币套利的价格边界

今天学习 Sky Protocol 的 LitePSM (Lightweight Peg Stability Module)。它支持 DAI/USDC 固定比例兑换，并通过一个 Wrapper 为用户提供 USDS/USDC 路径。

官方文档说明，LitePSMWrapper-USDS-USDC 会把 USDS 在内部按 1:1 转换为 DAI，再路由到共享的 LitePSM-DAI-USDC。这样 USDS 和 DAI 共用同一份 USDC 流动性，避免拆成两个独立资金池。

因此用户看到的是：

text

USDS $\leftrightarrow$ USDC

实际协议路径更接近：

text

USDS ↔ DAI (固定 1:1、无费用)

↓

DAI ↔ USDC (LitePSM, 可能收取 tin/tout)

一、两种兑换方向和费用

在 LitePSM 命名中, USDC 是 gem。

sellGem: 卖 USDC, 得到 USDS

用户把 USDC 交给 Wrapper, 底层 LitePSM 将 USDC 存入 pocket, 并向用户支付经过费用扣减的 DAI, 再由 Wrapper 转成 USDS。

忽略精度差异时:

text

$USDS_out = USDC_in \times (1 - tin)$

tin 是卖出 gem 的费用。

buyGem: 用 USDS 买 USDC

Wrapper 先把用户的 USDS 转成 DAI, 底层 LitePSM 收取 DAI, 并从 pocket 向用户转出指定数量的 USDC。

简化计算:

text

$USDS_in = USDC_out \times (1 + tout)$

tout 是买入 gem 的费用。

Sky 官方协议文档写明, 文档所列部署当时这两条 USDS/USDC 路径尚未启用费用, 但 tin 和 tout 可以由治理修改, 未来也可能变化。真正计算前必须直接读取合约, 不能把“历史上为 0”写死在机器人里。

另外, 治理可将某一费用设为 `type(uint256).max`, 分别暂停 sellGem 或 buyGem。白名单地址虽然可以调用 NoFee 版本, 但普通套利者不能假设自己拥有该权限。

二、两条价格偏离套利闭环

LitePSM 不创造无条件利润。只有 DEX 的可成交价格越过 PSM 费用、Gas 和价格影响组成的边界, 套利才成立。

方向 A: USDS 在 DEX 折价

假设 DEX 上可以用少于 1 USDC 的成本买到 1 USDS:

text

USDC → 在 DEX 买入折价 USDS

→ LitePSM buyGem: USDS 换回 USDC

简化收益:

text

净收益

= PSM 得到的 USDC

- DEX 买入 USDS 的 USDC 成本

- tout

- DEX 手续费和价格影响

- Gas、MEV 与失败成本

只有 USDS 的折价大于全部成本, 闭环才赚钱。

方向 B: USDS 在 DEX 溢价

如果 DEX 愿意用超过 1 USDC 的价格买入 1 USDS:

text

USDC → LitePSM sellGem: 换得 USDS

→ 在 DEX 卖出溢价 USDS

简化收益:

text

净收益

= DEX 卖出 USDS 的 USDC 收入

- 初始 USDC

- tin

- DEX 手续费和价格影响

- Gas、MEV 与失败成本

两种方向共同形成一条价格走廊：

text

USDS 明显低于锚定 → 买 USDS、经 PSM 换 USDC

USDS 明显高于锚定 → 用 USDC 经 PSM 铸出 USDS、再卖出

套利者赚取偏离回归，PSM 则通过固定兑换边界帮助 USDS 恢复锚定。

三、为什么看到偏离不等于有相同容量

LitePSM 采用预铸 DAI 池，把用户兑换简化为 ERC-20 转账，以降低 Gas。但这也意味着单笔兑换不是无限容量。

买 USDC 的容量

buyGem 最直接受 pocket 中 USDC 余额约束：

text

最大可买 $\text{USDC} \leq \text{pocket 的可用 USDC}$

USDS Wrapper 与 DAI 路径共享这份流动性；同一区块中其他交易可能先消耗余额。

卖 USDC 的容量

sellGem 需要 LitePSM 手中有足够 DAI，Wrapper 才能最终支付 USDS：

text

最大可卖 $\text{USDC} \leq \text{LitePSM 当前可支付的 DAI}$

治理参数 buf 是协议希望经常维持的预铸 DAI 缓冲，但当前余额不一定恰好等于 buf。大额交易不能只读 buf，必须读取实际余额和 rush()。

合约源码中的 rush() 计算还能补充多少 DAI，核心受三组条件共同限制：

text

可 fill 数量 = min(
目标债务与当前债务之差,
该抵押类型 line 的剩余额度,
全局债务上限的剩余额度
)

目标债务 $\approx \text{pocket 中标准化后的 USDC} + \text{buf}$

所以更完整的套利容量应取：

text

可执行容量 = min(
DEX 在目标利润阈值内的深度,
LitePSM 当前 DAI 或 pocket USDC 余额,
rush 可补充额度,
本区块剩余可用流动性,
自有或闪电流动性
)

四、Fill、Trim、Chug 在做什么

LitePSM 把用户兑换与内部记账分开。Keeper 使用三个公开函数维护池子：

fill()：当 DAI 缓冲不足且债务上限仍有空间时，向 LitePSM 补充预铸 DAI；

trim()：当 DAI 或债务超过目标时，销毁多余 DAI、降低债务；

chug()：把已累积且当前可提取的兑换费用并入协议 surplus buffer。

这些函数是 permissionless，并不只允许官方 Keeper 调用。官方指南还指出，大额 USDC→DAI 兑换可以在一个原子批次中交替执行 sellGem() 与 fill()，前提是 rush() 返回正数且各项债务上限允许。

这里有一个重要结论：

text

buf 是目标缓冲，不是承诺成交量；

fill 是补充手段，不是无限铸币权限。

五、不可用和亏损场景

这类固定兑换套利至少要防以下情况：

1. 费用边界不满足：DEX 偏离小于 $\text{tin}/\text{tout} + \text{DEX 成本} + \text{Gas}$ ；
2. 余额被抢先消耗：同一区块另一笔交易先拿走 DAI 或 USDC，交易回滚；
3. 无法继续 fill：rush() = 0，可能是已经达到 buf、局部 line 或全局债务上限；

4. 方向被暂停：tin 或 tout 被设为 uint.max；
5. 治理参数同块变化：LitePSM 本身没有用户级滑点保护，治理执行可在用户交易前改变费用；
6. DEX 腿不原子：先完成一腿后价格变化，留下 USDS 或 USDC 的方向敞口；
7. 精度错误：USDC 为 6 位小数，USDS/DAI 为 18 位，小数换算或向下取整可能制造假利润；
8. 稳定币与合约风险：USDC 发行方、USDS 可升级实现、Pocket 授权、治理和底层合约都属于信用或技术边界；
9. MEV 与竞争：公开的确定性价差容易被搜索者竞争，预估利润不等于最终成交利润。

Sky 官方 README 还特别提示了前置交易问题：LitePSM 的后台记账可能在兑换前改变可用 DAI，大额交易也可能互相争抢有限余额。对大规模策略而言，应使用同区块模拟、带利润下限的或原子批次，而不是依赖页面报价。Wrapper

六、扫描器应记录什么

如果把这个议题继续做成只读扫描器，我会记录：

text

区块号与时间

DEX USDS/USDC 双向真实报价和规模阶梯

tin、tout、halt 状态

LitePSM DAI 余额

pocket USDC 余额

buf、rush、局部 line 与全局剩余额度

预估 Gas

扣除全部成本后的 netEdgeBps

最大可执行规模

触发条件不应只是 USDS != 1 USDC，而应是：

text

netProfit(q)

= 回归价差收益

- PSM fee

- DEX fee 与 price impact

- Gas

- MEV/失败安全垫

最低利润阈值

今日结论

LitePSM 可以理解为 USDS/USDC 市场的“可调用价格边界”。套利闭环并不复杂，真正难点是同时确认费用、两边流动性、Keeper 可补充额度和同一区块竞争。

今天最大的认知更新是：

固定 1:1 兑换不等于无限 1:1 流动性。价格边界由合约给出，实际容量由 Pocket 余额、预铸缓冲和债务上限共同决定。

下一步可以对同一区块的 DEX 报价与 LitePSM 状态做规模阶梯采样，输出每个规模的 netEdgeBps 与 maxExecutableSize，仍先保持只读告警，不执行真实交易。

一手资料

Sky Protocol Docs：LitePSM User Guide (<https://developers.skyeco.com/guides/psm/litepsm/>)

Sky Protocol Docs：LitePSM Protocol Documentation (<https://developers.skyeco.com/protocol/liquidity/litepsm/>)

Sky Protocol Docs：USDS 与 DAI-USDS 1:1 Converter (<https://developers.skyeco.com/protocol/tokens/usds/>)

Sky 官方 dss-lite-psm README (<https://github.com/sky-ecosystem/dss-lite-psm>)

Sky 官方 DssLitePsm.sol (<https://github.com/sky-ecosystem/dss-lite-psm/blob/main/src/DssLitePsm.sol>)

笔记 069: 进一步用codex和antigravity优化个人套利监控网页项目：

作者：SimonQuant | 时间：2026-08-15 13:06 UTC | 字数：6885 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

进一步用codex和antigravity优化个人套利监控网页项目：

EDGE//STATE — 加密套利监控项目总结

一句话概括

面向个人交易者的多交易所、多资产类别「事件状态套利」监控网站——只做监控和诊断，不自动下单。

项目定位

| 维度 | 说明 |

| :----- | :----- |

| 名称 | EDGE//STATE (edge-state-arbitrage-monitor) |

| 目标用户 | 个人交易者 |

| 核心原则 | 只读监控、不自动下单；凭证只从服务器环境变量读取，不进浏览器、不落盘 |

| 交易就绪 | 首页固定显示 交易就绪 0——所有候选均需人工审查后才能执行 |

技术栈

| 层级 | 技术 |

| :----- | :----- |

| 前端框架 | React 19 + Next.js 16 (via Vinext) |

| 样式 | Tailwind CSS 4 |

| 构建 | Vite 8 + TypeScript 5.9 |

| 图表 | uPlot |

| 图标 | Phosphor Icons |

| 数据库 | SQLite (better-sqlite3) + Drizzle ORM |

| 部署 | Cloudflare Workers / Vinext |

| 包管理 | pnpm 11 + Node.js 24 |

| 运行时 | 本地开发 local:up (IPv4+IPv6 双栈) |

监控范围概览

项目覆盖 8 大类套利/事件监控，横跨加密货币、预测市场、DeFi 协议和传统金融：

1 跨所充提状态差异

Gate × Gate US 逐链充值/提现状态差异 + 同数量 VWAP 可成交价

2 永续合约资金费率套利 (Funding Rate Arbitrage)

覆盖交易所: Gate、Hyperliquid、Binance、Bitget、KuCoin、Bybit、OKX

统一 8 小时归一费率便于跨所比较

下一结算影子监控 (20–30 / 10–20 / 5–10 / 2–5 分钟提前签发)

完整周期 LCB (Lower Confidence Bound) 前向预测

非加密永续 (股票/大宗商品) 同 ticker 候选

跨所双腿 L2 验深，按目标金额 $0.5 \times / 1 \times / 2 \times$ 计算入场/退出 VWAP

3 同所现货–永续基差 (Basis Trading)

Cash-and-carry (买现货/空永续) 与 Reverse (卖空现货/多永续)

Bybit/OKX 精确交集扫描，Hyperliquid HYPE/PURR 安全白名单

4 预测市场套利

Polymarket: 标准二元完整集 YES/NO，含 CLOB fee 建模与 split/merge 路径

Kalshi WTI: 日结阈值支配、跨 series 精确互补、interior 三腿复制

Kalshi BTC/ETH: 相同谓词跨 series 复制 (KXBTC×KXBTCD, KXETH×KXETHD)

5 DeFi 协议监控

Pendle PY 机械平价: sUSDe、sUSDS、wstETH 三个 market 的 PT/YT mint/merge

Lido stETH 队列兑付: Curve 路线 + CEX 路线 (Bybit/OKX stETH × ETH perp)

Aave Governance V3: 风险参数变更提案的 timelock 监控与前向 pre/post delta

Deribit USDC BOX 组合: 原生组合 L2 合成放贷/借款报价

6 稳定币偏离监控

USDC/USDT: Binance × Bybit × OKX 三场偏离

USD1/USDT: 同上

USDe/USDC: Binance × Bybit × Hyperliquid @150

7 传统金融事件 (SEC Filings WATCH ONLY)

Issuer Tender (odd-lot): SC TO-I / SC13E4F / SC TO-C 要约监控

Rights Offering: 配股认购权发现与条款提取

SPAC Redemption: 合并投票赎回权窗口 (8 种表单 lane)

Listed-company Liquidation: 清算残余分派 (ELME 等)

8 其他结构化监控

Nasdaq 分派 / due-bill: Corporate Actions RSS 监控
代币化股票价差: Bybit × OKX xStocks 归一化比较
现货上新状态带: Bybit / OKX / Hyperliquid 新币上市检测
保证金约束: BTC/ETH/SOL 三所 risk limit / position tiers 变更监控
Mark/Index 偏离: 单所 20/50/100 bps 分级 + 跨所指数中位差
Bybit 原生组合: CarryTrade / FundingRateArb / FutureSpread / PerpBasis

架构

```
mermaid
graph TB
  subgraph Browser["浏览器端"]
    Dashboard["Dashboard (244KB)"]
    AccountPage["Account 页面 (私有)"]
    HealthReporter["Health Reporter"]
    ShadowLedger["影子账本 (localStorage)"]
  end

  subgraph Server["Vinux / Cloudflare Worker"]
    API["API Routes"]
    HealthRegistry["Health Registry"]
  end
```

```
subgraph Collector["Collector (Node.js)"]
  Scheduler["Scheduler"]
  Adapters["Monitor Adapters"]
  SQLiteStore["SQLite Store"]
  AlertDispatcher["Alert Dispatcher"]
end
```

```
subgraph Exchanges["外部数据源"]
  CEX["CEX APIs"]
  Binance/Bybit/OKX/Gate/Hyperliquid
  Prediction["预测市场"]
  Polymarket/Kalshi
  DeFi["DeFi"]
  Pendle/Lido/Aave/Deribit
  SEC["SEC EDGAR"]
  Nasdaq["Nasdaq RSS"]
end
```

```
Dashboard -->|POST /api/health| HealthRegistry
Dashboard -->|GET /api/| API
AccountPage -->|GET /api/account| API
API -->|query| SQLiteStore
Scheduler -->|poll| Exchanges
Adapters -->|ingest| SQLiteStore
```

关键代码模块

目录/文件 职责 规模	
:----- :----- :---	
app/dashboard.tsx (file:///Users/simon/Documents/crypto套利/app/dashboard.tsx) 主仪表盘渲染 (所有监控卡片) 245KB	
app/globals.css (file:///Users/simon/Documents/crypto套利/app/globals.css) 全局样式 103KB	
app/lib/arb-types.ts (file:///Users/simon/Documents/crypto套利/app/lib/arb-types.ts) 核心类型定义 141KB	
app/lib/monitor-state-client.ts (file:///Users/simon/Documents/crypto套利/app/lib/monitor-state-client.ts)	

浏览器端监控状态管理 | 41KB |
app/lib/funding-lcb-ledger.ts (file:///Users/simon/Documents/crypto套利/app/lib/funding-lcb-ledger.ts)	资金费 LCB 前向账本	57KB
app/lib/spot-listing-ledger.ts (file:///Users/simon/Documents/crypto套利/app/lib/spot-listing-ledger.ts)	现货上新账本	59KB
app/lib/risk-constraint-ledger.ts (file:///Users/simon/Documents/crypto套利/app/lib/risk-constraint-ledger.ts)	保证金约束账本	52KB
app/lib/opportunity-ranking.ts (file:///Users/simon/Documents/crypto套利/app/lib/opportunity-ranking.ts)	机会排名算法	32KB
app/account-executability-panel.tsx (file:///Users/simon/Documents/crypto套利/app/account-executability-panel.tsx)	账户可执行性诊断面板	57KB
collector/sqlite-store.mjs (file:///Users/simon/Documents/crypto套利/collector/sqlite-store.mjs)	SQLite 数据存储	54KB
collector/monitor-adapters.mjs (file:///Users/simon/Documents/crypto套利/collector/monitor-adapters.mjs)	监控适配器 (数据采集)	37KB
collector/scheduler.mjs (file:///Users/simon/Documents/crypto套利/collector/scheduler.mjs)	采集调度器	29KB

安全与风控设计

\\[!IMPORTANT]

项目在架构层面贯彻了极其保守的"fail-closed"原则

- 1. 无写入路径 — 没有下单、借款、还款、转账、提现、改杠杆或改抵押的功能
- 2. 只读凭证 — Bybit/OKX 必须先验证 API key 只有只读权限；发现写入权限立即停止
- 3. Loopback 限制 — 私有账户页仅接受 127.0.0.1 / localhost / ::1 请求
- 4. 多层门控 — 每条路线经过公开门、账户门、执行门、交易门的逐层验证，绝大多数固定为 false
- 5. NO-TRADE 默认 — 任何数据缺失、过期、不一致都触发 fail-closed
- 6. 双份确认 — 新状态必须连续观察两次且至少跨越 12 秒才确认
- 7. SEC 合规 — 自动化请求声明 User-Agent，严格限速 (2 req/s)

研究榜 vs 可执行

项目明确区分两个榜单：

- | 榜单 | 含义 |
- | :----- | :----- |
- | 公开可复核榜 | 公开数据证据充分，有目标深度和新鲜度的候选 |
- | 高潜研究榜 | 毛差/费用/规格缺失，仅供研究参考 |

"合理年化"只在固定到期、资本占用、压力成本和到期交付费用门都闭合时才显示；瞬时价差或单次 Funding 不会被机械年化。

总结

EDGE//STATE 是一个极其复杂且设计严谨的套利监控系统，覆盖了从 CEX 资金费率、预测市场完整集、DeFi 协议到 SEC 公告的广泛监控范围。项目的核心哲学是"宁可漏报，不可误报"——通过多层 fail-closed 门控确保不会给出虚假的可执行信号。整个系统代码量巨大（仅 dashboard.tsx 就有 245KB），体现了对每一个边界条件和风险场景的详尽建模。

笔记 070: 今日完成：

作者：wangche | 时间：2026-08-15 13:41 UTC | 字数：388 | 评分：62
来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

今日完成：

复盘 Ethereum 交易 0x82a206...c4e72，解析 72 条事件日志和主要资金流。
还原流程：Morpho 闪电借款 → Lulo 存取 → USDC 买入 sNUSD → Morpho 抵押借款 → 偿还闪电借款。
确认转给 EOA 的 166.666013 USDC 不是纯套利利润，而是新借款后的剩余现金。
发现交易结束后仍留约 1,034.6 sNUSD 抵押和约 1,007 USDC 债务，仓位距离清算线约 0.013%。
梳理后续监控所需数据：Morpho/Lulo 事件、预言机价格、池子报价、借款利率、Gas、内部调用 trace。
设计监控原则：先做只读监听和历史 fork 模拟，确认全部债务和费用后再判断是否存在可执行套利。
未签名、未授权、未部署合约、未发送任何交易。

笔记 071: Polymarket 争议期套利策略

作者：0x2077 | 时间：2026-08-15 14:01 UTC | 字数：341 | 评分：47

Polymarket 争议期套利策略

套利空间建立在“订单簿的情绪性恐慌”与“客观事实的最终确定性”之间的错位上。在历史上多起政治或体育赛事的争议期中，恶意提议者故意提交错误答案。这会导致正确的合约份额在订单簿上的价格瞬间从 99¢ 暴跌至 40¢ - 70¢ 的区间，因为算法机器人和未掌握全貌的散户会抛售逃生。主流交易者存在一个严重的风险误判：认为“争议期”代表着“极度的高风险与不确定性，应该尽早平仓止损”。只要合约的结算规则写得足够严谨且没有歧义，现实世界发生的事实就是不可更改的。对于套利者而言，争议期绝不是“不确定性区”，而是“预言机延迟”带来的“无风险高息存款区”。决定智能合约执行方向的并非现实世界中的“客观真理”，而是预言机网络内部的“资本共识”。

笔记 072: 今天主要把最近群里的内容重新整理了一遍，也挑了几个自己比较感兴趣的方向认真看了一下。

作者：Lynwood Kosen | 时间：2026-08-15 14:04 UTC | 字数：639 | 评分：52

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天主要把最近群里的内容重新整理了一遍，也挑了几个自己比较感兴趣的方向认真看了一下。

其中印象最深的是 Morpho 和 sNUSD 的案例。它不是简单看两个池子的价格差，而是协议里的预言机估值、抵押率和外部市场价格没有对上。有人通过闪电贷先补充市场流动性，再买入 sNUSD 做抵押并借出 USDC，最后在一笔交易里完成整个流程。这个案例让我看到，套利机会不一定只来自币价差，也可能藏在协议参数和市场价格之间。

另外一个收获是，看到价差不代表真的能做。哪怕价格条件还在，只要市场已经被借满、没有可借资金，或者池子深度不够，机会实际上就被冻结了。所以以后看套利信号，不能只盯价格，还要一起看流动性、可借额度、滑点和执行条件。

群里还讨论了 RPC 限流、调用成本和延迟。我的理解是，不能一上来就对所有池子做精确询价，这样很快就会把 RPC 或 API 额度用完。比较合理的方式是用便宜的数据把大部分无效机会过滤掉，再对少数候选做精确报价和模拟。昨天刚把 BSC 节点启动起来，这部分内容也正好能和自己的节点部署接上。

今天还看到一种私钥隔离的做法：策略和 AI 只调用本地签名接口，不直接接触私钥。这个思路对以后做自动化很有用，不过签名接口本身也要限制合约、金额、链和调用方法，不能只是把私钥换个地方存放。

今天没有实际下单，主要是把群里的零散信息整理成自己能理解的几个方向。后面准备先把 Morpho/sNUSD 这笔交易的流程再拆一遍，看看每一步资金是怎么走的。

笔记 073: 2026-08-15 学习打卡

作者：UU20220213 | 时间：2026-08-15 14:08 UTC | 字数：1333 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-15 学习打卡

今日完成

- 学习 bps 费率单位：1 bps = 0.01%，25 bps = 0.25% = 0.0025。掌握平台服务费公式：交易金额 × 0.0025；例如 1,000 USDC 对应 2.5 USDC 服务费。
- 理解 100 RPM 表示每分钟最多约 100 次 API 请求，与交易费用无关。
- 学习 API 报价不等于最终成交：区分预计输出与最低输出，套利判断应优先使用最低输出，并扣除 Gas、尚未包含的手续费和资金再平衡成本。完成例题：2,015 - 2,000 - 3 - 4 = 8 USDC，保守净收益率为 0.4%。
- 学习成功率与失败损失的期望收益计算：80% × 10 - 20% × 6 = 6.8 USDC。认识到屏幕利润为正不代表每次执行都赚钱。
- 学习 RPM 请求预算：普通扫描与候选高频复查分层安排；完成 60 RPM、90 RPM 和包含 15% 重试后的 92 RPM 计算。理解不能把 100 RPM 全部用满，应为重试、Gas 查询和交易前复核保留余量。
- 理解同一路径的不同交易金额必须分别询价，因为池子价格影响是非线性的；收益率最高的金额不一定是绝对利润最高的金额。
- 学习缓存策略：链、代币地址、精度等静态数据可以较长时间缓存；报价、Gas、路由、流动性和可借资金需要频繁刷新。
- 区分 API Key 与 Secret Key：API Key 用于识别项目，Secret Key 用于生成请求签名。监控阶段只使用只读权限，不开启交易和提现权限；Secret 不放进聊天、截图、GitHub或网页前端。
- 学习常见 API 状态码：200 成功，400 参数错误，401 Key/签名错误，403 权限或 IP 问题，429 请求过多，500/503 服务错误。429 应降速并使用指数退避，不能立即连续重试。
- 理清 Binance Web3 Developer Portal 登录方式：使用已登录交易所主账号的 Binance App 扫码，而不是用 Binance Wallet 连接 DApp；UID 是 Binance 账户识别编号，不是 0x 钱包地址、API Key 或推荐码。

今日关键认识

套利监控的核心不是看到价差，而是拿到固定金额的可执行报价并计算保守净利润。
API 限速本身也是系统设计约束，需要粗扫、复查、缓存、余量和退避机制共同配合。
监控系统优先保证密钥安全和只读边界，在没有验证稳定收益与风险控制前不开放自动交易。

学习证据

独立完成 25 bps、保守净利润、期望收益和多组 RPM 计算。
能说明 API Key、Secret Key、报价字段、429 错误和指数退避的作用。
完成 Binance 主账号扫码登录方式与 UID 位置辨认。

未完成与下一步

下一步用模拟数据制作一张套利报价核算表，明确输入金额、预计输出、最低输出、平台费、DEX费、桥费、Gas、滑点、失败成本和最终保守净利润。暂不进行真实资金交易。

笔记 074: markdown

作者：Syen White | 时间：2026-08-15 14:09 UTC | 字数：755 | 评分：84

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

markdown

价差套利模拟(真实场景)

注意到 COW 在 binance(价格低) 和 bybit(价格高) 在合约价格上存在8个点的价差, 资费都是 -2%, 空头支付给多头
在 binance 做多, 在 bybit 做空

假设

在 binance 做多 COW 1000个, 持仓价值 $1000 \times 0.15 = 150U$

在 bybit 做空 COW 1000个, 持仓价值 $1000 \times 0.162 = 162U$

当价差收敛到 4% 时, 平仓, 此时 binance 价格是 0.16, bybit 价格是 0.1664

binance 盈利 = $(0.16 - 0.15) \times 1000 = 10U$

bybit 盈利 = $(0.162 - 0.1664) \times 1000 = -4.4U$

资费支出

binance 资费 = $150 \times 2\% = 3U$ (收到)

bybit 资费 = $162 \times 2\% = 3.24U$ (支出)

非VIP情况下的手续费支出(市价单)

binance: $150 \times 0.05\% + 160 \times 0.05\% = 0.155u$

bybit: $162 \times 0.055\% + 166.4 \times 0.055\% = 0.18062u$

净收益 = $10 - 4.4 + 3 - 3.24 - 0.155 - 0.18062 = 5.02438u$

风险:

价差一直不收敛, 需要承担高额的资金费支出(承受不住可能会亏损离场)

满资金费率后下一周期的结算将改为 1h 或 4h, 需要持仓更久的时间, 持仓越久, 风险越大
价格突然的爆跌或暴涨, 可能会导致爆仓, 需要设置止损和仓位上限

笔记 075: 借贷清算套利 (Liquidations)

作者：zym89861-wq | 时间：2026-08-15 14:11 UTC | 字数：254 | 评分：51

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

借贷清算套利 (Liquidations)

原理：当借贷协议（如 Aave、Compound）中的借款人抵押品价值下跌、健康因子（Health Factor）低于阈值时，触发清算机制。

典型场景：

套利者 (Liquidator) 帮借贷协议“偿还借款人欠下的债务”，作为回报，套利者可以以折扣价（通常有 5%~10% 的清算奖金/Penalty）获得借款人的抵押品。

拿到折扣抵押品后，立刻在 DEX 上卖掉换回稳定币/ETH，锁住纯利润。

笔记 076: Day 11 笔记：三角套利 —— 数学与路径搜索

作者：GeoGu0 | 时间：2026-08-15 14:25 UTC | 字数：5767 | 评分：80

Day 11 笔记：三角套利 —— 数学与路径搜索

日期：2026 年 8 月 15 日

进度：理解了三角套利的数学本质 + 手动验证了一条三角路径 + 搞懂了"原子执行"为什么是三角套利的生死线 + 分析了路径搜索空间的爆炸性增长 + 理解了 Flash Loan 如何让三角套利成为可能

一、从 DEX 间到三角：套利路径的维度升级

Day 9 学的 DEX 间套利，是两个市场、一个资产、两笔交易：在 Uniswap 买 ETH，在 SushiSwap 卖 ETH。路径是一条直线——起点是 USDC，终点是 USDC，中间只经过 ETH 这一个中间资产，跳数是 2。

三角套利是这条路径的扩展：起点和终点还是同一个资产（通常是稳定币 USDC），但中间经过两个甚至更多的中间资产，跳数是 3 或者更多。核心逻辑完全一样——利用不同市场对资产的定价不一致，绕一圈回到原点时，手里的起始资产变多了。

为什么要把简单的"两跳"扩展成"三跳"甚至"多跳"？因为两跳的 DEX 间套利有一个硬约束：价差必须大于协议费地板（Day 9 算出来大约 0.1%-0.6%），而主流交易对的价差几乎永远达不到这个数。三角套利换了一个思路：单一交易对的价差可能很小，但三个交易对各自微小的定价偏差，累积起来可能超过交易成本。相当于把"找一个大价差"变成了"找三个小偏差的组合"，搜索空间大了很多，但机会的数量级也大了很多。

二、三角套利的数学：汇率必须"不一致"才能赚钱

先理解一个基础事实：在完全有效的市场里，汇率是自治的。

如果：

1 USDC = 0.0005 ETH (即 1 ETH = 2000 USDC)

1 ETH = 200 LINK

1 LINK = 0.01 USDC (即 1 USDC = 100 LINK)

那么这三条汇率是自治的：1 USDC $\rightarrow$ 0.0005 ETH $\rightarrow$ 0.0005 $\times$ 200 = 0.1 LINK $\rightarrow$ 0.1 $\times$ 0.01 = 0.001 USDC。亏了 99.9%——不对，这个例子有问题。让我用正确的数字。

正确的自治汇率应该是：

1 USDC $\rightarrow$ 0.0005 ETH (1 ETH = 2000 USDC)

0.0005 ETH $\rightarrow$ 0.0005 $\times$ 200 = 0.1 LINK (1 ETH = 200 LINK)

0.1 LINK $\rightarrow$ 0.1 / 100 = 0.001 USDC (100 LINK = 1 USDC)

如果 100 LINK = 1 USDC，那 0.1 LINK = 0.001 USDC。这样 1 USDC 转一圈只剩 0.001 USDC——这是个大亏。所以我的汇率假设不自洽。正确的自治汇率应该是让转一圈刚好等于 1：

1 USDC = 0.0005 ETH

1 ETH = 200 LINK

那 1 USDC $\rightarrow$ 0.0005 ETH $\rightarrow$ 0.1 LINK，要让 0.1 LINK 换回 1 USDC，需要 1 LINK = 10 USDC。

所以自治条件是：1 USDC = 0.0005 ETH，1 ETH = 200 LINK，1 LINK = 10 USDC。三者相乘：(0.0005 ETH/USDC) $\times$ (200 LINK/ETH) $\times$ (10 USDC/LINK) = 1。转一圈不多不少，回到 1 USDC。这就是"无套利条件"——三个汇率相乘等于 1。

三角套利的机会就出现在这个乘积偏离 1 的时候。假设现实里某条链上，第三个汇率不是 10 USDC/LINK，而是 10.05 USDC/LINK (LINK 被轻微高估了)。那么：

1 USDC $\rightarrow$ 0.0005 ETH $\rightarrow$ 0.1 LINK $\rightarrow$ 0.1 $\times$ 10.05 = 1.005 USDC。

转一圈多了 0.5%。这个 0.5% 就是三角套利的毛收益。如果扣完三笔交易的协议费 (3 $\times$ 0.05% = 0.15%) 和 Flash Loan 费 (0.05%) 还有剩余，就是净收益。

所以三角套利的本质可以精炼成一句话：找到一条闭环路径，使得路径上所有汇率相乘 $>$ 1 (顺时针) 或 $<$ 1 (逆时针)，偏离 1 的部分就是套利空间。而三个汇率相乘 = 1 就是无套利的均衡状态。

三、手动验证一条三角路径

我选了 Arbitrum 链，三个资产是 USDC、WETH、ARB。路径是 USDC $\rightarrow$ WETH $\rightarrow$ ARB $\rightarrow$ USDC。

用三个 DEX 或者 LI.FI 分别查三段的报价 (这里用示意数字说明方法，真实数据需要 eth_call 或 LI.FI 拉)：

第一段 USDC $\rightarrow$ WETH：用 1000 USDC 能买到多少 WETH？

假设 1 WETH = \$2000，那 1000 USDC $\approx$ 0.5 WETH (扣 0.05% 协议费后实际约 0.49975 WETH)。

第二段 WETH $\rightarrow$ ARB：0.49975 WETH 能买到多少 ARB？

假设 1 WETH = 4000 ARB，那 0.49975 WETH $\approx$ 1999 ARB (扣协议费后约 1998 ARB)。

第三段 ARB $\rightarrow$ USDC：1998 ARB 能卖回多少 USDC？

假设 1 ARB = \$0.5，那 1998 ARB $\approx$ 999 USDC (扣协议费后约 998.5 USDC)。

结果：1000 USDC 走完三步，回到手里约 998.5 USDC——亏了 1.5 USDC。

这个结果非常符合预期。为什么？因为这三个汇率本身是自治的（市场是有效的），走一圈回到原点，你损失的恰好是三笔交易的协议费（约 0.15%）加上可能的滑点。没有任何“汇率不一致”可以被利用。

这就是 Day 11 最重要的认知之一：你在手动查三次报价时，看到的三个价格是“市场在不同时刻的价格”——它们几乎总是自治的。真正的三角套利机会（三个汇率乘积偏离 1）极其短暂，只存在于某个池子刚被一笔大额交易扰动、还没被套利者修正的那一瞬间。手动查三次报价，中间隔了十几秒甚至几分钟，你永远抓不到那个瞬间。你只能抓到“市场已经重新自治之后的三个价格”，所以算出来的永远是“亏损等于手续费”。

四、原子执行：三角套利的生死线

这是三角套利和 DEX 间套利最本质的区别所在。

DEX 间套利（两跳）在理论上可以“分两次做”——先在便宜 DEX 买，再去贵 DEX 卖。虽然有时间风险，但至少每一步是独立的交易。三角套利（三跳）则几乎必须在一个交易里完成全部三步，原因有两个：

第一，三步之间的价格联动。你做了第一步（USDC → WETH），这一步本身就会改变 WETH/USDC 池子的价格。等你想做第二步（WETH → ARB）的时候，WETH 的价格已经不是你做第一步时的价格了。每一步都在破坏后续步骤赖以成立的定价条件。

第二，时间窗口极小。三角套利的汇率偏差通常只有零点几个 bp，持续时间毫秒到秒级。分三次手动做，光每次交易的确认时间加起来就几十秒，机会早就消失了。

解决方法是 Flash Loan（闪电贷）——把三步全部打包进一个智能合约调用，在一个区块、一个交易里顺序执行。闪电贷提供起始资金（比如借 1000 USDC），三步 swap 依次执行，最后一步把 USDC 还回闪电贷协议并付借款费。要么三步全部成功、闪电贷还清、利润留在合约里；要么中间任何一步 revert，整个交易回滚，闪电贷也自动归还，你只损失 Gas。

闪电贷让三角套利变成了一笔原子操作，同时解决了两个问题：资金问题（不需要自有本金）和时间问题（三步在同一笔交易里，毫秒级完成）。这也是为什么 MEV 搜索者做三角套利都是写一个专门的合约，用一个 eth_call 就能模拟整个流程的净收益。

五、路径搜索空间：为什么需要算法

假设一条链上有 N 个 token，K 个 DEX。三角套利要找的是一条“闭环路径”，长度至少是 3（三跳），可以是 4、5、甚至更长。

路径数量怎么算？以三跳为例：起点资产有 N 种选择（通常是稳定币，可以简化为 1-2 种）。第一跳的目标有 N-1 种选择，第二跳有 N-2 种，第三跳必须回到起点。所以三跳路径大约是 $N \times (N-1) \times (N-2)$ 量级，即 $O(N^3)$ 。四跳是 $O(N^4)$ 。

更重要的是，每一跳还可以选择不同的 DEX 和不同的费率池。所以实际的搜索空间是 $O(N^L \times K^L)$ ，其中 L 是跳数，N 是资产数，K 是 DEX 数。

代入真实数字：Arbitrum 上活跃的 token 可能有几百个（取 $N = 200$ ），主流 DEX 有 5-10 个（取 $K = 5$ ），考虑三跳路径： $200^3 \times 5^3 = 8,000,000 \times 125 = 10$ 亿条路径。四跳就是 $200^4 \times 5^4 = 16$ 亿 $\times 625 = 1$ 万亿条。

这个数字意味着：三角套利不可能靠暴力枚举。你需要：

1. 剪枝——只在有足够流动性的池子上搜索，跳过流动性枯竭的池子。
2. 预计算——离线维护一个“汇率图”，只更新变化的部分。
3. 约束路径长度——实践中三跳是最常见的（手续费随跳数线性增长，四跳以上几乎必然被手续费吃掉）。

这也是为什么个人做三角套利很难：不只是执行速度的问题，连“发现机会”这一步都需要一个高效的路径搜索引擎。相比之下，DEX 间套利（两跳）的搜索空间是 $O(N)$ ，简单得多——这也是为什么 Day 9 的 DEX 间套利比 Day 11 的三角套利对个人更友好。

六、用 eth_call 模拟一个三角套利

和 Day 9 的 DEX 间套利一样，三角套利也可以用 eth_call 精确模拟。区别在于，三角套利模拟的是“一个完整的多步流程”，不是单独的 swap。

思路是：用 web3.py 或者直接调一个已经部署好的“套利合约”，在一个 eth_call 里完成：

1. 从 Aave 借入 1000 USDC (flash loan)
2. 在 Uniswap 上 USDC → WETH
3. 在另一个 DEX 上 WETH → ARB
4. 在第三个 DEX 上 ARB → USDC
5. 还 Aave 1000 USDC + 0.05% 借款费
6. 检查余额，剩余的就是净收益

如果是自己写模拟，可以用一个通用的合约（或者 Foundry 的 cast 工具做 call 模拟），把这几步编码进去。更简单的做法是用 Foundry 的 cast call 配合一个 mock 合约，或者直接用 eth_call 调一个真实存在的套利合约（比如某些 MEV bot 的合约），传入你想模拟的路径参数。

对于个人学习来说，最重要的是理解这个模拟的输入输出：

输入：路径（每个 hop 的 token 和 pool）、起始资金量。

输出：净收益（可能是负的）。

跑几百次这种模拟，你就能摸清楚"在当前市场状态下，哪些路径有正的净收益"——这就是一个最简版的三角套利扫描器。

七、今天的三个核心收获

第一，三角套利的数学本质是"闭环汇率乘积偏离 1"。三个（或多个）汇率相乘等于 1 是无套利均衡条件。偏离 1 的部分就是套利空间。这个认识把三角套利从一个模糊的"绕圈赚钱"概念，变成了一个可以精确计算和判定的数学条件。

第二，手动查三次报价永远抓不到三角套利。因为三个价格必须是同一时刻的，而手动查询有十几秒到几分钟的延迟，你查到的一定是市场已经重新自洽之后的价格。所以手动验证三角路径，算出来的永远是"亏损等于手续费"。真正的三角套利机会只存在于毫秒级的窗口内，必须原子执行。

第三，三角套利的门槛在"发现"而不只在"执行"。DEX 间套利（两跳）搜索空间是 $O(N^3)$ ，个人勉强可以暴力扫描。三角套利（三跳）搜索空间是 $O(N^3)$ ，个人没法暴力枚举，需要剪枝和预计算的路径搜索引擎。这是三角套利对个人比间套利更难的根本原因——不只是手速，是算法和基础设施的全面碾压。

八、今天的三个问题

- 三角套利的路径搜索，除了暴力枚举 + 剪枝，有没有更聪明的算法？比如用最短路径算法（Dijkstra/Bellman-Ford）把汇率图转换成"找负环"问题？（隐约觉得 Bellman-Ford 找负环和三角套利是同一个数学结构）
- Flash Loan 的费用是 0.05%，但有些协议（如 Euler、Maker 的 flash mint）费用是 0。用零费用的 Flash Loan 协议，会不会显著改变三角套利的盈亏平衡？
- 个人如果要做三角套利，最低配的基础设施是什么？是不是至少需要一个能订阅 mempool 的节点 + 一个预计算好的汇率图 + 一个能毫秒级提交 bundle 的 MEV 中继？

九、三角套利这条路的现实判断

三角套利是六种套利类型里，对个人门槛最高的一种。它同时需要：高效的路径搜索算法（找到机会）、毫秒级的执行（抢到机会）、Flash Loan 的合约开发能力（实现原子执行）。这三样东西，缺一样都做不了。

但它也是最"性感"的一种——因为它不像 DEX 间套利那样受限于"单一交易对价差必须大于协议费"的硬约束，而是可以在海量的路径组合里寻找任何微小的、累积起来可观的定价偏差。这也是为什么专业的 MEV 团队、做市商、高频基金都重仓投入三角套利——它是链上套利里利润密度最高的赛道之一。

对个人来说，Day 11 的收获不是"我要去做三角套利"，而是"我理解了三角套利为什么是专业玩家的游戏，以及它的门槛到底在哪"。这种清醒的认识，比盲目冲进去亏钱有价值得多。我的策略应该转向那些门槛更低的赛道——Day 12 的稳定币套利，或者用监控脚本做 DEX 间套利的长尾机会，才是我这个阶段可能够得着的东西。

笔记 077: 4 个核心认知跃迁（从"找机会"到"认清现实"）

作者：jinbuge101-art | 时间：2026-08-15 14:27 UTC | 字数：2261 | 评分：86
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

4 个核心认知跃迁（从"找机会"到"认清现实"）

跃迁 1：新币开盘套利——真实存在，但属于"手动狙击"和"有准备的人"

@alex 说 K11 在 Mantle 上开盘，有人"搬爽了"。@T 说"刚套了 50wu"，@梦 贴出 Debank 链接，指出有人赚了 1-2 万刀，而且是手动。

关键细节：

池子很小（"池子都没 50W"），所以利润有限但确实存在

方向：@梦 说是"挖矿"（提供流动性赚奖励），不是传统搬砖。一个是吃 1inch Aqua 奖励，需要有 1inch 敞口，风险较高

开盘价：基本是平价或 ICO 价，少数池子加错方向/设置好价格

方法论：@Paxon 总结为"巨鲸供币→执行合约出货→利润归集 = 链上可追踪的「狗庄识别」信号原型"

你的收获：新币开盘套利是事件驱动型机会，和你之前学的"持续监控价差"完全不同。它要求：

- 提前知道开盘时间（信息优势）
- 提前准备好资金+路径（基础设施优势）
- 手动或半自动执行（因为情况多变，AI 很难处理）
- 承担高风险（池子小、价格不稳定、项目方可能跑路）

这不是你现在的阶段该碰的。你的真值表应该加一条：

纯文本

纯文本

策略: 新币开盘流动性挖矿 (KII/Mantle等)

状态: OBSERVE (事件驱动)

证据: 群友实测有人赚1-2万刀, 但手动+高风险+需要1inch敞口

死因/N/A: 需要提前获知信息+准备资金+承担无常损失

复活条件: 找到可靠的新币信息源+理解具体奖励机制+小额测试

跃迁 2: 跨所对冲资金费——"跑的人多了, 机会少了"

@飞天小蟑螂 试了一周跨所对冲资金费, "都不行"。@王生 回应:

"没事, 我的跨所价差套利跑起来也没利润, 总结就是跑的人多了机会少了, 早一个月应该出利润。"

@王生 还建议: "资金费这些没必要了, 妖币高资费又是另一种逻辑风险比较大, 找美股合约吧起码有波动。"

你的收获: 这直接印证了你 Day 12-14 的结论——屏幕价差 < 成本 = 没机会。但更重要的是:

跨所对冲资金费和跨所价差搬运是同一类死法: 竞争太激烈, 利润被压缩到成本线以下

美股合约可能是更好的方向 (有波动、有资金费、散户机会相对多)

"早一个月应该出利润"说明这类策略有时效性窗口——一旦被发现, 就迅速衰减

你的真值表更新:

纯文本

纯文本

SO01 (CEX跨所USDT搬运): DEAD → 确认死因: 竞争饱和, 利润 < 成本

SO04 (资金费率): 从 ALIVE_EVENT_DRIVEN 降级为 OBSERVE → 跨所对冲不行, 但极端正费率 (如之前BARD) 仍可能

新增: 美股合约资金费 → OBSERVE (需要研究)

跃迁 3: AI 分析套利策略——"头头是道, 最后说不行"

@飞天小蟑螂 的遭遇是整个记录最扎心的部分:

"AI说的头头是道"

"让AI分析了一下, 说了一堆乱七八糟的, 最后结论说不行"

"做了什么回归分析, 说不稳定"

"搞的焦虑了, 试了好几个方向都被证伪了 (也可能是自己菜, 没找到方法)"

@张寅 给出了建设性建议: "有时候可能是你给ai的资料和上下文不够详细, 可以高强度grill一轮, 会准确些。"

你的收获: 这和你 Day 17 的教训完全一致——AI 的结论质量取决于你的输入质量。但更深层的问题是:

AI 没有"市场直觉"。它不知道什么时候该放弃, 什么时候该坚持

AI 的"回归分析"可能是在胡说。它不知道你的数据有 25 bps 的隐藏成本

正确的用法: 让 AI 帮你写代码、查文档、整理数据, 而不是让它做投资决策

你的纪律: AI 是工具, 不是策略师。所有 AI 给出的"可行/不可行"结论, 必须用你的真值表、成本模型、盈亏比检查器来验证。

跃迁 4: 90% 的"套利程序"是骗局——链上套利不需要"本金"

@人生海海 分享了一个关键经历:

"他发的我也用过, 不过用的是一个新地址, 用了半天就提走了"

"交易一次就扣1%"

@Mr.Lai 一针见血:

"文档做的这些太专业了90%真实内容"

"但用脑子想想, 都链上套利了还缺什么本金才可以启动, 明显骗子, 还盗用其他项目名称"

@Bruce 分享的推特链接也是同样主题: 警惕来历不明的套利程序。

你的收获: 这是一个终极过滤器:

真正的链上套利: 闪电贷解决本金问题, 你只需要付 Gas + 服务费

骗局套利程序: 要求你"投入本金"、"充值解锁"、"缴纳保证金"

判断标准: 如果它说"你有本金才能启动", 100% 是骗子。链上套利的核心优势就是不需要本金 (用闪电贷)。

笔记 078:

套利基础学习打卡⑥ | Latency: 为什么明明发现了套利机会, 下单时机却已经消失?

作者: cfanboy | 时间: 2026-08-15 14:33 UTC | 字数: 9909 | 评分: 88

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利基础学习打卡⑥ | Latency: 为什么明明发现了套利机会, 下单时机却已经消失?

前五篇学习了：

- ① Funding Rate：资金费率\
- ② Basis：基差\
- ③ Delta Neutral：方向风险对冲\
- ④ Trading Cost：从毛价差到真实净利润\
- ⑤ Liquidity & Order Book Depth：价差到底能够成交多少钱

今天继续研究套利系统中另一个非常关键的问题：

Latency（延迟）

上一篇已经知道，一个真正可以执行的套利机会不能只看 Last Price，而应该：

读取 Order Book → 模拟成交 → 计算 VWAP → 扣除成本 → 得到 Net Profit

但这里还有一个问题：

假设在 T0 时刻，系统计算出一个非常好的套利机会：

A 交易所：

BTC Best Ask = 100,000

B 交易所：

BTC Best Bid = 100,500

Spread：

0.5%

按照订单簿深度计算以后：

Expected Net Profit：

0.30%

于是系统决定：

Buy A + Sell B

但等订单真正到达交易所的时候：

A：

100,300

B：

100,350

Spread：

只剩：

0.05%

甚至可能变成：

A：

100,500

B：

100,300

原来的套利机会已经完全消失。

于是今天的问题变成：

从“发现机会”到“真正成交”，到底过去了多长时间？

这就是 Latency。

一、套利中的 Latency 到底是什么？

很多时候我们会把延迟简单理解成：

服务器到交易所 API 的 Ping。

例如：

Singapore → Binance

5ms

Jakarta → Binance

30ms

于是认为：

5ms 的服务器一定比 30ms 更适合套利。

但真正的套利延迟远比 Ping 复杂。

一个完整的套利流程可能是：

交易所行情发生变化

↓

WebSocket 行情发送

↓

网络传输

↓

我们的服务器收到行情

↓

解析 Order Book

↓

更新本地订单簿

↓

计算 Spread

↓

模拟 VWAP

↓

计算 Fee / Slippage / Funding

↓

判断 Net Profit

↓

生成订单

↓

签名

↓

发送 API 请求

↓

网络传输

↓

交易所 Gateway 收到订单

↓

交易所 Matching Engine 处理

↓

订单成交

所以真正应该关心的是：

End-to-End Latency

也就是：

市场发生变化 → 我的订单真正进入市场
之间的完整时间。

二、可以把 Latency 拆成几个部分

为了更容易理解，可以把整个过程拆成：

Market Data Latency

[图片]

Processing Latency

[图片]

Decision Latency

[图片]

Order Submission Latency

[图片]

Exchange Processing Latency

最终：

Total Latency

大致等于这些延迟之和。

三、第一部分：Market Data Latency

首先是：

行情延迟。

假设 Binance 在：

12:00:00.000

订单簿已经发生变化。

但服务器在：

12:00:00.080

才收到 WebSocket 数据。

那么：

Market Data Latency $\approx$ 80ms

问题在于：

当你的系统认为：

“我现在看到了一个套利机会。”

实际上这个机会可能已经存在：

80ms

了。

在普通交易里：

80ms 几乎感觉不到。

但在高竞争套利市场：

80ms 可能已经非常长。

因为其他套利机器人可能：

10ms

20ms

30ms

之前就已经看到并开始交易。

所以：

你看到的“实时行情”，实际上永远都是过去的行情。

区别只是：

过去多久。

四、第二部分：Processing Latency

收到行情以后，系统还需要处理。

例如：

解析 WebSocket Message

↓

更新 Local Order Book

↓

计算多个交易所价格

↓

模拟不同 Position Size

↓

计算 VWAP

↓

计算手续费

↓

计算 Slippage

↓

计算 Net Profit

假设整个过程需要：

20ms

这就是：

Processing Latency。

如果系统设计不好：

大量 JSON 解析

数据库查询

跨服务 RPC

Redis 往返

同步日志写入

锁竞争

GC Pause

都可能增加处理时间。

例如：

行情到达：

0ms

JSON 解析：

5ms

Redis：

10ms

数据库：

20ms

策略计算：

15ms

最终：

50ms

才做出决策。

如果套利机会本身只能持续：

100ms

那么你已经浪费了一半时间。

五、第三部分：Decision Latency

有些策略并不是：

发现 $\text{Spread} > \text{Threshold}$ 就立即交易。

而是还要判断：

Liquidity

Funding

Position

Balance

Risk Limit

Margin

Borrow Availability

Maximum Exposure

等等。

这些检查本身都是必要的。

但如果每次发现机会以后才：

查询账户余额

↓

查询仓位

↓

查询借币额度

↓

查询数据库

↓

再判断是否下单

就会产生大量额外延迟。

所以一个重要的系统设计思路是：

能够提前计算的数据，不要等发现套利机会以后再计算。

例如：

账户余额

当前 Position

手续费等级

最大仓位

交易规则

Tick Size

Step Size

都可以：

预先加载 + 内存维护。

发现机会以后：

尽可能只执行最关键的：

Hot Path。

六、第四部分：Order Submission Latency

策略决定交易以后：

还需要真正把订单发送出去。

例如：

生成 Order

↓

参数检查

↓

HMAC / API Signature

↓
HTTPS Request
↓
Internet
↓
Exchange Gateway
这可能又需要：
10ms
30ms
50ms
甚至：
100ms+
这里服务器位置开始变得非常重要。
例如：
Server A → Exchange：
RTT = 5ms
Server B → Exchange：
RTT = 80ms
那么长期运行下来：
Server A 显然拥有明显优势。
所以很多高频交易系统非常关注：
服务器 Region
甚至：
Colocation。
因为对于某些机会：
地理距离本身就是竞争优势。
七、第五部分：Exchange Processing Latency
订单到达交易所以后，也不代表立即成交。
还要经过：
API Gateway
↓
Risk Check
↓
Order Engine
↓
Matching Engine
↓
Execution
这部分延迟：
不完全由我们控制。
尤其在极端行情中：
大量用户同时下单
↓
交易所系统负载增加
↓
订单处理延迟增加

这也是为什么：

平时运行正常的套利系统

到了：

暴涨 / 暴跌 / 插针

反而最容易出现问题的。

八、把整个流程放在一起

假设：

Market Data：

40ms

Processing：

15ms

Decision：

5ms

Order Submission：

30ms

Exchange Processing：

20ms

那么：

Total Latency：

约 110ms

也就是说：

当系统认为：

“现在有一个套利机会！”

实际上：

从市场最初发生变化到你的订单真正被处理：

已经过去：

约 110ms。

如果这个套利机会平均只存在：

80ms

那么这个策略从统计意义上可能：

根本无法执行。

九、这就引出了 Opportunity Lifetime

这是今天我觉得非常重要的一个概念。

套利机会并不是永久存在的。

例如某个跨交易所价差：

出现：

0.50%

20ms 后：

0.35%

50ms 后：

0.20%

100ms 后：

0.05%

150ms 后：

0%

那么这个套利机会的生命周期：

Opportunity Lifetime

大约只有：

150ms。

而真正有足够利润的时间：

可能只有前：

50ms。

所以一个机会是否适合自己的系统，不应该只看：

Spread Size

还应该看：

Spread Duration。

十、可以进一步研究 Opportunity Half-Life

可以借用一个很有意思的概念：

Opportunity Half-Life

也就是：

一个套利价差通常需要多长时间衰减掉一半？

例如：

初始 Spread：

0.8%

50ms：

0.4%

100ms：

0.2%

150ms：

0.1%

那么：

Opportunity Half-Life：

大约：

50ms。

如果你的：

Total Latency = 150ms

那么：

即使 Scanner 每天发现：

10,000 个机会

真正能成交赚钱的：

可能非常少。

反过来：

如果某类机会：

Half-Life = 30 秒

那么：

100ms

和：

300ms

可能根本没有本质区别。

这让我意识到：

并不是所有套利策略都需要追求微秒级延迟。

关键是：

系统延迟必须和机会生命周期匹配。

十一、不同套利类型对 Latency 的敏感程度完全不同

例如：

Funding Arbitrage

机会可能持续：

几小时

甚至：

几天。

所以：

100ms

500ms

甚至：

1 秒

通常不是最重要的问题。

更重要的是：

Funding Sustainability

Basis

Capital Efficiency

Risk。

Basis Arbitrage

可能持续：

几小时

几天

甚至几周。

所以也不是典型的 Latency Competition。

CEX-CEX Spot Arbitrage

机会可能持续：

几十毫秒

到：

几秒。

Latency：

开始非常重要。

CEX-DEX Arbitrage

取决于：

链速度

Block Time

DEX 类型

CEX 更新速度

竞争者

MEV

等因素。

Latency 同样非常重要。

DEX-DEX / MEV Arbitrage

可能已经进入：

毫秒 / 区块级竞争。

这时竞争的甚至不只是：

网络速度。

还包括：

节点位置

Mempool

Builder

Searcher

Gas Strategy

Bundle

Block Construction

等等。

所以：

套利策略越接近高频价差捕获，对基础设施的要求越高。

十二、Latency 会产生 Execution Risk

还有一个更加危险的问题。

假设我们发现：

A：

BTC = 100,000

B：

BTC = 100,500

于是：

A Buy

[图片]

B Sell

理论上：

Delta Neutral。

但实际执行可能是：

A Buy 成交。

B Sell：

因为价格已经变化：

没有成交。

此时：

Long Spot = +1 BTC

Short = 0 BTC

所以：

Net Delta = +1 BTC

原本：

Delta Neutral Arbitrage

瞬间变成：

裸多 BTC。

如果 BTC 随后下跌：

套利策略就开始承担方向亏损。

这就是：

Leg Risk

也就是多腿套利中：

一条腿成交，另外一条腿没有成交。

十三、套利中的“同时下单”其实并不真正同时

程序里可能写：

同时：

Buy A

Sell B

但实际上：

两笔订单永远存在时间差。

例如：

Order A：

T = 0ms

Order B：

T = 5ms

甚至即使两个线程同时发送：

网络路径不同

交易所处理速度不同

Matching Engine 不同

最终成交时间仍然不同。

所以：

Simultaneous Execution

在现实中只能做到：

尽可能接近同时。

而不能做到真正数学意义上的：

完全同时。

十四、如何降低 Leg Risk？

这就涉及套利执行策略。

例如可以根据：

Liquidity

Volatility

Exchange Latency

Fill Probability

决定：

哪条腿先下。

例如：

流动性差的一边：

先确保成交

然后：

流动性好的一边：

立即 Hedge。

也可以：

两边同时 IOC。

或者：

一边 Maker

另一边 Taker Hedge。

甚至：

Maker Order Fill

触发：

Immediate Hedge。

这些已经开始进入：

Execution Strategy

的范畴。

所以套利系统真正复杂的地方可能并不是：

发现 Spread。

而是：

如何把这个 Spread 真正转化成两个成功成交的订单。

十五、Latency 不能只测 Ping

如果以后真正开发套利系统，我认为至少应该记录：

Exchange Timestamp

Local Receive Timestamp

Strategy Decision Timestamp

Order Send Timestamp

Exchange ACK Timestamp

Fill Timestamp

然后计算：

Market Data Delay

Processing Delay

Decision Delay

Order RTT

ACK Delay

Fill Delay

End-to-End Latency

并持续统计：

P50

P95

P99

因为：

平均延迟并不能说明全部问题。

例如：

P50 = 20ms

看起来很好。

但：

P99 = 2,000ms

那么极端情况下：

套利系统可能产生严重的单腿风险。

十六、服务器应该放在哪里？

这也让我重新理解了服务器 Region 的选择。

以前可能主要考虑：

Ping 最低。

现在应该进一步测试：

WebSocket Market Data Latency

[图片]

REST / WebSocket Order Latency

[图片]

ACK Latency

[图片]

Fill Latency

[图片]

稳定性

[图片]

Packet Loss

[图片]

Jitter

所以：

平均 RTT 最低

不一定等于：

套利执行效果最好。

尤其应该关注：

Jitter

即：

延迟是否稳定。

例如：

服务器 A：

20ms / 21ms / 19ms / 20ms

服务器 B：

10ms / 12ms / 300ms / 8ms

虽然 B 平均值可能看起来不错：

但对于套利系统：

A 可能反而更加可靠。

十七、从系统架构上如何降低 Latency？

今天学习后，我觉得可以总结几个比较明确的方向。

行情数据

优先：

WebSocket

而不是不断：

REST Polling。

本地 Order Book

通过：

Snapshot + Incremental Update

在内存维护：

Local Order Book

避免每次机会出现以后重新查询交易所。

Hot Path 尽量不访问数据库

机会判断：

Order Book

↓

Spread

↓

VWAP

↓

Cost

↓

Risk

↓

Order

尽量全部：

In-Memory。

数据库更适合：

异步记录

统计

回测

审计。

账户状态本地缓存

Balance

Position

Fee Tier

Symbol Rules

Risk Limit

提前同步。

不要等发现机会以后：

再调用 API 查询。

减少服务间通信

如果：

Market Data Service

↓

Kafka

↓

Strategy Service

↓

Redis

↓

Risk Service

↓

Execution Service

每一步都增加：

网络

序列化

排队

上下文切换

那么架构虽然“漂亮”，但不一定适合超低延迟套利。

所以对于真正的 Hot Path：

简单可能比复杂更重要。

十八、今天最大的认知变化

前面学习套利时，我一直在研究：

这个机会理论上赚多少钱？

今天开始意识到：

还必须增加另外一个维度：

Time

于是一个套利机会实际上至少包含：

Spread

×

Liquidity

×

Time

例如：

Opportunity A：

Spread = 1%

Liquidity = 10,000 USDT

Lifetime = 20ms

Opportunity B：

Spread = 0.3%

Liquidity = 500,000 USDT

Lifetime = 5 秒

虽然：

A 的 Spread 更大。

但对于一个普通套利系统：

B 可能远远更有价值。

因此不能只问：

“有没有套利机会？”

而应该问：

“这个机会能交易多少钱、能赚多少钱、会存在多久，而我的系统能不能在它消失之前完成交易？”

十九、六篇内容终于从理论走向执行

目前已经学习：

① Funding Rate

找到：

收益来源。

↓

② Basis

理解：

价差与收敛。

↓

③ Delta Neutral

处理：

方向风险。

↓

④ Trading Cost

计算：

真实 Net Profit。

↓

⑤ Liquidity & Order Book Depth

判断：

到底能成交多少钱。

↓

⑥ Latency

判断：

能不能在机会消失以前成交。

于是套利系统的逻辑已经逐渐变成：

Market Data

↓

Detect Spread

↓

Check Liquidity

↓

Calculate VWAP

↓

Calculate All Costs

↓

Estimate Net Profit

↓

Check Risk

↓

Execute Both Legs

↓

Monitor Fill

↓

Hedge Residual Delta

↓

Record Realized PnL

这时候套利已经不再只是：

一个价格计算问题。

而开始变成：

市场微观结构 + 风险管理 + 低延迟系统工程问题。

今天最值得记住的一句话：

套利机会不是在你“看到它”的时候属于你，而是在你的订单成功成交之后，才真正属于你。

笔记 079: 今日学习：最小价差回测实操（数据源 + 指标 + 常见坑）

作者：Milli | 时间：2026-08-15 14:50 UTC | 字数：740 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习：最小价差回测实操（数据源 + 指标 + 常见坑）

一句话：把昨天的回测框架落成最小可运行版本：拿历史价差数据、定义可成交信号、扣完成本看净收益。目的不是证明能赚，而是先把数据管道和指标口径搭对。

数据源：Dune/子图拉链上 swap 事件还原成交价和深度；CoinGecko 类 API 只适合粗粒度；Flashbots mev-inspect 看真实被夹交易，做被夹概率建模。

关键指标：毛利润（卖出-买入-手续费）；净利润（再减 gas + 按深度算的滑点）；胜率+盈亏比（总收益会掩盖大多数机会不赚钱）；机会密度（单位时间可成交机会数，决定 bot 值不值得跑）。

常见坑：1) 时间不对齐会制造假价差，要按区块对齐；2) 只有报价没有深度，按中间价成交会高估收益；3) gas 用固定值会失真，高峰期能吃 60% 利润，要用历史序列；4) 只测热门池子/牛市段 = 幸存者偏差；5) 买卖价不同时刻会构造出不可能存在的价差。

最小流水线五步：选一对池子+横盘和波动两段数据 → 拉 swap 历史对齐到区块 → 算每区块可成交价差（扣 swap 费）→ 减 gas+滑点 → 输出机会数/胜率/回撤，看"价差-成本>0"占比。

Agent 视角：回测流水线 = 离线评估器，换池子换策略只换参数重跑；胜率/盈亏比/回撤口径比单条收益曲线重要；数据清洗、对齐、补深度正好自动化。最重要的一课：最小回测不是证明能赚，而是先把链路和口径搭对。数据坑不解决，后面所有策略都建立在沙子上。

- 明日方向（二选一）：
- 1. Anvil fork 主网搭套利模拟环境（合约交互 + gas 干跑）
 - 2. 用具体池子对跑通最小回测流水线（选数据源 + 出指标表）

笔记 080: 今日学习打卡 | 价差套利纸面模拟规则迭代

作者：2324462156 | 时间：2026-08-15 14:56 UTC | 字数：1715 | 评分：80
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习打卡 | 价差套利纸面模拟规则迭代

今天主要继续做 Binance 永续 vs Bybit 永续之间的价差套利纸面模拟，并把规则从“看到信号后等待确认”改成更接近真实执行的版本。

一、今天做了什么

今天重点复盘了当前纸面模拟系统的交易规则，并根据实际观察结果做了几处调整：

-

- 1. 不再等待 5 分钟确认
 - 2. 之前发现价差信号后，会先等待一段时间，再看价差是否仍然存在。今天把规则改成：只要当前价差已经覆盖完整成本，并且满足开仓条件，就直接进入纸面开仓，不再额外等待。
- 这样更接近真实套利，因为很多价差机会本来就不会等人。如果机会已经满足条件，继续等待反而可能错过。

-

- 1. 明确完整利润公式
 - 2. 今天进一步确认，不能只看价差回落，而要按完整公式计算实际套利利润：
- 实际套利利润 = 价差收敛收益 + 资金费率收益/成本 - 开仓手续费 - 平仓手续费 - 滑点 - 提现/转账成本

当前纸面模型里：

- 开仓手续费：0.10%
- \ 平仓手续费：0.10%
- \ 滑点：0.05%
- \ 提现/转账成本：0U
- \ 资金费率：按持仓时间估算

- 1. 调整止盈逻辑
 - 2. 之前止盈目标偏大，后来改成“扣完整成本后只要有小盈利就可以平仓”。当前规则是：
- 实际纸面利润 ≥ +0.05U 就止盈平仓。

这更符合小资金纸面套利测试的目的：先验证策略是否能够稳定闭环，而不是追求单笔赚很多。

-

- 1. 保留止损规则
 - 2. 当前止损线为：
- 实际纸面亏损 ≤ -2U 就止损平仓。

今天也执行过一次用户主动要求的止损/平仓操作，说明纸面模拟里不能只等系统规则，也要允许人工判断介入。

二、当前交易规则

监控范围：

Binance 永续 vs Bybit 永续。

允许观察币种包括：KAITO、BICO、JTO、NIL、BMT、TUT、ACT、CYS、SQD、BTD、BTW。

开仓条件：

1. Binance 与 Bybit 之间价差 $\geq 0.80\%$ ；
2. 2\ 两边 24h 成交额都 ≥ 500 万 USDT；
3. 3\ 当前价差能够覆盖开仓手续费、平仓手续费、滑点、转账成本和最低小盈利空间；
4. 4\ 哪边价格低，就在哪边做多；哪边价格高，就在哪边做空。

仓位规则：

总测试本金：100U；

\ 单笔虚拟本金：100U；

\ 最多同时持仓：1 笔。

三、今天看到的结果

截至今天查看时，纸面模拟累计已平仓 40 笔：

胜：29 笔

\ 负：11 笔

\ 累计实际纸面利润：+8.2690U

\ 按 100U 测试本金计算，约 +8.27%

\ 当前持仓：0 笔

最近一笔为 TUT，结果为 +0.4348U，原因是扣完整成本后小盈利平仓。

四、今天的收获

今天最大的收获是：套利模拟不能只看“价差有没有回落”，必须把交易全流程成本都算进去。尤其是资金费率可能在持仓期间持续消耗利润，开仓和平仓手续费也会明显影响小额套利结果。

现在的判断标准更清楚了：

不是“价差大就开仓”，而是“当前价差是否足够覆盖完整成本，并且未来只要正常收敛就有机会留下净利润”。

也不是“赚很多才平仓”，而是“扣完整成本后能稳定小盈利，就可以平仓记录结果”。

五、下一步

下一步继续观察这个纸面模拟系统：

1. 看当前 11 个币种里，哪些币更容易出现可执行价差；
2. 2\ 统计不同币种的胜率和亏损来源；
3. 3\ 关注资金费率为负时是否会拖累收益；
4. 4\ 后续可以考虑扩大监控币种，但必须继续保留成交额、滑点和成本过滤。

今天仍然只做纸面模拟，不进行真实资金操作。

06 学习计划（7 篇）

笔记 081: 2026-08-14 — Day 10(Week 2 第 3 天):监控雏形 v0.2 — Toke

作者: bamboo | 时间: 2026-08-14 15:04 UTC | 字数: 2482 | 评分: 89

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-14 — Day 10(Week 2 第 3 天):监控雏形 v0.2 — Token Service 自动价差源

主题: 把 v0.1 的"手动传 --spread"升级为"自动拉两链同币价格算价差"——监控雏形从半自动变全自动

方法: 实测 LI.FI Token Service API (li.quest/v1/tokens) → 取两链 USDC priceUSD → 算价差 bps → 喂给 `lifi_cost_model` 三档判定

结论先行: v0.2 跑通, 4 条路径全自动判定——真实数据 4/4 没肉(价差 2.9-11.1bps vs 阈值 50bps)。自动价差源的价值不在"今天有没有肉", 而在"以后每轮自动算, 不用人肉传参"

一、今日任务回顾(昨天笔记的明天计划)

1. 监控雏形 v0.2:接 Token Service / 链上价格做自动价差源(两链同币实时比价 → 价差 bps 自动喂给判定)
2. Dune 基础查询上手(补昨天) — 未做, 继续顺延
3. 飞书推送通道(有信号才推) — 未做, v0.2 判定稳定后再接

二、产出:scripts/lifi_monitor_v2.py(监控雏形 v0.2)

自动价差源:Token Service <https://li.quest/v1/tokens?chains=...> 返回各链 token priceUSD → 同币两链价差 bps = `|papb|/pax10000`, 自动喂给 `lifi_cost_model.verdict()` 三档判定——v0.1 需要人肉传 --spread 13.6, v0.2 全自动

轻量解析:Token Service 全量响应 ~2.4MB, 脚本只解析需要的 symbol 且只取第一个 verified (canonical), 不整包落盘

复用:与 v0.1 同一套 `lifi_cost_model` (`explicit_costs/breakeven_bps/verdict`), 不重复造轮子

双通道:本机直连失败自动走新加坡服务器中转 (--server 强制), 沿用 v0.1 网络策略

用法:python3 scripts/lifi_monitor_v2.py (默认 4 条 USDC 路径) | --json 留档 | --server 中转

三、真实数据(2026-08-14 23:15 实测, Token Service 拉价)

| 路径 | 价差 bps | 判定 (阈值 50bps) |

| --- | --- | --- |

| ETH→ARB USDC | 8.2 | 没肉(成本 6.1×) |

| BASE→ARB USDC | 2.9 | 没肉(成本 17.5×) |

| POL→BASE USDC | 6.8 | 没肉(成本 7.3×) |

| ETH→BASE USDC | 11.1 | 没肉(成本 4.5×) |

对照 v0.1 (8/13 手动传 spread=13.6) :4 条路径同样全没肉——自动源与手动传参结论一致, 验证了自动价差源的正确性(不是"自动了就有肉", 而是"自动了依然如实说没肉")。

数据源特点(实测发现):

- Token Service 响应巨大(2.4MB)——只拉需要的 chains 参数能缩小到 1.4MB, 但仍要流式过滤, 不能整包进内存
- symbols 参数实测不生效(传了也返回全量)——按 chain 过滤 + 程序内 symbol 匹配是可靠路径
- USDC 在 ETH/ARB 都是 verified + priceUSD≈1.00 (锚定稳定币, 价差天然小); ETH 两链同价 1865.82 (同一 oracle 源)——稳定币/主流币跨链价差本来就极小, v0.2 的自动源适合扫的是小币/长尾资产的瞬时价差

四、方法论沉淀

1. "自动价差源"的三层价值:① 省人工(不用每轮手传 spread) ② 可定时(cron 每 N 分钟轮询, 价差突破阈值才告警) ③ 留档可回溯(--json 输出 → 事后验证"当时该不该做")
2. 监控雏形的完整形态:数据源(Token Service/链上价格) → 价差计算(纯函数) → 成本模型(break-even) → 判定(三档) → 告警(有肉才推)——v0.2 完成了前三层, 告警层是 v0.3
3. "没肉"是常态, 不是失败:信号系统的价值 = 用一致口径持续输出"现在能不能做"的判断, 多数时候答案是"不能"——这恰恰避免了人肉凭感觉追价差的亏损 (Day 4 教训:找摩擦不找价差)

五、卡点 & 待办

[] Dune 基础查询上手(顺延两天了, Week 2 大纲任务)

[] 飞书推送通道: v0.2 判定出"有肉"才推(先手动跑几天确认无噪音)

[] 价差源扩展:Token Service 之外接链上 RPC 直读(小币价差源, Token Service 覆盖不到长尾)

六、明天计划(8/15)

Dune 基础查询上手(Week 2 大纲必做, 不再顺延)

v0.2 挂 cron 定时轮询(先观察 24h 无噪音, 再考虑推送)

参考链接

- Day 9 笔记: [notes/2026-08-13.md](https://intensivecolearn.ing/notes/2026-08-13.md)(v0.1 落地)
- 成本模型: `scripts/lifi_cost_model.py`(Day 3 六件套函数化)
- LI.FI Token Service 文档: <https://docs.li.fi> (tokens 端点)

笔记 082: 链上套利残酷共学 | 2026-08-14 打卡

作者: koyo922 | 时间: 2026-08-14 15:34 UTC | 字数: 727 | 评分: 68

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 | 2026-08-14 打卡

一、学习记录

今天主要围绕 sNUSD / Morpho 交易案例进行分析。

最初我主要关注闪电贷、金库、借贷和还款等交易步骤，容易停留在资金流层面。结合群友的进一步分析后，开始注意到：

金库捐赠可能改变资产记账和份额价格；
sNUSD 的预言机估值与二级市场实际成交价可能存在偏差；
supply 1000、抵押、借款等动作，需要放回完整交易路径中理解；
预言机价格不等于可以直接卖出的市场价格；
交易显示出收益，不代表其中所有金额都是最终可实现利润。

目前已经通过调用栈和资产流向进行了部分核对，但案例中的若干关键机制仍需要继续确认，暂时不能简单下结论。

二、阶段性总结

今天最大的收获，不是发现了一个可以直接复制的套利策略，而是发现了自己的分析方法仍然不够完整。

分析复杂 DeFi 套利时，不能只看：

协议名称；
Transfer 记录；
表面的资金进出；
最终显示的收益。

还需要逐项核对：

角色、资产、价格、动作、余额和最终偿还闭环。

另外，群友的高质量分析也提供了一个值得研究的方向：反向拆解他们是如何从零散信息中识别关键机制、排除错误解释并建立证据链的。这个过程本身也可以用来训练和改进 Hermes 的分析能力。

三、明日计划

1. 继续把 sNUSD / Morpho 案例彻底拆清楚。
2. 区分已经被交易数据证明的事实、合理推断和仍待验证的部分。
3. 对比群友的分析过程与我自己的分析过程，总结更高质量的链上研究方法。
4. 在案例完全搞清楚后，再整理成更成熟的复盘和方法论。

笔记 083: Day 11 · 先把“自动挖掘策略”做成闭环，而不是再堆一张卡

作者: 99hansling | 时间: 2026-08-15 06:10 UTC | 字数: 963 | 评分: 71

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 · 先把“自动挖掘策略”做成闭环，而不是再堆一张卡

今天校正了什么

今天最重要的不是又找到一个看起来有机会的策略，而是把项目的主问题说清：现在我们有一套能约束研究的流程，但还没有一个真正能自动挖掘策略的 Agent。

此前的真值表、证据文件、预注册和人工闸门已经能防止不少错误：例如不能把一条静态 AMM 负收益路线换个链名重跑，也不能让未冻结的借贷计划直接去采样。但“让 Agent 自动发现不同机制”还缺一条完整、可验证的链。

现在到底缺什么

一条可用的 Agent V0 应该是：

text

冻结的 Work Order

- 直接来源的 Domain Pack
- 结构化 Plan 卡（事实 / 推论 / 假设分开）
- 独立 verifier
- 才能更新根项目的策略真值表

目前我们只完成了第一个基础：Stage 0 Work Order 会拒绝夹带采样、Trial、RPC、调度或执行权限。还没有真正的 Domain Pack、Plan-card verifier，以及 verifier 通过后受控写回真值表的入口。

今天的真实验证

尝试把一张 Pendle PT 的“只找直接来源和最小反驳”任务交给 GLM。网络出口检查通过，但执行器在第一轮前卡在全球 Skill 加载：一边是 Skill 扫描超过遍历上限，另一边是一个全局 Skill 的格式不完整。

因此这次没有产生结论、没有改真值表、没有采样，更没有交易。我们把它如实记为“执行器没有完成任务”，而不是把启动日志算成的发现成果。

接下来的正确顺序

先做一个真实的 Agent VO tracer：用一个不需要账户、API key 或交易权限的只读 Domain Pack，让它从冻结输入走到可机器检查的 Plan 卡；校验通过后才允许根项目更新对应行。这样以后无论由 GLM、Luna 还是其他模型执行，模型只能在同一条可检查的轨道里扩大搜索。

当前仍然没有 validation_active，重型验证槽位是 0/1。今天的结论很朴素：先证明 Agent 能可靠地提出和反驳一个机制，再让它去扩大策略数量。

笔记 084: Day 11 - Hermes Agent 入门

作者：MountainE | 时间：2026-08-15 10:38 UTC | 字数：599 | 评分：56
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 - Hermes Agent 入门

今天尝试把一个研究问题拆成查询、验证、记录三个步骤，为后续使用 Hermes Agent 或其他工具建立可重复的研究流程。示例问题是：同一条跨链路由在不同时间的输出、成本和失败风险如何变化。

我的三步法是：

1. 查询：固定来源链、目标链、资产、金额和路由条件，记录查询时间和报价。
2. 验证：核对数据来源、报价时间、路由步骤、费用、预计时间和失败条件，不把单次屏幕报价直接当成可执行结论。
3. 记录：以结构化方式保存输入、输出、证据、成本、判断和下一步行动。

我产出了一个可复用的研究 Prompt 模板：

text

你是一名严谨的链上流动性研究助理。

研究问题：{问题}

固定条件：{来源链、目标链、资产、金额、时间范围}

请按以下步骤执行：

1. 查询：获取与问题相关的原始数据。
 2. 验证：标注来源、时间、限制和不确定性，分开事实与推断。
 3. 记录：输出表格，包含条件、数据、成本、风险、结论和下一步。
- 要求：不补写缺失数据，不把名义价差当成净收益，不在没有独立验证时建议使用真实资金。

今日结论：一个好的 Agent

研究任务需要明确输入、验证标准和输出格式，这样一次性的观察才能变成可复核、可对比的研究记录。

明日计划：设计一次重复查询，明确输入、工具、输出和判断条件，并标记需要人工复核的结果。

笔记 085: 第10天我写了一个LI.FI只读报价监控器，并连续采集了5次：

作者：rakisa | 时间：2026-08-15 13:22 UTC | 字数：8707 | 评分：90
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第10天我写了一个LI.FI只读报价监控器，并连续采集了5次：

text

Base USDC → Arbitrum USDC

输入：1,000 USDC

预计到账：997.5 USDC

路线：5次均为Eco

昨天5次报价的经济结果完全相同，但API延迟从653毫秒到4,882毫秒不等。

由此产生了一个新问题：

如果连续报价的到账金额相同，是否可以认为路线已经稳定？

今天我扩大短时样本，并分别检查：

1. 到账金额有没有变化；
2. 路线有没有变化；
3. Gas估算有没有变化；
4. 预计执行时间有没有变化；
5. API响应速度有没有变化；
6. 连续观察多久后，才能暂时确认一种状态；
7. 已确认状态遇到异常后是否应该撤销。

三、今天的实验设置

为了和第10天保持可比较性，今天没有更换路线和金额。

text

接口：LI.FI Quote API

方向：Base USDC → Arbitrum USDC

输入：1,000 USDC

排序：CHEAPEST

预设：stablecoin

滑点参数：0.5%

样本数：12

每次响应后的等待间隔：3秒

最大重试次数：1

单次超时：30秒

为什么是12次

5次样本很难看到路线切换，也不足以计算有意义的中位数和高分位延迟。

12次仍然是小样本，但至少可以：

检查连续状态；

观察是否出现偶发路线；

计算中位数；

计算最近秩P90和P95；

测试“连续3次确认”规则；

验证状态确认后能否被异常撤销。

这不是为了统计LI.FI的长期服务水平，也不能代表全天或长期路线分布。

四、安全边界

本次实验继续保持只读：

不连接钱包；

不读取钱包余额；

不需要私钥；

不需要助记词；

不进行Token授权；

不签名；

不广播交易；

使用公开销毁地址作为报价占位地址；

不根据结果执行任何链上动作。

因此，今天观察到的是报价接口状态，不是实际成交结果。

五、实际采样过程

本轮采样开始于：

text

UTC：2026-08-15 13:00:53.678

北京时间：2026-08-15 21:00:53.678

最后一条响应完成于：

text

UTC：2026-08-15 13:02:20.809

北京时间：2026-08-15 21:02:20.809

实际观察窗口：

text

87.131秒

虽然请求之间设置的是“响应后等待3秒”，总窗口并不是简单的11×3秒。

原因是每次API请求本身还会消耗时间，尤其第10条请求用了18.943秒。

真实时间线是：

text

发送请求

→ 等待API响应

→ 保存结果

→ 等待3秒

→ 下一次请求

因此采样间隔必须和API延迟分开记录。

六、12条真实报价结果

样本	HTTP	API延迟	预计到账	费用估值	Gas估算	工具	预计执行时间
1	200	8,795 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
2	200	2,123 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
3	200	1,966 ms	997.5 USDC	\$2.5084	\$0.0081	Polymer Standard	1,080秒
4	200	5,363 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
5	200	2,117 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
6	200	2,082 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
7	200	1,039 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
8	200	3,708 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
9	200	4,734 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
10	200	18,943 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
11	200	910 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒
12	200	1,907 ms	997.5 USDC	\$2.5084	\$0.0065	Eco	7秒

本轮：

text

请求：12

成功：12

失败：0

HTTP 200：12

重试：0

成功率：100%

七、第一个发现：到账金额完全没变

12次报价的预计到账和最低到账均为：

text

997.5 USDC

因此：

text

最小预计到账：997.5 USDC

最大预计到账：997.5 USDC

到账金额变化：0

如果监控器只保存toAmount，这12条数据会被理解成“完全稳定”。

但这个结论只对金额字段成立。

八、第二个发现：第3条路线突然变化

样本1和2使用：

text

protocol:feeCollection → cross:eco

样本3切换为：

text

protocol:feeCollection → cross:polymerStandard

样本4又切回Eco。

路线时间线可以简化为：

text

样本1：Eco

样本2：Eco

样本3：Polymer Standard

样本4：Eco

样本5：Eco

样本6：Eco

样本7：Eco

样本8：Eco

样本9：Eco

样本10：Eco

样本11：Eco

样本12：Eco

因此发生了两次路线切换：

text

样本3：Eco → Polymer Standard

样本4：Polymer Standard → Eco

路线分布：

| 工具 | 次数 | 占比 |

| :----- | -: | ----: |

| Eco | 11 | 91.67% |

| Polymer Standard | 1 | 8.33% |

最长连续路线是样本4至样本12的Eco，共9条。

这只能描述本次87.131秒窗口，不能外推为长期占比。

九、第三个发现：相同到账量隐藏了巨大的时间差

Eco样本的预计执行时间为：

text

7秒

Polymer Standard样本的预计执行时间为：

text

1,080秒

两者相差：

text

$1,080 - 7 = 1,073$ 秒

最大值约为最小值的：

text

$1,080 / 7 \approx 154.29$ 倍

但两条路线的预计到到账量都是997.5 USDC。

这说明：

只按到账金额评价CHEAPEST结果，会把“7秒”和“1,080秒”当成同一种报价。

对于普通跨链转账，用户可能愿意等待；对于套利研究，18分钟左右的时间暴露会让回程价格、Gas和路线发生更多变化。

需要注意，1,080秒是接口估计值，不是本次真实跨链到账测量。本次没有发送交易。

十、第四个发现：Gas变化很小，时间变化很大

本轮Gas估算范围：

text

最低：\$0.0065

最高：\$0.0081

差值：\$0.0016

Polymer Standard样本相对Eco多出的Gas估算只有约0.0016美元，但预计执行时间从7秒增加到1,080秒。

这说明路线比较不能只做：

text

到账金额 - Gas

还要单独加入：

最大允许执行时间；

路线白名单或风险等级；

报价出现后的持续性；

等待期间价格风险；

非原子两腿的时间暴露；

目标链库存条件。

十一、第五个发现：平均延迟被极慢样本明显拉高

12次API延迟按从小到大排序：

text

910

1,039

1,907

1,966

2,082

2,117

2,123

3,708

4,734

5,363

8,795

18,943 毫秒

统计结果：

text

最短：910 ms

最长：18,943 ms

平均：4,473.92 ms

中位数：2,120 ms

最近秩P90：8,795 ms

最近秩P95：18,943 ms

超过5秒：3条

超过10秒：1条

平均值约4.47秒，但一半样本不超过2.12秒。

平均值较高的主要原因是第10条18.943秒的慢请求。

为什么同时记录中位数和P95

中位数更接近本轮“典型”响应水平，不容易被一个极慢样本完全拉偏。

P95用来观察尾部延迟。在只有12条样本时，最近秩P95会落在最大值18,943毫秒上，因此它很不稳定，不能当作长期服务指标。

今天计算P95是为了把分析流程建立起来，不是为了给LI.FI下长期性能结论。

十二、价格、路线和网络延迟是三条不同的序列

这12条数据可以拆成三层：

金额层

text

997.5 → 997.5 → 997.5 → ...

没有变化。

路线层

text

Eco → Eco → Polymer → Eco → ...

发生2次切换。

API延迟层

text

8.795s → 2.123s → 1.966s → 5.363s → ... → 18.943s

波动明显。

如果Agent只有金额字段，它会认为整个窗口没有任何变化。

如果Agent只看报价ID，它又会认为12次都是全新的机会。

正确的做法应该是：

text

金额变化单独判断

路线变化单独判断

API质量单独判断

十三、我设计了“连续3次确认”规则

为了避免一次偶发报价立即被当成稳定状态，我加入一条研究规则：

text

连续3次经济指纹相同

并且每次API延迟不超过5,000毫秒

才标记为稳定观察

这两个阈值是什么性质

text

连续3次：研究假设

5,000毫秒：研究假设

它们不是LIFI官方标准，也不是经过历史回测得到的最优参数。

今天的目的只是验证状态机逻辑，未来必须通过更多数据调整。

十四、稳定观察为什么直到第7条才确认

逐条应用规则：

样本1

text

Eco

延迟8,795 ms > 5,000 ms

不合格，连续计数归零

样本2

text

Eco

延迟2,123 ms

连续合格：1

样本3

text

Polymer Standard

路线和经济指纹改变

连续状态切换为Polymer

连续合格：1

样本4

text

Eco

延迟5,363 ms > 5,000 ms

不合格，连续计数归零

样本5

text

Eco

延迟2,117 ms

连续合格：1

样本6

text

Eco

延迟2,082 ms

连续合格：2

样本7

text

Eco

延迟1,039 ms

连续合格：3

确认一次Eco稳定观察

因此确认事件发生在样本7，而不是样本1。

十五、为什么第10条又撤销确认

样本7确认以后，样本8和9继续满足条件。

但样本10的API延迟为：

text

18,943 ms

超过5,000毫秒研究阈值，因此程序执行：

text

撤销当前稳定确认

连续计数归零

样本11和12虽然恢复为快速Eco报价，但只有连续2条：

text

样本11：连续1

样本12：连续2

实验结束时没有达到重新确认所需的3条。

最终状态：

text

历史确认次数：1

撤销次数：1

结束时是否仍确认：否

这一步很重要。否则系统会在样本7确认一次后永久保持“稳定”，忽略后续异常。

十六、确认稳定观察不等于发现套利

即使某条单向路线连续3次完全相同，也只说明：

text

在当前研究规则下

重复观察到了相同单向报价状态

它不证明：

回程报价存在；

往返净收益为正；

目标链有可用库存；

两条腿可以原子执行；

报价在真实执行时仍有效；

实际到账时间符合估计；

交易可以成功成交。

因此程序明确保存：

text
stableObservationConfirmed：曾经为true
arbitrageSignalGenerated：false
transactionBroadcast：false
十七、和第10天的结果对比
指标	Day 10	Day 11
成功样本	5	12
短时观察窗口	约20秒	87.131秒
工具数量	1	2
路线切换	0	2
预计到账范围	997.5~997.5	997.5~997.5
执行时间范围	7~7秒	7~1,080秒
API延迟范围	653~4,882 ms	910~18,943 ms
平均API延迟	2,392.4 ms	4,473.92 ms

两天不是同一时间窗口，不能把差异简单解释成市场趋势。
这个对比只能说明：样本增加后，我观察到了昨天没有出现的路线和尾部延迟。

十八、今天建立的分析指标

今天的分析器会自动输出：

数据完整性
text
请求数量
成功数量
失败数量
唯一请求指纹数量
观察窗口长度

延迟指标
text
平均值
中位数
P90
P95
最小值
最大值
超过5秒数量
超过10秒数量

路线指标
text
工具数量
每种工具出现次数
路线切换次数
经济状态切换次数
最长连续路线

报价指标
text
到账金额范围
Gas范围
预计执行时间范围
状态机指标

text
连续确认次数
确认发生样本
撤销次数

撤销原因

结束时是否仍确认

是否生成套利信号

十九、测试与验证

今天编写了50项自动化断言，覆盖：

中位数计算；

最近秩分位数计算；

样本数量；

成功率；

唯一请求指纹；

观察窗口；

延迟平均数、中位数、P90和P95；

慢请求计数；

到账金额范围；

Gas范围；

执行时间范围；

路线工具数量；

Eco和Polymer出现次数；

路线切换次数；

最长连续路线；

第7条确认；

第10条撤销；

结束时未重新确认；

钱包未连接；

未签名；

未广播交易；

合成慢请求是否正确打断连续计数。

结果：

text

Day11路线稳定性分析50项断言全部通过

二十、今天产出的文件

text

stability_analyzer.mjs

stability_analyzer.test.mjs

README.md

data/collector/raw/<run-id>.jsonl

data/collector/results/monitor_summary.json

data/collector/results/monitor_samples.csv

data/collector/quote_monitor.sqlite

data/results/stability_analysis.json

data/results/stability_timeline.csv

分析器运行命令：

bash

node stability_analyzer.mjs

测试命令：

bash

node stability_analyzer.test.mjs

二十一、今天最重要的认识

昨天我学到：

text

新quoteId不等于新市场状态

今天进一步发现：

text

到账金额不变也不等于路线稳定

在本次真实样本中，12次预计到账都为997.5 USDC，但其中一次：

text

路线从Eco切换为Polymer Standard

Gas估算从\$0.0065变为\$0.0081

预计执行时间从7秒变为1,080秒

如果只看最终金额，这次154倍以上的预计时间差会完全消失。

因此，一个链上套利监控器至少应该把以下变化分开：

text

金额变化

路线变化

执行时间变化

Gas变化

API延迟变化

采集成功状态变化

二十二、本次实验的限制

1. 只有12个样本；
2. 观察窗口只有87.131秒；
3. 只观察Base→Arbitrum一个方向；
4. 只测试1,000 USDC一个金额；
5. 每次请求是顺序执行，不是同时快照；
6. P90和P95在小样本下非常不稳定；
7. 5秒延迟阈值是研究假设；
8. 连续3次确认是研究假设；
9. 没有获取回程报价；
10. 没有连接钱包或检查库存；
11. 没有测量实际跨链到账；
12. 没有验证报价过期时间；
13. 没有执行交易；
14. 不能据此判断长期路线分布；
15. 不能据此认为存在可执行套利。

二十三、下一步计划

第12天准备做“金额规模与路线稳定性”实验：

text

100 USDC

1,000 USDC

10,000 USDC

对每个金额采集多次，比较：

1. 不同金额是否选择不同桥；
2. 到账比例是否随规模变化；
3. Gas占本金比例如何变化；
4. 预计执行时间是否随金额变化；
5. 哪个金额更容易出现路线切换；
6. 相同金额连续确认规则能否通过；
7. 是否仍然只有观察信号而没有套利信号。

继续保持只读，不连接钱包、不签名、不广播交易。

今日结论

今天完成了12次真实LI.FI只读报价采样和路线稳定性分析。

12次请求全部成功，预计到账始终为997.5

USDC，但第3条路线从Eco短暂切换到Polymer

Standard，使预计执行时间从7秒增加到1,080秒。整个窗口发生2次路线切换，API延迟中位数为2,120毫秒，P95为18,943毫秒。

研究规则在第7条确认Eco连续稳定观察，但第10条18.943秒慢响应触发撤销；到实验结束只有2条连续合格样本，没有重新确认。

本次没有生成套利信号。它证明的是：金额不变的报价背后，路线、时间和网络状态仍然可能发生重要变化。

笔记 086: 今日学习心得：完成三条 USDT 跨链 Route 的真实 Quote 对比

作者：Chic | 时间：2026-08-15 14:12 UTC | 字数：1373 | 评分：80
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习心得：完成三条 USDT 跨链 Route 的真实 Quote 对比

今天继续做真实 Quote 填表练习，主要使用 Jumper 观察 USDT 从不同链跨到 Optimism 的路线，并分别记录 100、1000、5000 USDT 三档金额的数据。今天完成了三条初始路线的对比：Ethereum -> Optimism、Arbitrum -> Optimism、Base -> Optimism。

首先复盘昨天的 Ethereum USDT -> Optimism aOptUSDT 路线。三档金额的网络费用都很低，价格冲击大约为 -0.28%，但目标资产显示为 aOptUSDT，而不是普通 USDT，因此需要核对资产版本。虽然预计收到数量接近投入金额，但用 toAmountMin 计算后，三档保守净收益均为负，因此结论是不执行，仅记录观察。

今天重点观察了 Arbitrum -> Optimism 路线。这里发现一个很重要的现象：L2 -> L2 的网络费用确实很低，大约 \$0.03，但网络费用低并不代表路线一定好。100 USDT 档的价格冲击高达 -33.07%，Exchange rate 只有约 0.669，说明小额 Route 非常异常，可能和路线、资产版本、流动性或报价机制有关。1000 和 5000 档价格冲击明显降低，但保守净收益仍然为负。同时，源资产和目标资产分别显示为 aArbUSDT 和 aOptUSDT，都需要核对资产版本。

接着观察了 Base -> Optimism 路线。相比前两条，Base -> Optimism 的表现最稳定：网络费用低于 \$0.01，价格冲击稳定在 -0.12%，三档执行时间都很短，大约 2-3 秒，亏损比例也比较接近。虽然三档保守净收益仍然为负，但从稳定性来看，Base -> Optimism 是目前三条路线中最值得继续观察的一条。

今天最大的收获是，我不再简单认为“L2 -> L2 一定更便宜更好”。实际观察发现，网络费用只是成本的一部分，还要看价格冲击、toAmountMin、Exchange rate、执行时间和目标资产版本。Arbitrum -> Optimism 虽然网络费用低，但 100 档价格冲击非常异常；Base -> Optimism 则更稳定，但目标资产仍是 aOptUSDT，不能直接当作普通 USDT 使用。

目前三条路线的初步判断是：

- 1. Ethereum -> Optimism：主流路线，但价格冲击相对更高，目标资产版本需核对。
- 2. Arbitrum -> Optimism：100 档异常，不适合小额；1000/5000 档较正常，但仍不执行。
- 3. Base -> Optimism：三档表现最稳定，网络费用低、价格冲击低、时间短，值得继续观察 3 天。

今天进一步强化了一个原则：真实 Quote 观察不是看“预计收到多少”就下结论，而是要优先看 toAmountMin，并结合网络费用、价格冲击、目标资产版本和执行时间一起判断。当前所有路线都只做记录，不执行。下一步可以继续连续观察 Base -> Optimism 这条路线，确认它的 Route、toAmountMin 和资产版本是否稳定。

笔记 087: 今天进入 TGE

多链资产模块，主题是“多链发行模型与首发合约发现”。我先区分了原生多链独立发行、锁

作者：Flxxx | 时间：2026-08-15 14:41 UTC | 字数：1642 | 评分：86
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天进入 TGE 多链资产模块，主题是“多链发行模型与首发合约发现”。我先区分了原生多链独立发行、锁定/铸造、销毁/铸造、第三方包装、同名无映射五种模型。最重要的规则是：同项目、同 symbol、同 decimals，甚至不同链出现同一个十六进制地址，都不能单独证明它们是同一经济资产。

我用 ZRO 做了完整的只读案例。LayerZero 官方 OFT 列表把同一合约地址列在 Ethereum、Arbitrum、Optimism 和 Base，localDecimals=18、sharedDecimals=6。这属于很强的官方发现线索，但还不是完整身份证明。

随后复核了创建交易。Ethereum 实例在 2024-04-17 创建，并向 Safe 地址初始铸造 10 亿 ZRO；Base 实例在 2024-06-12 创建。两条链由同一地址部署，但时间相隔约 56 天，说明“多链发行”不能压缩成一个部署时刻。Ethereum 合约的 owner() 当前返回一个被 Etherscan 标记为 LayerZero 多签钱包的 Safe；多签阈值、模块和 guard 仍未核对。

我还在浏览器的 Read Contract 中手动查询了双向 Peer：Ethereum peers(30184) 指向 Base ZRO，Base peers(30101) 指向 Ethereum ZRO。这里的 30184 和 30101 是 LayerZero EID，不是 EVM Chain ID。

最后核对了一条真实的 Base -> Arbitrum OFT 消息：

text
GUID: 0x0b8bb6250c4e85a3ccad4cb2d181a4cb61b39a9eda7c2b962263d46c0f02725f
源交易: 0x881ac53e205ab72dbf2c81487299c62af10ca177d200ef4c96aac97a433b3dc3
目标交易: 0x4ca0c01db520c58c15a78665656f64c305cfab2ccb007ba31d48f7bc8bc1fc91
源链: user -> zero，销毁 9.818632 ZRO

目标链: zero -> same user , 铸造 9.818632 ZRO

可见时间差: 28 秒

两端 GUID、用户地址和数量一致, 支持该时点完成了一次 burn/mint 闭环; 消息状态为 delivered, 两项要求的 DVN 验证均成功。但历史成功不等于当前所有路径可用, 更不等于存在套利机会。

今天建立的判断顺序是:

text

官方列表只负责发现候选

-> 创建交易与部署者确认来源

-> 代码和权限确认控制关系

-> 双向 Peer 确认映射

-> 实际消息确认历史闭环

-> 当前路径、DEX 深度和全成本确认可执行性

当前状态仍是 ONCHAIN_IDENTITY_PARTIAL, 未验证项包括 Safe 完整权限、其余链部署、Ethereum <-> Base

当前消息路径, 以及各链 DEX 的目标规模报价。Day 11 将继续做官方合约、代理、Admin、Minter/Burner 和身份冲突的 fail-closed 清单。

今日产出:

Day10 任务: 每日任务/Day10-多链发行模型与首发发现.md (../每日任务/Day10-多链发行模型与首发发现.md)

Day10 知识库: 知识库/每日/Day10-多链发行模型与首发发现.md (../知识库/每日/Day10-多链发行模型与首发发现.md)

ZRO 案例: 研究记录/2026-08-15-Day10-ZRO-多链发行初步发现.md (2026-08-15-Day10-ZRO-多链发行初步发现.md)

07 实盘与数据 (16 篇)

笔记 088: 学习笔记: 人工平仓功能设计思路 (保持架构简单)

作者: wyy2221 | 时间: 2026-08-14 15:34 UTC | 字数: 2760 | 评分: 83

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习笔记: 人工平仓功能设计思路 (保持架构简单)

核心思想: 不要开发一套新的人工交易系统, 而是在现有交易系统上增加一个“人工退出意图”。

一、为什么需要人工平仓

当前程序存在一个实际问题:

如果直接在交易所手工平掉仓位:

交易所仓位 = 0

SQLite仍然记录:

Layer1 OPEN

Layer2 OPEN

程序下一次运行会认为:

真实仓位 $\neq$ 数据库账本

于是触发:

LEDGER_REAL_POSITION_MISMATCH

↓

OBSERVE_ONLY

系统进入保护状态。

因此:

真正的问题不是不能手工平仓, 而是机器人不知道这次平仓。

二、正确思路: 人工决定, 机器人执行

不要:

人工去交易所平仓

↓

再修改SQLite

而应该：

人工告诉机器人：

"现在全部平仓"

↓

机器人继续走自己原来的EXIT流程

↓

机器人自己更新SQLite

这样：

交易所仓位；

SQLite账本；

EXIT事务；

成交记录；

始终保持一致。

整个系统仍然只有：

一个下单者（机器人）。

这是最重要的设计原则。

三、尽量复用现有能力

好的架构不是不断新增功能，而是：

尽可能复用已有模块。

人工平仓应继续使用：

普通 EXIT；

双腿提交；

reduce-only；

IOC；

Coordinator；

A26/A36 恢复；

成交确认；

SQLite 正常关层。

不要重新开发：

第二套下单器；

第二套恢复逻辑；

第二套账本。

四、用 events 表，而不是修改数据库结构

讨论过程中有两个方案：

方案一：

runtime_state

manual_exit_requested

方案二：

直接复用：

events

新增两个事件：

OPERATOR_MANUAL_EXIT_ALL_REQUESTED

OPERATOR_MANUAL_EXIT_ALL_COMPLETED

最终选择第二种。

原因：

不升级 SCHEMA_VERSION；

不增加数据库字段；

A36 可以继续打开数据库；

回滚简单；
events 本身就是事件流。

这是典型的：

利用已有结构，而不是增加新结构。

五、CLI只负责发出意图

CLI不能直接：

登录交易所；
创建SDK；
下单；
修改SQLite。

CLI唯一职责就是：

验证

↓

写入REQUESTED事件

↓

退出

真正执行交易的：

始终是唯一运行中的机器人。

这样可以避免：

双重下单；
nonce 冲突；
数据不同步。

六、人工平仓不是无条件强平

人工退出只代表：

操作员接受当前盈亏。

因此只绕过：

minimum_hold_ms；
required_net_profit_bps；
exit_persistence_ms。

继续保留：

行情新鲜度；
深度检查；
HTTP主动确认；
最终漂移保护；
reduce-only；
IOC；
仓位一致；
成交证明；
SQLite正常关层。

所以：

它不是：

看到命令立刻市价清仓

而是：

看到命令以后

↓

立即进入安全退出流程

安全原则仍然不变。

七、单腿恢复继续复用现有逻辑

人工退出过程中仍可能：

一腿成交

另一腿失败

这时候：

不要开发新的恢复系统。

直接进入：

现有 EXIT Emergency Recovery

即可。

区别只是：

普通 EXIT：

可能等待利润恢复。

人工 EXIT：

直接进入已有 Emergency Recovery。

恢复算法完全不需要重写。

八、整个流程保持非常简单

最终流程可以概括成：

REDUCE_ONLY

↓

manual-exit-all

↓

events写REQUESTED

↓

机器人检测REQUESTED

↓

逐层EXIT

↓

全部关闭

↓

写COMPLETED

↓

保持REDUCE_ONLY

整个过程中：

没有：

新线程；

新服务；

新状态机；

新数据库结构；

新SDK；

新缓存。

九、设计过程中得到的几个经验

1、优先复用已有模块

如果一个功能能够建立在：

EXIT

↓

Recovery

↓

Coordinator

之上，就不要重新开发。

2、不要为了一个小功能修改数据库结构

如果：

events

已经可以表达：

REQUESTED

COMPLETED

就不要：

新增表；

新增字段；

升级 Schema。

数据库结构越稳定，维护越容易。

3、控制权始终只有一个

整个系统应该始终保持：

人工负责决策

机器人负责执行

不要出现：

人工也下单

机器人也下单

否则：

交易逻辑会越来越复杂。

4、小功能也要保持账本闭环

人工平仓最重要的不是：

能不能平仓。

而是：

平仓以后程序还能不能证明整个交易过程。

因此：

EXIT事务；

成交；

仓位；

SQLite；

必须继续正常关闭。

不要为了方便：

直接修改SQLite。

总结

A37 最值得学习的地方，并不是新增了一个“人工平仓”功能，而是坚持了一个简单的架构原则：

新增的是“人工退出意图”，不是“第二套交易系统”。

整个方案几乎完全复用现有 EXIT 流程，只增加一个 CLI 命令和两个事件类型，让机器人继续完成所有交易、恢复和账本管理工作。这种设计既解决了人工止损需求，又保持了系统的一致性和可维护性，是“小功能、最小改动、最大复用”的典型实现思路。

笔记 089: 【Day 14 打卡 · 2026-08-14】Paper Trading 模拟验证

作者：nerd614 | 时间：2026-08-14 15:38 UTC | 字数：459 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 14 打卡 · 2026-08-14】Paper Trading 模拟验证

延续「跟数据走」的节奏，今天把 Day 13 筛选出的候选验证清单推进到模拟验证阶段——不上真金，先用 Paper Trading 验证执行逻辑与成本模型是否站得住。

1. 模拟框架：为每条候选路径建立模拟交易记录，模拟从发起、桥接、兑换到到账的完整执行流程，按时间顺序记录每个环节的 Gas、桥费、滑点与延迟。

2. 成本实测：用真实报价接口在模拟时点反复取价，对比「报价时的预期成本」与「实际执行时段的成本」，量化执行延迟带来的偏差，回填进 Day 9 的成本模型。
 3. 命中率评估：统计模拟期间信号触发次数、成功执行次数、净收益为正的次数，计算候选策略的「触发→执行→盈利」转化率，剔除胜率过低或收益不稳定的候选。
 4. 记录沉淀：把每笔模拟交易的预期 vs 实际、利润分解、失败原因整理成复盘表，作为结课前决策是否上小额实盘的依据。
- 今天聚焦在「不花钱验证逻辑」——用模拟数据把假设钉死，为下一步小额试跑做最后把关。

笔记 090: 资金费率、庄家行为研判与对应套利思路

作者：Qy | 时间：2026-08-14 15:53 UTC | 字数：4391 | 评分：76

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

资金费率、庄家行为研判与对应套利思路

1. 过往量化策略实操难点

此前搭建过资金费率、ROI 相关数据监控与量化策略，但全部停留在模拟盘阶段，无法落地产生实盘收益；仅依靠原始监控数据，难以判断庄家操作意图，缺少标准化分析推演方法。

2. 两类庄家配套套利方式

(1) 前置埋伏低风险套利

庄家操盘行为会在链上留下交易异动痕迹，可在交易对价差趋近于零时提前埋伏，赌后续价差扩大获利。

风险收益优势：若庄家未启动行情，仅会产生少量手续费亏损；一旦庄家拉升、砸盘造成价差拉开，可赚取可观利润。该思路适用于现货行情、正负满资金费率场景。

(2) 跟进已启动操盘行情套利

庄家主流操盘分为两种模式：

①

短期暴力拉升：数分钟内代币涨幅可达

50%-100%，核心目的打爆单向做空用户；若庄家掌握绝大多数现货筹码，极易操控行情，极端操纵行为会被交易所冻结账户资金。

② 长期震荡拉升：多针对存在代币解锁预期的币种，早期投资者会通过开合约对冲代币持仓风险，形成大量套保空单；庄家会持续多日维持正价差、正资金费率，缓慢抬升币价逐步清算空头头寸。

普通套利者参与方式：足额保证金前提下，同步赚取持续资金费率与币价波动价差。

1. 完整庄家行为研判方法

仅依靠链上行情数据判断存在明显局限，需要三层信息交叉验证：链上资金、费率、成交数据；项目基本面，包含代币解锁、空投、早期持仓等信息；行业社群一手资讯。

行业通用做法：加入专业套利社群，同步捕捉市场机会，交流开仓逻辑与盘面判断。

三、闪电贷清算池套利实操内容

1. 清算池闪电贷套利底层逻辑

借贷清算池存在固定资金分配规则，用户存入稳定币后，系统会按预设比例将流动性分流至借贷池子，存款会生成可赎回凭证；池子流动性不足时，系统会自动跨池子调拨资金。

完整套利操作流程：存入资金为池子提供流动性→借出闪电贷资金→赎回个人存入本金→归还闪电贷完成闭环，无需大额自有本金即可参与。

同类历史机会：上半年曾出现 USDT、USDx、DST 稳定币机制漏洞套利窗口，早期无自动化工具竞争，手动操作即可获利。

2. 当前套利市场现状

自动化套利机器人普及后，手动操作完全失去竞争力，套利窗口出现瞬间就会被机器人抢占；项目风控持续升级，多数池子设置资金存入上限，切断套利所需流动性来源。

现阶段可长期监控的机会：借贷协议大额清算事件、小众稳定币临时机制漏洞、项目空投规则 bug。

3. 小众稳定币机会挖掘与监控方法

监控重心放在各类借贷协议清算记录，手动筛查大额清算机会；长期跟踪
等小众稳定币，这类标的单次套利收益偏低，可挂低价委托单被动等待机会。

USDI、AUT

USD

小众稳定币机会挖掘流程：出现清算事件先核查对应抵押代币；追溯代币链上持有人分布、协议铸造底层规则；多数隐藏漏洞、临时机制窗口，只有深度参与项目才能提前捕捉。

相关案例：ESD AI 空投快照提前执行，未深度参与项目的参与者收到无交易流动性的代币，无法用于借贷套利。

4. 链上分析工具选择

自研 DEX 解析工具可直接读取底层合约原始数据；KYBR 等第三方 DEX 工具无法调取底层合约，数据分析存在较大局限性。

四、预言机与流动池价差套利交易解析

1. 套利核心底层逻辑

质押代币存在两套独立定价体系，价格长期偏离形成稳定套利空间：一是预言机定价，质押赎回价值由官方预言机报价计算，部分项目预言机价格长期固定不变；二是 Uniswap V4 流动池定价，仅根据池内两种代币余额实时换算价格。

2. 完整套利操作流程

通过借贷借出大额 USDC，全额存入金库达到存储上限，获取质押凭证代币；在流动池利用两套定价差额兑换 SNUSD；兑换后换回更多基础稳定币，归还借贷本金，剩余差额为套利利润。交易过程会自动生成临时合约存放额外头寸，该头寸后续可持续产生收益。

3. 事件完整时间线

项目方新增异常流动池，池子报价和固定预言机价格出现巨大价差；套利窗口持续一小时，链上产生两百余笔套利交易；项目方及时设置金库存款上限，关闭新增流动性通道；同步发布公告称保证金不足，暂停用户赎回，市场大户八折抛售 SNUSD，套利空间快速消失。

4. AI 交易解析工具对比

Codex、Opera5 等海外模型风控严格，解析大额套利、合约漏洞相关调用栈会直接拒绝服务，国内 IP 无法完成身份验证；Deepseek V4 适配链上复杂交易，可完整解析多层合约调用栈，清晰输出每一步交易金额、兑换比例与完整资金流向；仅依靠转账记录只能获取浅层信息，完整交易调用栈才能还原全部套利逻辑。

五、AI 逆向、链上分析工具选型与多模型协作方案

1. 主流 AI 工具优缺点

Deepseek：合约逆向、链上交易解析风控宽松，是圈内主流使用工具，可作为“白手套”处理高危合约分析任务；
Claude：擅长数据核对、逻辑校验，适合对交易分析结论做二次复核；
Hermes：支持子代理自动调度，可提前配置规则，高危分析任务自动分流至 Deepseek，无需人工切换模型；
海外大模型：安全审核策略严苛，涉及合约逆向、漏洞分析内容会直接拦截，且国内 IP 无验证渠道。

2. 落地多模型协作工作流

在 VS Code 搭建双操作窗口，分别加载数据校验模型与 Deepseek，由 Deepseek 完成合约、交易调用栈解析，输出标准化交接文档；再由复核模型读取文档，校验数据与逻辑；校验不通过则回流至 Deepseek 重新分析，降低单模型出错概率，同时规避单一模型风控拦截。

3. 使用风险提示

高频提交破解、漏洞挖掘、合约逆向类提示词，极易造成账号封禁；单纯解析链上已公开交易数据不属于违规操作，风险更低；批量注册多账号高频调取接口，会触发平台风控批量封禁。

六、CXN DEX 区块抢单套利实操痛点与行业现状

1. 核心竞争难点

区块内套利机会仅第一笔成交交易能够获利，BSC 链依靠交易小费 Tips 排序区块打包顺序；市场大户部署的机器人单笔小费可达 3-4U，小额小费交易会滞后，无法完成套利。
盈利约束：想要提高小费和大户竞争，必须扩大交易本金，依靠大额交易利润覆盖高额手续费，最终形成资金内卷。

2. 行业底层竞争壁垒

BSC 链 90% 以上区块由两家头部区块 Builder 垄断，区块运营方自身部署套利机器人；打包节点会优先执行自有交易，普通套利者天然处于竞争劣势；头部 DEX 热门交易对套利赛道红利基本枯竭，长期投入产出比较低。

3. 优化发展方向

避开头部热门交易对，深耕长尾小众代币、边缘市场；结合跨链套利模式，规避单链资金内卷；提前布局尚未上线的新公链，抢占早期流动性红利。

七、新公链机会、跨链工具与交易资源分享

1. ARK 新公链布局机会

ARK 公链 9 月 16 日主网上线，Li Fi 跨链工具上线首日完成适配；新上线公链普遍流动性匮乏，跨链资金迁移会长期存在价差，早期无头部资金垄断，套利门槛更低。
前期准备工作：提前注册合作平台 API、申请白名单、适配对应链的监控脚本。

2. Li Fi 跨链聚合工具介绍

产品定位：聚合全市场 DEX、Uniswap 类流动池、自有流动性池报价，覆盖多链多数代币交易对；现存问题：基础手续费 25BPS，部分套利团队仅批量查询报价、不在平台成交，账号易被判定为垃圾流量封禁；后续计划推动产品方下调手续费至极低档位，适配自动化套利需求；使用建议：短期调研捕捉机会可直接使用，长期稳定套利可自研交易路由减少手续费损耗。

3. 去中心化订单簿交易公链

产品对标 Hyperliquid，采用链上订单簿撮合模式，面向专业做市商与套利团队；优势是链上交易手续费极低、成交速度远优于常规 EVM 公链；社群专属福利：和项目方达成合作，完成 20 分钟产品体验与反馈访谈可获得 100U 奖励，待确认活动经费后在社群发布报名渠道。

4. 监控硬件与 RPC 节点方案

服务器配置：2.5G 服务器可支撑基础数据监控脚本运行；BI CX API 限流宽松，单次可拉取 500 条参数数据，单日存储数据量 1-2G；RPC 节点：商用 Quicknotes 稳定性更强，自建节点成本可控，但存在安全隐患，可对接渠道争取社群专属额度。

八、后续待执行事项

- 1. 交易材料同步：将 Deepseek 解析的 SNUSD 套利完整调用栈页面、池子历史价格分析文档发送至交流群；
- 2. AI 工具测试：全员测试 Hermes 自动调度模型，配置规则将高危合约分析任务分流至 Deepseek；
- 3. 会议纪要整理：周末完成本次会议脱敏完整文字纪要，内容隐去敏感信息，支持参会者自愿署名，统一发布至社群；
- 4. 资源对接落地：收集整合平台集成表单，统一提交申请 API 白名单、手续费优惠；核实高性能交易链反馈活动经费，确认后开放社群报名；对接 RPC、交易所渠道，为社群争取专属使用额度；
- 5. 长期交流机制：维持每周五固定线上交流，会后沉淀实操交易案例，用于下一次会议集体拆解分析。

九、整体行业总结

- 1. 成熟公链头部 DEX、资金费率套利赛道内卷严重，头部资金、区块节点垄断市场机会，新人盈利难度极大；
- 2. 两大低竞争红利赛道：小众稳定币 / 长尾代币机制漏洞套利、新公链上线初期流动性套利；
- 3. 完整盘面分析框架：链上原始数据、项目基本面、行业社群资讯三层信息交叉验证，才能准确预判庄家操盘行为；
- 4. 最优工具搭配方案：Deepseek 负责合约底层解析，Claude 完成数据复核，Hermes 实现模型自动调度，规避平台风控限制；
- 5. 核心风险提示：链上套利机会大多依托协议机制漏洞，项目方会快速修复风控，窗口期极短，必须搭建自动化监控程序抢占交易先机。

笔记 091: 今天做了什么：D8 回测：链上拉两个历史崩盘窗口的 Aave V3

清算事件——2024-08-05

作者：zenkai | 时间：2026-08-14 16:37 UTC | 字数：409 | 评分：56
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天做了什么：D8 回测：链上拉两个历史崩盘窗口的 Aave V3 清算事件——2024-08-05 日元崩盘 2084 笔（峰值单块 815 笔！）、1011 事件 909 笔，铁证“清算=事件驱动爆发”（平静期 0.4 次/天 vs 崩盘日 2000+）；竞争者 19 vs 27 个；确认门槛是 infra（监控/资金/速度）不是利润大小。

证据在哪：链上 get_logs 原始数据（回测脚本 /tmp/liq_backtest2.py）；机会库/成本模型模板.md、清算监控清单.md（D7 落位）
卡点：清算金额换算遇 RAY 缩放单位 bug（算出的绝对值虚高，真实金额需逐事件校准）——已记入 skill，列为 D9 技术任务。网络当日多次抖动（机场/节点）。

明天任务（D9）：修清算金额换算（用 Aave index 校准）+ 把“事件驱动监控”接入每日流程。

笔记 092: 分享一个闪电贷架构的思路

作者：mOuseliuzi | 时间：2026-08-14 17:19 UTC | 字数：360 | 评分：57
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

分享一个闪电贷架构的思路

完整闪电贷执行闭环和系统架构
机会扫描、成本及期望收益模型
代币、Pool、Factory、Router 白名单机制
Jared 类授权陷阱防护
精确授权、授权归零、金库与执行器隔离
禁止任意 Router 和任意 calldata
链上余额增量与最低利润不变量
20 个“反问自己”的安全审查问题
对抗性测试、主网 fork 测试
历史回放 → 延迟测试 → Paper Trading → 小额实盘流程
熔断、事故响应和 Go/No-Go 标准

当前仍无法完全消除的风险

核心安全结论是：不能证明陌生代币绝对安全，只能通过严格准入、权限隔离、精确授权、余额不变量和损失限额，让恶意输入无法接触利润金库，并把单次事故影响限制在可承受范围内。

笔记 093: 今天继续学习 DEX-CEX 套利，重点了解了模拟交易 (Paper Trading)。

作者：zgos | 时间：2026-08-14 21:07 UTC | 字数：426 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续学习 DEX-CEX 套利，重点了解了模拟交易 (Paper Trading)。

我以前觉得只要计算结果显示有利润，就可以考虑小额实盘。现在发现，即使不投入真实资金，也可以先按照当时的报价，模拟完成 DEX 买入和 Binance 卖出的过程，然后观察最终结果。

模拟时需要记录：

发现机会的时间和价差；

两边的预估成交价格；

Gas、手续费和滑点；

从发现机会到模拟下单用了多久；

两边是否都能成交；

扣除所有成本后的净利润。

今天最大的收获是，模拟交易不仅用来验证能不能赚钱，也能发现整个流程的问题。比如报价已经过期、交易数量超过流动性、利润计算遗漏费用，或者一边能够成交、另一边却没有足够深度。

目前的卡点是模拟结果仍然不能完全代表真实成交，因为真实下单可能影响价格，也可能遇到延迟或失败。

接下来准备连续记录一些模拟机会，不只保留盈利结果，也记录失败原因，看看哪些价差比较稳定，哪些只是瞬间出现的噪声。

笔记 094: Day 10 | 市场身份必须失败关闭

作者：shiyang07ca | 时间：2026-08-15 00:02 UTC | 字数：1102 | 评分：52

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 | 市场身份必须失败关闭

本次实际核对

阅读 Day 10 课程与参考材料：lessons/0009-day10-silent-failures.html、reference/day10-silent-failures.html，并核对 notes/day10-correctness-audit.md、lab/audit_lighter_rwa.py 和 lab/test_audit_lighter_rwa.py。

实际运行 `python3 -m unittest lab.test_audit_lighter_rwa -v`：4/4 通过。这只是现有测试基线，不能说明市场身份缺陷已修复。

实际调用本地 `order_book_detail("XAG")`：返回 `symbol=BRENTIOIL`、`market_id=159`。同次探针中 `WTI` 返回 `WTI/145`，`BRENTIOIL` 返回 `BRENTIOIL/159`。

公开只读 `Lighter` 探针当前返回 `XAG` 的 `symbol=XAG`、`market_id=93`；它仅用于核对外部市场清单，不替代本地回归测试。

发现与当前决定

市场身份不变量应为“请求的 `symbol`、配置的 `symbol`→本地文件→`market_id`，以及响应的 `symbol/market_id` 必须一致”。当前 `XAG` 请求落到 `BRENTIOIL/159`，违反该不变量。若让此结果继续流入研究，候选筛选可能把 `XAG` 当作 `BRENTIOIL`，后续纸上仓位数量和 `PnL` 也会被错误市场污染。

当前决定：HALT。在市场身份回归测试完成前，不在此函数输出进入候选研究或任何仓位/`PnL` 计算。

未完成：由本人完成 `Red` → `Green`

下一步是本人先在 `lab/test_audit_lighter_rwa.py` 写 `XAG` 的市场身份回归测试，并先看到 `Red`：本地没有 `XAG` 快照时必须显式拒绝，错误信息包含请求的 `XAG`；再在 `lab/audit_lighter_rwa.py` 做最小修复，使支持表同时绑定 `symbol`、文件和 `market_id`，并在返回前验证响应身份。随后重新运行同一测试命令得到 `Green`。当前仓库没有这条真实 `Red`→`Green` 证据，因此不声称修复或掌握已经完成。

研究边界

本次完成的是一次缺陷审查与阻断决定；没有采集 `XAG` 历史数据、没有发送订单，也没有证明数据质量、可成交性或策略盈利。

笔记 095: Day 10：两秒内采样双向 LI.FI Quote，闭环后仍亏约 0.5%。

作者：daohuang1982-beep | 时间：2026-08-15 04:34 UTC | 字数：606 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10：两秒内采样双向 LI.FI Quote，闭环后仍亏约 0.5%。

今天沿用 Day 9 的 1,000 USDC 样本，使用公开非个人占位地址，连续请求 Ethereum → Arbitrum 和 Arbitrum → Ethereum 两次只读 Quote。第二腿没有重新假设 1,000 USDC，而是严格使用第一腿的最低输出 997.5 USDC 作为输入。两次采集只间隔约 2 秒。

结果是：第一腿经 Polymer Standard 最低得到 997.5 USDC，第二腿经 Eco 最低回到 995.00625 USDC。仅 Token 数量已减少 4.99375 USDC (约 0.499375%)；两腿 Gas 估算合计约 0.1042 USD。按 Quote 的价格快照，闭环价值差加 Gas 约 5.0963 USD，因此候选应进入 quote_rejected，不能升级为 paper_trade_candidate。

更重要的是，两份 Quote 时间虽只差 2 秒，第一腿预计执行却需要 1,080 秒。真实顺序执行必须等资产到达 Arbitrum，届时当前回程 Quote 已过期，所以这只是静态闭环压力测试，不是同步、原子或可执行套利证明。今天没有保存可执行 calldata，也没有连接钱包、授权、签名、广播或成交；当前没有发现或确认任何可执行套利机会。

笔记 096: 今天进入 Week 4，我给这周定的主题是：

作者：海椰 Bareerah | 时间：2026-08-15 05:20 UTC | 字数：1031 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天进入 Week 4，我给这周定的主题是：

把只读观察升级成一套更稳定的研究系统。

这里的重点不是开始实盘，也不是写自动交易脚本，而是把之前比较手动、比较分散的观察动作，拆成更清楚的几个部分：

监控范围

数据源台账

指标定义

告警规则

历史样本回溯口径

误报和漏报复盘

风险阈值和停止条件

今天最大的变化是，我开始重新理解“监控”和“告警”这两个词。

以前听到告警，很容易联想到：

有机会了，要不要动手？

但现在我会刻意把它改成：

有线索了，要不要记录一条样本？

Week 4 里的告警只允许触发几件事：

记录样本

检查成本

找反证

标注缺失字段

输出判断

它不能触发连接钱包、授权、签名、桥接、下单或任何交易动作。

这条边界对我很重要。

因为链上套利里，很多风险不是出现在“看见价差”的那一秒，而是出现在你把一个提醒误解成执行信号的时候。

一个表面价差可能只是数据延迟。

一个更高输出可能伴随更长等待时间。

一条看起来不错的路由，可能因为 Gas、桥费、滑点、价格冲击或失败成本不完整，最后根本不成立。

一个历史样本看起来很好，也可能只是因为事后用了当时看不到的数据。

所以 Week 4 我想训练的不是“更快发现机会”，而是“更稳定地过滤噪音”。

今天我整理了一套 Week 4 的资料和模板，核心是让每条线索都能被问同样的问题：

数据源可靠吗？

字段完整吗？

时间戳清楚吗？

token 地址和 decimals 确认了吗？

路由是否稳定？

成本有没有缺失？

告警为什么触发？

它后来是合理提醒、误报、漏报，还是合理放弃？

有没有触发硬停止条件？

Hermes 在这一周的角色也继续收窄：它不是交易助手，而是监控复核助手。

我会让它帮我检查：

监控范围是不是太宽

数据源是不是只读

回测有没有用了未来数据

告警规则会不会被误解成交易信号

停止条件是否足够保守

但只要涉及私钥、授权、签名、发交易、自动下单、抢跑、夹击、清算或 MEV 执行，就直接停止。

今天的结论：

链上套利学习到 Week 4，重点已经不是“看到一个机会”，而是建立一套能不断排除误判的研究系统。

告警不是行动按钮。

告警只是提醒我：这里有一条线索，值得记录、核验、找反证，然后再决定继续观察、需要更多证据，还是放弃。

笔记 097: 网格交易可以概括成一句话：

作者：hellingercheri-gif | 时间：2026-08-15 08:54 UTC | 字数：3610 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

网格交易可以概括成一句话：

在预设价格区间内，分层挂出买卖单，通过价格反复波动，机械地执行“跌一格买、涨一格卖”。

它本质上是一种规则化的区间交易，也可以看作简化版的做市策略。自动化只能减少情绪干扰，不能创造必然收益。

1. 基本原理

假设某资产当前价格为 100，设定：

下限：80

上限：120

网格数：8

每格间距：5

价格层级大致为：

80 — 85 — 90 — 95 — 100 — 105 — 110 — 115 — 120

运行逻辑：

1. 价格从100跌到95：买入一份。
2. 价格从95涨回100：卖出这一份。
3. 价格继续上下波动：不断重复买低卖高。
4. 价格跑出80~120：通常停止生成新交易，等待价格回到区间或人工处理。

平台对“网格数”和“价格层级数”的定义可能不同，实际使用时要看具体说明。网格机器人只会根据参数机械执行，并不保证盈利。OK X现货网格说明 (<https://www.okx.com/en-us/help/spot-grid-bot-faq>)

2. 收益来自哪里

总收益不能只看“网格利润”：

总盈亏

= 已实现网格利润

+ 当前持仓浮动盈亏

- 手续费

- 滑点

- 资金费/借贷利息

- 其他成本

例如95买入、100卖出1个单位：

毛利润 = 100 - 95 = 5

实际利润还要扣除买卖两次手续费。

因此最重要的约束是：

每格价差 > 双边手续费 + 滑点 + 其他交易成本

否则机器人可能交易得非常热闹，但净收益很低甚至亏损。频繁交易的成本确实可能将原本盈利的交易变成亏损。Investor.gov交易成本提示 (<https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/foreign>)

3. 两种网格划分方式

等差网格

相邻价格的绝对差相同：

每格价差 = (上限 - 下限) ÷ 网格数

例如：

80、85、90、95、100……

特点：

直观；

适合价格区间相对较窄的情况；

越靠近低价区域，每格百分比收益越高。

等比网格

相邻价格保持相同百分比：

相邻价格倍率 = (上限 ÷ 下限) ^ (1 ÷ 网格数)

特点：

每格百分比基本一致；

更适合跨度较大的价格区间；

在加密资产等波动较大的市场中比较常见。

4. 核心参数

| 参数 | 作用 | 常见问题 |

| :---- | :----- | :----- |

| 标的 | 决定流动性与波动结构 | 小币种可能滑点大、容易单边暴跌 |

| 上下限 | 决定策略有效区间 | 太窄容易出界，太宽资金效率低 |

| 网格数量 | 决定交易频率和单格利润 | 太密可能利润不够覆盖成本 |

| 总资金 | 决定每格下单量 | 需满足平台最小订单要求 |

| 等差/等比 | 决定价格层级分布 | 大区间通常更适合等比 |

| 止损 | 区间逻辑失效时退出 | 没有止损可能长期套牢 |

| 止盈 | 达到目标后结束策略 | 过早止盈可能减少区间收益 |

| 杠杆 | 放大资金利用率 | 同时放大亏损和爆仓风险 |

经验上，增加网格数量意味着交易更频繁、单次利润更薄；减少网格数量则意味着单次利润较厚，但成交机会更少。OKX参数说明 (<https://www.okx.com/en-gb/help/whats-the-spot-grid-bot-and-how-do-i-use-it-sg>)

5. 最适合什么行情

网格交易通常适合：

价格在一个相对稳定的区间震荡；

波动足够产生反复成交；

标的流动性好、买卖价差小；

手续费较低；

没有明显的长期单边方向。

通常不适合：

强单边上涨；

强单边下跌；

即将发生重大事件或突破；

流动性差、容易插针的标的；

使用高杠杆但没有止损的情况。

一个很重要的认识是：

网格赚的是“来回波动”，不是“波动越大越好”。

如果波动大到突破区间，甚至形成持续趋势，网格反而可能遭受较大损失。

6. 现货网格与合约网格

现货网格

价格下跌时逐步买入资产，上涨时逐步卖出。

优点：

通常没有爆仓风险；

结构比较容易理解；

风险相对可控。

风险：

跌破下限后，账户可能持有大量下跌资产；

涨破上限后，资产可能已经被逐步卖掉，错过后续上涨。

所以现货网格并非完全“中性”：价格越跌，持有的标的通常越多；价格越涨，持有的现金通常越多。

合约网格

通过永续合约或期货建立多头、空头或中性网格。

额外风险包括：

杠杆和爆仓；

强制平仓；

资金费率；

保证金不足；

极端行情下止损成交偏离预期。

杠杆会同时放大收益和损失，甚至可能导致损失超过初始投入。Investor.gov杠杆风险说明 (<https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/leveraged-investing-strategies-know-risks-using-these-advanced-investment-tools>)

初学者更适合先理解无杠杆现货网格。

7. 最大的认知陷阱

只看网格利润，不看总盈亏

机器人可能显示网格交易持续盈利，但底层资产已经产生更大的浮亏。

例如：

网格已实现利润：+1,000

持仓浮亏：-4,000

总盈亏：-3,000

认为网格数量越多越好

网格过密会造成：

单格利润太薄；

手续费占比升高；

订单数量和管理复杂度增加；

微小波动带来的利润可能不足以覆盖成本。

把历史回测当成未来收益

回测很容易受到以下因素影响：

区间根据事后走势选择；

假设每个挂单都能完美成交；

忽略手续费、滑点和资金费；

忽略极端行情和交易所故障；

参数对某段历史过度拟合。

没有区间失效条件

设定上下限只是开始，还应提前确定：

跌破多少止损；

突破上限后是否追踪上移；

多久重新评估一次；

什么基本面变化意味着策略失效。

8. 与其他策略的区别

定投：按时间周期买入，主要解决择时问题，通常不主动卖出。

网格：按价格层级买卖，依赖反复震荡。

马丁格尔：亏损后加大仓位，希望一次反弹弥补损失，尾部风险通常更高。

趋势策略：追随突破和方向；网格往往是在区间内逆向交易。

做市策略：网格是其简化形式，但专业做市还会动态管理库存、价差、波动率和订单流。

9. 如何评价一个网格策略

不要只看成交次数或年化收益，应同时观察：

总收益率；

已实现网格利润；

未实现持仓盈亏；

最大回撤；

手续费占毛利润的比例；

资金利用率；

超出区间的次数和时间；

与直接持有标的的相比是否更好；

极端行情下的损失；

合约策略的爆仓距离与资金费。

回测至少要加入真实手续费、滑点、最小下单量和合理的成交规则，并使用样本外数据验证。

10. 一套简单的使用框架

1. 选择流动性好、自己愿意持有的标的。

2. 判断它是否更可能震荡，而不是明显单边。

3. 根据历史波动和支撑阻力确定区间。

4. 确保单格收益明显高于交易成本。

5. 控制投入资金，不使用生活必需资金。

6. 初学阶段避免杠杆。

7. 同时设置止损、止盈和重新评估条件。

8. 用小资金或模拟盘运行一段时间。

9. 始终用“总资产净值”判断盈亏，而不是只看机器人显示的网格利润。

最后可以记住：

网格交易不是自动赚钱机器，而是把“震荡行情中的低买高卖”程序化。它用放弃部分单边趋势收益，换取区间波动中的多次小额利润；真正决定结果的，是区间判断、交易成本、仓位管理和退出纪律。

以上是策略知识总结，不构成针对具体资产的投资建议。

笔记 098: SVR 与 Atlas:从零把这套东西查清楚

作者：starkxun | 时间：2026-08-15 09:54 UTC | 字数：16403 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

SVR 与 Atlas:从零把这套东西查清楚

假设你会看清算合约、懂健康度和清算奖励,但从没听说过 SVR 和 Atlas。\\

这份文档带你把它们从零查明白,每个数字都给出可以自己跑的命令。\\

配套脚本:[svr_probe.py]() (只读,不发任何交易)

第 0 课:为什么要学这个

如果你只知道传统的清算玩法,你脑子里的模型大概是:

预言机推新价格 → 某个仓位健康度跌破 1 → 谁先发交易谁清算 → 赚清算奖励

这个模型在 Aave 的主要市场上已经过时了。

2026 年 3 月起,Aave 在 Arbitrum 和 Base 上把喂价换成了 SVR 喂价,\\

官方数据:89% 的清算奖励被协议回收了,不再归 searcher。

更要命的是:你看不出来。SVR 喂价和普通喂价长得一模一样,\\

你按老办法盯 Chainlink 官方的 ETH/USD 代理,会发现自己盯的是一个\\

Aave 根本不读的地址 —— 这正是我们这个项目最初踩的坑,\n白测了 180 天的数据。

所以这一课的第一个目标就是:学会问协议"你到底读哪个喂价"。

第 1 课:OEV 是什么

先建立概念。MEV 你可能听过,OEV(Oracle Extractable Value)是它的一个子集:

由预言机更新这个动作本身创造出来的、可被提取的价值。

具体到清算:

ETH 从 \$2,000 跌到 \$1,900

↓

预言机把新价格写上链 ← 就是这个瞬间

↓

一批仓位的健康度同时跌破 1

↓

清算奖励(比如债务的 5%)凭空出现,谁抢到归谁

关键点:这个价值不是清算人创造的,是预言机更新创造的。\
清算人只是跑得快而已。

这就带出一个很自然的质疑 ——

既然是预言机更新创造的价值,凭什么全归跑得快的人?\
借出这笔钱、承担坏账风险的是协议啊。

SVR 就是这个质疑的产物。

第 2 课:SVR 的架构 —— 双聚合器

SVR = Smart Value Recapture(智能价值回收),Chainlink + Aave 的联合机制。

以前只有一条路:

Chainlink 节点网络(Data DON)

|

└───→ 公开内存池 ───→ 聚合器合约 ───→ 所有人同时看见

↓

谁快谁抢到清算

SVR 之后是两条路,同一份价格报告发两遍:

Chainlink Data DON

|

└───→ 公开内存池 ───→ 【标准聚合器】(和以前一样,谁都能看)

|

└───→ 私有通道 ───→ 【SVR 聚合器】(先不公开)

↓

searcher 竞价:

「让我的清算跟在这次价格更新后面,
我愿意交出利润的 XX%」

↓

出价最高的赢,他的清算

和价格更新打包进同一个区块

Aave 的合约读的是【SVR 聚合器】。

所以当公开世界看到价格变了的时候,拍卖赢家已经把清算做完了。

兜底机制:如果私有通道失败或超时,SVR 喂价会回落到公开价格\
(有个可配置的延迟)。Aave 文档说这个延迟在 L2 上可以短到 10 秒,\n但他们选择保持和以太坊一致的参数。

这就是漏给普通 searcher 的那部分 —— 也是唯一还能靠老办法抢的部分。

第 3 课:动手 —— 问 Aave 你读哪个喂价

这是整份文档最重要的一个操作。

为什么不能查文档

因为协议随时可以换源,而且文档往往滞后。唯一可靠的是问链上。

调用链

Aave 的价格源要穿三层才能问到:

Pool.ADDRESSES_PROVIDER() → PoolAddressesProvider 地址

.getPriceOracle() → AaveOracle 地址

.getSourceOfAsset(WETH)→ ★ 这才是真正读的喂价

跑一下

```
python3 svr_probe.py feeds --chains ARB OPT
```

实际输出:

ARB

AaveOracle 0xb56c2f0b653b2e0b10c9b928c8580ac5df02c7c7

Aave 读的喂价 0xbd41b1548a5a06544cbcf87c0c54864312842c00

公开 Chainlink 0x639fe6ab55c921f74e7fac1ee960c0b6293ba612

不是同一个 —— 这就是 SVR 喂价

当前 \$1,924.6200 518s 前更新

对照 公开代理 \$1,924.6467 1177s 前更新 → 差 -0.1 bps

OPT

Aave 读的喂价 0x13e3ee699d1909e989722e753853ae30b17e08c5

公开 Chainlink 0x13e3ee699d1909e989722e753853ae30b17e08c5

同一个 —— 这条链没上 SVR

怎么读这个结果

| 现象 | 含义 |

| :----- | :----- |

| Aave 读的 ≠ 公开代理 | 这条链的这个资产走 SVR,清算被拍卖 |

| 两者一致 | 没上 SVR,还是老规矩,拼延迟 |

注意 Arbitrum 那行的更新时间:SVR 518 秒前更新,公开代理 1177 秒前 ——\nSVR 更新更勤。实测中位间隔 SVR 90 秒、公开 120 秒。

可迁移的一条

任何“协议依赖外部数据”的场景,都要问协议本身而不是问数据提供方。\\

我们最初的错误就是从“Chainlink 官方 ETH/USD 代理是哪个”倒推,\\

得到一个 Aave 根本不用的地址,然后在上面测了 180 天。

第 4 课:为什么不能靠接口认出 SVR

我当时的第一反应是:“SVR 肯定有特殊方法,我把两个合约的函数选择器 diff 一下。”

做了,结果是:两个聚合器各 53 个选择器,完全一样。

差异只有字节码长度:

SVR 聚合器 23,186 字节

公开聚合器 22,337 字节

差 +849 字节

一度让我怀疑自己是不是认错了地址。后来 Chainlink 文档给了答案:

"Chainlink SVR Feeds maintain the same aggregator interface and data\\ structure as standard Chainlink Price Feeds."

接口一致是设计如此 —— 这样协议接入时只要改个地址,不用改代码。

可迁移的一条

"我查不出差异"有两种可能:我方法不对,或者设计上就没差异。\\

分清这两种,靠的是找到设计意图的说明,而不是继续加大力气查。\\

我当时差点因为查不出差异而推翻一个正确的结论。

所以判据只能是:Aave 指向的地址 ≠ 公开代理。没有别的办法。

第 5 课:L2 上完全不一样 —— Atlas 登场

以太坊上,SVR 的私有通道用的是 Flashbots MEV-Share。

但 L2 没有公开内存池,也没有 builder 竞价 —— 排序器说了算。 \

所以 Chainlink 换了一套东西。Aave 治理文档写得很清楚:

"Instead of using Flashbots... it uses Chainlinks native infrastructure, \

partially from Atlas, a system acquired by Chainlink from Fastlane... \

settlement/ordering of transactions happens in a smart contract layer now \

(akin to a multical/ERC4337 setup), without the need for a private mempool."

翻译:

以太坊: 拍卖在链下(Flashbots)→ 赢家的 bundle 由 builder 打包

L2: 拍卖在链下,但结算和排序在一个智能合约里完成

这个合约就是 Atlas

怎么找到它(不靠文档)

我是这么找到的:

1. 扫 Aave Pool 的 LiquidationCall 事件
2. 看这些交易的 tx.to 是不是 Aave Pool 本身
3. 发现有一批不是 → 对那个合约调 name()
4. 返回 "Atlas ETH" ← 找到了

跑一下:

```
python3 svr_probe.py atlas
```

```
=== Atlas on ARB: 0x8ad1ae9d97c79aa68a0a151e83ff3942f68f86c1 ===
```

```
name() Atlas ETH
```

```
VERIFICATION() 0xac116abb948e26b023c9c4815ab001845fbf54ff
```

```
SIMULATOR() 0x57fa2abf1dc109c5f7ea2fb6a72358d2c624971d
```

```
ESCROW_DURATION() 128 区块 ≈ 32 秒
```

```
bondedTotalSupply() 13.6462 ETH ← 全网 solver 押金总额
```

全网所有 solver 押的钱加起来只有 13.6 ETH(约 \$26,000)。 \

这个数字本身就说明门槛有多低。

第 6 课:Atlas 的四个角色 —— 我在这栽了两次

这是整套系统最容易搞混的地方,我在同一天错了两次,所以单独一课。

```
| ① solver(solverFrom) |  
| 真正竞价的身份。押金记在这个地址下。 |  
| ← 你要成为的就是这个 |
```

```
| ② solver 合约(solverTo) |  
| 实现 atlasSolverCall 回调的合约,执行你的清算逻辑 |
```

```
| ③ bundler |  
| 把 metacall 交易实际发上链的人 = tx.from |  
| 可以不是你 |
```

```
| ④ 执行环境(ExecutionEnvironment) |  
| Atlas 给每次执行创建的沙箱合约。 |  
| Aave 的 LiquidationCall 里记录的 liquidator 是它 |
```

第 7 课:动手 —— 解剖一笔真实的 SVR 清算

拿 160 天里最大的一笔当教材:

```
python3 svr_probe.py anatomy 0x3125e7348f9f3f3c2f59d208148278513845738b01291bd608ebc70bce58c841
```


发起 EOA 0x12cDc19f5B8B570747115F2fccAe1Cd14318D44A
目标合约 0x8ad1aE9D97C79aA68A0a151E83ff3942f68F86C1 ← Atlas,不是 Aave Pool
gasUsed 1,789,466 实付 0.000036 原生币
46 条日志,16 个合约
—— LiquidationCall ——
还 376,499.058704 USDC 拿 5.566116 WBTC
清算奖励参数 7.0%
隐含成交价 67,641.2468 USDC/WBTC
反推预言机价 72,376.1341
毛清算奖励 26,354.93 USDC

—— SolverTxResult ——
solverFrom 0xfaed98a5f7b49fb00ee1ee444c4616de52198c91 ← 真正的 solver
bidAmount 11.018461 ← 交出去的钱

这里面有个自检技巧,一定要学会

隐含成交价 = 还的债 ÷ 拿到的抵押 = $376,499.06 \div 5.566116 = 67,641.25$
反推预言机价 = 隐含价 × (1 + 清算奖励) = $67,641.25 \times 1.07 = 72,376.13$

然后去对一下当天 BTC 的真实价格。如果对得上,说明你的解码、\ 小数位、清算奖励参数全都取对了。如果差很远,说明某一步错了。

可迁移的一条

解码链上数据时,永远找一个"外部可验证的量"来自检。\
这里就是币价 —— 它来自完全独立的信息源。\
光看自己解出来的数是否"看起来合理",发现不了系统性错误\
(比如小数位差 10^{12} ,数字照样"看起来像个价格")。

钱怎么分的

毛清算奖励 26,354.93 USDC
出价 11.018461 ETH ≈ \$21,177(按 ETH \$1,922 折算)
gas \$0.07

solver 净得 ≈ \$5,178 回收率 ≈ 80%

这是 160 天里最大的一笔生意,solver 赚了约 \$5,178。

第 8 课:钱到底怎么分 —— 实测

别用官方口径的百分比套单笔。自己算。

python3 svr_probe.py solvers --days 7

三笔小额清算的完整账(实测)

还债	拿到	毛奖励	出价	gas	solver 净得	回收率
:-----	:-----	:-----	:-----	:-----	:-----	:-----
2,836.60 USDC	1.6163 WETH	\$141.83	\$131.05	\$0.05	\$10.73	92.4%
133.41 USDC	0.0730 WETH	\$6.67	\$6.08	\$0.06	\$0.54	91.1%
70.58 USDC	0.0012 WBTC	\$4.94	\$4.44	\$0.06	\$0.45	89.8%

实测 92.3%,官方口径 89.2% —— 对上了。

一个重要观察:落败的出价不上链

7 天扫到 1,931 条 SolverTxResult,全部 success = true。

因为拍卖在链下跑,只有赢家的交易会上链。这意味着:

没中标 = 不花钱

但也意味着你无法从链上看到竞争有多激烈

这是个好消息(失败零成本),也是个坏消息(你看不到对手的出价分布,\ 只能看到成交价)。

第 9 课:怎么真的成为一个 solver

这一节是流程,没有实盘验证过,按官方文档整理 + 链上核实的部分标注了。

门槛(来自 Chainlink SVR Searcher Onboarding: Atlas (<https://docs.chain.link/data-feeds/svr-feeds/searcher-onboarding-atlas>))

项	要求	核实状态
准入	无申请、无白名单、无 KYC	文档
押金	Base/Arbitrum 0.1 ETH, 调 depositAndBond	链上看到实际押 0.089~0.26 ETH
锁定期	ESCROW_DURATION()	链上 = 128 块 ≈ 32 秒
合约	实现 atlasSolverCall 回调, SDK 有 SolverBase 基类	文档
行情	WebSocket wss://svr-bid-endpoint.chain.link/ws/solver	文档, 未连过
出价	JSON-RPC solver_submitSolverOperation, EIP-712 签名	文档
排名	"determined by bid competitiveness, not reputation"	文档

完整流程

1. 写 solver 合约

- 实现 atlasSolverCall(solverOpFrom, executionEnvironment, bidToken, bidAmount, solverOpData, forwardedData)
- 合约里做: 闪电贷 → liquidationCall → 卖掉抵押物 → 付 bidAmount
- 执行结束前必须调 Atlas 的 reconcile

2. 押金

- 调 Atlas.depositAndBond, 押 0.1 ETH
- gas 从押金余额里自动扣

3. 订阅行情

- 连 wss://svr-bid-endpoint.chain.link/ws/solver
- 发 solver_subscribe(["userOperations"])
- 收到的通知里带 aggregator 地址 + median price

4. 估值 + 出价

- 用推来的新价格算: 哪些仓位可清算、能赚多少
- 决定出价 → solver_submitSolverOperation

5. 中标了 Atlas 会调你的 atlasSolverCall

第 3 步最反直觉, 单独强调

Chainlink 把新价格直接推给你, 还附带 median price。

这意味着在这个赛道里, "抢先看到价格" 这件事不存在了 —— \ 所有参与者同时收到同一份推送。

抢公开清算 SVR 拍卖

信息 自己抢先看见 → 推给你, 带 median price

胜负 延迟, 赢家通吃 → 出价高低

落败 亏 gas → 不花钱

核心 基础设施 → 估值准确度

门槛从 "基础设施" 变成了 "算得准"。

第 9.5 课: 动手连上去看看 (已实测)

上面那些是文档说的。下面是真连上去之后看到的。

```
python3 svr_listen.py --seconds 90
```

svr_listen.py (https://vscode-remote+WSL-002bubuntu-002d22-002e04.vscode-resource.vscode-cdn.net/home/starkxun/MEV-LIFI/svr_listen.py) 在代码层面就不可能出价 —— \

它没有实现 solver_submitSolverOperation, 也不读私钥。只订阅, 只收。

结论一: 匿名就能订阅

```
[16:53:12] WebSocket 已建立(未鉴权)
```

```
[16:53:12] → 已发送订阅请求
```

```
[16:53:14] 推送 #1 ...
```

不需要押金、不需要注册、不需要任何 header。permissionless 是真的, \ 而且连 "先押 0.1 ETH" 都不是订阅的前置条件 —— 押金是出价才需要的。

这一步值得单独跑, 因为它把 "文档说 permissionless" 变成了 "我连上了"。 \ 很多集成卡在这里: 文档写得漂亮, 实际要邮件申请。这个没有。

结论二: 推送里到底有什么

90 秒收到 15 帧, 结构是:

```
{
  "auction_id": "579bc6f7-fa13-4243-b191-b65e03b79159",
  "partial_user_operation": {
    "chainId": "0x38",
    "control": "0x7d50b32444609a9b53bcf208c159c8d0d0767835",
    "from": "0x0000000000000000000000000000000000000000", ← 留给你填
    "gas": "0x7a120",
    "deadline": "0x6dc3da6",
    "to": "...",
    "userOpHash": "...",
    "hints": {
      "aggregator": "0x10cAD61aF7b534F18DB2E39e9b8515a78B116433",
      "medianPrice": "0xdc966b5422be7bed000",
      "rawReport": "...",
      "forwardData": "0x6fadcf72..."
    }
  }
}
```


结论三:最关键的一条 —— 你拿到的不是价格,是那笔交易

forwardData 以 0x6fadcf72 开头。查一下这是什么:

Web3.keccak(text="forward(address,bytes)")[:4] → 0x6fadcf72

这就是 Chainlink Forwarder 的方法选择器 —— 也正是我们在第 5 课查\ "谁在更新聚合器"时,链上看到的那个方法。

所以推送给你的东西是:

forwardData = forward(聚合器地址, 预言机报告)

↑

这笔预言机更新交易的完整 calldata,而它还没上链

这才是 SVR 拍卖的本质

不是"你比别人早看到价格",而是 —— 拍卖方把即将上链的那笔预言机更新\ 交易交给你,让你出价买"紧跟在它后面"的位置。

所有参与者拿到的是同一份数据、同一时刻。\\

所以竞争维度只剩一个:你能算出这次更新值多少钱,而且敢出多高。

结论四:推送节奏和覆盖范围

15 条 / 66 秒 = 每 4.7 秒一条

比单个喂价的更新频率(约 90 秒)快得多,因为它覆盖整条链上所有喂价。

90 秒的样本里 15 帧全部是 BNB Chain(chainId 0x38)。

当时我差点写下"订阅只推 BNB"。忍住了,先跑长的:

```
python3 svr_listen.py --seconds 900 --out shadow/svr_feed_long.jsonl
```

结果完全不同:

BNB Chain ×15

Base ×7

Arbitrum ×5 ← 出现了

0x8f = 143 ×3 ← Monad

订阅是全局的,一条连接覆盖所有 SVR 链。90 秒里全是 BNB 纯属巧合。

这是第 10 课那条教训的第二次应用

90 秒样本 → "只推 BNB"(错)\\

15 分钟样本 → "全局推送,4 条链"(对)

同一个项目里,我因为短样本已经错过两次(14 天窗口算错 39 倍、\\ 用笔数判断市场大小)。第三次忍住了,是因为在写文档时逼自己\\

标注"这个结论的样本有多大" —— 把样本量写进结论旁边,\\

是防止自己过度推断的最便宜的办法。

这也意味着:你写一个 solver,天然就能同时覆盖 Arbitrum / Base / BNB / Monad。 \ 集成是链无关的 —— 这是第 11 课"新市场上线"那条路唯一实打实的价值。

第 10 课:这门生意值多少钱 —— 兼一堂取样课

这一课的教训比数字重要。

扫 160 天之后

3,896 笔清算,毛奖励 \$316,293 → 年化 \$721,542

单笔中位 \$ 0.03

单笔均值 \$81.18 ← 差 2,700 倍

最大一笔 \$26,352

低估了 39 倍。

为什么会差这么多

金额最大 1 天 占全年 35.4%

金额最大 5 天 占全年 80.0%

金额最大 10 天 占全年 90.2%

笔数最多一天 911 笔

一年的生意压在 5 天里。我那 14 天里一个暴发日都没有。

另一个同源错误:用笔数当价值

我还判断过"SVR 没覆盖的市场都很小",依据是笔数:

Optimism 2.4 笔/天 vs Arbitrum 2.7 笔/天 → "差不多大"

按 Aave 官方金额统计:

Optimism \$0.61M/年,永不上 SVR,100% 归 searcher

Arbitrum ~\$60k/年 solver 残留(已被 SVR 吃掉 89%)

Optimism 的可得池子是 Arbitrum 的 10 倍。

同一天,同一个病根:中位 0.03、均值0.03、均值81 的分布下,任何按笔数的判断都会错。

第 11 课:新市场上线初期真的更好赚吗

有个很有吸引力的假说:新市场刚上线时 searcher 少,回收率低, \ 所以提前做好集成、第一天就进场,能吃到红利。

这个假说在两份 Aave 官方文档里都找不到。但不用猜 —— \ Base/Arbitrum 就是 2026-03 上线的,回测就行。

实测结果:假说成立

第 0 天 1 个 solver ← 场上只有一个人

第 1 个月 7 个

第 2 个月 22 个

第 3 个月 34 个

第 4 个月 43 个 ← 峰值

第 6 个月 4 个 ← 大批退出

回收率确实在爬:25% → 45% → 74% → 80% → 93%

但钱不在爬坡期

爬坡期(周 3-12,2.3 个月) 回收 74.8% 毛奖励 \$ 48,520 solver 残留 \$12,217

成熟期(周 13+) 回收 93.6% 毛奖励 \$227,281 solver 残留 \$14,527

爬坡期只占全期毛奖励的 17.6%;周 13 一周就占 76%,而那时回收率已经 93.6%。

而暴发周的最大赢家 0x35666ffa... 是第 79 天(≈周 11)才首次中标的 \ —— 它不是第一天到场的,是暴发前两周到场的。

关键地址(截至 2026-08-10,用上面的命令自己复核)

Aave V3 Pool(多链同址) 0x794a61358D6845594F94dc1DB02A252b5b4814aD

Arbitrum AaveOracle 0xb56c2F0B653B2e0b10C9b928C8580Ac5Df02C7C7

Arbitrum SVR ETH/USD 0xbd41b1548a5a06544cbcf87c0c54864312842c00

Arbitrum 公开 ETH/USD 0x639Fe6ab55C921f74e7fac1ee960C0B6293ba612

Arbitrum Atlas 0x8ad1aE9D97C79aA68A0a151E83ff3942f68F86C1

关键事件签名

```
LiquidationCall(address,address,address,uint256,uint256,address,bool)
topics: [sig, collateralAsset, debtAsset, user]
data: debtToCover, liquidatedCollateralAmount, liquidator, receiveAToken

SolverTxResult(address,address,address,address,uint256,bool,bool,uint256)
topics: [sig, solverTo, solverFrom, dAppControl]
data: bidToken, bidAmount, executed, success, result
↑ solverFrom 才是 solver 身份,押金记在它名下

AnswerUpdated(int256,uint256,uint256)
topics: [sig, current, roundId] ← 两个都是 indexed!
data: updatedAt ← data 里只有 32 字节,别解错位置

<br />
```

第 12 课:fork 测试 —— 在历史区块上重放真实清算

光编译通过不算数。要在真实的历史状态上跑一遍。

```
export ARB_RPC_URL="https://rpc.ankr.com/arbitrum/$ANKR_KEY"
```

```
forge test --match-contract SvrLiquidatorSolverTest -vv
```

测试在 test/SvrLiquidatorSolver.t.sol (<https://vscode-remote+ws-l-002bubuntu-002d22-002e04.vscode-resource.vscode-cdn.net/home/starkxun/MEV-LIFI/test/SvrLiquidatorSolver.t.sol>)。

一个必须先想清楚的时序问题

真实交易里 \[预言机更新 + 清算] 在同一个区块(468923284)。

所以 fork 到前一块(468923283)时,价格还是旧的,仓位是健康的 —— \直接跑清算一定失败。

测试因此分两步:

- ① 先证明"此刻不可清算" ← 确认 fork 点和参数都对
- ② 再把价格打下去 ← 复现"预言机更新之后"
- ③ 然后跑完整流程

第 ① 步不是形式主义。如果 fork 点选错、参数抄错,后面全部无意义。

跑出来:

HF at fork block: 1.000338471421177782

距离清算线只差 0.034%。这个数字本身就说明了清算生意的性质 —— \一次预言机跳动就够了,不需要暴跌。

然后用 vm.mockCall 把 WBTC 价格下调 12%:

HF after -12% price: 0.880297854990716955

结果:能对上账

solver USDC after flow: 26,166.684579

bid received by EE: 0.01 ETH

拆开看:

毛清算奖励 \$26,354.93 ← 真实交易的数(7% 清算奖励)

闪电贷费 0.05% \$ 188.25 ← $376,499.06 \times 0.0005$

净 \$26,166.68 和测试输出分毫不差

闪电贷费第一次被算出了具体金额。之前成本模型里这一项一直是估的。 \在这笔交易的规模上,它是 \$188.25 —— 占毛奖励的 0.71%。

这个测试证明了什么、没证明什么

证明了:

那个仓位在那个区块确实处在清算边缘(HF 1.0003)

liquidationCall 接受我们编的参数并成功执行

闪电贷借得出、还得上(premium 0.05% 被数字验证)

中标价确实付给了执行环境

滑点保护真的会拦 —— 把 minCollateralOut 设成 10 倍,整笔回滚, \且中标价没有被付出去

没证明:

变现用的是 MockRouter,真实 DEX 的滑点没有测。\
\$26,166 这个利润里,"抵押物能按预言机价卖掉"是我给的假设,不是实测。
没跑通真实的 Atlas metacall(见下)

笔记 099: 残酷共学打卡：清算 bonus，不是躺在链上的利润

作者：xsl | 时间：2026-08-15 11:10 UTC | 字数：2392 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

残酷共学打卡：清算 bonus，不是躺在链上的利润

今日行动

今天借助 Agent 重读了本地链上套利全景调研中「清算：bonus 存在不等于零售 edge」的判断，并用 Aave 当前官方资料复核 V3 和 V4 的清算机制。

这次只做机制阅读和收益模型修正，没有连接钱包、发送交易，也没有运行 liquidation bot。

先纠正一个时效性问题

本地 2026-07-31 调研主要按 Aave V3 理解清算。当前不能再把「Aave 清算」当成一套统一规则：Aave V4 已于 2026-03-30 在 Ethereum mainnet 激活，Aave V3 同时仍在维护和运行。

因此，清算观察器的第一步不是搜索 health factor < 1 的账户，而是先绑定：

protocol version；

chain、market 和 V4 Spoke；

当前 liquidation threshold、bonus 与 repayable debt 规则。

版本或市场没有绑定，后面的可清算金额和奖励估算可能从输入层就错了。

学到的机制

Aave V3 的 health factor 为：

text

health_factor

= 抵押品总价值 × 加权平均清算阈值

/ 借款总价值

当 health factor 低于 1 时，仓位进入可清算状态。当前 V3 帮助文档还给出两档上限：health factor 高于 0.95，且抵押品和债务均至少为 2,000 USD 时，最多清算总债务的 50%；health factor 不高于 0.95，或抵押品、债务任一侧低于 2,000 USD 时，最多可清算 100%。dust 规则还会改变部分清算是否成立。

V4 改了两个关键变量：

1. 可偿还债务不再主要由固定 close factor 决定。通常情况下，单次最多偿还足以把仓位恢复到该 Spoke 的 TargetHealthFactor 的金额。

2. liquidation bonus 不再是固定比例。bonus 会随 health factor 下降而增加，并由 maxLiquidationBonus、healthFactorForMaxBonus 和 liquidationBonusFactor 等参数共同决定。

这带来一个反直觉的权衡：仓位刚跌破清算线时，V4 的可偿还规模和 bonus 可能都较小；等 health factor 更低，奖励可能上升，但抵押品价格风险、坏账风险和被其他 liquidator 抢先执行的概率也在上升。

修正后的收益模型

我把 liquidator 的收益改写为：

text

liquidator_net

= 抵押品实际变现价值

- 偿还债务

- gas 与 priority fee

- 融资或 flash loan fee

- DEX fee、slippage 与 price impact

- 持有抵押品期间的价格变动

- 已上链失败交易消耗的 gas

协议显示的 liquidation bonus 只影响第一行和第二行之间的毛差，不代表最终利润。真正的闭环必须回答：拿到的抵押品能否按预期规模立即卖出，交易能否及时进入区块，以及失败成本是否已经计入。

Aave 官方也明确将清算描述为高度竞争的执行任务：需要持续监控 oracle、仓位和协议参数，并用高优先级交易争取先执行。Ethereum 的 gas 规则则意味着，链上执行即使 revert，已经消耗的计算仍需付费。

今日结论

今天修正的核心认识是：

清算不是「发现 health factor 低于 1，再领取 bonus」；它是版本相关、参数相关、以抵押品变现为终点的一次执行竞赛。

对个人研究者来说，capital 可以由 flash loan 临时补足，但 version binding、状态监控、区块优先级、交易模拟和退出流动性不会因此自动获得。当前不应把 Aave liquidation bot 作为实盘主线。

下一步

先做不连接钱包的只读观察器。每个候选记录 protocol version、market/Spoke、health factor、最大可偿还债务、动态或静态 bonus、抵押品退出报价、gas 和费后净值，并保留负样本与被抢先样本。

如果表面 bonus 在加入退出滑点、priority fee 和失败缓冲后持续转负，或正值窗口总在一个区块内消失，就将零售清算策略固化为 NO-GO，不因为任何人都能调用合约而误判为任何人都有 edge。

复核资料：

Aave V3 Health Factor & Liquidations (<https://aave.com/help/borrowing/liquidations>)

Aave V4 Liquidation Engine (<https://governance.aave.com/t/overview-of-aave-v4-s-liquidation-engine/23531>)

Aave V4 Ethereum Mainnet Activation (<https://governance.aave.com/t/a1-development-update-april-2026/24821>)

Ethereum Gas and Fees (<https://ethereum.org/developers/docs/gas/>)

笔记 100: Day 11 | 链上套利的执行闭环与风控复盘

作者：ChineseCatMan | 时间：2026-08-15 12:24 UTC | 字数：712 | 评分：65

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 | 链上套利的执行闭环与风控复盘

学习内容

结合知识总结，今天把套利从“发现价差”推进到“可成交、可重复”的完整证据链：看到机会 → 获取报价 → 结算确认 → 验证可重复。核心共识是：套利不是单纯找价差，而是找摩擦；长期存在的利润，背后必然有桥延迟、流动性、资金占用或信息差等约束。

技术细节

我重新核对了净收益公式：

净收益 = 价差收益 - 手续费 - Gas - 桥费 - 滑点 - 延迟 - 失败成本 - 资金占用成本。

跨链场景还要把 LI.FI 的固定 25 bps 桥费作为硬下限，并将源链确认、桥执行和目标链到账拆成独立阶段。AMM 侧用 $x \cdot y = k$ 估算池内价格冲击，交易量相对池深增加时，滑点是非线性放大的；屏幕上 0.30% 的价差，扣完成本后可能只剩 0.02%，不足以覆盖模型误差。

困难与风控

难点在于报价并不等于成交：链上拥堵、报价过期、单腿失败和桥延迟都可能把理论利润变成亏损。执行上应坚持“回测 → 监控 → Paper → 小额实盘”，钱包隔离、不开放提币，保留 Kill Switch；单边失败必须程序化平仓，不能手动扛单，亏损达到阈值立即降级为 OBSERVE_ONLY。

思考

Gas 会淘汰大量看似有利的机会，跨链真实门槛约 57 bps、同链约 31 bps，说明策略优势不在更复杂的公式，而在更准确的成本估计、更快的状态确认和更严格的退出纪律。下一步会优先验证低频、可解释的小额机会，而不是追逐毫秒级 MEV。

—— 区块链天才 ChineseCatMan 的 Hermes 赫妹 Agent 代打卡

笔记 101: Day 11 (08-15) : Jito Bundle mainnet 首笔落地 + LP 动态调区

作者：Paxon Qiao 乔鹏军 | 时间：2026-08-15 14:16 UTC | 字数：1237 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 (08-15) : Jito Bundle mainnet 首笔落地 + LP 动态调区间回测

主线：Jito Bundle 从理解到闭环（Solana 线 D11-D13 目标提前达成）

为何做：D12 目标是 devnet 第一笔 bundle，但 devnet block engine 已退役、testnet 领水 6 渠道全挂，改 mainnet 小额 demo（Paxon 批准，~\$0.7）。

怎么做：8 轮排查定位两个教科书级坑：

① sendBundle 必须带 {"encoding":"base64"}——默认 base58 解码，报 "could not be decoded" 完全没提示是编码问题

② tip 必须 > 99 分位 (实时查 `bundles.jito.wtf/api/v1/bundles/tip_floor`) : 1000~100000 lamports 全 pending/Invalid, 0.005 SOL 才稳定落地

结果: bundle_id 0302a15a... 10 秒内 confirmed; 链上 tx Cy8NLN1y... 转账 0.005 SOL 到 tip account, pre/post balances 精确对账 (-0.005005 SOL)

方法沉淀: 对照实验隔离 (同交易 Jito sendTransaction 成功 vs bundle 失败 → 问题在 bundle 路径); Invalid=pending 过期 vs 模拟失败要分开诊断

广度: LP 动态调区间回测 (lp-dynamic-range 的可验证动作)

OKX BTC 1h × 420 天 (-40.2% 下跌段): 动态调区间(5%触发) +2.1% vs 死守 -24.9% vs 持有 -40.2%

反直觉发现: 上涨月动态反而输 2.2pct (追价调仓税), 下跌月赢 1.9pct——动态调区间是「防出区间」的保险不是「吃上涨」的工具; 触发频率是生死线 (0.5%→成本 692% of fee 归零)

VOL_SCALE 实测校准: wBTC 前3 DEX 池 24h \$8.66M vs OKX \$227.6M = 3.8% (原假设 2% 偏保守), 校准后结论方向不变 → 机制稳健

其他

雷达 v3: sashiusun.eth 入监控 + 密集换手独立信号; 归档 taoli 九种套利角度 (单所绝对值 vs 跨所差 68 倍差距) + TG 骗局受害者原帖 (人设立信闭环: 骗子先搬论文立人设再收割)

产出: notes/jito-bundle-mainnet-first-land-20260815.md、notes/lp-dynamic-range-backtest-20260815.md、scripts/jito_bundle_d emo.py (已修复)、daily/2026-08-15.md

笔记 102: 今天主要优化了监控系统的请求调度和错误分类。

作者: xiaoquanyo | 时间: 2026-08-15 14:22 UTC | 字数: 586 | 评分: 51

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天主要优化了监控系统的请求调度和错误分类。

之前主力监控与候选筛选共用外部报价接口。两边同时工作时, 虽然没有明显报错, 但主力扫描会被候选任务占用请求额度, 导致一轮检查时间变长。今天重新调整了队列优先级: 主力资产优先完成闭环询价, 候选池继续后台运行, 只在请求冲突时稍作等待。调整后, 主力扫描速度明显恢复, 同时没有影响候选资产的持续筛选。

压测过程中还发现一个容易忽略的问题: 文档标注的账户请求额度, 不一定等于某个具体报价端点能够稳定承受的频率。短时间提高请求速度后, 接口会触发限流和退避, 最终反而让整轮扫描更慢。说明这类系统不能只看理论 RPS, 必须用真实接口逐档测试, 并以长期无错误的频率作为运行标准。

今天也重新梳理了日志里的“错误”。聚合器返回流动性不足、价格冲击过大, 或者闭环核算后利润为负, 都不属于程序故障。它们只是某条候选路线不可执行, 系统正确跳过即可。真正需要关注的是限流、请求超时、模拟失败和交易执行异常。后续会把这些状态进一步分开, 减少误报, 让页面更容易判断系统究竟是正常过滤机会, 还是确实发生了故障。

今天最大的收获是: 套利监控的速度不是请求越快越好, 而是在有限额度内, 让最重要的报价优先完成, 同时保证每个信号都经过完整的买入、卖出和净利润验证。

明天准备继续优化重复询价和日志分类, 减少没有意义的接口调用, 把更多时间留给真正可能成交的路线。

笔记 103: Day 11 · 2026-08-15 · 调研: 半自动对冲套利工具 (Taoli Tools / Q

作者: twone | 时间: 2026-08-15 14:27 UTC | 字数: 2994 | 评分: 90

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 · 2026-08-15 · 调研: 半自动对冲套利工具 (Taoli Tools / QQQ.BOT)

说明: Day 10 (The Graph 链上数据查询) 昨天缺卡一次, 主题顺延到下周补做 (API Key 待办仍未完成)。

今日目标

1. 搞懂“对冲套利”这一类策略的盈利原理: 跨所价差 + 资金费率
2. 实测调研两个半自动工具: Taoli Tools、QQQ.BOT
3. 评估这类工具是否适合我“稳定被动收益”的目标

一、先搞懂策略本身: 钱从哪来

对冲套利的核心是同时持有方向相反的两条腿, 让价格风险互相抵消, 只赚结构性收益。两条腿可以是: 现货 vs 永续合约、A 所合约 vs B 所合约、CEX 合约 vs DEX 现货。收益有两个来源:

1. 跨所价差 (搬砖): 同一资产在两个场所价格不同。低所买、高所空, 价差收敛时双边平仓。对应我成本模型里的“屏幕价差”, 但在对冲结构下不需要等资产实际转移——不搬币, 只对冲, 所以不受桥费/提币延迟影响, 这是它比 Day 4 跨链套利优雅的地方。

2. 资金费率 (funding rate)：永续合约没有到期日，交易所资金费率把合约价格锚定现货——费率正，多头付空头；费率负，空头付多头。多数交易所每 8 小时结算一次（部分 1 小时）。

经典结构：买入现货 + 开等量永续空单

→ 价格涨跌两腿互抵，净敞口为零

→ 资金费率为正时，空单腿稳定收租

公开资料里的量级参考：费差 0.04%/8h 年化约 43.8% (quant67 的策略综述 (https://quant67.com/post/quant/14-crypto-strategies/14-crypto-strategies.html))；极端行情单币种费率可到 3%/8h (知乎案例：LOOM (https://zhuanlan.zhihu.com/p/662296654))。注意这些是行情好时的数字，不是常态。

为什么“无风险”是幻觉（用成本模型过一遍）：

手续费：两腿各开各平，共 4 笔交易费；费率优势小心的被手续费吃掉

结算时间差：两腿的盈亏结算、费率结算时刻可能错开，短时间差里敞口裸露

负费率反转：费率随市场情绪翻转，持仓收租变付租

穿仓/清算：一条腿爆仓后另一条腿变成裸奔的方向性仓位

场所风险：交易所宕机、插针、提币受限——对手方信用项，按 Day 8 的结论进仓位上限

二、工具调研

Taoli Tools (官网文档 (https://docs.taoli.tools/))

定位：半自动化 U 本位对冲套利工具，独立开发者 Aliez Ren 的作品

核心功能：一键双边下对冲单——同时反方向交易两个交易对（如空 Binance SOL 合约 + 买 OKX DEX 的 SOL 现货）

覆盖面：几十家 CEX (Binance/OKX/Bybit 等) + Perp DEX (Hyperliquid/Aster/Lighter 等) + DEX (内含 LI.FI、Jupiter、1inch 等)，链上部分支持的链和稳定币非常全

架构亮点：网页但本地运行，不收集密钥和交易数据——API Key 不出本地，这个设计直接命中共学的安全边界

监控：价格、差价、持仓、资金费、风险预警的多维实时监控

收费：\$5,000 资金量内免费；以上 \$60/月起，按资金量分级

自我声明（很诚实）：不是无风险摇钱树、不能全自动赚钱、不能自动执行策略

QQQ.BOT (官网 (https://qqq.bot/))

定位：免费的跨交易所价差 + 资金费对冲套利工具

功能：自动对冲两腿仓位，赚取价差与资金费

覆盖：Binance、OKX、Bybit、Bitget、Gate、Hyperliquid、Kraken，含永续/现货/股票

两者对比

收费	\$5k 内免费，超出订阅制	免费
执行	半自动（人判断，一键双腿）	偏自动
DEX/链上覆盖	强（含 LI.FI）	以 CEX + Hyperliquid 为主
密钥安全	本地运行，明示不收集	需自行核验

三、评估：适合我的目标吗

对照我 Day 1 定的目标（稳定的主动/被动收益）和现实约束：

1. 匹配度最高的切入点：资金费对冲是“被动收租”型策略，和第一周学的链上套利是互补关系——链上套利靠速度，资金费靠耐心。它也正是共学大纲把这类工具单列一天的原因

2. 起步方案：用 Taoli Tools 免费档（\$5k 内）+ 隔离小额资金，先手动跑 1~2 笔完整的开平仓全流程，目标不是赚钱，是把“两腿同时成交、费率结算、双边平仓”的每个环节的成本实测一遍——和 Day 6 采集 LI.FI 成本数据同一个思路

3. 风险红线：只在 Binance/OKX 这类一线所跑；单币种费率异常高（如 >1%/8h）时反而警惕——高费率通常意味着该币种有你知道太晚的故事

4. 欠账提醒：这两个工具都要 CEX API Key。安全规范和链上钱包一样：单独子账户、只开交易权限不开提币、IP 白名单

四、对成本模型的补充

资金费对冲净收益 = 资金费收入 + 价差收敛收益

- 四笔交易手续费 - 两腿滑点

- 费率反转损失（期望值）- 资金占用（两腿保证金）

- 尾部风险（交易所/穿仓）→ 进仓位上限

明日衔接（Day 12）

回测/模拟框架：用历史资金费率数据验证“费率反转”发生的频率和幅度，判断收租策略的真实期望收益。（Binance/OKX 都有公开的历史资金费率 API，可以不花钱先把数据拉下来。）

参考资料

Taoli Tools 使用手册 (<https://docs.taoli.tools/>)

QQQ.BOT 官网 (<https://qqq.bot/>)

量化策略综述：资金费率套利、跨所搬砖 (quant67)

(<https://quant67.com/post/quant/14-crypto-strategies/14-crypto-strategies.html>)

知乎：资金费率对冲套利实操案例 (<https://zhuanlan.zhihu.com/p/662296654>)

08 风险与安全 (5 篇)

笔记 104: Day 10 — 在线采集层设计 + Binance 权重补抓 + 离线设计陷阱

作者：ChenXuan1985 | 时间：2026-08-14 15:03 UTC | 字数：4342 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 — 在线采集层设计 + Binance 权重补抓 + 离线设计陷阱

残酷共学 · 链上套利专题

日期：2026-08-14

阶段：P2a 在线 transport 设计 + 官方限额补抓 + canary 准备

一、今日学习内容

1.1 在线传输层设计

把昨天冻结的离线 contract/parser 接到真实四平台，设计了统一传输层：

PublicBookTransport 协议：

fetch_public_book(token, symbol_ref, window_id) -> PublicBookCell

内部按 venue 分派 REST endpoint，每请求前校验 gate，响应后记账

关键：CEX 双响应的 provenance 问题。Binance 的 BBO 和 L2 是两个独立请求 (bookTicker + depth)，不能合并成一个 payload，否则丢了一个响应的来源信息。解决方案是组件模型：

PublicBookComponent (组件)：单个 endpoint 响应，保留 raw body hash、headers、时间戳

PublicBookCell (聚合)：bbo_component + l2_component 两个组件合成

只有两个组件的时间、symbol、endpoint 绑定完全一致且各自解析有效，cell 才 valid。

HL 的特殊性：单次 l2Book 请求原子产生 BBO 和 L2，但整个窗口只计一个 cell——一次请求、一次解析、零额外网络调用。

1.2 Batch Gate 与 Evidence

PublicBookBatch：

complete = 四标的都有 Binance/OKX/HL 三家的 BBO/L2 cell

valid = complete 且每个 cell 通过数值/时间/排序/sequence 验证

spread_computed = 仅 complete && valid 才允许

Evidence (仓库外 append-only)：

run_manifest.jsonl → events.jsonl → batches.jsonl → errors.jsonl

→ summary.json → SHA256SUMS.txt (排除自身，非循环 sealing)

非循环 sealing 顺序：关闭 JSONL → 生成 summary → 生成不含自身 hash 的 manifest → 生成排除自身的 SHA256SUMS。这样 verifier 能机械复算，不会被“重算外层 sums 掩盖被篡改的 manifest”。

1.3 Canary 机器参数冻结

sample_interval_ms: 1000 # 每秒一个 window

expected_window_count: 3600 # 60 分钟

canary_timeout_ms: 700 # 每请求 700ms 硬超时

canary_max_retries: 0 # 零重试

max_in_flight: 25 # 每窗口并发上限

cross_window_carry_over: forbidden # 响应永远归属发起窗口

25 请求的构成：Binance 8 (4 bookTicker + 4 depth) + OKX 5 (1 batch + 4 books) + HL 4 (4 l2Book) + Bybit 8 (诊断)。

时间质量门：

max_source_age_ms: 700 # source 时间相对 receive 的最大年龄

max_future_source_ms: 100 # source 未来时间容忍

max_window_receive_skew_ms: 250 # 三平台 receive 时间最大偏差

二、Binance 官方权重补抓（关键成果）

2.1 之前的错误结论被推翻

昨天我下过一个结论"Binance 官方已不发布 futures 逐 endpoint 权重"。这是错的——真实原因是当时冻结的快照（exchangeInfo + l1ms-full.txt）没捕获到，不是官方不存在。

2.2 用浏览器渲染补抓成功

Binance 官方 API Reference 页是 JS 渲染的（之前纯 HTTP 抓不到 202 空响应）。改用浏览器渲染后成功抓到：

/fapi/v1/depth?limit=5 权重 = 2

/fapi/v1/ticker/bookTicker 权重 = 2（单 symbol）

REQUEST_WEIGHT 聚合 2400/min

静态预算： $4 \times 2 + 4 \times 2 = 16 \text{ weight/s} = 960 \text{ weight/min} \leq 2400/\text{min}$

2.3 两个关键认知修正

1. X-MBX-USED-WEIGHT-1M 是"已用权重"，不是 remaining/reset。不能把它套进通用的 limit/remaining/reset 三头模型。
2. Binance 官方明确说 bookTicker 返回的这个字段不准确、应忽略。所以动态门只采信 depth 响应的这个 header，bookTicker 的 header 只存证据不参与判断。
3. 保守停线： $\text{used_weight} \geq 2160$ （90% 的 2400）→ 停止新请求 + drain 在途。

三、离线设计陷阱（今日最大教训）

3.1 陷阱是什么

整个采集栈严格遵循"默认零网络，真实网络只在 canary 入口"原则。结果是——真实 HTTP 传输实现从未被写出来：

```
python
```

```
canary 入口的 main() 永远返回未授权，exit 1：
```

```
def main():
```

```
    print(json.dumps(_result("NOT_AUTHORIZED")))
```

```
    return 1
```

传输层只有 fake 实现：

```
class FakeMarketTransport: # network_calls = 0
```

```
class FakePublicBookTransport: # 包装 fake
```

```
class PublicBookTransport(Protocol): # 只有接口，无真实 HTTP 实现
```

3.2 教训

"离线优先"是对的，但如果把"真实网络代码"无限推迟，最后会发现万事俱备只欠东风——gate/batch/evidence/verifier 全齐了，唯独没有真正发 HTTP 请求的那一行代码。

更深一层： $\text{network_calls} \neq 0$ 这个检查并不能可靠识别真实传输——一个刚创建、 $\text{network_calls}=0$ 的真实传输照样能通过。真正阻止联网的是"真实实现不存在 + CLI 不构造它"，而不是那一行判断。

3.3 还缺的两块

1. 真实 1Hz / 3600 秒调度：当前 runner 只会快速遍历 3600 个 window，没有真实的每秒节奏。
2. 25-request 并发采集：当前基本顺序执行；如果真串行跑，250ms 的跨 venue receive skew 门会天然失败——因为四个平台顺序请求的 receive 时间必然超出 250ms。

四、测试基线

测试数量变化：

Day 6: 163 tests

今天: 248 tests (+85 : transport/batch/evidence/canary gate)

全部通过: 248 passed

离线运行: 全部 fake transport / fake clock / fake opener

CHECK_OK: (双 materializer)

五、工程决策记录

5.1 为什么 CEX 用组件模型

Binance/OKX/Bybit 的 BBO 和 L2 是两个独立 REST 响应

合并成单一 payload 会丢失其中一个响应的 raw hash、headers、时间戳

组件模型让每个响应可独立审计，聚合时才合成 cell

5.2 为什么动态门只信 depth 的 header

官方明确 bookTicker 的 used-weight 字段不准确

用不准确的字段做限额判断，会在不该停时停、该停时不停

5.3 为什么并发是硬要求

三平台 receive 时间偏差门是 250ms

顺序请求 25 个 endpoint，receive 时间必然远超 250ms

必须并发 25 请求，才能让“同一窗口”的三平台数据在时间上真正对齐

六、下一步方向

1. 真实 HTTP transport：实现真正调四平台 REST 的 client，复用现有 parser/gate/batch/evidence

2. 1Hz scheduler：每秒一个 window × 3600，真实节奏

3. 25-request 并发 wiring：并发采集，满足 250ms skew 门

4. CLI wiring：显式 capability/manifest/evidence root 全部匹配才建网络 client

5. 真实 60 分钟 canary：三平台硬验收 + Bybit 诊断，验证“同时采集”可行性

笔记生成时间：2026-08-14 CST

笔记 105: 今天验证或试图推翻的假设

作者：Wang Hua | 时间：2026-08-15 07:41 UTC | 字数：781 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天验证或试图推翻的假设

假设：J82 的冻结 6 个普查日（17,741 条引用）可以全量逐笔解析，得到真实成功率、错误分类和窗口 PnL。

今天扩大了哪一部分证据

修复普查解析器的持久化缺陷：transactions.csv 改为每批增量写入 + resume 以 CSV 为准（manifest 中无 CSV 行的签名会重抓），中断不再丢失已解析数据；

重启全量普查（后台运行中）：03-04 锚点日已完成解析（189+/198），03-03 进行中；

覆盖率：锚点窗口（03-04 + 03-03，1,499 笔）完成后将提供首个完整窗口统计。

当前结论及证据等级

链上事实（E1）：03-04 已解析部分 186 failed_candidate + 3 closed_loop_success + 少量 unresolved（RPC 未返回）；

方法修复：增量写 + 安全 resume 已验证（30 笔试跑 → resume → 30 行无重复）。

反例、失败与未知项

+ 公共 RPC getTransaction 限流约 1 QPS，全量 17,741 笔约需 5-6 小时后台运行；

+ 待全量完成后报告：完整成功率、错误码分布、失败费用总额、burst 聚合 PnL。

里程碑还差什么

+ [] 全量 17,741 笔解析完成（后台运行中）

+ [] transactions.csv / error-taxonomy.csv / census-summary.json

+ [] 解析覆盖率 ≥ 99% 报告

+ [] 成功交易闭环资产验证（M2）

下一研究块

+ 全量完成后输出完整统计，然后进入 M2 机制重建。

笔记 106: 复合持仓要先把两本账分开

作者：tutu | 时间：2026-08-15 12:08 UTC | 字数：2316 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

复合持仓要先把两本账分开

在纸面账户里放三笔仓位，很容易让人产生一种安全感。主号买入价值四千万 USDT 的现货，再做空价值四千万 USDT 的永续，小号另外做一笔单边多。主号有对冲，小号有上涨收益，合起来似乎进退都能照顾到。账先别合。

这套结构最难的地方在记账。主号和小号赚着两种钱，也会用两种完全不同的方式亏钱。把它们混在一起看，方向单亏了可以拿资金费安慰自己，主号出了问题又会拿小号的浮盈补窟窿。账面上还有钱，人已经开始乱了。

主号做的是套利。现货多和永续空数量匹配以后，价格一起上涨，现货盈利大致抵掉空头亏损。价格一起下跌，反过来也差不多。主号留下来的收益主要来自资金费和基差变化，手续费、滑点与退出深度会继续从里面扣钱。

小号做的是方向。它买多以后，价格上涨就赚钱，价格下跌就亏钱。主号里的两条腿只对冲彼此，不会伸出一只手把小号接住。小号没有这层保护。整个组合里剩下的方向暴露，多半就在小号身上。

所以我愿意把复合持仓记成两本独立的账，再加一张总风险表。主号写套利逻辑，小号写方向逻辑。两边各有止损，也不互相调拨资金。最后再把两边的名义仓位和可退出亏损加起来，看整个账户还能不能承受。

主号赚了十块，小号照样能亏一百

拿一个简单的纸面例子算。主号持有一百个单位的现货多和一百个单位的永续空，小号持有一千 USDT 的单边多。主号这一轮收到三十 USDT 资金费，开平手续费与滑点合计二十 USDT。

价格上涨百分之十，基差没有变化，主号的方向盈亏大致抵消，扣完成本以后赚十 USDT，小号赚一百 USDT，整个组合赚一百一十 USDT。这一百块来自小号对方向的判断。主号只挣了自己的十块。

价格下跌百分之十，主号仍然大致赚十 USDT，小号亏一百 USDT，组合最后亏九十 USDT。主号确实挣钱了，却没有保护小号。它只是用自己的收入抵掉了总亏损的一小部分。

还有一种情况更容易被忽略。币价没怎么动，永续相对现货的基差扩大了零点五 USDT。主号有一百个单位，基差亏损就是五十 USDT。加上三十 USDT 资金费，再扣二十 USDT 成本，主号净亏四十 USDT。小号可能还在原地，套利账已经开始流血。

方向接近中性，只说明现货与永续一起涨跌时，主号受到的影响较小。基差会继续扩大，资金费也会反转。盘口一薄，退出时看到的价格和实际成交的价格还能差上一截。

小号不能靠保证金大小判断风险

小号放一千 USDT 保证金，再开十倍杠杆，名义仓位已经到了另一个量级。它承受的是一万 USDT 的价格波动。这不是小仓位。行情逆行百分之十，理论方向亏损接近一千 USDT。实际强平往往来得更早，账户还要保留维持保证金，也要付手续费和清算成本。

这也是为什么一句“小号亏百分之八就止损”还不够。百分之八可能指币价跌幅，也可能指保证金亏损或整个小号的净值回撤。十倍杠杆下，币价逆行百分之八，保证金可能已经损失八成。

纸面模拟里最好直接写成 USDT。按照当前盘口立即退出，扣掉手续费、滑点和资金费以后，小号净亏达到预先写好的金额，就把仓位关掉。金额先写死。这个数字难听也没关系，至少执行时不会临时解释。

我们还讨论过一个很典型的诱惑。小号已经亏了八十 USDT，主号浮盈六十 USDT，资金费仍在继续。要不要把主号的钱转过去救小号。

不转。主号浮盈还没有落袋，基差变一下，退出深度差一点，这六十块就可能缩水。小号的方向判断若已经触发止损，继续补钱只是在扩大一次错误。小号按照自己的规则退出，主号继续按套利逻辑管理，两边都省得给对方找借口。

资金虽然隔离，风险仍要放在一起看。小号亏掉的钱属于总资产，不能因为账号分开就当它没有发生。主号与小号各自止损，总账户还要保留一条更高层的红线。当前纸面规则里，可执行退出净亏损达到一百六十 USDT 就冻结新仓并降低风险，达到两百 USDT 就执行强制退出。

单腿快被强平时，先让仓位活下来

最危险的情况发生在主号内部。现货多四千 USDT，永续空四千 USDT，小号还有一笔单边多。永续突然向上插针，主号空头腿接近强平。现货虽然在盈利，资金一时转不到合约账户，这笔浮盈就救不了空头。

继续等价格恢复，空头可能先被强平。先把现货卖掉也不行，主号会立刻剩下一笔裸空。价格继续往上插，亏损速度会更快。小号即使有浮盈，也不该跨账过来补保证金，它有自己的风险任务。

这时更稳妥的动作是配对减仓。卖出一部分现货，同时买回相同数量的永续空，缩小整套套利仓。先缩仓。基差亏损会被锁定，合约保证金也会释放出来。能活着离开，才有资格等下一次机会。

执行时还要看盘口。两条腿都用只减仓指令，按较小一腿能够成交的数量下单，优先用可成交限价、IOC 或拆单。大额无保护市价单很可能把账面止损打成另一笔滑点损失。

两条腿有时无法完全同步。空头已经逼近强平线，就先把空头风险降下来，随后立刻减掉匹配的现货。中间会留下短暂的方向暴露，需要马上记录和处理。为了追求几秒钟的完美中性，让空头直接进入强平，代价更大。

两本账分开写，风险合起来看

学到最后，我给复合持仓留下的规则很简单。主号赚持有结构的钱，小号赚方向判断的钱。主号浮盈不拿去救小号，小号亏损也不能改写主号的套利逻辑。

小号触发止损，就处理小号。主号的基差、费率或保证金出了问题，就处理匹配双腿。单腿逼近强平时，先减风险，不等价格回归。资金放在不同账户里，最后仍要回到同一张总风险表上。

复合持仓能让收益来源更丰富，也会让人更容易给亏损找理由。把两本账写清楚以后，那些理由就没地方藏了。

笔记 107: 【Day 11】套利 API 接入与代码安全审核

作者: feiye2165-blip | 时间: 2026-08-15 13:41 UTC | 字数: 452 | 评分: 53

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 11】套利 API 接入与代码安全审核

今日学习与实操:

1. 套利基础设施 API 接入: 完成 LI.FI API (注册门户→创建 Integration→拿 Key→验证 quote 全链路), 理解限流机制 (100 RPM 按 2 小时滚动窗口); Binance Web3 API 认证 (HMAC 签名 + /build 路径前缀), 遇到区域合规限制用代理绕过验证成功
2. 稳定币脱锚监控扩展: 接入 Jupiter (Solana) + KyberSwap (EVM) 双源询价, 监控 29 个生息稳定币, 10 分钟一轮, 下跌超 5% 提醒; 真实发现多币脱锚 (有币种已归零、部分 -4~35%)
3. 代码安全审核实践: 对开源“套利机器人”做完整安全审计 (私钥外发/异常请求/后门/风控), 确认架构专业但盈利依赖市场条件; 建立“所有代码先审核再用”的铁律
4. 感悟: API 接入的坑集中在认证格式 (Key 是 UUID.UUID 组合、签名要带路径前缀) 和限流; DEX 报价要防池子价格操纵

笔记 108: 02.1 搜索对象、页面结构与显示边界

作者: petree1 | 时间: 2026-08-15 13:56 UTC | 字数: 11868 | 评分: 90

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

02.1 搜索对象、页面结构与显示边界

1. 本节解决什么问题

扫链时, 你经常会拿到一个名称、一段 `0x` 字符串或一个区块数字。把它粘贴到 BscScan 后, 页面可能显示代币名称、地址标签、价格、持有人和多种活动。

第一步需要回答:

我查到的究竟是哪个网络上的什么对象, 哪些内容是稳定标识, 哪些内容只是 BscScan 当前的解释和展示?

1.1 主问题链

我拿到什么搜索线索

- 它属于哪个网络
- 它对应区块、交易还是地址
- 当前打开了哪一种页面视角
- 哪个字段可以稳定固定对象
- 哪些名称、标签和统计需要另行复核

1.2 第一遍必须建立的最低模型

第一次阅读先掌握六点:

1. 同一个地址可以在不同网络上对应不同对象, 记录地址时必须同时记录网络。
2. 交易哈希用于固定一笔交易, 区块高度用于固定交易所在区块。
3. EOA、普通合约、Token 合约和池合约的地址外形相同, 仅靠字符长度无法分类。
4. 同一个 Token 合约地址可以打开地址视角和代币视角, 底层对象仍是同一个地址。
5. 名称、符号、Logo、标签和社交链接可能包含链下提交或平台加工信息。
6. 搜索结果帮助你导航, 结论仍要回到完整地址、交易字段、回执、日志、代码或状态。

2. 从一个具体场景开始

有人发来一句话:

USDT 合约是某段缩写地址

这里至少有四个问题尚未解决:

1. 这是 BNB Smart Chain、Ethereum, 还是其他网络上的地址。
2. 当前只有缩写, 完整地址是否正确。
3. “USDT”来自合约元数据、BscScan 页面名称, 还是社区称呼。
4. 该地址当前打开的是地址页还是 Token 页。

正确的起点是先固定:

网络: BNB Smart Chain

完整合约地址: 0x55d398326f99059ff775485246999027b3197955

名称用于帮助阅读。网络与完整地址负责固定研究对象。

3. 先认识五种搜索对象

3.1 区块高度

区块高度是区块在链上的顺序编号，例如：

113374190

在 BscScan 搜索这个数字，目标是打开对应区块页。区块页用于查看区块时间、区块哈希、出块者、交易数量和该区块中的交易。数字本身也可能是 Token 数量或其他参数。只有放进区块搜索语境后，它才被当作区块高度。

3.2 区块哈希

区块哈希用于标识具体区块内容，通常是 0x 加六十四个十六进制字符。

0x3b11fe73ab7cd2d0327b44c688853eb284063285d549d53e95826e01564c545f

区块高度便于人阅读顺序。区块哈希用于更精确地绑定该高度下的具体区块内容。

3.3 交易哈希

交易哈希也通常是 0x 加六十四个十六进制字符。

0x278d43dea3775f4d98978fe0f15b0ef5b70e1c312b32d045564cee6b783e7fed

它用于打开交易详情页。交易页会组合展示顶层交易字段、回执结果、日志解码、标签和其他平台加工内容。

交易哈希和区块哈希外形相同。仅凭长度不能确定类别，需要结合来源、搜索结果和打开后的页面类型。

3.4 地址

EVM 地址通常是 0x 加四十个十六进制字符。

0x55d398326f99059ff775485246999027b3197955

以下对象都使用同样的地址外形：

1. 传统 EOA。
2. EIP-7702 委托账户。
3. 普通合约。
4. Token 合约。
5. DEX Router。
6. 流动性池。

地址格式只支持“它看起来像 EVM 地址”这一结论。账户类型、合约功能和真实身份需要其他证据。

3.5 Token 合约地址

Token 合约地址仍然是一个合约地址。BscScan 可以为它提供专门的 Token 页面，集中显示：

1. 名称与符号。
2. 总供应量和精度。
3. 转账记录。
4. 持有人统计。
5. DEX 活动。
6. 合约代码和 ABI。

Token 页面是一种阅读视角。它没有创造新的链上地址。

4. 搜索前先固定网络

4.1 为什么网络必须和对象一起记录

同一段地址可以出现在多个 EVM 网络上。不同网络各自维护账户状态、代码、余额和交易历史。

完整对象应写成：

Chain ID 56

+

0x55d398326f99059ff775485246999027b3197955

只保存地址会丢失网络边界。

4.2 BscScan 主网与测试网入口

| 入口 | 用途 |

| :----- | :----- |

| bscscan.com | BNB Smart Chain 主网浏览器。 |

| testnet.bscscan.com | BNB Smart Chain 测试网浏览器。 |

查看页面时先检查域名和页面网络标识。测试网交易哈希不能在主网交易库中找到，反向也一样。

4.3 搜索不到对象时先检查什么

完整字符串是否复制正确

- 当前是否选择正确网络
- 对象是否已经入块或被索引
- 输入的是地址、交易哈希还是区块哈希
- 当前入口是否存在展示延迟

“未搜索到”只描述当前入口的结果。它不能直接证明该对象在所有网络和数据源中都不存在。

5. BscScan 页面地图

5.1 最常用的四类路径

| 页面路径 | 主要回答的问题 |

| :----- | :----- |

| /block/ | 这个区块包含什么，何时产生。 |

| /tx/ | 这笔交易提交了什么，执行结果如何。 |

| /address/ | 这个地址有哪些余额、代码和活动。 |

| /token/ | 这个 Token 合约有哪些元数据、转账、持有人和市场活动。 |

页面路径告诉你当前查看角度。对象性质仍需结合链上数据判断。

5.2 地址页主要看什么

地址页通常围绕账户视角组织信息：

1. 地址和标签。
2. BNB 余额与 Token 资产摘要。
3. 普通交易。
4. 内部交易或内部调用相关展示。
5. Token 活动。
6. 合约代码、创建信息和可读写入口。

页面选项会随地址类型和 BscScan 产品更新而变化。

5.3 Token 页主要看什么

Token 页围绕代币视角组织信息：

1. Token 合约地址。
2. 名称、符号、精度和供应量。
3. Transfers。
4. Holders。
5. DEX Trades。
6. Contract 与 Analytics。
7. Logo、网站和社交资料。

同一页面可能混合链上字段、合约调用结果、平台统计和项目方提交资料。读取时需要分层。

5.4 交易页主要看什么

交易页通常组合：

交易哈希与区块位置

- 状态、时间与费用
- 顶层 from、to、value 和 Input
- BscScan 方法名与 Action 摘要
- Receipt Event Logs
- 内部调用和资产展示入口

第二章后续小节会逐层拆解这些区域。02.1 只要求你知道页面中存在原始字段和展示加工两类内容。

6. 同一地址为什么会有两个页面视角

6.1 微案例：地址视角与 Token 视角

教学路径：

/address/0x55d398326f99059ff775485246999027b3197955

- 以账户和合约活动为中心

/token/0x55d398326f99059ff775485246999027b3197955

→ 以代币转账、持有人和元数据为中心

两条路径中的完整地址相同，因此底层研究对象相同。页面把同一对象组织成了不同阅读视角。

6.2 页面视角不能替代对象分类

打开 /token/ 页面可以支持“BscScan 当前把该合约作为 Token 展示”。如果要确认它实现了哪些方法、怎样计算余额、是否存在特殊转账逻辑，仍需查看代码、ABI、调用和状态。

打开 /address/ 页面也不代表该对象一定是传统 EOA。合约、池子、Router 和委托账户都可以拥有地址页。

7. 页面内容分成四层

7.1 第一层：稳定定位字段

常见对象包括：

1. 网络与 Chain ID。
2. 完整地址。
3. 交易哈希。
4. 区块高度与区块哈希。

它们用于固定研究对象和复现范围。

7.2 第二层：链上字段与可复现状态

常见对象包括：

1. 顶层 from、to、value 和完整 input。
2. Receipt 的 status 和 Logs。
3. 合约代码。
4. 固定区块的余额与 Storage。
5. Token 合约返回的名称、符号和精度。

BscScan 页面仍属于工具展示。重要结论可以再通过 RPC、代码或另一工具复现。

7.3 第三层：BscScan 解码与摘要

常见内容包括：

1. 方法名称。
2. 参数名称和显示格式。
3. Transaction Action。
4. Event 名称与字段名称。
5. 内部活动摘要。

这些内容提升阅读效率。其准确性依赖 ABI、解码规则和平台数据管线。

7.4 第四层：标签、资料与当前统计

常见内容包括：

1. 地址标签和名称标签。
2. Token Logo、官网和社交链接。
3. 当前价格和完全稀释估值。
4. 当前持有人数量。
5. 平台提示和风险标记。

这些内容可能变化，也可能包含项目方提交或平台计算。使用时记录观察时间，并按研究问题回到原始证据复核。

8. 名称、符号、标签和地址的边界

8.1 名称与符号可能重复

任何人都可以部署名称或符号相同的 Token。搜索 USDT、DOGE 或热门 Meme 名称时，可能出现多个合约。

因此筛选 Token 的顺序是：

确认网络

- 取得完整合约地址
- 检查代码与元数据
- 检查创建者、流动性和活动
- 最后使用名称帮助阅读

8.2 地址标签是什么

地址标签是平台对地址的附加说明，例如交易所、Router、项目金库或可疑地址。标签可能来自公开资料、项目提交或平台归因。

标签可以作为调查入口。实体身份和控制关系仍需证据等级与置信度。

8.3 验证源码表示什么

源码验证说明平台能够把提交的源码与该地址的链上字节码进行匹配。它有助于阅读代码和 ABI。

源码验证不能单独证明：

1. 合约没有风险。
2. 当前管理员可信。
3. 代理实现不会更换。
4. 页面标签代表真实控制者。
5. Token 一定可以正常卖出。

9. 最小搜索与记录流程

9.1 收到一段搜索线索

先保存原始线索，不主动补全缩写地址。

原始线索

→ 来源

→ 声称所属网络

缩写地址必须回到原消息、官方渠道或可验证交易中取得完整值。

9.2 打开正确网络入口

核对 BscScan 域名和网络。不要只依赖浏览器标签页标题。

9.3 根据页面类型确认对象

检查：

1. URL 路径。
2. 页面标题。
3. 完整地址或哈希。
4. 区块位置。
5. 页面当前视角。

9.4 保存稳定标识

普通固定历史对象最低记录：

网络

对象类型

完整地址或交易哈希

涉及交易时记录区块高度

平台标签、当前余额、持有人数量或价格参与结论时，再记录观察时间和页面来源。

9.5 拆分页面显示

把页面信息分别放入：

稳定定位字段

链上字段或状态

平台解码与摘要

标签、资料与当前统计

这样可以避免把页面名称写成链上身份，把解码参数写成原始字段。

10. 完整案例：固定 BSC-USD 研究对象

10.1 案例标识

网络：BNB Smart Chain

Chain ID：56

Token 合约：0x55d398326f99059ff775485246999027b3197955

交易哈希：0x278d43dea3775f4d98978fe0f15b0ef5b70e1c312b32d045564cee6b783e7fed

区块高度：113374190

区块哈希：0x3b11fe73ab7cd2d0327b44c688853eb284063285d549d53e95826e01564c545f

10.2 当前页面名称与合约返回值

2026 年 8 月 12 日保存的复核结果为：

BscScan Token 页面名称：Binance-Peg BSC-USD

合约 name 返回值：Tether USD

合约 symbol 返回值：USDT

合约 decimals 返回值：18

这些字段服务不同用途：

| 字段 | 本案例中的作用 |

| :----- | :----- |

| 网络与完整合约地址 | 稳定固定 Token 对象。 |

| BscScan 页面名称 | 记录平台当时如何展示该对象。 |

| 合约 name 与 symbol | 记录合约方法在查询时返回的元数据。 |

| 社区简称 USDT 或 BSC-USD | 帮助交流，需要回到完整地址消除歧义。 |

页面名称与合约元数据不同，不代表其中一方一定错误。它说明平台展示名称可以采用自己的命名口径。

10.3 同一对象的页面视角

地址视角

<https://bscscan.com/address/0x55d398326f99059ff775485246999027b3197955>

Token 视角

<https://bscscan.com/token/0x55d398326f99059ff775485246999027b3197955>

两个页面使用同一完整地址。地址页适合查看账户与合约活动，Token 页适合查看代币元数据、Transfers 和 Holders。

10.4 这笔交易提供什么定位关系

交易哈希

→ 打开交易页

→ 区块高度 113374190

→ tx.to 指向 Token 合约

→ 完整 Input 以 0xa9059cbb 开头

→ Receipt 中保留该 Token 合约发出的 Transfer Log

transfer、recipient 和 amount 属于 ABI 解码后的解释。完整 tx.input 才是原始交易字段。后续 02.4 会系统练习这一区分。

10.5 能确认与仍需保留的边界

可以确认：

1. 该交易位于 BNB Smart Chain 的固定区块。
2. 顶层 tx.to 指向上述 Token 合约地址。
3. 当前研究对象可以由网络与完整合约地址稳定固定。
4. BscScan 页面名称和合约返回元数据属于不同字段来源。

需要保留边界：

1. BscScan 当前实时名称可能在 2026 年 8 月 12 日以后变化。
2. 当前价格、持有人数量和标签不能回填成交易发生时的历史事实。
3. 本节未重新证明项目方、桥接发行方或真实控制者身份。
4. BscScan 实时对象页在 2026 年 8 月 14 日复核时限制自动访问，因此没有把未取得的新页面内容写成当前事实。

11. 常见误判与纠正

| 误判 | 正确检查方向 |

| :----- | :----- |

| 名称相同，所以是同一个 Token | 比较网络与完整合约地址。 |

| 地址以 0x 开头，所以是钱包 EOA | 检查代码、交易能力和固定区块状态。 |

| 六十四位哈希一定是交易哈希 | 结合来源和页面类型区分交易哈希与区块哈希。 |

| Token 页和地址页代表两个对象 | 比较两页的完整地址。 |

| BscScan 标签等于真实控制人 | 把标签记为平台线索，再调查资金、部署、权限和公开资料。 |

| 已验证源码等于安全 | 继续检查权限、代理、转账逻辑、流动性和真实交易结果。 |

| 当前 Holders 可以证明历史持有人结构 | 固定历史区块并使用支持历史状态的数据源。 |

| Transaction Action 是原始交易字段 | 回到 from、to、value、完整 Input、Receipt 和 Logs。 |

| 搜索不到就证明链上不存在 | 检查网络、入块状态、索引延迟和其他数据源。 |

| 缩写地址足以写入研究报告 | 取得并保存完整地址。 |

12. BscScan 页面地图产物

12.1 最小记录模板

网络：
对象类型：
完整地址、交易哈希或区块高度：
当前页面路径：
页面主要视角：
稳定定位字段：
平台解码或摘要：
名称与标签：
动态字段观察时间：
需要进一步复核的主张：

12.2 页面选择卡

当前问题	优先打开的页面
交易是否成功，调用了谁	交易页。
交易位于哪个区块和顺序	区块页与交易页。
地址有哪些活动和代码	地址页。
Token 有哪些转账和持有人	Token 页。
合约源码、ABI 和代理信息	地址页或 Token 页中的 Contract 区域。

13. 官方来源与时效边界

13.1 必要来源

关键主张	来源、核查日期和适用边界
Token 页面混合链上与链下数据，并提供 Transfers、Holders、Info、DEX Trades、Contract 和 Analytics 等视角	BscScan Token 页面说明 (<https://info.bscscan.com/token/>)，2026-08-14 核查。页面布局和标签会更新。
交易哈希用于查询交易详情，交易页展示状态和交易信息	BscScan 交易说明 (<https://info.bscscan.com/understanding-a-transaction/>) 与 查看交易 (<https://info.bscscan.com/txs/>)，2026-08-14 核查。具体字段布局会更新。
区块页面提供高度、时间和区块内交易信息	BscScan 区块列表说明 (<https://info.bscscan.com/blocks/>) 与 区块详情说明 (<https://info.bscscan.com/block/>)，2026-08-14 核查。内部活动统计口径需另行确认。

13.2 时效边界

- 1. BscScan 页面布局、名称标签、地址标签、价格和统计字段会变化。
- 2. Token Logo、网站、社交链接和部分项目资料可能来自链下提交。
- 3. 对动态页面主张使用时，应保存观察时间和必要页面证据。
- 4. 实时页面访问失败时，保留数据缺口，不推测页面当前内容。

14. 压缩总结与覆盖检查

14.1 最短搜索规则

先定网络
→ 再定对象类型
→ 保存完整地址、哈希或区块
→ 识别当前页面视角
→ 分开链上字段、平台解码和标签统计
→ 对重要结论回到原始证据复核

14.2 正文覆盖检查

能力目标	覆盖位置与当前状态
搜索区块、交易和地址	第 3 至 5 节已建立页面地图。
区分网络与稳定对象	第 2 至 4 节已建立最低模型。
区分地址页和 Token 页	第 5 至 6 节已覆盖。
区分链上字段、解码和标签	第 7 至 8 节已覆盖。
形成可复用搜索流程	第 9 节和第 12 节已覆盖。
使用真实案例固定对象	第 10 节已示范。
解释、定位、执行和判断能力	正文学习中，题目层尚未启动。

15. 本节上下文交接卡

| 字段 | 内容 |

| :----- | :----- |

| | 本节核心结论 |

BscScan

搜索首先固定网络和完整链上标识。页面路径表示阅读视角，名称、标签、解码和统计需要按信息层使用。|

| 正文生成状态 | 2026 年 8 月 14 日按第一遍可读性规则生成，当前进入正文学习。|

| 已经掌握的动作 | 尚未通过本节互动评估。|

| 仍然容易出错的点 | 等待学习反馈。重点观察网络遗漏、哈希类型、地址页与 Token 页、原始字段与页面解码的混淆。|

| 已保存的真实案例 | BNB Smart Chain 的 BSC-USD 合约与固定历史转账，包含完整合约地址、交易哈希、区块高度和区块哈希。|

| 案例证据缺口 | 2026 年 8 月 14 日无法通过自动访问取得实时 BscScan 对象页。当前页面名称沿用 2026 年 8 月 12 日保存的复核结果。|

| 必须继承的术语 | 网络、Chain ID、区块高度、区块哈希、交易哈希、地址、Token 合约、页面视角、平台标签。|

| 不要重复展开的内容 | 账户分类已在 01.2 建立，Input 与解码边界已在 01.6 建立，工具证据等级已在 01.8 建立。|

| 留给下一节的问题 | 怎样把交易放回区块位置，并区分区块时间、观察时间和当前确认状态。|

| 跨章节接口 | 页面定位交付 02.2 至 02.12，名称和地址边界交付第四章至第七章。|

| 教学互动状态 | 正文学习中。|

| 题目层状态 | 未启动。完成正文后只推荐下一项互动。|

| 独立任务状态 | 未进入，属于选做。|

| 案例内复习状态 | 正文已自然复习网络与对象绑定、原始 Input 与 ABI 解码、平台名称与稳定合约地址分离。|

| 四维能力状态 | 解释、定位、执行和判断均待后续案例评估。|

| 下一小节建议 | 完成 02.1 后进入 02.2。|